Sliper Kod Dökümü

Oluşturulma tarihi: 04.11.2026 11:45
Dosya sayısı: 37
Toplam satır: 13605

Dosyalar

1. CMakeLists.txt
2. esp32\sliper_esp32\sliper_esp32.ino
3. main.cpp
4. qml\components\NavIkon.qml
5. qml\components\SuTerazisi.qml
6. qml\components\TarihSecici.qml
7. qml\Main.qml
8. qml\pages\DashboardPage.qml
9. qml\pages\GecmisPage.qml
10. qml\pages\KalibrasyonPage.qml
11. qml\pages\LoginPage.qml
12. qml\pages\OlcumPage.qml
13. qml\pages\RehberMockup.qml
14. qml\pages\RehberPage.qml
15. qml\pages\SonuclarPage.qml
16. qml\Translations.qml
17. src\Backend.cpp
18. src\Backend.h
19. src\Calculator.cpp
20. src\Calculator.h
21. src\Database.cpp
22. src\Database.h
23. src\ReportManager.cpp
24. src\ReportManager.h
25. src\SensorManager.cpp
26. src\SensorManager.h
27. src\SliperModel.cpp
28. src\SliperModel.h
29. src\VoiceCommandManager.cpp
30. src\VoiceCommandManager.h
31. src\VoiceRecognitionWorker.cpp
32. src\VoiceRecognitionWorker.h
33. src\WifiManager.cpp
34. src\WifiManager.h
35. tools\esp32_baglanti_testi.py
36. tools\esp32_seri_dinle.py
37. tools\pmaks_yeniden_hesapla.py

CMakeLists.txt

04.11.2026 11:45

```
cmake_minimum_required(VERSION 3.16)
project(sliper VERSION 1.0)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_PREFIX_PATH "C:/Qt/6.7.3/mingw_64")

find_package(Qt6 REQUIRED COMPONENTS Core Quick Charts Widgets Sql PrintSupport Network Multimedia)

qt_standard_project_setup()
qt_policy(SET QTP0001 NEW)

qt_add_executable(sliper
    main.cpp
    src/Backend.h
    src/Backend.cpp
    src/SensorManager.h
    src/SensorManager.cpp
    src/Calculator.h
    src/Calculator.cpp
    src/SliperModel.h
    src/SliperModel.cpp
    src/Database.h
    src/Database.cpp
    src/ReportManager.h
    src/ReportManager.cpp
    src/WifiManager.h
    src/WifiManager.cpp
    src/VoiceCommandManager.h
    src/VoiceCommandManager.cpp
    src/VoiceRecognitionWorker.h
    src/VoiceRecognitionWorker.cpp
)

set_source_files_properties(qml/Translations.qml PROPERTIES
    QT_QML_SINGLETON_TYPE TRUE
)

qt_add_qml_module(sliper
    URI sliper
    VERSION 1.0

    QML_FILES
        qml/Main.qml
        qml/Translations.qml
        qml/pages/LoginPage.qml
        qml/pages/DashboardPage.qml
        qml/pages/OlcumPage.qml
        qml/pages/SonuclarPage.qml
        qml/pages/GecmisPage.qml
        qml/pages/KalibrasyonPage.qml
        qml/pages/RehberPage.qml
        qml/pages/RehberMockup.qml
        qml/components/SuTerazisi.qml
        qml/components/TarihSecici.qml
        qml/components/NavIkon.qml

    RESOURCES
        qml/assets/logo.png
        qml/assets/egim_sensor.png
        qml/assets/loadcell.png
        qml/assets/mesafe_sensor.png
        qml/assets/slipergif.mp4
)

target_link_libraries(sliper PRIVATE
    Qt6::Core
    Qt6::Quick
    Qt6::Charts
```

```
    Qt6::Widgets
    Qt6::Sql
    Qt6::PrintSupport
    Qt6::Network
    Qt6::Multimedia
)

# --- Sesli komut (Vosk, offline Turkce konusma tanima) ---
set(VOSK_DIR "${CMAKE_SOURCE_DIR}/third_party/vosk")
set(VOSK_MODEL_DIR "${CMAKE_SOURCE_DIR}/resources/vosk-model-small-tr-0.3")

target_include_directories(sliper PRIVATE ${VOSK_DIR})
target_link_libraries(sliper PRIVATE "${VOSK_DIR}/libvosk.lib")

# libvosk.dll kendi MinGW calisma zamanini (libstdc++/libwinpthread/libgcc_s)
# gerektirir. Vosk paketiyle gelen bu DLL'leri DEGIL, projenin kullandigi
# Qt MinGW derleyicisiyle ayni surumden gelenleri kopyaliyoruz; aksi halde
# surum uyusmazligi uygulamanin ENTRYPOINT_NOT_FOUND ile cokmesine yol acar.
get_filename_component(MINGW_BIN_DIR "${CMAKE_CXX_COMPILER}" DIRECTORY)

add_custom_command(TARGET sliper POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
        "${VOSK_DIR}/libvosk.dll"
        "${MINGW_BIN_DIR}/libstdc++-6.dll"
        "${MINGW_BIN_DIR}/libwinpthread-1.dll"
        "${MINGW_BIN_DIR}/libgcc_s_seh-1.dll"
        "$<TARGET_FILE_DIR:sliper>"
    COMMENT "Vosk kutuphaneleri kopyalaniyor"
)

add_custom_command(TARGET sliper POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory
        "${VOSK_MODEL_DIR}"
        "$<TARGET_FILE_DIR:sliper>/vosk-model-small-tr-0.3"
    COMMENT "Vosk Turkce ses tanima modeli kopyalaniyor"
)
```

esp32\sliper_esp32\sliper_esp32.ino

04.11.2026 11:45

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <HX711.h>
#include <Adafruit_ADS1X15.h>
#include <ArduinoJson.h>
#include <WiFi.h>
#include <lwip/sockets.h>
#include <esp_system.h>

#define I2C_SDA 17
#define I2C_SCL 18

#define HX711_DT 39
#define HX711_SCK 38

const uint8_t TOF250_I2C_ADRESI = 0x52;
const uint8_t TOF250_MESAFE_REGISTERI = 0x00;
const long TOF250_MM_CARPANI = 1;
const unsigned long TOF250_OKUMA_ARALIGI_MS = 100;
const unsigned long TOF250_ZAMAN_ASIMI_MS = 500;
const long TOF250_MAKSIMUM_CM = 250;
const unsigned long HX711_ZAMAN_ASIMI_MS = 500;

#define BATARYA_ADS_KANALI 1

Adafruit_MPU6050 mpu;
HX711 loadCell;
Adafruit_ADS1115 ads;

bool mpuHazir = false;
unsigned long sonMpuDenemeMs = 0;
const unsigned long MPU_DENEME_ARALIGI_MS = 2000;

bool adsHazir = false;
unsigned long sonAdsDenemeMs = 0;
const unsigned long ADS_DENEME_ARALIGI_MS = 2000;

const uint8_t ADS1115_I2C_ADRESI = 0x48;
const uint8_t MPU6050_I2C_ADRESI = 0x68;
const unsigned long ADS_DONUSUM_ZAMAN_ASIMI_MS = 20;

unsigned long i2cHataSayaci = 0;
unsigned long i2cYenidenKurmaSayisi = 0;
const unsigned long I2C_YENIDEN_KURMA_ESIGI = 10;

const char *WIFI_SSID = "SLIPER-ESP32";
const char *WIFI_SIFRE = "sliper1234";
const uint16_t TCP_PORT = 8888;

WiFiServer tcpSunucu(TCP_PORT);
WiFiClient bagliIstemci;

void yenidenBaslatmaSebebiniYazdir() {
    const esp_reset_reason_t sebep = esp_reset_reason();
    Serial.print("Yeniden baslatma sebebi: ");
    switch (sebep) {
        case ESP_RST_POWERON:  Serial.println("Normal guc verme."); break;
        case ESP_RST_SW:       Serial.println("Yazilimsal reset."); break;
        case ESP_RST_PANIC:    Serial.println("PANIC/exception - kod cokmus!"); break;
        case ESP_RST_TASK_WDT: Serial.println("TASK WATCHDOG - loop() kilitlenmis!"); break;
        case ESP_RST_INT_WDT:  Serial.println("INTERRUPT WATCHDOG - kesme kilitlenmis!"); break;
        case ESP_RST_WDT:      Serial.println("WATCHDOG."); break;
        case ESP_RST_BROWNOUT: Serial.println("BROWNOUT - besleme voltaji dustu!"); break;
        case ESP_RST_DEEPSLEEP:Serial.println("Derin uykudan uyanma."); break;
        default:               Serial.println(sebep); break;
    }
}
```

```
void setup() {
    Serial.begin(115200);
    delay(200);

    yenidenBaslatmaSebebiniYazdir();

    Wire.begin(I2C_SDA, I2C_SCL);
    Wire.setClock(100000);
    Wire.setTimeOut(50);
    delay(100);

    if (!i2cCihazKararliCevapVeriyor(MPU6050_I2C_ADRESI) || !mpu.begin()) {
        Serial.println("MPU6050 bulunamadi! Loop icinde tekrar denenecek.");
        mpuHazir = false;
    } else {
        mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
        mpu.setGyroRange(MPU6050_RANGE_250_DEG);
        mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
        Serial.println("MPU6050 hazir.");
        mpuHazir = true;
    }

    loadCell.begin(HX711_DT, HX711_SCK);
    if (loadCell.wait_ready_timeout(2000)) {
        Serial.println("HX711 hazir.");
    } else {
        Serial.println("HX711 bulunamadi!");
    }

    if (!i2cCihazKararliCevapVeriyor(ADS1115_I2C_ADRESI) || !ads.begin()) {
        Serial.println("ADS1115 bulunamadi!");
        adsHazir = false;
    } else {
        ads.setGain(GAIN_ONE);
        ads.setDataRate(RATE_ADS1115_860SPS);
        Serial.println("ADS1115 hazir.");
        adsHazir = true;
    }

    WiFi.softAP(WIFI_SSID, WIFI_SIFRE);
    WiFi.setSleep(false);
    Serial.print("SoftAP baslatildi. IP adresi: ");
    Serial.println(WiFi.softAPIP());

    tcpSunucu.begin();
    Serial.println("TCP sunucu baslatildi, baglanti bekleniyor...");
}

const unsigned long ORNEK_ARALIGI_MS = 20;
const unsigned long BATARYA_ARALIGI_MS = 1000;
const unsigned long MPU_ARALIGI_MS = 100;

#define SERI_DEBUG 0

unsigned long sonOrnekMs = 0;
unsigned long sonBataryaMs = 0;
unsigned long sonMpuMs = 0;
long sonHamAgirlik = 0;
unsigned long sonAgirlikZamaniMs = 0;
unsigned long atlananPaketSayisi = 0;
unsigned long gonderilenPaketSayisi = 0;
unsigned long sonDurumYazisiMs = 0;
const unsigned long DURUM_YAZISI_ARALIGI_MS = 1000;
int16_t sonHamBatarya = 0;
float sonAccelX = 0, sonAccelY = 0, sonAccelZ = 0;

bool soketYazmayaHazir(NetworkClient &istemci) {
    const int soket = istemci.fd();
    if (soket < 0) return false;
```

```
    fd_set yazmaKumesi;
    FD_ZERO(&yazmaKumesi);
    FD_SET(soket, &yazmaKumesi);
    struct timeval beklemeYok = {0, 0};
    return select(soket + 1, NULL, &yazmaKumesi, NULL, &beklemeYok) > 0;
}

void hamAgirlikGuncelle() {
    if (loadCell.is_ready()) {
        sonHamAgirlik = loadCell.read();
        sonAgirlikZamaniMs = millis();
    }
}

bool hx711Hazir() {
    return sonAgirlikZamaniMs != 0 && millis() - sonAgirlikZamaniMs < HX711_ZAMAN_ASIMI_MS;
}

bool i2cCihazCevapVeriyor(uint8_t adres) {
    Wire.beginTransmission(adres);
    Wire.write((uint8_t)0x00);
    return Wire.endTransmission() == 0;
}

bool i2cCihazKararliCevapVeriyor(uint8_t adres) {
    if (!i2cCihazCevapVeriyor(adres)) return false;
    delay(2);
    return i2cCihazCevapVeriyor(adres);
}

void i2cVeriYoluSagliginiKontrolEt() {
    if (i2cHataSayaci < I2C_YENIDEN_KURMA_ESIGI) return;

    Serial.println("I2C hatti cevapsiz, veri yolu yeniden kuruluyor...");
    Wire.end();
    delay(50);
    Wire.begin(I2C_SDA, I2C_SCL);
    Wire.setClock(100000);
    Wire.setTimeOut(50);
    i2cHataSayaci = 0;
    i2cYenidenKurmaSayisi++;
}

int16_t adsKanalOku(uint8_t kanal) {
    if (!adsHazir) return 0;

    if (!i2cCihazCevapVeriyor(ADS1115_I2C_ADRESI)) {
        Serial.println("ADS1115 cevap vermiyor, yeniden baslatilacak.");
        adsHazir = false;
        i2cHataSayaci++;
        return 0;
    }

    ads.startADCReading(MUX_BY_CHANNEL[kanal], false);

    const unsigned long baslangicMs = millis();
    while (!ads.conversionComplete()) {
        if (millis() - baslangicMs > ADS_DONUSUM_ZAMAN_ASIMI_MS) {
            Serial.println("ADS1115 donusumu zaman asimina ugradi.");
            adsHazir = false;
            return 0;
        }
    }

    return ads.getLastConversionResults();
}

bool tofHamMesafeOku(uint16_t &hamMesafe) {
    Wire.beginTransmission(TOF250_I2C_ADRESI);
```

```
    Wire.write(TOF250_MESAFE_REGISTERI);
    if (Wire.endTransmission(false) != 0) {
        Wire.beginTransmission(TOF250_I2C_ADRESI);
        Wire.write(TOF250_MESAFE_REGISTERI);
        if (Wire.endTransmission(true) != 0) return false;
    }
    if (Wire.requestFrom((int)TOF250_I2C_ADRESI, 2) != 2) return false;
    const uint8_t yuksekBayt = Wire.read();
    const uint8_t dusukBayt = Wire.read();
    hamMesafe = ((uint16_t)yuksekBayt << 8) | dusukBayt;
    return true;
}

long sonMesafeMm = 0;
unsigned long sonMesafeZamaniMs = 0;
unsigned long sonTofOkumaMs = 0;

void tofMesafeGuncelle() {
    if (millis() - sonTofOkumaMs < TOF250_OKUMA_ARALIGI_MS) return;
    sonTofOkumaMs = millis();

    uint16_t hamMesafe;
    if (!tofHamMesafeOku(hamMesafe)) {
        i2cHataSayaci++;
        return;
    }
    const long mesafeMm = (long)hamMesafe * TOF250_MM_CARPANI;
    if (mesafeMm <= TOF250_MAKSIMUM_CM) {
        sonMesafeMm = mesafeMm;
        sonMesafeZamaniMs = millis();
    }
}

bool tofHazir() {
    return sonMesafeZamaniMs != 0 && millis() - sonMesafeZamaniMs < TOF250_ZAMAN_ASIMI_MS;
}

int16_t hamBataryaOku() {
    return adsKanalOku(BATARYA_ADS_KANALI);
}

void loop() {
    if (bagliIstemci && !bagliIstemci.connected()) {
        bagliIstemci.stop();
        Serial.println("PC baglantisi kapandi.");
    }

    WiFiClient yeniIstemci = tcpSunucu.available();
    if (yeniIstemci) {
        if (bagliIstemci) {
            bagliIstemci.stop();
        }
        bagliIstemci = yeniIstemci;
        bagliIstemci.setNoDelay(true);
        Serial.println("PC baglandi.");
    }

    if (!mpuHazir && millis() - sonMpuDenemeMs > MPU_DENEME_ARALIGI_MS) {
        sonMpuDenemeMs = millis();
        if (!i2cCihazKararliCevapVeriyor(MPU6050_I2C_ADRESI)) {
            i2cHataSayaci++;
        } else if (mpu.begin()) {
            mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
            mpu.setGyroRange(MPU6050_RANGE_250_DEG);
            mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
            Serial.println("MPU6050 sonradan hazir oldu.");
            mpuHazir = true;
        } else {
            Serial.println("MPU6050 hala bulunamadi, tekrar denenecek...");
        }
```

```
}

if (!adsHazir && millis() - sonAdsDenemeMs > ADS_DENEME_ARALIGI_MS) {
    sonAdsDenemeMs = millis();
    if (!i2cCihazKararliCevapVeriyor(ADS1115_I2C_ADRESI)) {
        i2cHataSayaci++;
    } else if (ads.begin()) {
        ads.setGain(GAIN_ONE);
        ads.setDataRate(RATE_ADS1115_860SPS);
        Serial.println("ADS1115 sonradan hazir oldu.");
        adsHazir = true;
        i2cHataSayaci = 0;
    } else {
        Serial.println("ADS1115 ACK veriyor ama baslatilamadi.");
    }
}

i2cVeriYoluSagliginiKontrolEt();

hamAgirlikGuncelle();
tofMesafeGuncelle();

const unsigned long simdi = millis();
if (simdi - sonOrnekMs < ORNEK_ARALIGI_MS) {
    return;
}
sonOrnekMs = simdi;

if (simdi - sonBataryaMs >= BATARYA_ARALIGI_MS) {
    sonBataryaMs = simdi;
    sonHamBatarya = hamBataryaOku();
}

if (mpuHazir && simdi - sonMpuMs >= MPU_ARALIGI_MS) {
    sonMpuMs = simdi;
    if (!i2cCihazCevapVeriyor(MPU6050_I2C_ADRESI)) {
        Serial.println("MPU6050 cevap vermiyor, yeniden baslatilacak.");
        mpuHazir = false;
        i2cHataSayaci++;
    } else {
        sensors_event_t ivme, gyro, sicaklik;
        mpu.getEvent(&ivme, &gyro, &sicaklik);
        sonAccelX = ivme.acceleration.x;
        sonAccelY = ivme.acceleration.y;
        sonAccelZ = ivme.acceleration.z;
    }
}

StaticJsonDocument<256> doc;
doc["t"] = simdi;
doc["hamAgirlik"] = sonHamAgirlik;
doc["hamMesafe"] = sonMesafeMm;
doc["mesafeMm"] = sonMesafeMm;
doc["tofGecerli"] = tofHazir();
doc["hxGecerli"] = hx711Hazir();
doc["hamBatarya"] = sonHamBatarya;
doc["accelX"] = round(sonAccelX * 1000) / 1000.0;
doc["accelY"] = round(sonAccelY * 1000) / 1000.0;
doc["accelZ"] = round(sonAccelZ * 1000) / 1000.0;

String json;
serializeJson(doc, json);
json += "\n";

if (bagliIstemci && bagliIstemci.connected()) {
    if (soketYazmayaHazir(bagliIstemci)) {
        bagliIstemci.print(json);
        gonderilenPaketSayisi++;
    } else {
        atlananPaketSayisi++;
```

```
        }
    }

    if (simdi - sonDurumYazisiMs >= DURUM_YAZISI_ARALIGI_MS) {
        sonDurumYazisiMs = simdi;
        Serial.printf("[%lus] istemci=%d gonderilen=%lu atlanan=%lu tof=%d mesafe=%ldmm ads=%d mpu=%d hx=%d i2cHata=%lu
i2cReset=%lu\n",
                      simdi / 1000,
                      (bagliIstemci && bagliIstemci.connected()) ? 1 : 0,
                      gonderilenPaketSayisi, atlananPaketSayisi,
                      tofHazir() ? 1 : 0, sonMesafeMm,
                      adsHazir ? 1 : 0, mpuHazir ? 1 : 0,
                      hx711Hazir() ? 1 : 0,
                      i2cHataSayaci, i2cYenidenKurmaSayisi);
    }

#if SERI_DEBUG
    Serial.print(json);
#endif
}
```

main.cpp

04.11.2026 11:45

```
#include <QQmlContext>
#include <QApplication>
#include <QQmlApplicationEngine>
#include <QDebug>
#include <QtGlobal>
#include "src/Backend.h"
#include "src/SensorManager.h"
#include "src/Calculator.h"
#include "src/Database.h"
#include "src/ReportManager.h"
#include "src/WifiManager.h"
#include "src/VoiceCommandManager.h"

int main(int argc, char *argv[])
{
    qputenv("QT_QUICK_CONTROLS_STYLE", "Basic");

    QApplication app(argc, argv);
    QApplication::setApplicationName("SLIPER");

    Backend backend;
    SensorManager sensorManager;
    Calculator calculator;
    Database database;
    ReportManager reportManager(&database);
    WifiManager wifiManager(&sensorManager, &database);
    VoiceCommandManager voiceCommandManager;

    const QVariantMap konumSinirlari = database.kalibrasyonGetir("konum_sinirlari");
    if (konumSinirlari.value("mevcut").toBool()) {
        calculator.setUstKonumMm(konumSinirlari.value("deger1").toDouble());
        calculator.setAltKonumMm(konumSinirlari.value("deger2").toDouble());
    }

    QQmlApplicationEngine engine;

    engine.rootContext()->setContextProperty("backend", &backend);
    engine.rootContext()->setContextProperty("sensorManager", &sensorManager);
    engine.rootContext()->setContextProperty("calculator", &calculator);
    engine.rootContext()->setContextProperty("database", &database);
    engine.rootContext()->setContextProperty("reportManager", &reportManager);
    engine.rootContext()->setContextProperty("wifiManager", &wifiManager);
    engine.rootContext()->setContextProperty("voiceCommandManager", &voiceCommandManager);

    QObject::connect(
        &engine,
        &QQmlApplicationEngine::objectCreationFailed,
        &app,
        []() {
            qWarning() << "QML nesnesi olusturulamadi!";
            QCoreApplication::exit(-1);
        },
        Qt::QueuedConnection);

    engine.loadFromModule("sliper", "Main");

    return app.exec();
}
```

qml\components\NavIkon.qml

04.11.2026 11:45

```
import QtQuick 6.7

Canvas {
    id: root
    width: 18
    height: 18
    property string tur: "olcum"
    property color renk: "#9ca3af"
    property real kalinlik: 1.6

    onRenkChanged: requestPaint()
    onTurChanged: requestPaint()
    onWidthChanged: requestPaint()
    onHeightChanged: requestPaint()

    onPaint: {
        var ctx = getContext("2d")
        ctx.reset()
        var s = Math.min(width, height) / 24
        ctx.save()
        ctx.scale(s, s)
        ctx.strokeStyle = renk
        ctx.fillStyle = renk
        ctx.lineWidth = kalinlik / s
        ctx.lineCap = "round"
        ctx.lineJoin = "round"

        switch (tur) {
        case "olcum": {
            var bars = [ [5, 20, 10], [12, 20, 4], [19, 20, 13] ]
            for (var i = 0; i < bars.length; i++) {
                ctx.beginPath()
                ctx.moveTo(bars[i][0], bars[i][1])
                ctx.lineTo(bars[i][0], bars[i][2])
                ctx.stroke()
            }
            break
        }
        case "sonuclar": {
            ctx.beginPath()
            ctx.moveTo(3, 17)
            ctx.lineTo(9.5, 10.5)
            ctx.lineTo(13.5, 14.5)
            ctx.lineTo(21, 6)
            ctx.stroke()

            ctx.beginPath()
            ctx.moveTo(15, 6)
            ctx.lineTo(21, 6)
            ctx.lineTo(21, 12)
            ctx.stroke()
            break
        }
        case "gecmis": {
            ctx.beginPath()
            ctx.arc(12, 12, 8.5, 0, Math.PI * 2)
            ctx.stroke()

            ctx.beginPath()
            ctx.moveTo(12, 7)
            ctx.lineTo(12, 12.5)
            ctx.lineTo(16, 14.5)
            ctx.stroke()
            break
        }
        case "kalibrasyon": {
            var cx = 12, cy = 12, rIn = 5.5, rOut = 8.5, teeth = 8
            ctx.beginPath()
```

```
            for (var t = 0; t < teeth; t++) {
                var a = (t / teeth) * Math.PI * 2
                var ax = cx + Math.cos(a) * rOut
                var ay = cy + Math.sin(a) * rOut
                var bx = cx + Math.cos(a) * rIn
                var by = cy + Math.sin(a) * rIn
                ctx.moveTo(bx, by)
                ctx.lineTo(ax, ay)
            }
            ctx.stroke()

            ctx.beginPath()
            ctx.arc(cx, cy, rIn, 0, Math.PI * 2)
            ctx.stroke()

            ctx.beginPath()
            ctx.arc(cx, cy, 2.2, 0, Math.PI * 2)
            ctx.stroke()
            break
        }
        case "rehber": {
            ctx.beginPath()
            ctx.moveTo(12, 6.5)
            ctx.bezierCurveTo(10.2, 5, 6.5, 4.5, 3.5, 5.2)
            ctx.lineTo(3.5, 17.5)
            ctx.bezierCurveTo(6.5, 16.8, 10.2, 17.3, 12, 18.8)
            ctx.stroke()

            ctx.beginPath()
            ctx.moveTo(12, 6.5)
            ctx.bezierCurveTo(13.8, 5, 17.5, 4.5, 20.5, 5.2)
            ctx.lineTo(20.5, 17.5)
            ctx.bezierCurveTo(17.5, 16.8, 13.8, 17.3, 12, 18.8)
            ctx.stroke()

            ctx.beginPath()
            ctx.moveTo(12, 6.5)
            ctx.lineTo(12, 18.8)
            ctx.stroke()
            break
        }
        }

        ctx.restore()
    }
}
```

qml\components\SuTerazisi.qml

04.11.2026 11:45

```
import QtQuick 6.7
import sliper

Item {
    id: root
    property real egimX: 0
    property real egimY: 0
    property real maxAci: 15

    readonly property var metinler: ({
        duzMetni: { tr: "Düz", en: "Level" },
        ayarlaniyorMetni: { tr: "Ayarlanıyor...", en: "Adjusting..." }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    width: 200
    height: 200

    readonly property real yaricap: (width / 2) - 20
    readonly property bool duz: Math.abs(egimX) < 0.5 && Math.abs(egimY) < 0.5

    readonly property real kabarcikOfsetX: Math.max(-1, Math.min(1, egimY / maxAci)) * yaricap
    readonly property real kabarcikOfsetY: Math.max(-1, Math.min(1, egimX / maxAci)) * yaricap

    Rectangle {
        anchors.fill: parent
        radius: width / 2
        color: "#12121a"
        border.color: "#1e2a3f"
        border.width: 2
    }

    Rectangle {
        anchors.centerIn: parent
        width: root.yaricap * 1.2
        height: width
        radius: width / 2
        color: "transparent"
        border.color: "#1e2a3f"
        border.width: 1
    }

    Rectangle {
        anchors.centerIn: parent
        width: parent.width - 20
        height: 1
        color: "#1e2a3f"
    }

    Rectangle {
        anchors.centerIn: parent
        width: 1
        height: parent.height - 20
        color: "#1e2a3f"
    }

    Rectangle {
        id: kabarcik
        width: 28
        height: 28
        radius: 14
        color: root.duz ? "#16a34a" : "#dc2626"
        border.color: root.duz ? "#4ade80" : "#f87171"
        border.width: 2
```

```
        x: (root.width / 2) - (width / 2) + root.kabarcikOfsetX
        y: (root.height / 2) - (height / 2) + root.kabarcikOfsetY

        Behavior on x { NumberAnimation { duration: 150; easing.type: Easing.OutQuad } }
        Behavior on y { NumberAnimation { duration: 150; easing.type: Easing.OutQuad } }
        Behavior on color { ColorAnimation { duration: 200 } }
        Behavior on border.color { ColorAnimation { duration: 200 } }
    }

    Text {
        anchors.top: parent.bottom
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.topMargin: 8
        text: root.duz ? root.txt("duzMetni") : root.txt("ayarlaniyorMetni")
        color: root.duz ? "#4ade80" : "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 12
        font.bold: root.duz
    }
}
```

qml\components\TarihSecici.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import sliper

Item {
    id: root
    width: 130
    height: 36

    readonly property var metinler: ({
        placeholderMetni: { tr: "gg.aa.yyyy", en: "dd.mm.yyyy" },
        temizleButon: { tr: "Temizle", en: "Clear" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    property string secilenTarih: ""
    property string placeholderText: txt("placeholderMetni")
    property date gorunenAy: new Date()

    readonly property int hucreBoyutu: 28
    readonly property int hucreAraligi: 4
    readonly property int gridGenisligi: 7 * hucreBoyutu + 6 * hucreAraligi

    signal tarihSecildi(string tarihStr)

    function ikiHane(n) {
        return n < 10 ? "0" + n : "" + n
    }

    function tarihStr(d) {
        return ikiHane(d.getDate()) + "." + ikiHane(d.getMonth() + 1) + "." + d.getFullYear()
    }

    function gunSayisi(yil, ay) {
        return new Date(yil, ay + 1, 0).getDate()
    }

    function ayGunleriniHesapla(ay) {
        var yil = ay.getFullYear()
        var ayIndeks = ay.getMonth()
        var ilkGun = new Date(yil, ayIndeks, 1)
        var haftaBaslangicKaymasi = (ilkGun.getDay() + 6) % 7
        var baslangic = new Date(yil, ayIndeks, 1 - haftaBaslangicKaymasi)

        var gunler = []
        for (var i = 0; i < 42; i++) {
            gunler.push(new Date(baslangic.getFullYear(), baslangic.getMonth(), baslangic.getDate() + i))
        }
        return gunler
    }

    property var gunHucreleri: ayGunleriniHesapla(gorunenAy)

    readonly property var ayAdlari: [
        { tr: "Ocak", en: "January" }, { tr: "Şubat", en: "February" }, { tr: "Mart", en: "March" },
        { tr: "Nisan", en: "April" }, { tr: "Mayıs", en: "May" }, { tr: "Haziran", en: "June" },
        { tr: "Temmuz", en: "July" }, { tr: "Ağustos", en: "August" }, { tr: "Eylül", en: "September" },
        { tr: "Ekim", en: "October" }, { tr: "Kasım", en: "November" }, { tr: "Aralık", en: "December" }
    ]
    readonly property var haftaGunleri: [
        { tr: "Pt", en: "Mo" }, { tr: "Sa", en: "Tu" }, { tr: "Ça", en: "We" },
        { tr: "Pe", en: "Th" }, { tr: "Cu", en: "Fr" }, { tr: "Ct", en: "Sa" }, { tr: "Pz", en: "Su" }
    ]

    Rectangle {
```

```
    id: kutu
    anchors.fill: parent
    radius: 8
    color: "#12121a"
    border.color: (alan.containsMouse || takvimPopup.visible) ? "#3b82f6" : "#1e2a3f"
    border.width: 1
    Behavior on border.color { ColorAnimation { duration: 120 } }

    Text {
        anchors.left: parent.left
        anchors.right: simgeMetni.left
        anchors.leftMargin: 10
        anchors.verticalCenter: parent.verticalCenter
        elide: Text.ElideRight
        text: root.secilenTarih.length > 0 ? root.secilenTarih : root.placeholderText
        color: root.secilenTarih.length > 0 ? "#dce8f5" : "#4b5563"
        font.family: "Segoe UI"
        font.pixelSize: 13
    }

    Text {
        id: simgeMetni
        anchors.right: parent.right
        anchors.rightMargin: 10
        anchors.verticalCenter: parent.verticalCenter
        text: "📅"
        font.pixelSize: 12
        color: "#6b7280"
    }

    MouseArea {
        id: alan
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onClicked: {
            if (root.secilenTarih.length === 10) {
                var parcalar = root.secilenTarih.split(".")
                root.gorunenAy = new Date(parseInt(parcalar[2], 10), parseInt(parcalar[1], 10) - 1, 1)
            } else {
                root.gorunenAy = new Date()
            }
            takvimPopup.open()
        }
    }
}

Popup {
    id: takvimPopup
    y: kutu.height + 6
    width: root.gridGenisligi + 28
    padding: 14
    modal: true
    focus: true
    closePolicy: Popup.CloseOnEscape | Popup.CloseOnPressOutside

    background: Rectangle {
        color: "#12121a"
        radius: 12
        border.color: "#1e2a3f"
        border.width: 1
    }

    Overlay.modal: Rectangle {
        color: "#a6000000"
    }

    contentItem: Column {
        spacing: 10
```

```
            Row {
                width: root.gridGenisligi
                height: 28

                Rectangle {
                    width: 28
                    height: 28
                    radius: 6
                    anchors.verticalCenter: parent.verticalCenter
                    color: oncekiAlan.containsMouse ? "#1e2a3f" : "transparent"
                    Behavior on color { ColorAnimation { duration: 120 } }

                    Text {
                        anchors.centerIn: parent
                        text: "‹"
                        color: "#9ca3af"
                        font.pixelSize: 16
                    }

                    MouseArea {
                        id: oncekiAlan
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onClicked: root.gorunenAy = new Date(root.gorunenAy.getFullYear(), root.gorunenAy.getMonth() - 1, 1)
                    }
                }

                Text {
                    width: parent.width - 56
                    anchors.verticalCenter: parent.verticalCenter
                    horizontalAlignment: Text.AlignHCenter
                    text: (Translations.turkish ? root.ayAdlari[root.gorunenAy.getMonth()].tr :
root.ayAdlari[root.gorunenAy.getMonth()].en) + " " + root.gorunenAy.getFullYear()
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                    font.bold: true
                }

                Rectangle {
                    width: 28
                    height: 28
                    radius: 6
                    anchors.verticalCenter: parent.verticalCenter
                    color: sonrakiAlan.containsMouse ? "#1e2a3f" : "transparent"
                    Behavior on color { ColorAnimation { duration: 120 } }

                    Text {
                        anchors.centerIn: parent
                        text: "›"
                        color: "#9ca3af"
                        font.pixelSize: 16
                    }

                    MouseArea {
                        id: sonrakiAlan
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onClicked: root.gorunenAy = new Date(root.gorunenAy.getFullYear(), root.gorunenAy.getMonth() + 1, 1)
                    }
                }
            }

            Row {
                width: root.gridGenisligi
                spacing: root.hucreAraligi

                Repeater {
```

```
        model: root.haftaGunleri

        delegate: Text {
            width: root.hucreBoyutu
            horizontalAlignment: Text.AlignHCenter
            text: Translations.turkish ? modelData.tr : modelData.en
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 10
            font.bold: true
        }
    }
}

Grid {
    width: root.gridGenisligi
    columns: 7
    rowSpacing: root.hucreAraligi
    columnSpacing: root.hucreAraligi

    Repeater {
        model: root.gunHucreleri

        delegate: Rectangle {
            id: gunHucresi
            width: root.hucreBoyutu
            height: root.hucreBoyutu
            radius: root.hucreBoyutu / 2
            property bool buAyMi: modelData.getMonth() === root.gorunenAy.getMonth()
            property bool seciliMi: root.secilenTarih === root.tarihStr(modelData)
            property bool bugunMu: root.tarihStr(modelData) === root.tarihStr(new Date())
            color: seciliMi ? "#3b82f6" : (gunAlani.containsMouse ? "#1e2a3f" : "transparent")
            border.color: (bugunMu && !seciliMi) ? "#3b82f6" : "transparent"
            border.width: 1

            Text {
                anchors.centerIn: parent
                text: modelData.getDate()
                color: gunHucresi.buAyMi ? (gunHucresi.seciliMi ? "#dce8f5" : "#dce8f5") : "#374151"
                font.family: "Segoe UI"
                font.pixelSize: 12
                font.bold: gunHucresi.seciliMi
            }

            MouseArea {
                id: gunAlani
                anchors.fill: parent
                hoverEnabled: true
                cursorShape: Qt.PointingHandCursor
                onClicked: {
                    root.tarihSecildi(root.tarihStr(modelData))
                    takvimPopup.close()
                }
            }
        }
    }
}

Rectangle {
    width: root.gridGenisligi
    height: 1
    color: "#1e2a3f"
}

Rectangle {
    width: root.gridGenisligi
    height: 30
    radius: 6
    color: temizleAlani.containsMouse ? "#1e2a3f" : "transparent"
    Behavior on color { ColorAnimation { duration: 120 } }
```

```
                Text {
                    anchors.centerIn: parent
                    text: root.txt("temizleButon")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                }

                MouseArea {
                    id: temizleAlani
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onClicked: {
                        root.tarihSecildi("")
                        takvimPopup.close()
                    }
                }
            }
        }
    }
}
```

qml\Main.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Window 6.7
import QtQuick.Layouts 6.7
import "pages"

ApplicationWindow {
    id: pencere
    visibility: Window.Maximized
    title: "SLIPER Analiz"

    background: Rectangle {
        color: "#07070a"
    }

    StackLayout {
        id: anaStack
        anchors.fill: parent
        currentIndex: 0

        LoginPage {
            onGirisBasarili: anaStack.currentIndex = 1
        }

        DashboardPage {
            onCikisYapildi: anaStack.currentIndex = 0
        }
    }
}
```

qml\pages\DashboardPage.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Layouts 6.7
import QtMultimedia
import sliper

Rectangle {
    color: "#0a0a0d"

    signal cikisYapildi()

    readonly property var navOgeleri: [
        { etiketTr: "Ölçüm", etiketEn: "Measurement", ikon: "olcum" },
        { etiketTr: "Sonuçlar", etiketEn: "Results", ikon: "sonuclar" },
        { etiketTr: "Geçmiş", etiketEn: "History", ikon: "gecmis" },
        { etiketTr: "Kalibrasyon", etiketEn: "Calibration", ikon: "kalibrasyon" },
        { etiketTr: "Rehber", etiketEn: "Guide", ikon: "rehber" }
    ]

    readonly property var metinler: ({
        baglandi: { tr: "SLIPER-ESP32 Bağlı", en: "SLIPER-ESP32 Connected" },
        baglaniyor: { tr: "Bağlanıyor...", en: "Connecting..." },
        bagliDegil: { tr: "Bağlı Değil", en: "Not Connected" },
        batarya: { tr: "🔋 Batarya: ", en: "🔋 Battery: " },
        sesliKomut: { tr: "🎙 Sesli Komut: ", en: "🎙 Voice Command: " },
        yukleniyor: { tr: "Yükleniyor...", en: "Loading..." },
        acik: { tr: "Açık", en: "On" },
        kapali: { tr: "Kapalı", en: "Off" },
        cikisYap: { tr: "Çıkış Yap", en: "Log Out" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    Row {
        anchors.fill: parent
        spacing: 0

        Rectangle {
            id: sidebar
            width: 230
            height: parent.height

            gradient: Gradient {
                GradientStop { position: 0.0; color: "#14141c" }
                GradientStop { position: 1.0; color: "#08080a" }
            }

            Image {
                anchors.top: parent.top
                anchors.horizontalCenter: parent.horizontalCenter
                anchors.topMargin: 24
                source: "../assets/logo.png"
                width: 144
                height: 144
                fillMode: Image.PreserveAspectFit
            }

            Rectangle {
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 24
                anchors.rightMargin: 24
                anchors.topMargin: 172
                height: 1
                color: "#1e2a3f"
```

```
}

Row {
    anchors.top: parent.top
    anchors.topMargin: 184
    anchors.horizontalCenter: parent.horizontalCenter
    spacing: 6

    Rectangle {
        width: 32
        height: 22
        radius: 5
        color: "#12121a"
        border.color: Translations.turkish ? "#3b82f6" : "#1e2a3f"
        border.width: 1

        Text {
            anchors.centerIn: parent
            text: "TR"
            color: Translations.turkish ? "#dce8f5" : "#6b7280"
            font.pixelSize: 10
            font.bold: Translations.turkish
        }

        MouseArea {
            anchors.fill: parent
            cursorShape: Qt.PointingHandCursor
            onClicked: Translations.turkish = true
        }
    }

    Rectangle {
        width: 32
        height: 22
        radius: 5
        color: "#12121a"
        border.color: !Translations.turkish ? "#3b82f6" : "#1e2a3f"
        border.width: 1

        Text {
            anchors.centerIn: parent
            text: "EN"
            color: !Translations.turkish ? "#dce8f5" : "#6b7280"
            font.pixelSize: 10
            font.bold: !Translations.turkish
        }

        MouseArea {
            anchors.fill: parent
            cursorShape: Qt.PointingHandCursor
            onClicked: Translations.turkish = false
        }
    }

    Rectangle {
        id: cikisButonu
        width: cikisMetni.implicitWidth + 16
        height: 22
        radius: 5
        color: cikisAlani.pressed ? "#4c1414" : (cikisAlani.containsMouse ? "#3a1414" : "#1f1414")
        border.color: "#7f1d1d"
        border.width: 1
        Behavior on color { ColorAnimation { duration: 120 } }

        Text {
            id: cikisMetni
            anchors.centerIn: parent
            text: txt("cikisYap")
            color: "#f87171"
            font.family: "Segoe UI"
```

```
                    font.pixelSize: 10
                    font.bold: true
                }

                MouseArea {
                    id: cikisAlani
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onClicked: cikisYapildi()
                }
            }
        }

        Rectangle {
            id: baglantiDurumu
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.leftMargin: 24
            anchors.rightMargin: 24
            anchors.topMargin: 224
            height: 36
            radius: 6
            color: baglantiAlani.hoverli ? "#141419" : "transparent"
            Behavior on color { ColorAnimation { duration: 120 } }

            Row {
                anchors.left: parent.left
                anchors.leftMargin: 8
                anchors.verticalCenter: parent.verticalCenter
                spacing: 8

                Rectangle {
                    width: 8
                    height: 8
                    radius: 4
                    anchors.verticalCenter: parent.verticalCenter
                    color: wifiManager.baglandi ? "#16a34a" : (wifiManager.baglaniyor ? "#f59e0b" : "#dc2626")
                }

                Text {
                    text: wifiManager.baglandi ? txt("baglandi") : (wifiManager.baglaniyor ? txt("baglaniyor") :
txt("bagliDegil"))
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                    anchors.verticalCenter: parent.verticalCenter
                }
            }

            MouseArea {
                id: baglantiAlani
                anchors.fill: parent
                hoverEnabled: true
                cursorShape: Qt.PointingHandCursor
                enabled: !wifiManager.baglandi && !wifiManager.baglaniyor
                onClicked: wifiManager.baglan()
            }
        }

        Rectangle {
            id: bataryaGostergesi
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.leftMargin: 24
            anchors.rightMargin: 24
            anchors.topMargin: 236
            height: 22
```

```
                radius: 4
                visible: wifiManager.baglandi && sensorManager.bataryaVoltaj > 0.1
                color: {
                    if (sensorManager.bataryaVoltaj >= 14.5) return "#0f2417"
                    if (sensorManager.bataryaVoltaj >= 14.0) return "#2a2414"
                    return "#2a1414"
                }

                Text {
                    anchors.left: parent.left
                    anchors.leftMargin: 8
                    anchors.verticalCenter: parent.verticalCenter
                    text: txt("batarya") + sensorManager.bataryaVoltaj.toFixed(2) + " V"
                    color: {
                        if (sensorManager.bataryaVoltaj >= 14.5) return "#4ade80"
                        if (sensorManager.bataryaVoltaj >= 14.0) return "#e8a020"
                        return "#f87171"
                    }
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                    font.bold: true
                }
            }

            Rectangle {
                id: sesKontrolu
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 24
                anchors.rightMargin: 24
                anchors.topMargin: 264
                height: 30
                radius: 6
                color: sesKontroluAlani.hoverli ? "#141419" : "transparent"
                Behavior on color { ColorAnimation { duration: 120 } }

                Row {
                    anchors.left: parent.left
                    anchors.leftMargin: 8
                    anchors.verticalCenter: parent.verticalCenter
                    spacing: 8

                    Rectangle {
                        width: 8
                        height: 8
                        radius: 4
                        anchors.verticalCenter: parent.verticalCenter
                        color: !voiceCommandManager.modelHazir ? "#6b7280" : (voiceCommandManager.etkin ? "#16a34a" : "#dc2626")
                    }

                    Text {
                        text: txt("sesliKomut") + (!voiceCommandManager.modelHazir ? txt("yukleniyor") :
(voiceCommandManager.etkin ? txt("acik") : txt("kapali")))
                        color: "#9ca3af"
                        font.family: "Segoe UI"
                        font.pixelSize: 12
                        anchors.verticalCenter: parent.verticalCenter
                    }
                }

                MouseArea {
                    id: sesKontroluAlani
                    property bool hoverli: false
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    enabled: voiceCommandManager.modelHazir
                    onEntered: hoverli = true
                    onExited: hoverli = false
```

```
        onClicked: voiceCommandManager.etkin = !voiceCommandManager.etkin
    }
}

Rectangle {
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.leftMargin: 24
    anchors.rightMargin: 24
    anchors.topMargin: 302
    height: 1
    color: "#1e2a3f"
}

Column {
    id: navColumn
    anchors.top: parent.top
    anchors.topMargin: 320
    width: parent.width
    spacing: 2

    Repeater {
        model: navOgeleri

        Rectangle {
            id: navOgesi
            width: parent.width
            height: 46
            property bool aktif: icerikStack.currentIndex === index
            property bool hoverli: false
            color: aktif ? "#17263d" : (hoverli ? "#141419" : "transparent")
            Behavior on color { ColorAnimation { duration: 120 } }

            Rectangle {
                width: 3
                height: parent.height * 0.56
                radius: 2
                anchors.verticalCenter: parent.verticalCenter
                color: "#3b82f6"
                visible: navOgesi.aktif
                Behavior on height { NumberAnimation { duration: 140 } }
            }

            Row {
                anchors.left: parent.left
                anchors.leftMargin: 24
                anchors.verticalCenter: parent.verticalCenter
                spacing: 12

                NavIkon {
                    tur: modelData.ikon
                    width: 17
                    height: 17
                    renk: navOgesi.aktif ? "#dce8f5" : (navOgesi.hoverli ? "#dce8f5" : "#9ca3af")
                    anchors.verticalCenter: parent.verticalCenter
                    Behavior on renk { ColorAnimation { duration: 120 } }
                }

                Text {
                    text: Translations.turkish ? modelData.etiketTr : modelData.etiketEn
                    color: navOgesi.aktif ? "#dce8f5" : (navOgesi.hoverli ? "#dce8f5" : "#9ca3af")
                    font.family: "Segoe UI"
                    font.pixelSize: 14
                    font.letterSpacing: 0.3
                    font.bold: navOgesi.aktif
                    anchors.verticalCenter: parent.verticalCenter
                    Behavior on color { ColorAnimation { duration: 120 } }
                }
            }
```

```
                MouseArea {
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onEntered: navOgesi.hoverli = true
                    onExited: navOgesi.hoverli = false
                    onClicked: icerikStack.currentIndex = index
                }
            }
        }
    }

    MediaPlayer {
        id: simulasyonOynatici
        source: "../assets/slipergif.mp4"
        videoOutput: simulasyonVideo
        loops: 1
    }

    Rectangle {
        id: simulasyonCercevesi
        anchors.top: navColumn.bottom
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        anchors.margins: 16
        anchors.topMargin: 20
        radius: 8
        color: "#08080a"
        border.color: "#1e2a3f"
        border.width: 1
        clip: true

        VideoOutput {
            id: simulasyonVideo
            anchors.fill: parent
            anchors.margins: 1
        }
    }
}

Rectangle {
    id: sidebarAyraci
    width: 3
    height: parent.height
    color: "#060607"

    Rectangle {
        anchors.right: parent.right
        anchors.top: parent.top
        anchors.bottom: parent.bottom
        width: 1
        color: "#403b82f6"
    }
}

StackLayout {
    id: icerikStack
    width: parent.width - sidebar.width - sidebarAyraci.width
    height: parent.height
    currentIndex: 0

    OlcumPage {
        onOlcumTamamlandi: function(id) {
            sonuclarSayfasi.olcumId = id
            icerikStack.currentIndex = 1
        }
        onAgirlikEklendi: {
            simulasyonOynatici.stop()
```

```
                simulasyonOynatici.play()
            }
        }

        SonuclarPage {
            id: sonuclarSayfasi
        }

        GecmisPage {
            onTestSecildi: function(id) {
                sonuclarSayfasi.olcumId = id
                icerikStack.currentIndex = 1
            }
        }

        KalibrasyonPage {}

        RehberPage {}
    }
  }
}
```

qml\pages\GecmisPage.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import "../components"
import sliper

Rectangle {
    id: gecmisSayfasi
    color: "#0a0a0d"

    signal testSecildi(int id)

    property var olcumListesi: []
    property string baslangicTarihi: ""
    property string bitisTarihi: ""

    readonly property var filtreAnahtarlari: ["tumTestler", "son7Gun", "son30Gun"]

    readonly property var metinler: ({
        aramaPlaceholder: { tr: "Müşteri veya reçete ara...", en: "Search by customer or recipe..." },
        tumTestler: { tr: "Tüm Testler", en: "All Tests" },
        son7Gun: { tr: "Son 7 Gün", en: "Last 7 Days" },
        son30Gun: { tr: "Son 30 Gün", en: "Last 30 Days" },
        ozelTarih: { tr: "Özel Tarih:", en: "Custom Date:" },
        temizle: { tr: "Temizle", en: "Clear" },
        musteriEtiket: { tr: "Müşteri: ", en: "Customer: " },
        receteEtiket: { tr: "  •  Reçete: ", en: "  •  Recipe: " },
        goruntule: { tr: "Görüntüle", en: "View" },
        kayitSilBaslik: { tr: "Kaydı Sil", en: "Delete Record" },
        kayitSilOnSoru: { tr: "'", en: "Are you sure you want to delete the measurement for '" },
        kayitSilSonSoru: { tr: "' ölçümünü silmek istediğinize emin misiniz? Bu işlem geri alınamaz.", en: "'? This action cannot
be undone." },
        sifreEtiket: { tr: "ŞİFRE", en: "PASSWORD" },
        sifrenizGirin: { tr: "Şifrenizi girin", en: "Enter your password" },
        sifreHatali: { tr: "Şifre hatalı, tekrar deneyin.", en: "Incorrect password, please try again." },
        sil: { tr: "Sil", en: "Delete" },
        iptal: { tr: "İptal", en: "Cancel" },
        gelismisAyarlarBaslik: { tr: "Gelişmiş Ayarlar", en: "Advanced Settings" },
        onayla: { tr: "Onayla", en: "Confirm" },
        veriTabaniDisaAktar: { tr: "Veritabanını Dışa Aktar", en: "Export Database" },
        disaAktarildi: { tr: "✓ Dışa aktarıldı: ", en: "✓ Exported: " },
        disaAktarilamadi: { tr: "✕ Dışa aktarılamadı", en: "✕ Export failed" },
        icaAktarYoluEtiket: { tr: "İÇE AKTARILACAK .DB DOSYA YOLU", en: ".DB FILE PATH TO IMPORT" },
        icaAktarPlaceholder: { tr: "ör. C:/Users/.../SliperRaporlari/sliper_yedek_....db", en: "e.g.
C:/Users/.../SliperRaporlari/sliper_yedek_....db" },
        veriTabaniIcaAktar: { tr: "Veritabanını İçe Aktar", en: "Import Database" },
        icaAktarildi: { tr: "✓ İçe aktarıldı, uygulamayı yeniden başlatmanız önerilir.", en: "✓ Imported, it is recommended to
restart the application." },
        icaAktarilamadi: { tr: "✕ İçe aktarılamadı, dosya yolunu kontrol edin.", en: "✕ Import failed, please check the file
path." },
        tumVeriyiSil: { tr: "Tüm Verileri Sil", en: "Delete All Data" },
        kapat: { tr: "Kapat", en: "Close" },
        tumVeriyiSilMesaj: { tr: "Tüm ölçüm ve stroke verileri kalıcı olarak silinecek. Sensör kalibrasyonları etkilenmez. Bu
işlem GERİ ALINAMAZ, emin misiniz?", en: "All measurement and stroke data will be permanently deleted. Sensor calibrations will
not be affected. This action CANNOT BE UNDONE, are you sure?" },
        evetHepsiniSil: { tr: "Evet, Hepsini Sil", en: "Yes, Delete All" },
        tumVerilerSilindi: { tr: "✓ Tüm veriler silindi.", en: "✓ All data deleted." }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    function listeyiYenile() {
        olcumListesi = database.tumOlcumleriGetir()
    }

    onVisibleChanged: {
        if (visible) {
```

```
        listeyiYenile()
    }
}

Component.onCompleted: listeyiYenile()

function tarihiAyristir(tarihStr) {
    if (!tarihStr) return null
    var tarihKismi = tarihStr.split(" ")[0].split(".")
    if (tarihKismi.length !== 3) return null
    var gun = parseInt(tarihKismi[0], 10)
    var ay = parseInt(tarihKismi[1], 10)
    var yil = parseInt(tarihKismi[2], 10)
    if (isNaN(gun) || isNaN(ay) || isNaN(yil)) return null
    return new Date(yil, ay - 1, gun)
}

function gecerliOzelTarih(metin) {
    if (!metin || metin.length !== 10) return null
    return tarihiAyristir(metin)
}

function filtrelenmisListeyiHesapla() {
    var sonuc = []
    var aramaMetni = aramaKutusu.text.toLowerCase()
    var ozelBaslangic = gecerliOzelTarih(gecmisSayfasi.baslangicTarihi)
    var ozelBitis = gecerliOzelTarih(gecmisSayfasi.bitisTarihi)
    var simdi = new Date()

    for (var i = 0; i < olcumListesi.length; i++) {
        var kayit = olcumListesi[i]
        var tarihObj = tarihiAyristir(kayit.tarih)

        if (aramaMetni.length > 0) {
            var musteriEslesir = kayit.musteri.toLowerCase().indexOf(aramaMetni) !== -1
            var receteEslesir = kayit.recete.toLowerCase().indexOf(aramaMetni) !== -1
            var yerEslesir = (kayit.yer || "").toLowerCase().indexOf(aramaMetni) !== -1
            if (!musteriEslesir && !receteEslesir && !yerEslesir) continue
        }

        if (ozelBaslangic || ozelBitis) {
            if (ozelBaslangic && tarihObj && tarihObj < ozelBaslangic) continue
            if (ozelBitis && tarihObj) {
                var bitisSonu = new Date(ozelBitis.getFullYear(), ozelBitis.getMonth(), ozelBitis.getDate(), 23, 59, 59)
                if (tarihObj > bitisSonu) continue
            }
        } else if (tarihObj) {
            if (filtreKutusu.currentIndex === 1) {
                var esik7 = new Date(simdi.getTime() - 7 * 24 * 60 * 60 * 1000)
                if (tarihObj < esik7) continue
            } else if (filtreKutusu.currentIndex === 2) {
                var esik30 = new Date(simdi.getTime() - 30 * 24 * 60 * 60 * 1000)
                if (tarihObj < esik30) continue
            }
        }

        sonuc.push(kayit)
    }

    return sonuc
}

property var filtrelenmisListe: filtrelenmisListeyiHesapla()

readonly property var metinlerBaslik: ({
    gecmisBaslik: { tr: "Geçmiş Kayıtlar", en: "History Records" },
    toplamKayit: { tr: "Toplam Kayıt", en: "Total Records" },
    sonucBulunamadi: { tr: "Eşleşen kayıt bulunamadı", en: "No matching records found" }
})
```

```
function txt2(anahtar) {
    return Translations.turkish ? metinlerBaslik[anahtar].tr : metinlerBaslik[anahtar].en
}

Item {
    anchors.fill: parent
    anchors.margins: 28

    Column {
        id: ustBolum
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        spacing: 20

    Row {
        width: parent.width

        Column {
            width: parent.width - 150
            spacing: 3

            Text {
                text: txt2("gecmisBaslik")
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 22
                font.bold: true
                font.letterSpacing: 0.3
            }
        }

        Rectangle {
            width: 150
            height: 48
            radius: 8
            color: "#12121a"
            border.color: "#1e2a3f"
            border.width: 1

            Column {
                anchors.centerIn: parent
                spacing: 2

                Text {
                    anchors.horizontalCenter: parent.horizontalCenter
                    text: gecmisSayfasi.olcumListesi.length
                    color: "#3b82f6"
                    font.family: "Segoe UI"
                    font.pixelSize: 20
                    font.bold: true
                }

                Text {
                    anchors.horizontalCenter: parent.horizontalCenter
                    text: txt2("toplamKayit")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 9
                    font.letterSpacing: 0.5
                }
            }
        }
    }

    Rectangle {
        width: parent.width
        height: 1
        color: "#1e2a3f"
    }
```

```
Row {
    width: parent.width
    spacing: 12

    TextField {
        id: aramaKutusu
        width: parent.width - filtreKutusu.width - gelismisAyarlarButonu.width - parent.spacing * 2
        height: 40
        placeholderText: txt("aramaPlaceholder")
        placeholderTextColor: "#4b5563"
        color: "#dce8f5"
        font.pixelSize: 13
        leftPadding: 34
        verticalAlignment: TextInput.AlignVCenter
        background: Rectangle {
            color: "#12121a"
            radius: 8
            border.color: aramaKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
            border.width: 1
        }

        Text {
            anchors.left: parent.left
            anchors.leftMargin: 12
            anchors.verticalCenter: parent.verticalCenter
            text: "🔍"
            color: "#4b5563"
            font.pixelSize: 13
        }
    }

    ComboBox {
        id: filtreKutusu
        width: 160
        height: 40
        model: filtreAnahtarlari.map(function(anahtar) { return txt(anahtar) })

        background: Rectangle {
            color: "#12121a"
            radius: 8
            border.color: filtreKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
            border.width: 1
        }

        contentItem: Text {
            text: filtreKutusu.displayText
            color: "#9ca3af"
            font.pixelSize: 13
            leftPadding: 10
            verticalAlignment: Text.AlignVCenter
        }
    }

    Button {
        id: gelismisAyarlarButonu
        width: 44
        height: 40
        text: "⚙"
        font.pixelSize: 15

        onClicked: {
            gelismisAyarlarPopup.pinDogrulandi = false
            pinKutusu.text = ""
            gelismisAyarlarBildirim.text = ""
            gelismisAyarlarPopup.open()
        }

        background: Rectangle {
            radius: 8
```

```
            color: parent.hovered ? "#191921" : "#12121a"
            border.color: "#1e2a3f"
            border.width: 1
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#9ca3af"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }
}

Row {
    width: parent.width
    spacing: 10

    Text {
        text: txt("ozelTarih")
        color: "#9ca3af"
        font.family: "Segoe UI"
        font.pixelSize: 12
        anchors.verticalCenter: parent.verticalCenter
    }

    TarihSecici {
        id: baslangicSecici
        anchors.verticalCenter: parent.verticalCenter
        secilenTarih: gecmisSayfasi.baslangicTarihi
        onTarihSecildi: function(t) { gecmisSayfasi.baslangicTarihi = t }
    }

    Text {
        text: "—"
        color: "#4b5563"
        font.pixelSize: 13
        anchors.verticalCenter: parent.verticalCenter
    }

    TarihSecici {
        id: bitisSecici
        anchors.verticalCenter: parent.verticalCenter
        secilenTarih: gecmisSayfasi.bitisTarihi
        onTarihSecildi: function(t) { gecmisSayfasi.bitisTarihi = t }
    }

    Rectangle {
        width: temizleMetni.implicitWidth + 20
        height: 36
        radius: 8
        color: temizleMouse.containsMouse ? "#191921" : "#12121a"
        border.color: "#1e2a3f"
        border.width: 1
        visible: gecmisSayfasi.baslangicTarihi.length > 0 || gecmisSayfasi.bitisTarihi.length > 0
        anchors.verticalCenter: parent.verticalCenter

        Text {
            id: temizleMetni
            anchors.centerIn: parent
            text: txt("temizle")
            color: "#9ca3af"
            font.family: "Segoe UI"
            font.pixelSize: 12
        }

        MouseArea {
            id: temizleMouse
```

```
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onClicked: {
                        gecmisSayfasi.baslangicTarihi = ""
                        gecmisSayfasi.bitisTarihi = ""
                    }
                }
            }
        }

        }

        Rectangle {
            id: sonucPaneli
            anchors.top: ustBolum.bottom
            anchors.topMargin: 20
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            radius: 12
            color: "#12121a"
            border.color: "#1e2a3f"
            border.width: 1
            visible: gecmisSayfasi.filtrelenmisListe.length > 0
            clip: true

            Row {
                id: tabloBasligi
                anchors.top: parent.top
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.margins: 18
                height: 20

                Text { width: 56; text: "#"; color: "#4b5563"; font.family: "Segoe UI"; font.pixelSize: 10; font.letterSpacing:
1; font.bold: true }
                Text { width: parent.width - 56 - 150 - 68 - 152; text: Translations.turkish ? "MÜŞTERİ / REÇETE" : "CUSTOMER /
RECIPE"; color: "#4b5563"; font.family: "Segoe UI"; font.pixelSize: 10; font.letterSpacing: 1; font.bold: true }
                Text { width: 150; text: Translations.turkish ? "TARİH" : "DATE"; color: "#4b5563"; font.family: "Segoe UI";
font.pixelSize: 10; font.letterSpacing: 1; font.bold: true }
                Text { width: 68; text: Translations.turkish ? "STROKE" : "STROKE"; color: "#4b5563"; font.family: "Segoe UI";
font.pixelSize: 10; font.letterSpacing: 1; font.bold: true }
                Text { width: 152; horizontalAlignment: Text.AlignRight; text: ""; color: "#4b5563"; font.pixelSize: 10 }
            }

            Rectangle {
                anchors.top: tabloBasligi.bottom
                anchors.topMargin: 12
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.leftMargin: 18
                anchors.rightMargin: 18
                height: 1
                color: "#1e2a3f"
            }

            ListView {
                anchors.top: tabloBasligi.bottom
                anchors.topMargin: 13
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.bottom: parent.bottom
                anchors.margins: 4
                clip: true
                spacing: 0

                model: gecmisSayfasi.filtrelenmisListe

                delegate: Rectangle {
```

```
id: gecmisKarti
width: parent ? parent.width : 0
height: 58
color: kartAlani.containsMouse ? "#16161e" : "transparent"
Behavior on color { ColorAnimation { duration: 120 } }

MouseArea {
    id: kartAlani
    anchors.fill: parent
    hoverEnabled: true
    acceptedButtons: Qt.NoButton
}

Rectangle {
    anchors.bottom: parent.bottom
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.leftMargin: 14
    anchors.rightMargin: 14
    height: 1
    color: "#1e1e18"
    visible: index !== gecmisSayfasi.filtrelenmisListe.length - 1
}

Row {
    anchors.fill: parent
    anchors.leftMargin: 14
    anchors.rightMargin: 14

    Text {
        width: 56
        anchors.verticalCenter: parent.verticalCenter
        text: (index + 1).toString().padStart(2, "0")
        color: "#4b5563"
        font.family: "Segoe UI"
        font.pixelSize: 12
    }

    Column {
        width: parent.width - 56 - 150 - 68 - 152
        anchors.verticalCenter: parent.verticalCenter
        spacing: 3

        Text {
            text: modelData.musteri
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
            elide: Text.ElideRight
            width: parent.width
        }

        Text {
            text: modelData.recete
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
            elide: Text.ElideRight
            width: parent.width
        }
    }

    Text {
        width: 150
        anchors.verticalCenter: parent.verticalCenter
        text: modelData.tarih
        color: "#9ca3af"
        font.family: "Segoe UI"
        font.pixelSize: 12
```

```
}

Rectangle {
    width: 68
    height: 22
    radius: 5
    anchors.verticalCenter: parent.verticalCenter
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    Text {
        anchors.centerIn: parent
        text: modelData.strokeSayisi
        color: "#4f8cf7"
        font.family: "Segoe UI"
        font.pixelSize: 11
        font.bold: true
    }
}

Row {
    width: 152
    anchors.verticalCenter: parent.verticalCenter
    spacing: 8
    layoutDirection: Qt.RightToLeft

    Rectangle {
        id: silButonu
        width: 30
        height: 30
        radius: 6
        color: silMouse.containsMouse ? "#3f1d24" : "transparent"
        border.color: silMouse.containsMouse ? "#ef4444" : "#1e2a3f"
        border.width: 1
        Behavior on color { ColorAnimation { duration: 120 } }

        Text {
            anchors.centerIn: parent
            text: "🗑"
            font.pixelSize: 13
        }

        MouseArea {
            id: silMouse
            anchors.fill: parent
            hoverEnabled: true
            cursorShape: Qt.PointingHandCursor
            onClicked: {
                silOnayPopup.hedefId = modelData.id
                silOnayPopup.hedefMusteri = modelData.musteri
                silOnayPopup.open()
            }
        }
    }

    Rectangle {
        id: goruntuleButonu
        width: 84
        height: 30
        radius: 6
        color: goruntuleMouse.containsMouse ? "#191921" : "transparent"
        border.color: goruntuleMouse.containsMouse ? "#3b82f6" : "#1e2a3f"
        border.width: 1
        Behavior on color { ColorAnimation { duration: 120 } }
        Behavior on border.color { ColorAnimation { duration: 120 } }

        Text {
            anchors.centerIn: parent
            text: txt("goruntule")
```

```
                                color: goruntuleMouse.containsMouse ? "#dce8f5" : "#9ca3af"
                                font.family: "Segoe UI"
                                font.pixelSize: 11
                                font.bold: true
                            }

                            MouseArea {
                                id: goruntuleMouse
                                anchors.fill: parent
                                hoverEnabled: true
                                cursorShape: Qt.PointingHandCursor
                                onClicked: testSecildi(modelData.id)
                            }
                        }
                    }
                }
            }
        }

        Column {
            anchors.top: ustBolum.bottom
            anchors.topMargin: 20
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            visible: gecmisSayfasi.filtrelenmisListe.length === 0
            spacing: 8

            Item { width: 1; height: parent.height / 2 - 40 }

            Text {
                anchors.horizontalCenter: parent.horizontalCenter
                text: "🔍"
                font.pixelSize: 28
                opacity: 0.4
            }

            Text {
                anchors.horizontalCenter: parent.horizontalCenter
                text: txt2("sonucBulunamadi")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 14
                font.bold: true
            }
        }
    }

    Popup {
        id: silOnayPopup
        anchors.centerIn: parent
        width: 380
        padding: 24
        modal: true
        focus: true
        closePolicy: Popup.CloseOnEscape

        property int hedefId: -1
        property string hedefMusteri: ""

        onOpened: {
            sifreGirisAlani.text = ""
            hataMetni.visible = false
            sifreGirisAlani.forceActiveFocus()
        }

        background: Rectangle {
            color: "#12121a"
            radius: 14
```

```
        border.color: "#1e2a3f"
        border.width: 1
    }

    contentItem: Column {
        width: silOnayPopup.availableWidth
        spacing: 16

        Text {
            text: txt("kayitSilBaslik")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 17
            font.bold: true
        }

        Text {
            width: parent.width
            wrapMode: Text.WordWrap
            text: txt("kayitSilOnSoru") + silOnayPopup.hedefMusteri + txt("kayitSilSonSoru")
            color: "#9ca3af"
            font.family: "Segoe UI"
            font.pixelSize: 13
        }

        Column {
            width: parent.width
            spacing: 6

            Text {
                text: txt("sifreEtiket")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.letterSpacing: 1
            }

            TextField {
                id: sifreGirisAlani
                width: parent.width
                height: 40
                echoMode: TextInput.Password
                placeholderText: txt("sifrenizGirin")
                placeholderTextColor: "#4b5563"
                color: "#dce8f5"
                font.pixelSize: 13
                leftPadding: 12
                verticalAlignment: TextInput.AlignVCenter
                background: Rectangle {
                    color: "#0a0a0d"
                    radius: 6
                    border.color: sifreGirisAlani.activeFocus ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
                }
                Keys.onReturnPressed: silOnaylaButonu.clicked()
            }

            Text {
                id: hataMetni
                text: txt("sifreHatali")
                color: "#ef4444"
                font.family: "Segoe UI"
                font.pixelSize: 11
                visible: false
            }
        }

        Row {
            width: parent.width
            spacing: 10
```

```
            layoutDirection: Qt.RightToLeft

            Button {
                id: silOnaylaButonu
                width: (parent.width - 10) / 2
                height: 40
                text: txt("sil")
                font.pixelSize: 13
                font.bold: true

                onClicked: {
                    if (backend.sifreDogrula(sifreGirisAlani.text)) {
                        database.olcumSil(silOnayPopup.hedefId)
                        gecmisSayfasi.listeyiYenile()
                        silOnayPopup.close()
                    } else {
                        hataMetni.visible = true
                        sifreGirisAlani.text = ""
                        sifreGirisAlani.forceActiveFocus()
                    }
                }

                background: Rectangle {
                    radius: 8
                    color: parent.pressed ? "#b91c1c" : (parent.hovered ? "#ef4444" : "#dc2626")
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#dce8f5"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: (parent.width - 10) / 2
                height: 40
                text: txt("iptal")
                font.pixelSize: 13

                onClicked: silOnayPopup.close()

                background: Rectangle {
                    radius: 8
                    color: parent.hovered ? "#191921" : "#12121a"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#9ca3af"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }
        }
    }
}

Popup {
    id: gelismisAyarlarPopup
    anchors.centerIn: parent
    width: 420
    padding: 24
```

```
modal: true
focus: true
closePolicy: Popup.CloseOnEscape

property bool pinDogrulandi: false

background: Rectangle {
    color: "#12121a"
    radius: 14
    border.color: "#1e2a3f"
    border.width: 1
}

contentItem: Column {
    width: gelismisAyarlarPopup.availableWidth
    spacing: 16

    Text {
        text: txt("gelismisAyarlarBaslik")
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 17
        font.bold: true
    }

    Column {
        width: parent.width
        spacing: 6
        visible: !gelismisAyarlarPopup.pinDogrulandi

        Text {
            text: txt("sifreEtiket")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 10
            font.letterSpacing: 1
        }

        TextField {
            id: pinKutusu
            width: parent.width
            height: 40
            echoMode: TextInput.Password
            placeholderText: txt("sifrenizGirin")
            placeholderTextColor: "#4b5563"
            color: "#dce8f5"
            font.pixelSize: 13
            leftPadding: 12
            verticalAlignment: TextInput.AlignVCenter
            background: Rectangle {
                color: "#0a0a0d"
                radius: 6
                border.color: pinKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                border.width: 1
            }
            Keys.onReturnPressed: pinDogrulaButonu.clicked()
        }

        Text {
            id: pinHataMetni
            text: txt("sifreHatali")
            color: "#ef4444"
            font.family: "Segoe UI"
            font.pixelSize: 11
            visible: false
        }

        Button {
            id: pinDogrulaButonu
            width: parent.width
```

```
        height: 40
        text: txt("onayla")
        font.pixelSize: 13
        font.bold: true

        onClicked: {
            if (backend.sifreDogrula(pinKutusu.text)) {
                gelismisAyarlarPopup.pinDogrulandi = true
                pinHataMetni.visible = false
            } else {
                pinHataMetni.visible = true
                pinKutusu.text = ""
                pinKutusu.forceActiveFocus()
            }
        }

        background: Rectangle {
            radius: 8
            color: parent.hovered ? "#4f8cf7" : "#3b82f6"
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#dce8f5"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }
}

Column {
    width: parent.width
    spacing: 10
    visible: gelismisAyarlarPopup.pinDogrulandi

    Button {
        width: parent.width
        height: 40
        text: txt("veriTabaniDisaAktar")
        font.pixelSize: 13

        onClicked: {
            var yol = database.veriTabaniDisaAktar()
            gelismisAyarlarBildirim.text = yol.length > 0
                ? txt("disaAktarildi") + yol
                : txt("disaAktarilamadi")
        }

        background: Rectangle {
            radius: 8
            color: parent.hovered ? "#191921" : "#12121a"
            border.color: "#1e2a3f"
            border.width: 1
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#dce8f5"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }

    Column {
        width: parent.width
        spacing: 6
```

```
    Text {
        text: txt("icaAktarYoluEtiket")
        color: "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 10
        font.letterSpacing: 1
    }

    TextField {
        id: icaAktarYoluKutusu
        width: parent.width
        height: 40
        placeholderText: txt("icaAktarPlaceholder")
        placeholderTextColor: "#4b5563"
        color: "#dce8f5"
        font.pixelSize: 12
        leftPadding: 12
        verticalAlignment: TextInput.AlignVCenter
        background: Rectangle {
            color: "#0a0a0d"
            radius: 6
            border.color: icaAktarYoluKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
            border.width: 1
        }
    }
}

Button {
    width: parent.width
    height: 40
    text: txt("veriTabaniIcaAktar")
    font.pixelSize: 13

    onClicked: {
        var basarili = database.veriTabaniIcaAktar(icaAktarYoluKutusu.text)
        gelismisAyarlarBildirim.text = basarili
            ? txt("icaAktarildi")
            : txt("icaAktarilamadi")
        if (basarili) {
            gecmisSayfasi.listeyiYenile()
        }
    }

    background: Rectangle {
        radius: 8
        color: parent.hovered ? "#191921" : "#12121a"
        border.color: "#1e2a3f"
        border.width: 1
        Behavior on color { ColorAnimation { duration: 120 } }
    }

    contentItem: Text {
        text: parent.text
        color: "#dce8f5"
        font: parent.font
        horizontalAlignment: Text.AlignHCenter
        verticalAlignment: Text.AlignVCenter
    }
}

Button {
    width: parent.width
    height: 40
    text: txt("tumVeriyiSil")
    font.pixelSize: 13
    font.bold: true

    onClicked: tumVeriyiSilOnayPopup.open()
```

```
                background: Rectangle {
                    radius: 8
                    color: parent.pressed ? "#b91c1c" : (parent.hovered ? "#ef4444" : "#dc2626")
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#dce8f5"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Text {
                id: gelismisAyarlarBildirim
                width: parent.width
                text: ""
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 10
                wrapMode: Text.WrapAnywhere
            }

            Button {
                width: parent.width
                height: 36
                text: txt("kapat")
                font.pixelSize: 12

                onClicked: gelismisAyarlarPopup.close()

                background: Rectangle {
                    radius: 8
                    color: parent.hovered ? "#191921" : "transparent"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#9ca3af"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }
        }
    }
}

Popup {
    id: tumVeriyiSilOnayPopup
    anchors.centerIn: parent
    width: 380
    padding: 24
    modal: true
    focus: true
    closePolicy: Popup.CloseOnEscape

    background: Rectangle {
        color: "#12121a"
        radius: 14
        border.color: "#dc2626"
        border.width: 1
    }

    contentItem: Column {
```

```
width: tumVeriyiSilOnayPopup.availableWidth
spacing: 16

Text {
    text: txt("tumVeriyiSil")
    color: "#dce8f5"
    font.family: "Segoe UI"
    font.pixelSize: 17
    font.bold: true
}

Text {
    width: parent.width
    wrapMode: Text.WordWrap
    text: txt("tumVeriyiSilMesaj")
    color: "#9ca3af"
    font.family: "Segoe UI"
    font.pixelSize: 13
}

Row {
    width: parent.width
    spacing: 10
    layoutDirection: Qt.RightToLeft

    Button {
        width: (parent.width - 10) / 2
        height: 40
        text: txt("evetHepsiniSil")
        font.pixelSize: 13
        font.bold: true

        onClicked: {
            database.tumVeriyiSil()
            gecmisSayfasi.listeyiYenile()
            tumVeriyiSilOnayPopup.close()
            gelismisAyarlarBildirim.text = txt("tumVerilerSilindi")
        }

        background: Rectangle {
            radius: 8
            color: parent.pressed ? "#b91c1c" : (parent.hovered ? "#ef4444" : "#dc2626")
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#dce8f5"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }

    Button {
        width: (parent.width - 10) / 2
        height: 40
        text: txt("iptal")
        font.pixelSize: 13

        onClicked: tumVeriyiSilOnayPopup.close()

        background: Rectangle {
            radius: 8
            color: parent.hovered ? "#191921" : "#12121a"
            border.color: "#1e2a3f"
            border.width: 1
            Behavior on color { ColorAnimation { duration: 120 } }
        }
```

```
                    contentItem: Text {
                        text: parent.text
                        color: "#9ca3af"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}
```

qml\pages\KalibrasyonPage.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Layouts 6.7
import "../components"
import sliper

Rectangle {
    color: "#0a0a0d"

    readonly property var metinler: ({
        baslikSensorSec: { tr: "KALİBRASYON — Sensör Seçin", en: "CALIBRATION — Select Sensor" },
        egimSensoru: { tr: "Eğim Sensörü", en: "Incline Sensor" },
        egimAciklama: { tr: "Yatay hizalama kalibrasyonu", en: "Level alignment calibration" },
        kalibreEdildiOnEk: { tr: "✓ Kalibre Edildi: ", en: "✓ Calibrated: " },
        loadCellAdi: { tr: "Load Cell", en: "Load Cell" },
        loadCellAciklama: { tr: "Basınç sensörü kalibrasyonu", en: "Load pressure sensor calibration" },
        mesafeSensoru: { tr: "Mesafe Sensörü", en: "Distance Sensor" },
        mesafeAciklama: { tr: "Konum ölçümü kalibrasyonu", en: "Position measurement calibration" },
        geriButon: { tr: "← Geri", en: "← Back" },
        egimBaslik: { tr: "Eğim Sensörü Kalibrasyonu", en: "Incline Sensor Calibration" },
        loadCellBaslik: { tr: "Load Cell Kalibrasyonu", en: "Load Cell Calibration" },
        mesafeBaslik: { tr: "Mesafe Sensörü Kalibrasyonu", en: "Distance Sensor Calibration" },
        sonKalibrasyonOnEk: { tr: "Son kalibrasyon: ", en: "Last calibration: " },
        cihaziYerlestir: { tr: "Cihazı düz ve sabit bir yüzeye yerleştirin", en: "Place the device on a flat, stable surface" },
        cihazBagliDegilKalibrasyon: { tr: "Cihaz bağlı değil — kalibrasyon kaydedilemez", en: "Device not connected — calibration
cannot be saved" },
        sifirlaButon: { tr: "Sıfırla (Zero)", en: "Zero (Reset)" },
        kalibrasyonKaydedildiMesaj: { tr: "✓ Kalibrasyon kaydedildi", en: "✓ Calibration saved" },
        kalibrasyonKaydedilemediMesaj: { tr: "✕ Kalibrasyon kaydedilemedi, tekrar deneyin", en: "✕ Calibration could not be
saved, try again" },
        kayitliOnEk: { tr: "Kayıtlı: ", en: "Saved: " },
        noktaSonEk: { tr: " nokta", en: " points" },
        kayitliKalibrasyonBulundu: { tr: "Bu sensör için kayıtlı bir kalibrasyon bulundu", en: "A saved calibration was found for
this sensor" },
        mevcutGoruntule: { tr: "Mevcut Kalibrasyonu Görüntüle", en: "View Current Calibration" },
        kayitliNoktalariIncele: { tr: "Kayıtlı noktaları incele", en: "Review saved points" },
        yenidenKalibreEt: { tr: "Yeniden Kalibre Et", en: "Recalibrate" },
        sifirdanYeniOlcum: { tr: "Sıfırdan yeni ölçüm al", en: "Take a fresh measurement" },
        kayitliNoktalarBaslik: { tr: "KAYITLI KALİBRASYON NOKTALARI", en: "SAVED CALIBRATION POINTS" },
        duzenleButon: { tr: "Düzenle", en: "Edit" },
        silinecekOnay: { tr: "Mevcut kalibrasyon silinecek, emin misiniz?", en: "The current calibration will be deleted, are you
sure?" },
        vazgecButon: { tr: "Vazgeç", en: "Cancel" },
        evetYenidenBaslaButon: { tr: "Evet, Yeniden Başla", en: "Yes, Start Over" },
        kalibrasyonTamamlandiBaslik: { tr: "Kalibrasyon Tamamlandı", en: "Calibration Complete" },
        toplamOnEk: { tr: "Toplam ", en: "Total " },
        noktaGirildiSonEk: { tr: " nokta girildi. Bu kalibrasyonu kaydetmek istiyor musunuz?", en: " points entered. Do you want
to save this calibration?" },
        noktalariKontrolEt: { tr: "Noktaları Kontrol Et", en: "Review Points" },
        kaydetVeBitir: { tr: "Kaydet ve Bitir", en: "Save and Finish" },
        kacNoktaSoru: { tr: "Kaç noktalı kalibrasyon yapmak istersiniz?", en: "How many calibration points would you like to
use?" },
        dahaFazlaNokta: { tr: "Daha fazla nokta, daha hassas kalibrasyon sağlar", en: "More points provide a more precise
calibration" },
        hamDegerOnEk: { tr: "Ham değer: ", en: "Raw value: " },
        hamDegerSifirUyari: { tr: "Cihaz bağlı değil — ham değer 0 olarak kaydedilecek", en: "Device not connected — raw value
will be saved as 0" },
        degisiklikleriKaydet: { tr: "Değişiklikleri Kaydet", en: "Save Changes" },
        kaydedilenNoktalarBaslik: { tr: "KAYDEDİLEN NOKTALAR", en: "SAVED POINTS" },
        duzeltmekIcinTikla: { tr: "Düzeltmek için bir noktaya tıkla", en: "Click a point to correct it" },
        kalibrasyonuDuzenle: { tr: "Kalibrasyonu Düzenle", en: "Edit Calibration" },
        noktaOnEk: { tr: "Nokta ", en: "Point " },
        duzenleniyorSonEk: { tr: " düzenleniyor", en: " being edited" },
        duzenlemekIcinTikla: { tr: "Düzenlemek istediğiniz noktaya sağdaki listeden tıklayın", en: "Click the point you want to
edit from the list on the right" },
        hedefAgirlikYonerge: { tr: "Hedef ağırlığı gir, sonra o ağırlığı load cell'e koy", en: "Enter the target weight, then
place that weight on the load cell" },
        hedefMesafeYonerge: { tr: "Hedef mesafeyi gir, sonra boruyu o mesafeye getir", en: "Enter the target distance, then move
```

```
the tube to that distance" },
        noktayiGuncelle: { tr: "Noktayı Güncelle", en: "Update Point" },
        kalibrasyonuTamamla: { tr: "Kalibrasyonu Tamamla", en: "Complete Calibration" },
        sonNoktayiKaydet: { tr: "Son Noktayı Kaydet", en: "Save Last Point" },
        buNoktayiKaydet: { tr: "Bu Noktayı Kaydet", en: "Save This Point" },
        konumSinirlari: { tr: "Stroke Konumları", en: "Stroke Positions" },
        konumSinirlariAciklama: { tr: "Üst / alt konum referansı", en: "Top / bottom position reference" },
        konumBaslik: { tr: "Üst / Alt Konum Kalibrasyonu", en: "Top / Bottom Position Calibration" },
        konumYonerge: { tr: "Stroke'ların otomatik algılanması için borunun üst ve alt konumu kaydedilmelidir. Boruyu tamamen üst
konuma kaldırıp kilitleyin ve \"Üst Konumu Kaydet\"e basın. Sonra boruyu en alt konuma indirip \"Alt Konumu Kaydet\"e basın.",
en: "For automatic stroke detection the top and bottom positions of the pipe must be saved. Lift the pipe fully to the top
position, lock it and press \"Save Top Position\". Then lower the pipe to the bottom and press \"Save Bottom Position\"." },
        anlikKonum: { tr: "ANLIK KONUM", en: "CURRENT POSITION" },
        ustKonum: { tr: "Üst konum", en: "Top position" },
        altKonum: { tr: "Alt konum", en: "Bottom position" },
        ustKaydet: { tr: "Üst Konumu Kaydet", en: "Save Top Position" },
        altKaydet: { tr: "Alt Konumu Kaydet", en: "Save Bottom Position" },
        varsayilanaDon: { tr: "Varsayılana Dön", en: "Reset to Default" },
        konumAralikHatasi: { tr: "✕ Üst ve alt konum arasında en az 10 cm fark olmalı", en: "✕ Top and bottom positions must
differ by at least 10 cm" },
        varsayilan: { tr: " (varsayılan)", en: " (default)" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    property int seciliSensor: -1
    property bool konumKayitliVar: false
    property string konumSonTarih: ""
    property string konumBildirim: ""
    property bool konumBildirimHata: false

    function konumSinirlariniKaydet(ust, alt) {
        if (Math.abs(ust - alt) < 100) {
            konumBildirim = txt("konumAralikHatasi")
            konumBildirimHata = true
            return
        }
        if (database.kalibrasyonKaydet("konum_sinirlari", ust, alt)) {
            database.kalibrasyonTarihiKaydet("konum_sinirlari")
            calculator.ustKonumMm = ust
            calculator.altKonumMm = alt
            wifiManager.kalibrasyonYenidenYukle()
            konumKayitliVar = true
            konumSonTarih = database.kalibrasyonTarihiGetir("konum_sinirlari")
            konumBildirim = txt("kalibrasyonKaydedildiMesaj")
            konumBildirimHata = false
        } else {
            konumBildirim = txt("kalibrasyonKaydedilemediMesaj")
            konumBildirimHata = true
        }
    }
    property int loadCellAdimi: 0

    property bool egimKayitliVar: false
    property string egimSonTarih: ""
    property bool egimBildirimGoster: false
    property bool egimBildirimBasarili: true
    property string egimBildirimMesaj: ""

    property bool loadCellKayitliVar: false
    property string loadCellSonTarih: ""
    property int loadCellNoktaSayisi: 0
    property var loadCellNoktalari: []
    onLoadCellNoktalariChanged: loadCellAdimi = loadCellNoktalari.length
    property int loadCellToplamNokta: 10
    property bool loadCellSayiSecildi: false
    property int loadCellDuzenlemeIndex: -1
```

```
property bool loadCellAraEkranGoster: false
property bool loadCellGoruntuleModu: false
property bool loadCellTamDuzenlemeModu: false
property bool loadCellYenidenOnayGoster: false
property bool loadCellTamamlandiOnayGoster: false

property bool loadCellBildirimGoster: false
property bool loadCellBildirimBasarili: true
property string loadCellBildirimMesaj: ""

property string mesafeSonTarih: ""

property bool mesafeOlcumKayitliVar: false
property int mesafeOlcumNoktaSayisi: 0
property var mesafeNoktalari: []
property int mesafeToplamNokta: 10
property bool mesafeSayiSecildi: false
property int mesafeDuzenlemeIndex: -1

property bool mesafeAraEkranGoster: false
property bool mesafeGoruntuleModu: false
property bool mesafeTamDuzenlemeModu: false
property bool mesafeYenidenOnayGoster: false
property bool mesafeTamamlandiOnayGoster: false

property bool mesafeBildirimGoster: false
property bool mesafeBildirimBasarili: true
property string mesafeBildirimMesaj: ""

function loadCellSiraliKopya(dizi) {
    var kopya = dizi.slice()
    kopya.sort(function(a, b) { return a.hamDeger - b.hamDeger })
    return kopya
}

function loadCellKaydetVeBildir() {
    var sirali = loadCellSiraliKopya(loadCellNoktalari)
    var basarili = database.loadCellNoktalariKaydet(sirali)
    if (basarili) {
        wifiManager.kalibrasyonYenidenYukle()
    }
    loadCellBildirimBasarili = basarili
    loadCellBildirimMesaj = basarili ? txt("kalibrasyonKaydedildiMesaj") : txt("kalibrasyonKaydedilemediMesaj")
    loadCellBildirimGoster = true
    loadCellBildirimTimer.restart()
}

Timer {
    id: loadCellBildirimTimer
    interval: loadCellBildirimBasarili ? 1800 : 2500
    repeat: false
    onTriggered: {
        loadCellBildirimGoster = false
        if (loadCellBildirimBasarili) {
            seciliSensor = -1
        }
    }
}

function mesafeSiraliKopya(dizi) {
    var kopya = dizi.slice()
    kopya.sort(function(a, b) { return a.hamDeger - b.hamDeger })
    return kopya
}

function mesafeKaydetVeBildir() {
    var sirali = mesafeSiraliKopya(mesafeNoktalari)
    var basarili = database.mesafeNoktalariKaydet(sirali)
    if (basarili) {
        wifiManager.kalibrasyonYenidenYukle()
```

```
    }
    mesafeBildirimBasarili = basarili
    mesafeBildirimMesaj = basarili ? txt("kalibrasyonKaydedildiMesaj") : txt("kalibrasyonKaydedilemediMesaj")
    mesafeBildirimGoster = true
    mesafeBildirimTimer.restart()
}

Timer {
    id: mesafeBildirimTimer
    interval: mesafeBildirimBasarili ? 1800 : 2500
    repeat: false
    onTriggered: {
        mesafeBildirimGoster = false
        if (mesafeBildirimBasarili) {
            seciliSensor = -1
        }
    }
}

function egimSifirlaVeBildir() {
    var biasX = sensorManager.hamAccelX
    var biasY = sensorManager.hamAccelY
    var basarili = database.egimKalibrasyonuKaydet(biasX, biasY, 0.0, 1.0, 1.0, 1.0)
    if (basarili) {
        wifiManager.kalibrasyonYenidenYukle()
    }
    egimBildirimBasarili = basarili
    egimBildirimMesaj = basarili ? txt("kalibrasyonKaydedildiMesaj") : txt("kalibrasyonKaydedilemediMesaj")
    egimBildirimGoster = true
    egimBildirimTimer.restart()
}

Timer {
    id: egimBildirimTimer
    interval: egimBildirimBasarili ? 1800 : 2500
    repeat: false
    onTriggered: {
        egimBildirimGoster = false
        if (egimBildirimBasarili) {
            seciliSensor = -1
        }
    }
}

function kalibrasyonDurumunuYukle() {
    var egimKal = database.egimKalibrasyonuGetir()
    egimKayitliVar = egimKal.mevcut
    egimSonTarih = egimKal.mevcut ? egimKal.tarih : ""

    var lcNoktalar = database.loadCellNoktalariGetir()
    loadCellKayitliVar = lcNoktalar.length > 0
    loadCellNoktaSayisi = lcNoktalar.length
    loadCellSonTarih = loadCellKayitliVar ? database.kalibrasyonTarihiGetir("loadcell") : ""

    var mNoktalar = database.mesafeNoktalariGetir()
    mesafeOlcumKayitliVar = mNoktalar.length > 0
    mesafeOlcumNoktaSayisi = mNoktalar.length
    mesafeSonTarih = mesafeOlcumKayitliVar ? database.kalibrasyonTarihiGetir("mesafe_olcum") : ""

    var konumKal = database.kalibrasyonGetir("konum_sinirlari")
    konumKayitliVar = konumKal.mevcut
    konumSonTarih = konumKal.mevcut ? database.kalibrasyonTarihiGetir("konum_sinirlari") : ""
}

Component.onCompleted: kalibrasyonDurumunuYukle()

onVisibleChanged: {
    if (visible) {
        kalibrasyonDurumunuYukle()
    }
```

```
}

onSeciliSensorChanged: {
    if (seciliSensor === 0) {
        var egimKal = database.egimKalibrasyonuGetir()
        egimKayitliVar = egimKal.mevcut
        egimSonTarih = egimKal.mevcut ? egimKal.tarih : ""
        egimBildirimGoster = false
    } else if (seciliSensor === 1) {
        var noktalar = database.loadCellNoktalariGetir()
        loadCellKayitliVar = noktalar.length > 0
        loadCellNoktaSayisi = noktalar.length
        loadCellSonTarih = loadCellKayitliVar ? database.kalibrasyonTarihiGetir("loadcell") : ""
        loadCellNoktalari = []
        loadCellSayiSecildi = false
        loadCellDuzenlemeIndex = -1
        loadCellTamDuzenlemeModu = false
        loadCellGoruntuleModu = false
        loadCellYenidenOnayGoster = false
        loadCellTamamlandiOnayGoster = false
        loadCellBildirimGoster = false
        loadCellAraEkranGoster = loadCellKayitliVar
    } else if (seciliSensor === 2) {
        var mNoktalar = database.mesafeNoktalariGetir()
        mesafeOlcumKayitliVar = mNoktalar.length > 0
        mesafeOlcumNoktaSayisi = mNoktalar.length
        mesafeSonTarih = mesafeOlcumKayitliVar ? database.kalibrasyonTarihiGetir("mesafe_olcum") : ""
        mesafeNoktalari = []
        mesafeSayiSecildi = false
        mesafeDuzenlemeIndex = -1
        mesafeTamDuzenlemeModu = false
        mesafeGoruntuleModu = false
        mesafeYenidenOnayGoster = false
        mesafeTamamlandiOnayGoster = false
        mesafeBildirimGoster = false
        mesafeAraEkranGoster = mesafeOlcumKayitliVar
    }
}

StackLayout {
    anchors.fill: parent
    currentIndex: seciliSensor === -1 ? 0 : 1

    Item {
        Text {
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.margins: 20
            text: txt("baslikSensorSec")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 12
            font.bold: true
            font.letterSpacing: 1
        }

        Row {
            anchors.centerIn: parent
            spacing: 24

            Rectangle {
                id: egimKarti
                width: 300
                height: 450
                radius: 12
                color: "#12121a"
                border.color: egimKarti.hovered ? "#14b8a6" : "#1e2a3f"
                border.width: 1
                property bool hovered: false
                scale: hovered ? 1.03 : 1.0
```

```
                    Behavior on border.color { ColorAnimation { duration: 120 } }
                    Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

                    Column {
                        anchors.centerIn: parent
                        spacing: 14

                        Item {
                            anchors.horizontalCenter: parent.horizontalCenter
                            width: 210
                            height: 150

                            Image {
                                anchors.centerIn: parent
                                width: parent.width
                                height: parent.height
                                source: "../assets/egim_sensor.png"
                                fillMode: Image.PreserveAspectFit
                            }
                        }

                        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("egimSensoru"); color: "#dce8f5";
font.family: "Segoe UI"; font.pixelSize: 24; font.bold: true }
                        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("egimAciklama"); color: "#6b7280";
font.family: "Segoe UI"; font.pixelSize: 14 }

                        Rectangle {
                            anchors.horizontalCenter: parent.horizontalCenter
                            visible: egimKayitliVar
                            width: kaliMetni1.implicitWidth + 16
                            height: 20
                            radius: 10
                            color: "#123321"

                            Text {
                                id: kaliMetni1
                                anchors.centerIn: parent
                                text: txt("kalibreEdildiOnEk") + egimSonTarih
                                color: "#4ade80"
                                font.pixelSize: 10
                                font.bold: true
                            }
                        }
                    }

                    MouseArea {
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onEntered: egimKarti.hovered = true
                        onExited: egimKarti.hovered = false
                        onClicked: seciliSensor = 0
                    }
                }

                Rectangle {
                    id: loadCellKarti
                    width: 300
                    height: 450
                    radius: 12
                    color: "#12121a"
                    border.color: loadCellKarti.hovered ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
                    property bool hovered: false
                    scale: hovered ? 1.03 : 1.0
                    Behavior on border.color { ColorAnimation { duration: 120 } }
                    Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

                    Column {
                        anchors.centerIn: parent
```

```
                        spacing: 14

                        Item {
                            anchors.horizontalCenter: parent.horizontalCenter
                            width: 210
                            height: 150

                            Image {
                                anchors.centerIn: parent
                                width: parent.width
                                height: parent.height
                                source: "../assets/loadcell.png"
                                fillMode: Image.PreserveAspectFit
                            }
                        }

                        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("loadCellAdi"); color: "#dce8f5";
font.family: "Segoe UI"; font.pixelSize: 24; font.bold: true }
                        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("loadCellAciklama"); color:
"#6b7280"; font.family: "Segoe UI"; font.pixelSize: 14 }

                        Rectangle {
                            anchors.horizontalCenter: parent.horizontalCenter
                            visible: loadCellKayitliVar
                            width: kaliMetni2.implicitWidth + 16
                            height: 20
                            radius: 10
                            color: "#123321"

                            Text {
                                id: kaliMetni2
                                anchors.centerIn: parent
                                text: txt("kalibreEdildiOnEk") + loadCellSonTarih
                                color: "#4ade80"
                                font.pixelSize: 10
                                font.bold: true
                            }
                        }
                    }

                    MouseArea {
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onEntered: loadCellKarti.hovered = true
                        onExited: loadCellKarti.hovered = false
                        onClicked: seciliSensor = 1
                    }
                }

                Rectangle {
                    id: mesafeKarti
                    width: 300
                    height: 450
                    radius: 12
                    color: "#12121a"
                    border.color: mesafeKarti.hovered ? "#9333ea" : "#1e2a3f"
                    border.width: 1
                    property bool hovered: false
                    scale: hovered ? 1.03 : 1.0
                    Behavior on border.color { ColorAnimation { duration: 120 } }
                    Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

                    Column {
                        anchors.centerIn: parent
                        spacing: 14

                        Item {
                            anchors.horizontalCenter: parent.horizontalCenter
                            width: 210
```

```
                            height: 150

                            Image {
                                anchors.centerIn: parent
                                width: parent.width
                                height: parent.height
                                source: "../assets/mesafe_sensor.png"
                                fillMode: Image.PreserveAspectFit
                            }
                        }

                        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("mesafeSensoru"); color: "#dce8f5";
font.family: "Segoe UI"; font.pixelSize: 24; font.bold: true }
                        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("mesafeAciklama"); color: "#6b7280";
font.family: "Segoe UI"; font.pixelSize: 14 }

                        Rectangle {
                            anchors.horizontalCenter: parent.horizontalCenter
                            visible: mesafeOlcumKayitliVar
                            width: kaliMetni3.implicitWidth + 16
                            height: 20
                            radius: 10
                            color: "#123321"

                            Text {
                                id: kaliMetni3
                                anchors.centerIn: parent
                                text: txt("kalibreEdildiOnEk") + mesafeSonTarih
                                color: "#4ade80"
                                font.pixelSize: 10
                                font.bold: true
                            }
                        }
                    }

                    MouseArea {
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onEntered: mesafeKarti.hovered = true
                        onExited: mesafeKarti.hovered = false
                        onClicked: seciliSensor = 2
                    }
                }

                Rectangle {
                    id: konumKarti
                    width: 300
                    height: 450
                    radius: 12
                    color: "#12121a"
                    border.color: konumKarti.hovered ? "#f59e0b" : "#1e2a3f"
                    border.width: 1
                    property bool hovered: false
                    scale: hovered ? 1.03 : 1.0
                    Behavior on border.color { ColorAnimation { duration: 120 } }
                    Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

                    Column {
                        anchors.centerIn: parent
                        spacing: 14

                        Item {
                            anchors.horizontalCenter: parent.horizontalCenter
                            width: 210
                            height: 150

                            Rectangle { x: 90; y: 20; width: 30; height: 110; radius: 4; color: "#1e2a3f"; border.color:
"#f59e0b"; border.width: 2 }
                            Rectangle { x: 40; y: 20; width: 130; height: 2; color: "#4ade80" }
```

```
                        Rectangle { x: 40; y: 128; width: 130; height: 2; color: "#f87171" }
                        Text { x: 176; y: 12; text: "▲"; color: "#4ade80"; font.pixelSize: 14 }
                        Text { x: 176; y: 120; text: "▼"; color: "#f87171"; font.pixelSize: 14 }
                    }

                    Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("konumSinirlari"); color: "#dce8f5";
font.family: "Segoe UI"; font.pixelSize: 24; font.bold: true }
                    Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("konumSinirlariAciklama"); color:
"#6b7280"; font.family: "Segoe UI"; font.pixelSize: 14 }

                    Rectangle {
                        anchors.horizontalCenter: parent.horizontalCenter
                        visible: konumKayitliVar
                        width: kaliMetni4.implicitWidth + 16
                        height: 20
                        radius: 10
                        color: "#123321"

                        Text {
                            id: kaliMetni4
                            anchors.centerIn: parent
                            text: txt("kalibreEdildiOnEk") + konumSonTarih
                            color: "#4ade80"
                            font.pixelSize: 10
                            font.bold: true
                        }
                    }
                }

                MouseArea {
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onEntered: konumKarti.hovered = true
                    onExited: konumKarti.hovered = false
                    onClicked: {
                        konumBildirim = ""
                        seciliSensor = 3
                    }
                }
            }
        }
    }

    Item {
        Row {
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.margins: 20
            spacing: 12

            Button {
                text: txt("geriButon")
                font.pixelSize: 13

                background: Rectangle {
                    radius: 6
                    color: parent.hovered ? "#1e2a3f" : "#12121a"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#dce8f5"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                    leftPadding: 10
```

```
            rightPadding: 10
        }

        onClicked: seciliSensor = -1
    }

    Rectangle {
        width: 8
        height: 8
        radius: 4
        anchors.verticalCenter: parent.verticalCenter
        color: {
            if (seciliSensor === 0) return "#14b8a6"
            if (seciliSensor === 1) return "#3b82f6"
            if (seciliSensor === 2) return "#9333ea"
            if (seciliSensor === 3) return "#f59e0b"
            return "#6b7280"
        }
    }

    Text {
        anchors.verticalCenter: parent.verticalCenter
        text: {
            if (seciliSensor === 0) return txt("egimBaslik")
            if (seciliSensor === 1) return txt("loadCellBaslik")
            if (seciliSensor === 2) return txt("mesafeBaslik")
            if (seciliSensor === 3) return txt("konumBaslik")
            return ""
        }
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 16
        font.bold: true
    }
}

Item {
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.bottom: parent.bottom
    anchors.topMargin: 70
    visible: seciliSensor === 3

    Column {
        anchors.centerIn: parent
        width: 620
        spacing: 18

        Text {
            width: parent.width
            text: txt("konumYonerge")
            color: "#9ca3af"
            font.family: "Segoe UI"
            font.pixelSize: 13
            wrapMode: Text.WordWrap
            horizontalAlignment: Text.AlignHCenter
        }

        Rectangle {
            width: parent.width
            height: 110
            radius: 12
            color: "#12121a"
            border.color: "#1e2a3f"
            border.width: 1

            Column {
                anchors.centerIn: parent
                spacing: 4
```

```
                        Text { anchors.horizontalCenter: parent.horizontalCenter; text: txt("anlikKonum"); color: "#6b7280";
font.family: "Segoe UI"; font.pixelSize: 11; font.bold: true; font.letterSpacing: 1 }
                        Text {
                            anchors.horizontalCenter: parent.horizontalCenter
                            text: sensorManager.veriGecerli ? (sensorManager.konum / 10).toFixed(1) + " cm" : "-- cm"
                            color: "#f59e0b"
                            font.family: "Segoe UI"
                            font.pixelSize: 40
                            font.bold: true
                        }
                    }
                }

                Repeater {
                    model: [
                        { ad: txt("ustKonum"), deger: calculator.ustKonumMm, buton: txt("ustKaydet"), renk: "#16a34a", ust:
true },
                        { ad: txt("altKonum"), deger: calculator.altKonumMm, buton: txt("altKaydet"), renk: "#dc2626", ust:
false }
                    ]

                    Rectangle {
                        width: 620
                        height: 60
                        radius: 10
                        color: "#12121a"
                        border.color: "#1e2a3f"
                        border.width: 1

                        Rectangle { width: 4; height: parent.height; radius: 2; color: modelData.renk }

                        Column {
                            anchors.left: parent.left
                            anchors.leftMargin: 20
                            anchors.verticalCenter: parent.verticalCenter
                            spacing: 2
                            Text { text: modelData.ad; color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12 }
                            Text {
                                text: (modelData.deger / 10).toFixed(1) + " cm" + (konumKayitliVar ? "" : txt("varsayilan"))
                                color: "#dce8f5"
                                font.family: "Segoe UI"
                                font.pixelSize: 18
                                font.bold: true
                            }
                        }

                        Button {
                            anchors.right: parent.right
                            anchors.rightMargin: 12
                            anchors.verticalCenter: parent.verticalCenter
                            width: 220
                            height: 38
                            text: modelData.buton
                            enabled: sensorManager.veriGecerli
                            font.pixelSize: 13
                            font.bold: true

                            onClicked: {
                                var k = sensorManager.konum
                                if (modelData.ust) konumSinirlariniKaydet(k, calculator.altKonumMm)
                                else konumSinirlariniKaydet(calculator.ustKonumMm, k)
                            }

                            background: Rectangle {
                                radius: 8
                                color: parent.enabled ? (parent.hovered ? Qt.lighter(modelData.renk, 1.2) : modelData.renk) :
"#1e2a3f"
                            }

                            contentItem: Text {
```

```
                    text: parent.text
                    color: "#dce8f5"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }
        }
    }

    Text {
        width: parent.width
        visible: !sensorManager.veriGecerli
        text: txt("cihazBagliDegilKalibrasyon")
        color: "#f87171"
        font.family: "Segoe UI"
        font.pixelSize: 12
        horizontalAlignment: Text.AlignHCenter
    }

    Text {
        width: parent.width
        visible: konumBildirim.length > 0
        text: konumBildirim
        color: konumBildirimHata ? "#f87171" : "#4ade80"
        font.family: "Segoe UI"
        font.pixelSize: 12
        font.bold: true
        horizontalAlignment: Text.AlignHCenter
    }

    Button {
        anchors.horizontalCenter: parent.horizontalCenter
        width: 180
        height: 34
        text: txt("varsayilanaDon")
        font.pixelSize: 12
        onClicked: konumSinirlariniKaydet(40, 540)

        background: Rectangle {
            radius: 8
            color: parent.hovered ? "#1e2a3f" : "transparent"
            border.color: "#1e2a3f"
            border.width: 1
        }

        contentItem: Text {
            text: parent.text
            color: "#9ca3af"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }
  }
}

Item {
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.bottom: parent.bottom
    anchors.topMargin: 70
    visible: seciliSensor === 0

    Column {
        anchors.centerIn: parent
        spacing: 20
        width: 360
```

```
Rectangle {
    anchors.horizontalCenter: parent.horizontalCenter
    visible: egimKayitliVar
    width: egimSonKaliMetni.implicitWidth + 20
    height: 26
    radius: 13
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    Text {
        id: egimSonKaliMetni
        anchors.centerIn: parent
        text: txt("sonKalibrasyonOnEk") + egimSonTarih
        color: "#6b7280"
        font.pixelSize: 11
    }
}

Text {
    width: parent.width
    horizontalAlignment: Text.AlignHCenter
    wrapMode: Text.WordWrap
    text: txt("cihaziYerlestir")
    color: "#dce8f5"
    font.family: "Segoe UI"
    font.pixelSize: 14
}

SuTerazisi {
    anchors.horizontalCenter: parent.horizontalCenter
    egimX: sensorManager.egimX
    egimY: sensorManager.egimY
}

Rectangle {
    anchors.horizontalCenter: parent.horizontalCenter
    width: 220
    height: 72
    radius: 12
    color: "#0f2e2a"
    border.color: "#14b8a6"
    border.width: 1

    Text {
        anchors.centerIn: parent
        text: "X: " + sensorManager.egimX.toFixed(1) + "°   Y: " + sensorManager.egimY.toFixed(1) + "°"
        color: "#2dd4bf"
        font.family: "Segoe UI"
        font.pixelSize: 22
        font.bold: true
    }
}

Text {
    width: parent.width
    horizontalAlignment: Text.AlignHCenter
    wrapMode: Text.WordWrap
    visible: !wifiManager.baglandi
    text: txt("cihazBagliDegilKalibrasyon")
    color: "#f87171"
    font.family: "Segoe UI"
    font.pixelSize: 11
}

Button {
    width: parent.width
    height: 44
    text: txt("sifirlaButon")
    font.pixelSize: 14
```

```
            font.bold: true
            enabled: wifiManager.baglandi

            onClicked: egimSifirlaVeBildir()

            background: Rectangle {
                radius: 8
                color: parent.enabled ? (parent.hovered ? "#2dd4bf" : "#14b8a6") : "#1e2a3f"
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#dce8f5"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }

    Rectangle {
        id: egimBildirimKutusu
        anchors.top: parent.top
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.topMargin: 12
        z: 100
        width: egimBildirimMetni.implicitWidth + 24
        height: 32
        radius: 16
        color: egimBildirimBasarili ? "#123321" : "#3a1414"
        opacity: egimBildirimGoster ? 1 : 0
        visible: opacity > 0
        Behavior on opacity { NumberAnimation { duration: 200 } }

        Text {
            id: egimBildirimMetni
            anchors.centerIn: parent
            text: egimBildirimMesaj
            color: egimBildirimBasarili ? "#4ade80" : "#f87171"
            font.family: "Segoe UI"
            font.pixelSize: 12
            font.bold: true
        }
    }
}

Item {
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.bottom: parent.bottom
    anchors.topMargin: 70
    visible: seciliSensor === 1

    Column {
        anchors.centerIn: parent
        spacing: 24
        width: 420
        visible: loadCellAraEkranGoster

        Rectangle {
            anchors.horizontalCenter: parent.horizontalCenter
            width: araEkranBadge.implicitWidth + 20
            height: 26
            radius: 13
            color: "#12121a"
            border.color: "#1e2a3f"
            border.width: 1
```

```
    Text {
        id: araEkranBadge
        anchors.centerIn: parent
        text: txt("kayitliOnEk") + loadCellNoktaSayisi + txt("noktaSonEk")
        color: "#6b7280"
        font.pixelSize: 11
    }
}

Text {
    width: parent.width
    horizontalAlignment: Text.AlignHCenter
    wrapMode: Text.WordWrap
    text: txt("kayitliKalibrasyonBulundu")
    color: "#dce8f5"
    font.family: "Segoe UI"
    font.pixelSize: 16
    font.bold: true
}

Row {
    anchors.horizontalCenter: parent.horizontalCenter
    spacing: 20

    Rectangle {
        id: goruntuleKarti
        width: 190
        height: 180
        radius: 14
        color: "#12121a"
        border.color: goruntuleKarti.hovered ? "#3b82f6" : "#1e2a3f"
        border.width: 1
        property bool hovered: false
        Behavior on border.color { ColorAnimation { duration: 120 } }

        Column {
            anchors.centerIn: parent
            spacing: 12
            width: parent.width - 28

            Canvas {
                anchors.horizontalCenter: parent.horizontalCenter
                width: 32
                height: 32
                onPaint: {
                    var ctx = getContext("2d")
                    ctx.clearRect(0, 0, width, height)
                    ctx.lineWidth = 1.8
                    ctx.lineCap = "round"
                    ctx.lineJoin = "round"
                    ctx.strokeStyle = "#3b82f6"

                    ctx.beginPath()
                    ctx.moveTo(3, 16)
                    ctx.bezierCurveTo(7.5, 8.5, 12.5, 6.8, 16, 6.8)
                    ctx.bezierCurveTo(19.5, 6.8, 24.5, 8.5, 29, 16)
                    ctx.bezierCurveTo(24.5, 23.5, 19.5, 25.2, 16, 25.2)
                    ctx.bezierCurveTo(12.5, 25.2, 7.5, 23.5, 3, 16)
                    ctx.stroke()

                    ctx.beginPath()
                    ctx.arc(16, 16, 4.4, 0, Math.PI * 2, false)
                    ctx.stroke()
                }
            }

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                wrapMode: Text.WordWrap
```

```
            text: txt("mevcutGoruntule")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
        }

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("kayitliNoktalariIncele")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
        }
    }

    MouseArea {
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onEntered: goruntuleKarti.hovered = true
        onExited: goruntuleKarti.hovered = false
        onClicked: {
            loadCellNoktalari = database.loadCellNoktalariGetir()
            loadCellAraEkranGoster = false
            loadCellGoruntuleModu = true
            loadCellDuzenlemeIndex = -1
        }
    }
}

Rectangle {
    id: yenidenKarti
    width: 190
    height: 180
    radius: 14
    color: "#12121a"
    border.color: yenidenKarti.hovered ? "#9ca3af" : "#1e2a3f"
    border.width: 1
    property bool hovered: false
    Behavior on border.color { ColorAnimation { duration: 120 } }

    Column {
        anchors.centerIn: parent
        spacing: 12
        width: parent.width - 28

        Canvas {
            anchors.horizontalCenter: parent.horizontalCenter
            width: 32
            height: 32
            onPaint: {
                var ctx = getContext("2d")
                ctx.clearRect(0, 0, width, height)
                ctx.strokeStyle = "#9ca3af"
                ctx.fillStyle = "#9ca3af"
                ctx.lineWidth = 3.4
                ctx.lineCap = "butt"

                function yayOku(startDeg, endDeg) {
                    var s = startDeg * Math.PI / 180
                    var e = endDeg * Math.PI / 180
                    var cx = 16, cy = 16, r = 10.5

                    ctx.beginPath()
                    ctx.arc(cx, cy, r, s, e, false)
                    ctx.stroke()
```

```
                            var ex = cx + r * Math.cos(e)
                            var ey = cy + r * Math.sin(e)
                            var tx = -Math.sin(e)
                            var ty = Math.cos(e)
                            var px = -ty
                            var py = tx
                            var geri = 6.2
                            var yaricap = 3.6
                            var bx = ex - tx * geri
                            var by = ey - ty * geri

                            ctx.beginPath()
                            ctx.moveTo(ex, ey)
                            ctx.lineTo(bx + px * yaricap, by + py * yaricap)
                            ctx.lineTo(bx - px * yaricap, by - py * yaricap)
                            ctx.closePath()
                            ctx.fill()
                        }

                        yayOku(195, 345)
                        yayOku(15, 165)
                    }
                }

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    wrapMode: Text.WordWrap
                    text: txt("yenidenKalibreEt")
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                    font.bold: true
                }

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    wrapMode: Text.WordWrap
                    text: txt("sifirdanYeniOlcum")
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                }
            }

            MouseArea {
                anchors.fill: parent
                hoverEnabled: true
                cursorShape: Qt.PointingHandCursor
                onEntered: yenidenKarti.hovered = true
                onExited: yenidenKarti.hovered = false
                onClicked: loadCellYenidenOnayGoster = true
            }
        }
    }
}

Column {
    anchors.centerIn: parent
    spacing: 16
    width: 340
    visible: loadCellGoruntuleModu

    Text {
        text: txt("kayitliNoktalarBaslik")
        color: "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 11
        font.bold: true
```

```
    font.letterSpacing: 1
}

Rectangle {
    width: parent.width
    height: 320
    radius: 14
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    ListView {
        anchors.fill: parent
        anchors.margins: 16
        clip: true
        spacing: 8

        model: loadCellNoktalari

        delegate: Rectangle {
            width: ListView.view.width
            height: 48
            radius: 8
            color: "#0a0a0d"
            border.color: "#1e2a3f"
            border.width: 1

            Row {
                anchors.fill: parent
                anchors.leftMargin: 12
                anchors.rightMargin: 12
                spacing: 8

                Rectangle {
                    width: 22
                    height: 22
                    radius: 11
                    anchors.verticalCenter: parent.verticalCenter
                    color: "#17263d"
                    border.color: "#3b82f6"
                    border.width: 1

                    Text {
                        anchors.centerIn: parent
                        text: index + 1
                        color: "#3b82f6"
                        font.pixelSize: 10
                        font.bold: true
                    }
                }

                Column {
                    anchors.verticalCenter: parent.verticalCenter
                    spacing: 1

                    Text {
                        text: modelData.hedefKg.toFixed(2) + " kg"
                        color: "#dce8f5"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                        font.bold: true
                    }

                    Text {
                        text: "ADC: " + modelData.hamDeger.toFixed(0)
                        color: "#6b7280"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                    }
                }
```

```
                }
            }
        }
    }

    Button {
        width: parent.width
        height: 46
        text: txt("duzenleButon")
        font.pixelSize: 14
        font.bold: true

        onClicked: {
            loadCellToplamNokta = loadCellNoktalari.length
            loadCellGoruntuleModu = false
            loadCellTamDuzenlemeModu = true
            loadCellSayiSecildi = true
            loadCellDuzenlemeIndex = -1
        }

        background: Rectangle {
            radius: 8
            color: parent.hovered ? "#4f8cf7" : "#3b82f6"
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#dce8f5"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }
}

Rectangle {
    anchors.fill: parent
    color: "#000000b3"
    visible: loadCellYenidenOnayGoster
    z: 200

    MouseArea { anchors.fill: parent }

    Rectangle {
        anchors.centerIn: parent
        width: 360
        height: 200
        radius: 14
        color: "#12121a"
        border.color: "#1e2a3f"
        border.width: 1

        Column {
            anchors.centerIn: parent
            spacing: 18
            width: parent.width - 40

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                wrapMode: Text.WordWrap
                text: txt("silinecekOnay")
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 14
                font.bold: true
            }

            Row {
```

```
                anchors.horizontalCenter: parent.horizontalCenter
                spacing: 12

                Button {
                    width: 150
                    height: 42
                    text: txt("vazgecButon")

                    onClicked: loadCellYenidenOnayGoster = false

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#1e2a3f" : "transparent"
                        border.color: "#1e2a3f"
                        border.width: 1
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#9ca3af"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    width: 150
                    height: 42
                    text: txt("evetYenidenBaslaButon")
                    font.bold: true

                    onClicked: {
                        loadCellYenidenOnayGoster = false
                        loadCellAraEkranGoster = false
                        loadCellGoruntuleModu = false
                        loadCellTamDuzenlemeModu = false
                        loadCellNoktalari = []
                        loadCellSayiSecildi = false
                        loadCellDuzenlemeIndex = -1
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#b91c1c" : "#991b1b"
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#dce8f5"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}

Rectangle {
    anchors.fill: parent
    color: "#000000b3"
    visible: loadCellTamamlandiOnayGoster
    z: 200

    MouseArea { anchors.fill: parent }
```

```
Rectangle {
    anchors.centerIn: parent
    width: 380
    height: 220
    radius: 14
    color: "#12121a"
    border.color: "#16a34a"
    border.width: 1

    Column {
        anchors.centerIn: parent
        spacing: 18
        width: parent.width - 40

        Column {
            width: parent.width
            spacing: 6

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                text: txt("kalibrasyonTamamlandiBaslik")
                color: "#4ade80"
                font.family: "Segoe UI"
                font.pixelSize: 17
                font.bold: true
            }

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                wrapMode: Text.WordWrap
                text: txt("toplamOnEk") + loadCellToplamNokta + txt("noktaGirildiSonEk")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 13
            }
        }

        Row {
            anchors.horizontalCenter: parent.horizontalCenter
            spacing: 12

            Button {
                width: 150
                height: 42
                text: txt("noktalariKontrolEt")

                onClicked: loadCellTamamlandiOnayGoster = false

                background: Rectangle {
                    radius: 8
                    color: parent.hovered ? "#1e2a3f" : "transparent"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#9ca3af"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: 150
                height: 42
```

```
                    text: txt("kaydetVeBitir")
                    font.bold: true

                    onClicked: {
                        loadCellTamamlandiOnayGoster = false
                        loadCellKaydetVeBildir()
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#22c55e" : "#16a34a"
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#dce8f5"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}

Rectangle {
    id: loadCellBildirimKutusu
    anchors.top: parent.top
    anchors.horizontalCenter: parent.horizontalCenter
    anchors.topMargin: 12
    z: 100
    width: loadCellBildirimMetni.implicitWidth + 24
    height: 32
    radius: 16
    color: loadCellBildirimBasarili ? "#123321" : "#3a1414"
    opacity: loadCellBildirimGoster ? 1 : 0
    visible: opacity > 0
    Behavior on opacity { NumberAnimation { duration: 200 } }

    Text {
        id: loadCellBildirimMetni
        anchors.centerIn: parent
        text: loadCellBildirimMesaj
        color: loadCellBildirimBasarili ? "#4ade80" : "#f87171"
        font.family: "Segoe UI"
        font.pixelSize: 12
        font.bold: true
    }
}

Column {
    anchors.centerIn: parent
    spacing: 24
    width: 360
    visible: !loadCellSayiSecildi && !loadCellAraEkranGoster && !loadCellGoruntuleModu

    Text {
        width: parent.width
        horizontalAlignment: Text.AlignHCenter
        wrapMode: Text.WordWrap
        text: txt("kacNoktaSoru")
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 16
        font.bold: true
    }

    Text {
```

```
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("dahaFazlaNokta")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 12
        }

        Grid {
            anchors.horizontalCenter: parent.horizontalCenter
            columns: 4
            spacing: 12

            Repeater {
                model: [4, 6, 8, 10]

                Rectangle {
                    id: secKarti
                    width: 72
                    height: 72
                    radius: 12
                    color: secKarti.hovered ? "#17263d" : "#12121a"
                    border.color: "#3b82f6"
                    border.width: 1
                    property bool hovered: false
                    Behavior on color { ColorAnimation { duration: 120 } }

                    Text {
                        anchors.centerIn: parent
                        text: modelData
                        color: "#dce8f5"
                        font.family: "Segoe UI"
                        font.pixelSize: 26
                        font.bold: true
                    }

                    MouseArea {
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onEntered: secKarti.hovered = true
                        onExited: secKarti.hovered = false
                        onClicked: {
                            loadCellToplamNokta = modelData
                            loadCellNoktalari = []
                            loadCellDuzenlemeIndex = -1
                            loadCellSayiSecildi = true
                        }
                    }
                }
            }
        }
    }

    Row {
        anchors.centerIn: parent
        spacing: 24
        visible: loadCellSayiSecildi

        Rectangle {
            width: 340
            height: 500
            radius: 14
            color: "#12121a"
            border.color: "#1e2a3f"
            border.width: 1

            Column {
                anchors.centerIn: parent
```

```
spacing: 16
width: parent.width - 40

Rectangle {
    anchors.horizontalCenter: parent.horizontalCenter
    visible: loadCellKayitliVar && !loadCellTamDuzenlemeModu
    width: sonKaliMetni2.implicitWidth + 20
    height: 24
    radius: 12
    color: "#0a0a0d"
    border.color: "#1e2a3f"
    border.width: 1

    Text {
        id: sonKaliMetni2
        anchors.centerIn: parent
        text: txt("kayitliOnEk") + loadCellNoktaSayisi + txt("noktaSonEk")
        color: "#6b7280"
        font.pixelSize: 11
    }
}

Text {
    anchors.horizontalCenter: parent.horizontalCenter
    text: loadCellTamDuzenlemeModu
          ? txt("kalibrasyonuDuzenle")
          : txt("noktaOnEk") + (loadCellNoktalari.length + 1) + " / " + loadCellToplamNokta
    color: "#3b82f6"
    font.family: "Segoe UI"
    font.pixelSize: 13
    font.bold: true
    font.letterSpacing: 1
}

Rectangle {
    width: parent.width
    height: 6
    radius: 3
    color: "#1e2a3f"

    Rectangle {
        width: parent.width * (loadCellNoktalari.length / loadCellToplamNokta)
        height: parent.height
        radius: 3
        color: "#3b82f6"
        Behavior on width { NumberAnimation { duration: 150 } }
    }
}

Text {
    width: parent.width
    horizontalAlignment: Text.AlignHCenter
    wrapMode: Text.WordWrap
    text: loadCellDuzenlemeIndex >= 0
          ? txt("noktaOnEk") + (loadCellDuzenlemeIndex + 1) + txt("duzenleniyorSonEk")
          : (loadCellTamDuzenlemeModu
             ? txt("duzenlemekIcinTikla")
             : txt("hedefAgirlikYonerge"))
    color: loadCellDuzenlemeIndex >= 0 ? "#3b82f6" : "#9ca3af"
    font.family: "Segoe UI"
    font.bold: loadCellDuzenlemeIndex >= 0
    font.pixelSize: 13
}

TextField {
    id: hedefKgKutusu
    anchors.horizontalCenter: parent.horizontalCenter
    width: 180
    height: 44
    placeholderText: "kg"
```

```
                                placeholderTextColor: "#4b5563"
                                color: "#dce8f5"
                                font.pixelSize: 16
                                horizontalAlignment: TextInput.AlignHCenter
                                validator: DoubleValidator { bottom: 0; top: 500; decimals: 2 }
                                background: Rectangle {
                                    color: "#0a0a0d"
                                    radius: 10
                                    border.color: hedefKgKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                                    border.width: 1
                                }
                            }

                            Rectangle {
                                anchors.horizontalCenter: parent.horizontalCenter
                                width: parent.width
                                height: 64
                                radius: 12
                                color: "#16161e"
                                border.color: wifiManager.baglandi ? "#3b82f6" : "#dc2626"
                                border.width: 1
                                Behavior on border.color { ColorAnimation { duration: 120 } }

                                Text {
                                    anchors.centerIn: parent
                                    text: txt("hamDegerOnEk") + sensorManager.hamAgirlik.toFixed(0)
                                    color: wifiManager.baglandi ? "#3b82f6" : "#f87171"
                                    font.family: "Segoe UI"
                                    font.pixelSize: 20
                                    font.bold: true
                                }
                            }

                            Text {
                                width: parent.width
                                horizontalAlignment: Text.AlignHCenter
                                wrapMode: Text.WordWrap
                                visible: !wifiManager.baglandi
                                text: txt("hamDegerSifirUyari")
                                color: "#f87171"
                                font.family: "Segoe UI"
                                font.pixelSize: 11
                            }

                            Button {
                                id: noktaKaydetButonu
                                width: parent.width
                                height: 46
                                property bool tumNoktalarGirildi: !loadCellTamDuzenlemeModu && loadCellDuzenlemeIndex < 0 &&
loadCellNoktalari.length >= loadCellToplamNokta
                                text: loadCellDuzenlemeIndex >= 0
                                      ? txt("noktayiGuncelle")
                                      : (tumNoktalarGirildi
                                         ? txt("kalibrasyonuTamamla")
                                         : (loadCellNoktalari.length >= loadCellToplamNokta - 1 ? txt("sonNoktayiKaydet") :
txt("buNoktayiKaydet")))
                                font.pixelSize: 14
                                font.bold: true
                                enabled: tumNoktalarGirildi || (hedefKgKutusu.text.length > 0 && (loadCellDuzenlemeIndex >= 0 ||
loadCellNoktalari.length < loadCellToplamNokta))

                                onClicked: {
                                    if (tumNoktalarGirildi) {
                                        loadCellTamamlandiOnayGoster = true
                                        return
                                    }

                                    var yeniNokta = { hedefKg: parseFloat(hedefKgKutusu.text), hamDeger: sensorManager.hamAgirlik
}
```

```
            if (loadCellDuzenlemeIndex >= 0) {
                var guncelDizi = loadCellNoktalari.slice()
                guncelDizi[loadCellDuzenlemeIndex] = yeniNokta
                loadCellNoktalari = guncelDizi
                loadCellDuzenlemeIndex = -1
                hedefKgKutusu.text = ""
                return
            }

            var yeniDizi = loadCellNoktalari.slice()
            yeniDizi.push(yeniNokta)
            loadCellNoktalari = yeniDizi
            hedefKgKutusu.text = ""

            if (loadCellNoktalari.length >= loadCellToplamNokta) {
                loadCellTamamlandiOnayGoster = true
            }
        }

        background: Rectangle {
            radius: 10
            color: parent.enabled ? (parent.hovered ? "#4f8cf7" : "#3b82f6") : "#1e2a3f"
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#dce8f5"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }

    Button {
        width: parent.width
        height: 46
        visible: loadCellTamDuzenlemeModu
        text: txt("degisiklikleriKaydet")
        font.pixelSize: 14
        font.bold: true

        onClicked: loadCellKaydetVeBildir()

        background: Rectangle {
            radius: 10
            color: parent.hovered ? "#22c55e" : "#16a34a"
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#dce8f5"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }

    Button {
        width: parent.width
        height: 38
        visible: loadCellDuzenlemeIndex >= 0
        text: txt("vazgecButon")
        font.pixelSize: 13

        onClicked: {
            loadCellDuzenlemeIndex = -1
            hedefKgKutusu.text = ""
        }
```

```
            background: Rectangle {
                radius: 8
                color: parent.hovered ? "#1e2a3f" : "transparent"
                border.color: "#1e2a3f"
                border.width: 1
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#9ca3af"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }
}

Rectangle {
    width: 300
    height: 500
    radius: 14
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    Column {
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.margins: 20
        spacing: 4

        Text {
            text: txt("kaydedilenNoktalarBaslik")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
            font.bold: true
            font.letterSpacing: 1
        }

        Text {
            text: txt("duzeltmekIcinTikla")
            color: "#4b5563"
            font.family: "Segoe UI"
            font.pixelSize: 10
        }
    }

    ListView {
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        anchors.margins: 16
        anchors.topMargin: 60
        clip: true
        spacing: 8

        model: loadCellNoktalari

        delegate: Rectangle {
            width: ListView.view.width
            height: 48
            radius: 8
            color: satirAlani.containsMouse ? "#17263d" : "#0a0a0d"
            border.color: index === loadCellDuzenlemeIndex ? "#3b82f6" : "#1e2a3f"
```

```
            border.width: index === loadCellDuzenlemeIndex ? 2 : 1
            Behavior on color { ColorAnimation { duration: 120 } }

            Row {
                anchors.fill: parent
                anchors.leftMargin: 12
                anchors.rightMargin: 12
                spacing: 8

                Rectangle {
                    width: 22
                    height: 22
                    radius: 11
                    anchors.verticalCenter: parent.verticalCenter
                    color: "#17263d"
                    border.color: "#3b82f6"
                    border.width: 1

                    Text {
                        anchors.centerIn: parent
                        text: index + 1
                        color: "#3b82f6"
                        font.pixelSize: 10
                        font.bold: true
                    }
                }

                Column {
                    anchors.verticalCenter: parent.verticalCenter
                    spacing: 1

                    Text {
                        text: modelData.hedefKg.toFixed(2) + " kg"
                        color: "#dce8f5"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                        font.bold: true
                    }

                    Text {
                        text: "ADC: " + modelData.hamDeger.toFixed(0)
                        color: "#6b7280"
                        font.family: "Segoe UI"
                        font.pixelSize: 10
                    }
                }
            }

            Text {
                anchors.right: parent.right
                anchors.verticalCenter: parent.verticalCenter
                anchors.rightMargin: 12
                text: "✎"
                color: "#6b7280"
                font.pixelSize: 13
                visible: satirAlani.containsMouse
            }

            MouseArea {
                id: satirAlani
                anchors.fill: parent
                hoverEnabled: true
                cursorShape: Qt.PointingHandCursor
                onClicked: {
                    hedefKgKutusu.text = modelData.hedefKg.toString()
                    loadCellDuzenlemeIndex = index
                }
            }
        }
    }
}
```

```
        }
    }
}

Item {
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.bottom: parent.bottom
    anchors.topMargin: 70
    visible: seciliSensor === 2

    Item {
        anchors.fill: parent

        Column {
            anchors.centerIn: parent
            spacing: 24
            width: 420
            visible: mesafeAraEkranGoster

            Rectangle {
                anchors.horizontalCenter: parent.horizontalCenter
                width: mesafeAraEkranBadge.implicitWidth + 20
                height: 26
                radius: 13
                color: "#12121a"
                border.color: "#1e2a3f"
                border.width: 1

                Text {
                    id: mesafeAraEkranBadge
                    anchors.centerIn: parent
                    text: txt("kayitliOnEk") + mesafeOlcumNoktaSayisi + txt("noktaSonEk")
                    color: "#6b7280"
                    font.pixelSize: 11
                }
            }

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                wrapMode: Text.WordWrap
                text: txt("kayitliKalibrasyonBulundu")
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 16
                font.bold: true
            }

            Row {
                anchors.horizontalCenter: parent.horizontalCenter
                spacing: 20

                Rectangle {
                    id: mesafeGoruntuleKarti
                    width: 190
                    height: 180
                    radius: 14
                    color: "#12121a"
                    border.color: mesafeGoruntuleKarti.hovered ? "#9333ea" : "#1e2a3f"
                    border.width: 1
                    property bool hovered: false
                    Behavior on border.color { ColorAnimation { duration: 120 } }

                    Column {
                        anchors.centerIn: parent
                        spacing: 12
                        width: parent.width - 28
```

```
        Canvas {
            anchors.horizontalCenter: parent.horizontalCenter
            width: 32
            height: 32
            onPaint: {
                var ctx = getContext("2d")
                ctx.clearRect(0, 0, width, height)
                ctx.lineWidth = 1.8
                ctx.lineCap = "round"
                ctx.lineJoin = "round"
                ctx.strokeStyle = "#9333ea"

                ctx.beginPath()
                ctx.moveTo(3, 16)
                ctx.bezierCurveTo(7.5, 8.5, 12.5, 6.8, 16, 6.8)
                ctx.bezierCurveTo(19.5, 6.8, 24.5, 8.5, 29, 16)
                ctx.bezierCurveTo(24.5, 23.5, 19.5, 25.2, 16, 25.2)
                ctx.bezierCurveTo(12.5, 25.2, 7.5, 23.5, 3, 16)
                ctx.stroke()

                ctx.beginPath()
                ctx.arc(16, 16, 4.4, 0, Math.PI * 2, false)
                ctx.stroke()
            }
        }

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("mevcutGoruntule")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
        }

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("kayitliNoktalariIncele")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
        }
    }

    MouseArea {
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onEntered: mesafeGoruntuleKarti.hovered = true
        onExited: mesafeGoruntuleKarti.hovered = false
        onClicked: {
            mesafeNoktalari = database.mesafeNoktalariGetir()
            mesafeAraEkranGoster = false
            mesafeGoruntuleModu = true
            mesafeDuzenlemeIndex = -1
        }
    }
}

Rectangle {
    id: mesafeYenidenKarti
    width: 190
    height: 180
    radius: 14
    color: "#12121a"
    border.color: mesafeYenidenKarti.hovered ? "#9ca3af" : "#1e2a3f"
```

```
border.width: 1
property bool hovered: false
Behavior on border.color { ColorAnimation { duration: 120 } }

Column {
    anchors.centerIn: parent
    spacing: 12
    width: parent.width - 28

    Canvas {
        anchors.horizontalCenter: parent.horizontalCenter
        width: 32
        height: 32
        onPaint: {
            var ctx = getContext("2d")
            ctx.clearRect(0, 0, width, height)
            ctx.strokeStyle = "#9ca3af"
            ctx.fillStyle = "#9ca3af"
            ctx.lineWidth = 3.4
            ctx.lineCap = "butt"

            function yayOku(startDeg, endDeg) {
                var s = startDeg * Math.PI / 180
                var e = endDeg * Math.PI / 180
                var cx = 16, cy = 16, r = 10.5

                ctx.beginPath()
                ctx.arc(cx, cy, r, s, e, false)
                ctx.stroke()

                var ex = cx + r * Math.cos(e)
                var ey = cy + r * Math.sin(e)
                var tx = -Math.sin(e)
                var ty = Math.cos(e)
                var px = -ty
                var py = tx
                var geri = 6.2
                var yaricap = 3.6
                var bx = ex - tx * geri
                var by = ey - ty * geri

                ctx.beginPath()
                ctx.moveTo(ex, ey)
                ctx.lineTo(bx + px * yaricap, by + py * yaricap)
                ctx.lineTo(bx - px * yaricap, by - py * yaricap)
                ctx.closePath()
                ctx.fill()
            }

            yayOku(195, 345)
            yayOku(15, 165)
        }
    }

    Text {
        width: parent.width
        horizontalAlignment: Text.AlignHCenter
        wrapMode: Text.WordWrap
        text: txt("yenidenKalibreEt")
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 13
        font.bold: true
    }

    Text {
        width: parent.width
        horizontalAlignment: Text.AlignHCenter
        wrapMode: Text.WordWrap
        text: txt("sifirdanYeniOlcum")
```

```
                    color: "#6b7280"
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                }
            }

            MouseArea {
                anchors.fill: parent
                hoverEnabled: true
                cursorShape: Qt.PointingHandCursor
                onEntered: mesafeYenidenKarti.hovered = true
                onExited: mesafeYenidenKarti.hovered = false
                onClicked: mesafeYenidenOnayGoster = true
            }
        }
    }
}

Column {
    anchors.centerIn: parent
    spacing: 16
    width: 340
    visible: mesafeGoruntuleModu

    Text {
        text: txt("kayitliNoktalarBaslik")
        color: "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 11
        font.bold: true
        font.letterSpacing: 1
    }

    Rectangle {
        width: parent.width
        height: 320
        radius: 14
        color: "#12121a"
        border.color: "#1e2a3f"
        border.width: 1

        ListView {
            anchors.fill: parent
            anchors.margins: 16
            clip: true
            spacing: 8

            model: mesafeNoktalari

            delegate: Rectangle {
                width: ListView.view.width
                height: 48
                radius: 8
                color: "#0a0a0d"
                border.color: "#1e2a3f"
                border.width: 1

                Row {
                    anchors.fill: parent
                    anchors.leftMargin: 12
                    anchors.rightMargin: 12
                    spacing: 8

                    Rectangle {
                        width: 22
                        height: 22
                        radius: 11
                        anchors.verticalCenter: parent.verticalCenter
                        color: "#241a38"
                        border.color: "#9333ea"
```

```
                        border.width: 1

                        Text {
                            anchors.centerIn: parent
                            text: index + 1
                            color: "#c084fc"
                            font.pixelSize: 10
                            font.bold: true
                        }
                    }

                    Column {
                        anchors.verticalCenter: parent.verticalCenter
                        spacing: 1

                        Text {
                            text: (modelData.hedefMm / 10).toFixed(1) + " cm"
                            color: "#dce8f5"
                            font.family: "Segoe UI"
                            font.pixelSize: 13
                            font.bold: true
                        }

                        Text {
                            text: "ADC: " + modelData.hamDeger.toFixed(0)
                            color: "#6b7280"
                            font.family: "Segoe UI"
                            font.pixelSize: 10
                        }
                    }
                }
            }
        }
    }

    Button {
        width: parent.width
        height: 46
        text: txt("duzenleButon")
        font.pixelSize: 14
        font.bold: true

        onClicked: {
            mesafeToplamNokta = mesafeNoktalari.length
            mesafeGoruntuleModu = false
            mesafeTamDuzenlemeModu = true
            mesafeSayiSecildi = true
            mesafeDuzenlemeIndex = -1
        }

        background: Rectangle {
            radius: 8
            color: parent.hovered ? "#a855f7" : "#9333ea"
            Behavior on color { ColorAnimation { duration: 120 } }
        }

        contentItem: Text {
            text: parent.text
            color: "#dce8f5"
            font: parent.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }
}

Rectangle {
    anchors.fill: parent
    color: "#000000b3"
    visible: mesafeYenidenOnayGoster
```

```
z: 200

MouseArea { anchors.fill: parent }

Rectangle {
    anchors.centerIn: parent
    width: 360
    height: 200
    radius: 14
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    Column {
        anchors.centerIn: parent
        spacing: 18
        width: parent.width - 40

        Text {
            width: parent.width
            horizontalAlignment: Text.AlignHCenter
            wrapMode: Text.WordWrap
            text: txt("silinecekOnay")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 14
            font.bold: true
        }

        Row {
            anchors.horizontalCenter: parent.horizontalCenter
            spacing: 12

            Button {
                width: 150
                height: 42
                text: txt("vazgecButon")

                onClicked: mesafeYenidenOnayGoster = false

                background: Rectangle {
                    radius: 8
                    color: parent.hovered ? "#1e2a3f" : "transparent"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#9ca3af"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: 150
                height: 42
                text: txt("evetYenidenBaslaButon")
                font.bold: true

                onClicked: {
                    mesafeYenidenOnayGoster = false
                    mesafeAraEkranGoster = false
                    mesafeGoruntuleModu = false
                    mesafeTamDuzenlemeModu = false
                    mesafeNoktalari = []
                    mesafeSayiSecildi = false
```

```
                        mesafeDuzenlemeIndex = -1
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#b91c1c" : "#991b1b"
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#dce8f5"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}

Rectangle {
    anchors.fill: parent
    color: "#000000b3"
    visible: mesafeTamamlandiOnayGoster
    z: 200

    MouseArea { anchors.fill: parent }

    Rectangle {
        anchors.centerIn: parent
        width: 380
        height: 220
        radius: 14
        color: "#12121a"
        border.color: "#16a34a"
        border.width: 1

        Column {
            anchors.centerIn: parent
            spacing: 18
            width: parent.width - 40

            Column {
                width: parent.width
                spacing: 6

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    text: txt("kalibrasyonTamamlandiBaslik")
                    color: "#4ade80"
                    font.family: "Segoe UI"
                    font.pixelSize: 17
                    font.bold: true
                }

                Text {
                    width: parent.width
                    horizontalAlignment: Text.AlignHCenter
                    wrapMode: Text.WordWrap
                    text: txt("toplamOnEk") + mesafeToplamNokta + txt("noktaGirildiSonEk")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                }
            }

            Row {
```

```
                anchors.horizontalCenter: parent.horizontalCenter
                spacing: 12

                Button {
                    width: 150
                    height: 42
                    text: txt("noktalariKontrolEt")

                    onClicked: mesafeTamamlandiOnayGoster = false

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#1e2a3f" : "transparent"
                        border.color: "#1e2a3f"
                        border.width: 1
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#9ca3af"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    width: 150
                    height: 42
                    text: txt("kaydetVeBitir")
                    font.bold: true

                    onClicked: {
                        mesafeTamamlandiOnayGoster = false
                        mesafeKaydetVeBildir()
                    }

                    background: Rectangle {
                        radius: 8
                        color: parent.hovered ? "#22c55e" : "#16a34a"
                        Behavior on color { ColorAnimation { duration: 120 } }
                    }

                    contentItem: Text {
                        text: parent.text
                        color: "#dce8f5"
                        font: parent.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}

Column {
    anchors.centerIn: parent
    spacing: 24
    width: 360
    visible: !mesafeSayiSecildi && !mesafeAraEkranGoster && !mesafeGoruntuleModu

    Text {
        width: parent.width
        horizontalAlignment: Text.AlignHCenter
        wrapMode: Text.WordWrap
        text: txt("kacNoktaSoru")
        color: "#dce8f5"
        font.family: "Segoe UI"
```

```
        font.pixelSize: 16
        font.bold: true
    }

    Text {
        width: parent.width
        horizontalAlignment: Text.AlignHCenter
        wrapMode: Text.WordWrap
        text: txt("dahaFazlaNokta")
        color: "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 12
    }

    Grid {
        anchors.horizontalCenter: parent.horizontalCenter
        columns: 4
        spacing: 12

        Repeater {
            model: [4, 6, 8, 10]

            Rectangle {
                id: mesafeSecKarti
                width: 72
                height: 72
                radius: 12
                color: mesafeSecKarti.hovered ? "#241a38" : "#12121a"
                border.color: "#9333ea"
                border.width: 1
                property bool hovered: false
                Behavior on color { ColorAnimation { duration: 120 } }

                Text {
                    anchors.centerIn: parent
                    text: modelData
                    color: "#dce8f5"
                    font.family: "Segoe UI"
                    font.pixelSize: 26
                    font.bold: true
                }

                MouseArea {
                    anchors.fill: parent
                    hoverEnabled: true
                    cursorShape: Qt.PointingHandCursor
                    onEntered: mesafeSecKarti.hovered = true
                    onExited: mesafeSecKarti.hovered = false
                    onClicked: {
                        mesafeToplamNokta = modelData
                        mesafeNoktalari = []
                        mesafeDuzenlemeIndex = -1
                        mesafeSayiSecildi = true
                    }
                }
            }
        }
    }
}

Row {
    anchors.centerIn: parent
    spacing: 24
    visible: mesafeSayiSecildi

    Rectangle {
        width: 340
        height: 500
        radius: 14
        color: "#12121a"
```

```
border.color: "#1e2a3f"
border.width: 1

Column {
    anchors.centerIn: parent
    spacing: 16
    width: parent.width - 40

    Rectangle {
        anchors.horizontalCenter: parent.horizontalCenter
        visible: mesafeOlcumKayitliVar && !mesafeTamDuzenlemeModu
        width: mesafeSonKaliMetni.implicitWidth + 20
        height: 24
        radius: 12
        color: "#0a0a0d"
        border.color: "#1e2a3f"
        border.width: 1

        Text {
            id: mesafeSonKaliMetni
            anchors.centerIn: parent
            text: txt("kayitliOnEk") + mesafeOlcumNoktaSayisi + txt("noktaSonEk")
            color: "#6b7280"
            font.pixelSize: 11
        }
    }

    Text {
        anchors.horizontalCenter: parent.horizontalCenter
        text: mesafeTamDuzenlemeModu
              ? txt("kalibrasyonuDuzenle")
              : txt("noktaOnEk") + (mesafeNoktalari.length + 1) + " / " + mesafeToplamNokta
        color: "#9333ea"
        font.family: "Segoe UI"
        font.pixelSize: 13
        font.bold: true
        font.letterSpacing: 1
    }

    Rectangle {
        width: parent.width
        height: 6
        radius: 3
        color: "#1e2a3f"

        Rectangle {
            width: parent.width * (mesafeNoktalari.length / mesafeToplamNokta)
            height: parent.height
            radius: 3
            color: "#9333ea"
            Behavior on width { NumberAnimation { duration: 150 } }
        }
    }

    Text {
        width: parent.width
        horizontalAlignment: Text.AlignHCenter
        wrapMode: Text.WordWrap
        text: mesafeDuzenlemeIndex >= 0
              ? txt("noktaOnEk") + (mesafeDuzenlemeIndex + 1) + txt("duzenleniyorSonEk")
              : (mesafeTamDuzenlemeModu
                 ? txt("duzenlemekIcinTikla")
                 : txt("hedefMesafeYonerge"))
        color: mesafeDuzenlemeIndex >= 0 ? "#9333ea" : "#9ca3af"
        font.family: "Segoe UI"
        font.bold: mesafeDuzenlemeIndex >= 0
        font.pixelSize: 13
    }

    TextField {
```

```
                                id: hedefCmKutusu
                                anchors.horizontalCenter: parent.horizontalCenter
                                width: 180
                                height: 44
                                placeholderText: "cm"
                                placeholderTextColor: "#4b5563"
                                color: "#dce8f5"
                                font.pixelSize: 16
                                horizontalAlignment: TextInput.AlignHCenter
                                validator: DoubleValidator { bottom: 0; top: 300; decimals: 1 }
                                background: Rectangle {
                                    color: "#0a0a0d"
                                    radius: 10
                                    border.color: hedefCmKutusu.activeFocus ? "#9333ea" : "#1e2a3f"
                                    border.width: 1
                                }
                            }

                            Rectangle {
                                anchors.horizontalCenter: parent.horizontalCenter
                                width: parent.width
                                height: 64
                                radius: 12
                                color: "#16161e"
                                border.color: wifiManager.baglandi ? "#9333ea" : "#dc2626"
                                border.width: 1
                                Behavior on border.color { ColorAnimation { duration: 120 } }

                                Text {
                                    anchors.centerIn: parent
                                    text: txt("hamDegerOnEk") + sensorManager.hamMesafe.toFixed(0)
                                    color: wifiManager.baglandi ? "#c084fc" : "#f87171"
                                    font.family: "Segoe UI"
                                    font.pixelSize: 20
                                    font.bold: true
                                }
                            }

                            Text {
                                width: parent.width
                                horizontalAlignment: Text.AlignHCenter
                                wrapMode: Text.WordWrap
                                visible: !wifiManager.baglandi
                                text: txt("hamDegerSifirUyari")
                                color: "#f87171"
                                font.family: "Segoe UI"
                                font.pixelSize: 11
                            }

                            Button {
                                id: mesafeNoktaKaydetButonu
                                width: parent.width
                                height: 46
                                property bool tumNoktalarGirildi: !mesafeTamDuzenlemeModu && mesafeDuzenlemeIndex < 0 &&
mesafeNoktalari.length >= mesafeToplamNokta
                                text: mesafeDuzenlemeIndex >= 0
                                      ? txt("noktayiGuncelle")
                                      : (tumNoktalarGirildi
                                         ? txt("kalibrasyonuTamamla")
                                         : (mesafeNoktalari.length >= mesafeToplamNokta - 1 ? txt("sonNoktayiKaydet") :
txt("buNoktayiKaydet")))
                                font.pixelSize: 14
                                font.bold: true
                                enabled: tumNoktalarGirildi || (hedefCmKutusu.text.length > 0 && (mesafeDuzenlemeIndex >= 0
|| mesafeNoktalari.length < mesafeToplamNokta))

                                onClicked: {
                                    if (tumNoktalarGirildi) {
                                        mesafeTamamlandiOnayGoster = true
                                        return
```

```
                    }

                    var yeniNokta = { hedefMm: parseFloat(hedefCmKutusu.text) * 10, hamDeger:
sensorManager.hamMesafe }

                    if (mesafeDuzenlemeIndex >= 0) {
                        var guncelDizi = mesafeNoktalari.slice()
                        guncelDizi[mesafeDuzenlemeIndex] = yeniNokta
                        mesafeNoktalari = guncelDizi
                        mesafeDuzenlemeIndex = -1
                        hedefCmKutusu.text = ""
                        return
                    }

                    var yeniDizi = mesafeNoktalari.slice()
                    yeniDizi.push(yeniNokta)
                    mesafeNoktalari = yeniDizi
                    hedefCmKutusu.text = ""

                    if (mesafeNoktalari.length >= mesafeToplamNokta) {
                        mesafeTamamlandiOnayGoster = true
                    }
                }

                background: Rectangle {
                    radius: 10
                    color: parent.enabled ? (parent.hovered ? "#a855f7" : "#9333ea") : "#1e2a3f"
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#dce8f5"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: parent.width
                height: 46
                visible: mesafeTamDuzenlemeModu
                text: txt("degisiklikleriKaydet")
                font.pixelSize: 14
                font.bold: true

                onClicked: mesafeKaydetVeBildir()

                background: Rectangle {
                    radius: 10
                    color: parent.hovered ? "#22c55e" : "#16a34a"
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#dce8f5"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Button {
                width: parent.width
                height: 38
                visible: mesafeDuzenlemeIndex >= 0
                text: txt("vazgecButon")
                font.pixelSize: 13
```

```
            onClicked: {
                mesafeDuzenlemeIndex = -1
                hedefCmKutusu.text = ""
            }

            background: Rectangle {
                radius: 8
                color: parent.hovered ? "#1e2a3f" : "transparent"
                border.color: "#1e2a3f"
                border.width: 1
                Behavior on color { ColorAnimation { duration: 120 } }
            }

            contentItem: Text {
                text: parent.text
                color: "#9ca3af"
                font: parent.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }
}

Rectangle {
    width: 300
    height: 500
    radius: 14
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    Column {
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.margins: 20
        spacing: 4

        Text {
            text: txt("kaydedilenNoktalarBaslik")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 11
            font.bold: true
            font.letterSpacing: 1
        }

        Text {
            text: txt("duzeltmekIcinTikla")
            color: "#4b5563"
            font.family: "Segoe UI"
            font.pixelSize: 10
        }
    }

    ListView {
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        anchors.margins: 16
        anchors.topMargin: 60
        clip: true
        spacing: 8

        model: mesafeNoktalari

        delegate: Rectangle {
```

```
width: ListView.view.width
height: 48
radius: 8
color: mesafeSatirAlani.containsMouse ? "#241a38" : "#0a0a0d"
border.color: index === mesafeDuzenlemeIndex ? "#9333ea" : "#1e2a3f"
border.width: index === mesafeDuzenlemeIndex ? 2 : 1
Behavior on color { ColorAnimation { duration: 120 } }

Row {
    anchors.fill: parent
    anchors.leftMargin: 12
    anchors.rightMargin: 12
    spacing: 8

    Rectangle {
        width: 22
        height: 22
        radius: 11
        anchors.verticalCenter: parent.verticalCenter
        color: "#241a38"
        border.color: "#9333ea"
        border.width: 1

        Text {
            anchors.centerIn: parent
            text: index + 1
            color: "#c084fc"
            font.pixelSize: 10
            font.bold: true
        }
    }

    Column {
        anchors.verticalCenter: parent.verticalCenter
        spacing: 1

        Text {
            text: (modelData.hedefMm / 10).toFixed(1) + " cm"
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
        }

        Text {
            text: "ADC: " + modelData.hamDeger.toFixed(0)
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 10
        }
    }
}

Text {
    anchors.right: parent.right
    anchors.verticalCenter: parent.verticalCenter
    anchors.rightMargin: 12
    text: "✎"
    color: "#6b7280"
    font.pixelSize: 13
    visible: mesafeSatirAlani.containsMouse
}

MouseArea {
    id: mesafeSatirAlani
    anchors.fill: parent
    hoverEnabled: true
    cursorShape: Qt.PointingHandCursor
    onClicked: {
        hedefCmKutusu.text = (modelData.hedefMm / 10).toString()
```

```
                                    mesafeDuzenlemeIndex = index
                                }
                            }
                        }
                    }
                }
            }

            Rectangle {
                id: mesafeBildirimKutusu
                anchors.top: parent.top
                anchors.horizontalCenter: parent.horizontalCenter
                anchors.topMargin: 12
                z: 100
                width: mesafeBildirimMetni.implicitWidth + 24
                height: 32
                radius: 16
                color: mesafeBildirimBasarili ? "#123321" : "#3a1414"
                opacity: mesafeBildirimGoster ? 1 : 0
                visible: opacity > 0
                Behavior on opacity { NumberAnimation { duration: 200 } }

                Text {
                    id: mesafeBildirimMetni
                    anchors.centerIn: parent
                    text: mesafeBildirimMesaj
                    color: mesafeBildirimBasarili ? "#4ade80" : "#f87171"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                    font.bold: true
                }
            }
        }
    }
}
```

qml\pages\LoginPage.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import sliper

Rectangle {
    id: sayfa

    property bool sifreGorunur: false
    property bool beniHatirla: false

    signal girisBasarili()

    readonly property var metinler: ({
        girisBasligi: { tr: "Sisteme Giriş", en: "System Login" },
        girisAltBaslik: { tr: "Yetkili hesabınızla devam edin", en: "Continue with your authorized account" },
        kullaniciAdiEtiket: { tr: "KULLANICI ADI", en: "USERNAME" },
        kullaniciAdiPlaceholder: { tr: "kullanici_adi", en: "username" },
        sifreEtiket: { tr: "ŞİFRE", en: "PASSWORD" },
        girisYapButon: { tr: "Giriş Yap", en: "Log In" },
        telifHakki: { tr: "© 2026 Liya Test A.Ş. SLIPER Analiz Yazılımı", en: "© 2026 Liya Test A.Ş. SLIPER Analysis Software" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    gradient: Gradient {
        GradientStop { position: 0.0; color: "#15121a" }
        GradientStop { position: 0.55; color: "#0c0b0e" }
        GradientStop { position: 1.0; color: "#07070a" }
    }

    Rectangle {
        anchors.centerIn: kart
        width: 640
        height: 640
        radius: width / 2
        opacity: 0.10
        gradient: Gradient {
            GradientStop { position: 0.0; color: "#3b82f6" }
            GradientStop { position: 1.0; color: "#00000000" }
        }
    }

    Row {
        anchors.top: parent.top
        anchors.right: parent.right
        anchors.margins: 24
        spacing: 8

        Rectangle {
            width: 44
            height: 30
            radius: 6
            color: "#12121a"
            border.color: Translations.turkish ? "#3b82f6" : "#1e2a3f"
            border.width: 1

            Text {
                anchors.centerIn: parent
                text: "TR"
                color: Translations.turkish ? "#dce8f5" : "#6b7280"
                font.pixelSize: 12
                font.bold: Translations.turkish
            }

            MouseArea {
                anchors.fill: parent
```

```
            cursorShape: Qt.PointingHandCursor
            onClicked: Translations.turkish = true
        }
    }

    Rectangle {
        width: 44
        height: 30
        radius: 6
        color: "#12121a"
        border.color: !Translations.turkish ? "#3b82f6" : "#1e2a3f"
        border.width: 1

        Text {
            anchors.centerIn: parent
            text: "EN"
            color: !Translations.turkish ? "#dce8f5" : "#6b7280"
            font.pixelSize: 12
            font.bold: !Translations.turkish
        }

        MouseArea {
            anchors.fill: parent
            cursorShape: Qt.PointingHandCursor
            onClicked: Translations.turkish = false
        }
    }
}

Rectangle {
    anchors.centerIn: kart
    width: kart.width + 48
    height: kart.height + 48
    radius: 28
    color: "#18000000"
}

Rectangle {
    anchors.centerIn: kart
    width: kart.width + 32
    height: kart.height + 32
    radius: 24
    color: "#25000000"
}

Rectangle {
    anchors.centerIn: kart
    width: kart.width + 18
    height: kart.height + 18
    radius: 20
    color: "#35000000"
}

Rectangle {
    anchors.centerIn: kart
    width: kart.width + 8
    height: kart.height + 8
    radius: 18
    color: "#45000000"
}

Rectangle {
    id: kart
    anchors.centerIn: parent
    width: 400
    color: "#12121a"
    radius: 16
    border.color: "#1e2a3f"
    border.width: 1
    height: govde.implicitHeight + 48
```

```
Rectangle {
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.topMargin: -1
    height: 3
    radius: 2
    gradient: Gradient {
        orientation: Gradient.Horizontal
        GradientStop { position: 0.0; color: "#00000000" }
        GradientStop { position: 0.5; color: "#3b82f6" }
        GradientStop { position: 1.0; color: "#00000000" }
    }
}

Column {
    id: govde
    anchors.top: parent.top
    anchors.horizontalCenter: parent.horizontalCenter
    anchors.topMargin: 32
    width: parent.width - 48
    spacing: 20

    Image {
        anchors.horizontalCenter: parent.horizontalCenter
        source: "../assets/logo.png"
        width: 144
        height: 144
        fillMode: Image.PreserveAspectFit
    }

    Row {
        anchors.horizontalCenter: parent.horizontalCenter
        spacing: 10

        Rectangle { width: 50; height: 1; color: "#1e2a3f"; anchors.verticalCenter: parent.verticalCenter }
        Rectangle { width: 5; height: 5; rotation: 45; color: "#3b82f6"; anchors.verticalCenter: parent.verticalCenter }
        Rectangle { width: 50; height: 1; color: "#1e2a3f"; anchors.verticalCenter: parent.verticalCenter }
    }

    Column {
        width: parent.width
        spacing: 4

        Text {
            anchors.horizontalCenter: parent.horizontalCenter
            text: txt("girisBasligi")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 22
            font.letterSpacing: 0.5
            font.bold: true
        }

        Text {
            anchors.horizontalCenter: parent.horizontalCenter
            text: txt("girisAltBaslik")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 12
        }
    }

    Column {
        width: parent.width
        spacing: 6

        Text {
            text: txt("kullaniciAdiEtiket")
```

```
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.letterSpacing: 1
            }

            TextField {
                id: kullaniciAdiKutusu
                width: parent.width
                height: 42
                placeholderText: txt("kullaniciAdiPlaceholder")
                placeholderTextColor: "#4b5563"
                color: "#dce8f5"
                font.pixelSize: 13
                leftPadding: 12
                verticalAlignment: TextInput.AlignVCenter
                background: Rectangle {
                    color: "#0a0a0d"
                    radius: 6
                    border.color: kullaniciAdiKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
                }
            }
        }

        Column {
            width: parent.width
            spacing: 6

            Text {
                text: txt("sifreEtiket")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.letterSpacing: 1
            }

            TextField {
                id: sifreKutusu
                width: parent.width
                height: 42
                placeholderText: "********"
                placeholderTextColor: "#4b5563"
                echoMode: sayfa.sifreGorunur ? TextInput.Normal : TextInput.Password
                color: "#dce8f5"
                font.pixelSize: 13
                leftPadding: 12
                rightPadding: 48
                verticalAlignment: TextInput.AlignVCenter
                background: Rectangle {
                    color: "#0a0a0d"
                    radius: 6
                    border.color: sifreKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
                }

                Rectangle {
                    id: sifreGosterButonu
                    anchors.right: parent.right
                    anchors.verticalCenter: parent.verticalCenter
                    anchors.rightMargin: 8
                    width: 28
                    height: 28
                    radius: 6
                    color: sifreGosterMouse.pressed ? "#191921" : (sifreGosterMouse.containsMouse ? "#14141a" :
"transparent")
                    border.color: sayfa.sifreGorunur ? "#334155" : "transparent"
                    border.width: 1

                    property color ikonRengi: sifreGosterMouse.containsMouse || sayfa.sifreGorunur ? "#dce8f5" : "#6b7280"
```

```
    Canvas {
        id: sifreIkonu
        anchors.centerIn: parent
        width: 18
        height: 18

        onPaint: {
            var ctx = getContext("2d")
            ctx.clearRect(0, 0, width, height)
            ctx.lineWidth = 1.7
            ctx.lineCap = "round"
            ctx.lineJoin = "round"
            ctx.strokeStyle = sifreGosterButonu.ikonRengi

            ctx.beginPath()
            ctx.moveTo(1.5, 9)
            ctx.bezierCurveTo(4.2, 4.8, 7.0, 3.8, 9, 3.8)
            ctx.bezierCurveTo(11.0, 3.8, 13.8, 4.8, 16.5, 9)
            ctx.bezierCurveTo(13.8, 13.2, 11.0, 14.2, 9, 14.2)
            ctx.bezierCurveTo(7.0, 14.2, 4.2, 13.2, 1.5, 9)
            ctx.stroke()

            ctx.beginPath()
            ctx.arc(9, 9, 2.5, 0, Math.PI * 2, false)
            ctx.stroke()

            if (sayfa.sifreGorunur) {
                ctx.beginPath()
                ctx.moveTo(3, 15)
                ctx.lineTo(15, 3)
                ctx.stroke()
            }
        }

        Connections {
            target: sayfa
            function onSifreGorunurChanged() {
                sifreIkonu.requestPaint()
            }
        }

        Connections {
            target: sifreGosterButonu
            function onIkonRengiChanged() {
                sifreIkonu.requestPaint()
            }
        }
    }

    MouseArea {
        id: sifreGosterMouse
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
        onClicked: sayfa.sifreGorunur = !sayfa.sifreGorunur
    }
}

Text {
    anchors.right: parent.right
    anchors.verticalCenter: parent.verticalCenter
    anchors.rightMargin: 12
    text: sayfa.sifreGorunur ? "👁" : "👁"
    color: "#6b7280"
    font.pixelSize: 14
    visible: false

    MouseArea {
        anchors.fill: parent
```

```
                            anchors.margins: -8
                            cursorShape: Qt.PointingHandCursor
                            onClicked: sayfa.sifreGorunur = !sayfa.sifreGorunur
                        }
                    }
                }
            }

            Button {
                width: parent.width
                height: 44
                text: txt("girisYapButon")
                font.family: "Segoe UI"
                font.pixelSize: 14
                font.letterSpacing: 0.5
                font.bold: true

                onClicked: {
                    var sonuc = backend.girisYap(kullaniciAdiKutusu.text, sifreKutusu.text)

                    if (sonuc) {
                        sayfa.girisBasarili()
                    } else {
                        console.log("Kullanici adi veya sifre hatali")
                    }
                }

                background: Rectangle {
                    radius: 8
                    gradient: Gradient {
                        orientation: Gradient.Horizontal
                        GradientStop { position: 0.0; color: parent.pressed ? "#1d4ed8" : (parent.hovered ? "#dce8f5" :
"#3b82f6") }
                        GradientStop { position: 1.0; color: parent.pressed ? "#1e3a8a" : (parent.hovered ? "#4f8cf7" :
"#1d4ed8") }
                    }
                }

                contentItem: Text {
                    text: parent.text
                    color: "#1e2a3f"
                    font: parent.font
                    horizontalAlignment: Text.AlignHCenter
                    verticalAlignment: Text.AlignVCenter
                }
            }

            Rectangle {
                width: parent.width
                height: 1
                color: "#17263d"
            }

            Text {
                anchors.horizontalCenter: parent.horizontalCenter
                text: txt("telifHakki")
                color: "#4b5563"
                font.family: "Segoe UI"
                font.pixelSize: 10
            }
        }
    }
}
```

qml\pages\OlcumPage.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtCharts 6.7
import sliper

Rectangle {
    color: "#0a0a0d"

    readonly property var metinler: ({
        testBilgileri: { tr: "TEST BİLGİLERİ", en: "TEST INFORMATION" },
        olcumYeri: { tr: "Ölçüm Yeri", en: "Place of Measure" },
        olcumYeriPlaceholder: { tr: "Laboratuvar / şantiye...", en: "Laboratory / site..." },
        musteri: { tr: "Müşteri", en: "Customer" },
        musteriPlaceholder: { tr: "Musteri adi girin...", en: "Enter customer name..." },
        betonRecetesi: { tr: "Beton Reçetesi", en: "Concrete Mix Design" },
        receteSeciniz: { tr: "Reçete Seçin...", en: "Select Mix Design..." },
        cimentoEtiket: { tr: "Çimento ", en: "Cement " },
        suCimentoEtiket: { tr: "  •  Su/Çimento ", en: "  •  Water/Cement " },
        eklenenAgirlik: { tr: "Eklenen Ağırlık (kg)", en: "Added Weight (kg)" },
        agirlikYok: { tr: "Ek ağırlık yok", en: "No added weight" },
        sonStroke: { tr: "SON STROKE", en: "LAST STROKE" },
        strokeYok: { tr: "Henüz stroke yok — boruyu üst konuma kaldırıp serbest bırakın", en: "No stroke yet — lift the pipe to
the top position and release it" },
        sure: { tr: "Süre", en: "Duration" },
        gecerliStroke: { tr: "GEÇERLİ", en: "VALID" },
        hataliStroke: { tr: "HATALI STROKE", en: "WRONG STROKE" },
        nedenHiz: { tr: "hız ≤ 0", en: "speed ≤ 0" },
        nedenBasinc: { tr: "p ≤ 0", en: "p ≤ 0" },
        nedenOrnek: { tr: "yetersiz örnek", en: "too few samples" },
        nedenKesinti: { tr: "veri kesintisi", en: "data gap" },
        geriButon: { tr: "←", en: "←" },
        sifirlaButon: { tr: "↺", en: "↺" },
        olcumuBaslatButon: { tr: "▶  Ölçümü Başlat", en: "▶  Start Measurement" },
        devamEtButon: { tr: "▶  Devam Et", en: "▶  Resume" },
        duraklatButon: { tr: "⏸  Duraklat", en: "⏸  Pause" },
        bitirButon: { tr: "■  Bitir", en: "■  Finish" },
        veriYok: { tr: "VERI YOK", en: "NO DATA" },
        duraklatildiDurum: { tr: "DURAKLATILDI", en: "PAUSED" },
        ustKonumBekleniyor: { tr: "ÜSTE TAKIN", en: "ATTACH AT TOP" },
        hazirlaButon: { tr: "●  Hazırla", en: "●  Prepare" },
        hazirlayinDurum: { tr: "HAZIRLAYIN", en: "PRESS PREPARE" },
        hazirlaniyorDurum: { tr: "HAZIRLANIYOR", en: "PREPARING" },
        hazirDurum: { tr: "HAZIR", en: "READY" },
        grafikDonduDurum: { tr: "STROKE BİTTİ", en: "STROKE FINISHED" },
        ustKonumIpucu: { tr: "Boruyu üste takın, \"Hazırla\"ya basın, HAZIR görününce serbest bırakın", en: "Attach the pipe at
the top, press \"Prepare\", release when READY" },
        anlikDegerler: { tr: "ANLIK DEĞERLER", en: "CURRENT VALUES" },
        basincEtiket: { tr: "BASINÇ", en: "PRESSURE" },
        konumEtiket: { tr: "KONUM", en: "POSITION" },
        hizEtiket: { tr: "HIZ", en: "SPEED" },
        debiEtiket: { tr: "DEBİ", en: "FLOW RATE" },
        basincZaman: { tr: "Basınç - Zaman", en: "Pressure - Time" },
        konumZaman: { tr: "Konum - Zaman", en: "Position - Time" },
        hizZaman: { tr: "Hız - Zaman", en: "Speed - Time" },
        debiZaman: { tr: "Debi - Zaman", en: "Flow Rate - Time" },
        sesliDinleniyor: { tr: "🎙 Dinleniyor — \"Başlat / Durdur / Devam Et / Bitir / Ekle\"", en: "🎙 Listening — \"Start /
Stop / Resume / Finish / Add\"" },
        sesliHazirlaniyor: { tr: "🎙 Sesli komut hazırlanıyor...", en: "🎙 Preparing voice commands..." },
        duyulanEtiket: { tr: "Duyulan: ", en: "Heard: " },
        bitirOnaySorusu: { tr: "Ölçümü bitirmek istediğinize emin misiniz?", en: "Are you sure you want to finish the
measurement?" },
        kaydetVeBitir: { tr: "💾  Kaydet ve Bitir", en: "💾  Save and Finish" },
        testiSilVeBitir: { tr: "🗑  Testi Sil ve Bitir", en: "🗑  Delete Test and Finish" },
        vazgecTesteDevam: { tr: "Vazgeç, Teste Devam Et", en: "Cancel, Continue Testing" },
        uyariMusteriAdi: { tr: "Lütfen müşteri adını girin.", en: "Please enter the customer name." },
        uyariReceteSec: { tr: "Lütfen reçeteyi seçin veya yazın.", en: "Please select or type the mix design." },
        recetePlaceholder: { tr: "Seçin veya yazın (örn. C30/37, K-12)...", en: "Select or type (e.g. C30/37, M-12)..." },
        veriAkisiKesildi: { tr: "⚠ Veri akışı kesildi — cihazı ve Wi-Fi bağlantısını kontrol edin", en: "⚠ Data transfer
```

```
interrupted — check the device and Wi-Fi connection" },
        mesafeSensoruArizali: { tr: "⚠ Mesafe sensörü cevap vermiyor", en: "⚠ Distance sensor not responding" },
        yukSensoruArizali: { tr: "⚠ Load cell cevap vermiyor", en: "⚠ Load cell not responding" },
        ikiSensorArizali: { tr: "⚠ Mesafe sensörü ve load cell cevap vermiyor", en: "⚠ Distance sensor and load cell not
responding" },
        uyariStrokeKaydedilemedi: { tr: "Stroke %1 veritabanına kaydedilemedi!", en: "Stroke %1 could not be saved to the
database!" },
        uyariCihazBagliDegil: { tr: "Cihaz bağlı değil. Lütfen önce sol alttan SLIPER-ESP32'ye bağlanın.", en: "Device not
connected. Please connect to SLIPER-ESP32 from the bottom left first." },
        uyariAktifOlcumYok: { tr: "Aktif bir ölçüm yok. Önce ölçümü başlatın.", en: "No active measurement. Please start a
measurement first." },
        daraAlButon: { tr: "DARA AL", en: "TARE" },
        daraSifirlaButon: { tr: "↺", en: "↺" },
        daraEtiket: { tr: "Dara: ", en: "Tare: " },
        daraYok: { tr: "Dara yok", en: "No tare" },
        uyariDaraVeriYok: { tr: "Dara alınamadı: cihazdan geçerli veri gelmiyor.", en: "Tare failed: no valid data from the
device." },
        uyariDaraCokBuyuk: { tr: "Dara alınamadı: basınç %1 mbar, sınır %2 mbar. Load cell yüklü görünüyor — boruyu boşaltıp
ağırlıkları kaldırın.", en: "Tare failed: pressure is %1 mbar, limit %2 mbar. The load cell appears loaded — empty the pipe and
remove the weights." },
        uyariDaraStrokeVar: { tr: "Dara bu ölçümde değiştirilemez: kaydedilmiş stroke'lar darasız ölçüldü, karıştırılırsa P-Q
doğrusu bozulur. Daranın değişmesi gerekiyorsa ölçümü bitirip yeni bir ölçüm başlatın.", en: "The tare cannot be changed in this
measurement: the recorded strokes were measured without it, and mixing them would corrupt the P-Q line. If the tare must change,
finish this measurement and start a new one." }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }
    property real zamanSayaci: 0
    property int aktifOlcumId: -1
    property string uyariMesaji: ""
    property bool bitirIcinDuraklatildi: false

    property real gosterilenBasinc: 0
    property real gosterilenKonum: 0
    property real gosterilenHiz: 0
    property real gosterilenDebi: 0
    property bool gosterilenVeriGecerli: false

    Timer {
        interval: 200
        running: true
        repeat: true
        onTriggered: {
            gosterilenBasinc = sensorManager.basinc
            gosterilenKonum = sensorManager.konum
            gosterilenHiz = sensorManager.hiz
            gosterilenDebi = sensorManager.debi
            gosterilenVeriGecerli = sensorManager.veriGecerli
        }
    }

    property real baslangicZamaniS: -1
    property var zamanGecmis: []
    property var basincGecmis: []
    property var konumGecmis: []
    property var hizGecmis: []
    property var debiGecmis: []

    property string kayitDurumu: "BEKLEMEDE"
    property bool inisBasladi: false
    property real kayitBaslangicKonumuMm: 0

    readonly property real ustYakalamaToleransiMm: calculator.ustYakalamaToleransiMm()
    readonly property real inisBaslamaYoluMm: calculator.minInisYoluMm()
    readonly property real beklemePenceresiS: 1.0

    property var agirlikListesi: []
    property real strokeAgirligi: -1
```

```
    readonly property int agirlikAdedi: agirlikListesi.length
    readonly property real agirlikToplam: {
        var t = 0
        for (var i = 0; i < agirlikListesi.length; i++) t += agirlikListesi[i]
        return t
    }

    function agirlikEkle(kg) {
        var yeni = agirlikListesi.slice()
        yeni.push(kg)
        agirlikListesi = yeni
        agirlikEklendi()
    }

    function agirlikGeriAl() {
        var yeni = agirlikListesi.slice()
        yeni.pop()
        agirlikListesi = yeni
    }

    function agirlikOzeti() {
        if (agirlikListesi.length === 0) return txt("agirlikYok")
        var kucukAgirlikSayisi = 0, buyukAgirlikSayisi = 0
        for (var i = 0; i < agirlikListesi.length; i++) {
            if (agirlikListesi[i] < 3) kucukAgirlikSayisi++; else buyukAgirlikSayisi++
        }
        var parcalar = []
        if (kucukAgirlikSayisi > 0) parcalar.push(kucukAgirlikSayisi + " x 1.6")
        if (buyukAgirlikSayisi > 0) parcalar.push(buyukAgirlikSayisi + " x 4.8")
        return parcalar.join(" + ") + " = " + agirlikToplam.toFixed(1) + " kg"
    }

    readonly property var receteListesi: [
        { sinifTr: "C16/20", sinifEn: "C16/20", cimentoTr: "min. 260 kg/m³", cimentoEn: "min. 260 kg/m³", suCimentoTr: "maks.
0.65", suCimentoEn: "max. 0.65", aciklamaTr: "Hafif yükte yalın/donatılı beton", aciklamaEn: "Plain/reinforced concrete for light
loads" },
        { sinifTr: "C20/25", sinifEn: "C20/25", cimentoTr: "min. 280 kg/m³", cimentoEn: "min. 280 kg/m³", suCimentoTr: "maks.
0.60", suCimentoEn: "max. 0.60", aciklamaTr: "Standart betonarme", aciklamaEn: "Standard reinforced concrete" },
        { sinifTr: "C25/30", sinifEn: "C25/30", cimentoTr: "min. 300 kg/m³", cimentoEn: "min. 300 kg/m³", suCimentoTr: "maks.
0.55", suCimentoEn: "max. 0.55", aciklamaTr: "Standart betonarme (TS EN 206)", aciklamaEn: "Standard reinforced concrete (TS EN
206)" },
        { sinifTr: "C30/37", sinifEn: "C30/37", cimentoTr: "min. 320 kg/m³", cimentoEn: "min. 320 kg/m³", suCimentoTr: "maks.
0.50", suCimentoEn: "max. 0.50", aciklamaTr: "Standart betonarme", aciklamaEn: "Standard reinforced concrete" },
        { sinifTr: "C30/37 SCC", sinifEn: "C30/37 SCC", cimentoTr: "min. 380 kg/m³", cimentoEn: "min. 380 kg/m³", suCimentoTr:
"maks. 0.45", suCimentoEn: "max. 0.45", aciklamaTr: "Kendiliğinden yerleşen beton (SF2)", aciklamaEn: "Self-compacting concrete
(SF2)" },
        { sinifTr: "C35/45", sinifEn: "C35/45", cimentoTr: "min. 340 kg/m³", cimentoEn: "min. 340 kg/m³", suCimentoTr: "maks.
0.45", suCimentoEn: "max. 0.45", aciklamaTr: "Orta-yüksek dayanım", aciklamaEn: "Medium-high strength" },
        { sinifTr: "C40/50", sinifEn: "C40/50", cimentoTr: "min. 360 kg/m³", cimentoEn: "min. 360 kg/m³", suCimentoTr: "maks.
0.40", suCimentoEn: "max. 0.40", aciklamaTr: "Yüksek dayanım", aciklamaEn: "High strength" },
        { sinifTr: "C45/55", sinifEn: "C45/55", cimentoTr: "min. 380 kg/m³", cimentoEn: "min. 380 kg/m³", suCimentoTr: "maks.
0.38", suCimentoEn: "max. 0.38", aciklamaTr: "Yüksek dayanım", aciklamaEn: "High strength" },
        { sinifTr: "C50/60", sinifEn: "C50/60", cimentoTr: "min. 400 kg/m³", cimentoEn: "min. 400 kg/m³", suCimentoTr: "maks.
0.35", suCimentoEn: "max. 0.35", aciklamaTr: "Yüksek dayanım (TS EN 206 normal beton sınırı)", aciklamaEn: "High strength (TS EN
206 normal concrete limit)" }
    ]

    signal olcumTamamlandi(int id)
    signal agirlikEklendi()

    function resetMeasurementScreen() {
        aktifOlcumId = -1
        uyariMesaji = ""
        bitirIcinDuraklatildi = false
        musteriKutusu.text = ""
        receteKutusu.currentIndex = -1
        receteKutusu.editText = ""
        agirlikListesi = []
        calculator.sifirla()
        kayitDurumunaDon()
```

```
}

function kayitDurumunaDon() {
    kayitDurumu = "BEKLEMEDE"
    inisBasladi = false
    grafikleriSifirla()
}

function baslatOlcumu() {
    if (aktifOlcumId > 0) return

    if (musteriKutusu.text.trim().length === 0) {
        uyariMesaji = txt("uyariMusteriAdi")
        return
    }

    if (receteKutusu.editText.trim().length === 0) {
        uyariMesaji = txt("uyariReceteSec")
        return
    }

    if (!wifiManager.baglandi) {
        console.warn("ESP32'ye baglanti yok, olcum baslatilmadi.")
        uyariMesaji = txt("uyariCihazBagliDegil")
        return
    }

    uyariMesaji = ""
    kayitDurumunaDon()

    calculator.sifirla()
    aktifOlcumId = database.olcumBaslat(
        musteriKutusu.text,
        receteKutusu.editText.trim(),
        agirlikToplam,
        yerKutusu.text,
        ""
    )
    console.log("Aktif olcum id:", aktifOlcumId)
    daraVeritabaninaYaz()
}

function daraDegistirilebilir() {
    return aktifOlcumId <= 0 || calculator.strokeSayisi === 0
}

function daraAl() {
    if (!daraDegistirilebilir()) {
        uyariMesaji = txt("uyariDaraStrokeVar")
        return
    }

    if (wifiManager.daraAl()) {
        uyariMesaji = ""
        daraVeritabaninaYaz()
        return
    }

    if (!sensorManager.veriGecerli) {
        uyariMesaji = txt("uyariDaraVeriYok")
        return
    }

    var darasizBasinc = sensorManager.basinc + wifiManager.daraBasinciMbar
    uyariMesaji = txt("uyariDaraCokBuyuk")
        .arg(darasizBasinc.toFixed(1))
        .arg(wifiManager.daraUstSiniriMbar.toFixed(1))
}

function daraSifirla() {
```

```
    if (!daraDegistirilebilir()) {
        uyariMesaji = txt("uyariDaraStrokeVar")
        return
    }
    wifiManager.daraSifirla()
    uyariMesaji = ""
    daraVeritabaninaYaz()
}

function daraVeritabaninaYaz() {
    if (aktifOlcumId > 0) {
        database.olcumDarasiKaydet(aktifOlcumId, wifiManager.daraBasinciMbar)
    }
}

function bitirTalebiGoster() {
    if (aktifOlcumId > 0) {
        if (!calculator.duraklatildi) {
            calculator.duraklat()
            bitirIcinDuraklatildi = true
        }
        bitirOnayPopup.open()
    } else {
        uyariMesaji = txt("uyariAktifOlcumYok")
    }
}

function grafikleriSifirla() {
    zamanSayaci = 0
    baslangicZamaniS = -1
    zamanGecmis = []
    basincSerisi.clear()
    konumSerisi.clear()
    hizSerisi.clear()
    debiSerisi.clear()
    basincGecmis = []
    konumGecmis = []
    hizGecmis = []
    debiGecmis = []
    zamanEkseniniUygula(0)
    yEkseni.min = 0
    yEkseni.max = 10
    konumYEkseni.min = 0
    konumYEkseni.max = 600
    hizYEkseni.min = 0
    hizYEkseni.max = 1
    debiYEkseni.min = 0
    debiYEkseni.max = 50
}

function zamanEkseniniUygula(sonZamanS) {
    var ilkZamanS = zamanGecmis.length > 0 ? zamanGecmis[0] : 0
    var genislik = Math.max(sonZamanS - ilkZamanS, 1.0)
    var bitis = ilkZamanS + genislik * 1.04
    var eksenler = [xEkseni, konumXEkseni, hizXEkseni, debiXEkseni]
    for (var i = 0; i < eksenler.length; i++) {
        eksenler[i].min = ilkZamanS
        eksenler[i].max = bitis
    }
}

function degerEkseniHesapla(dizi, sifirdanBasla, varsayilanTavan) {
    if (dizi.length === 0) return { min: 0, max: varsayilanTavan }
    var maxDeger = Math.max.apply(null, dizi)
    var minDeger = Math.min.apply(null, dizi)
    var kenarBoslugu = Math.max((maxDeger - minDeger) * 0.08, Math.abs(maxDeger) * 0.04, 0.5)
    var taban = sifirdanBasla ? Math.min(0, minDeger) : minDeger - kenarBoslugu
    return { min: taban, max: maxDeger + kenarBoslugu }
}
```

```
function konumYonu() {
    return calculator.ustKonumMm >= calculator.altKonumMm ? 1 : -1
}

function ustKonumda(konumMm) {
    var yukseklikMm = konumYonu() * konumMm
    return yukseklikMm > konumYonu() * calculator.ustKonumMm - ustYakalamaToleransiMm
}

function zamanEkseniniSifirla() {
    if (zamanGecmis.length === 0) return
    var kaydirma = zamanGecmis[0]
    if (kaydirma <= 0) return

    basincSerisi.clear()
    konumSerisi.clear()
    hizSerisi.clear()
    debiSerisi.clear()
    for (var i = 0; i < zamanGecmis.length; i++) {
        zamanGecmis[i] -= kaydirma
        basincSerisi.append(zamanGecmis[i], basincGecmis[i])
        konumSerisi.append(zamanGecmis[i], konumGecmis[i])
        hizSerisi.append(zamanGecmis[i], hizGecmis[i])
        debiSerisi.append(zamanGecmis[i], debiGecmis[i])
    }
    baslangicZamaniS += kaydirma
    zamanSayaci -= kaydirma
}

function kayitDurumunuGuncelle() {
    var konumMm = sensorManager.konum

    if (kayitDurumu !== "KAYITTA" && calculator.hazir && ustKonumda(konumMm)) {
        grafikleriSifirla()
        kayitDurumu = "KAYITTA"
        kayitBaslangicKonumuMm = konumMm
        inisBasladi = false
        return
    }

    if (kayitDurumu !== "KAYITTA") return

    if (!inisBasladi) {
        var inilenYolMm = konumYonu() * (kayitBaslangicKonumuMm - konumMm)
        if (inilenYolMm > inisBaslamaYoluMm) {
            inisBasladi = true
            zamanEkseniniSifirla()
        }
        return
    }

    if (calculator.durum === "TAMAMLANDI") kayitDurumu = "DONDU"
}

Connections {
    target: calculator
    function onDuraklatildiChanged() {
        if (!calculator.duraklatildi) kayitDurumunaDon()
    }
}

Connections {
    target: sensorManager
    function onVeriGuncellendi() {
        if (!sensorManager.veriGecerli || aktifOlcumId <= 0) return
        if (calculator.duraklatildi) return

        calculator.konumGuncelle(sensorManager.zamanS, sensorManager.konum, sensorManager.basinc)

        kayitDurumunuGuncelle()
```

```
        if (kayitDurumu !== "KAYITTA") return

        if (baslangicZamaniS < 0) baslangicZamaniS = sensorManager.zamanS
        zamanSayaci = sensorManager.zamanS - baslangicZamaniS

        basincSerisi.append(zamanSayaci, sensorManager.basinc)
        konumSerisi.append(zamanSayaci, sensorManager.konum)
        hizSerisi.append(zamanSayaci, sensorManager.hiz)
        debiSerisi.append(zamanSayaci, sensorManager.debi)

        zamanGecmis.push(zamanSayaci)
        basincGecmis.push(sensorManager.basinc)
        konumGecmis.push(sensorManager.konum)
        hizGecmis.push(sensorManager.hiz)
        debiGecmis.push(sensorManager.debi)

        var tutulacakSureS = inisBasladi ? 60 : beklemePenceresiS
        while (zamanGecmis.length > 1 && zamanGecmis[0] < zamanSayaci - tutulacakSureS) {
            basincSerisi.remove(0)
            konumSerisi.remove(0)
            hizSerisi.remove(0)
            debiSerisi.remove(0)

            zamanGecmis.shift()
            basincGecmis.shift()
            konumGecmis.shift()
            hizGecmis.shift()
            debiGecmis.shift()
        }

        zamanEkseniniUygula(zamanSayaci)

        var basincAralik = degerEkseniHesapla(basincGecmis, false, 10)
        yEkseni.min = basincAralik.min
        yEkseni.max = basincAralik.max
        var konumAralik = degerEkseniHesapla(konumGecmis, false, 600)
        konumYEkseni.min = konumAralik.min
        konumYEkseni.max = konumAralik.max
        var hizAralik = degerEkseniHesapla(hizGecmis, true, 1)
        hizYEkseni.min = hizAralik.min
        hizYEkseni.max = hizAralik.max
        var debiAralik = degerEkseniHesapla(debiGecmis, true, 50)
        debiYEkseni.min = debiAralik.min
        debiYEkseni.max = debiAralik.max
    }
}

Connections {
    target: voiceCommandManager

    function onBaslatKomutu() {
        if (!visible) return
        baslatOlcumu()
    }

    function onDurdurKomutu() {
        if (!visible || aktifOlcumId <= 0) return
        if (!calculator.duraklatildi) calculator.duraklat()
    }

    function onDevamKomutu() {
        if (!visible || aktifOlcumId <= 0) return
        if (calculator.duraklatildi) calculator.devamEt()
    }

    function onBitirKomutu() {
        if (!visible) return
        bitirTalebiGoster()
    }
```

```
    function onEkleKomutu() {
        if (!visible) return
        agirlikEkle(1.6)
    }
}

Row {
    anchors.fill: parent
    spacing: 0

    Flickable {
        id: testBilgileriPaneli
        width: 280
        height: parent.height
        contentWidth: width
        contentHeight: daraDurumu.y + daraDurumu.height + 20
        clip: true
        boundsBehavior: Flickable.StopAtBounds
        ScrollBar.vertical: ScrollBar { policy: ScrollBar.AsNeeded }

        Rectangle {
            z: -1
            width: testBilgileriPaneli.width
            height: Math.max(testBilgileriPaneli.height, testBilgileriPaneli.contentHeight)
            color: "#12121a"
        }

        Text {
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.margins: 20
            text: txt("testBilgileri")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 12
            font.bold: true
            font.letterSpacing: 1
        }

        Column {
            id: formAlanlari
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.margins: 20
            anchors.topMargin: 52
            spacing: 6

            Text {
                text: txt("olcumYeri")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 12
            }

            TextField {
                id: yerKutusu
                width: parent.width
                height: 34
                placeholderText: txt("olcumYeriPlaceholder")
                placeholderTextColor: "#4b5563"
                color: "#dce8f5"
                font.pixelSize: 13
                leftPadding: 10
                verticalAlignment: TextInput.AlignVCenter
                background: Rectangle {
                    color: "#0a0a0d"
                    radius: 6
                    border.color: yerKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
```

```
    }
}

Text {
    text: txt("musteri")
    color: "#9ca3af"
    font.family: "Segoe UI"
    font.pixelSize: 12
}

TextField {
    id: musteriKutusu
    width: parent.width
    height: 38
    placeholderText: txt("musteriPlaceholder")
    placeholderTextColor: "#4b5563"
    color: "#dce8f5"
    font.pixelSize: 13
    leftPadding: 10
    verticalAlignment: TextInput.AlignVCenter
    background: Rectangle {
        color: "#0a0a0d"
        radius: 6
        border.color: musteriKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
        border.width: 1
    }
}

Text {
    text: txt("betonRecetesi")
    color: "#9ca3af"
    font.family: "Segoe UI"
    font.pixelSize: 12
}

ComboBox {
    id: receteKutusu
    width: parent.width
    height: 38
    model: receteListesi
    textRole: Translations.turkish ? "sinifTr" : "sinifEn"
    editable: true
    currentIndex: -1

    background: Rectangle {
        color: "#0a0a0d"
        radius: 6
        border.color: receteKutusu.activeFocus ? "#3b82f6" : "#1e2a3f"
        border.width: 1
    }

    contentItem: TextField {
        text: receteKutusu.editText
        placeholderText: txt("recetePlaceholder")
        placeholderTextColor: "#4b5563"
        color: "#dce8f5"
        font.pixelSize: 13
        leftPadding: 10
        rightPadding: 30
        verticalAlignment: TextInput.AlignVCenter
        selectByMouse: true
        background: null
        onTextEdited: receteKutusu.editText = text
    }

    delegate: ItemDelegate {
        id: receteDelege
        width: receteKutusu.width
        height: modelData.cimentoTr.length > 0 ? 52 : 36
        highlighted: receteKutusu.highlightedIndex === index
```

```
                        background: Rectangle {
                            color: receteDelege.highlighted ? "#17263d" : "#0a0a0d"
                        }

                        contentItem: Column {
                            anchors.verticalCenter: parent.verticalCenter
                            anchors.left: parent.left
                            anchors.leftMargin: 10
                            anchors.right: parent.right
                            anchors.rightMargin: 10
                            spacing: 2

                            Text {
                                text: Translations.turkish ? modelData.sinifTr : modelData.sinifEn
                                color: "#dce8f5"
                                font.family: "Segoe UI"
                                font.pixelSize: 13
                                font.bold: true
                            }

                            Text {
                                visible: modelData.cimentoTr.length > 0
                                text: Translations.turkish
                                    ? (txt("cimentoEtiket") + modelData.cimentoTr + txt("suCimentoEtiket") +
modelData.suCimentoTr)
                                    : (txt("cimentoEtiket") + modelData.cimentoEn + txt("suCimentoEtiket") +
modelData.suCimentoEn)
                                color: "#6b7280"
                                font.family: "Segoe UI"
                                font.pixelSize: 10
                            }
                        }
                    }
                }

                Text {
                    text: txt("eklenenAgirlik")
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 12
                }

                Column {
                    width: parent.width
                    spacing: 8

                    Row {
                        width: parent.width
                        spacing: 8

                        Button {
                            id: agirlikEkleButonu
                            width: (parent.width - 3 * parent.spacing) / 4
                            height: 38
                            text: "+1.6"
                            font.family: "Segoe UI"
                            font.pixelSize: 12

                            onClicked: agirlikEkle(1.6)

                            background: Rectangle {
                                radius: 6
                                color: agirlikEkleButonu.pressed ? "#1e3a8a" : (agirlikEkleButonu.hovered ? "#1e3a8a" :
"#1d4ed8")
                                border.color: "#3b82f6"
                                border.width: 1
                            }

                            contentItem: Text {
```

```
                text: agirlikEkleButonu.text
                color: "#dce8f5"
                font: agirlikEkleButonu.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            id: agirlikEkle48Butonu
            width: (parent.width - 3 * parent.spacing) / 4
            height: 38
            text: "+4.8"
            font.family: "Segoe UI"
            font.pixelSize: 12

            onClicked: agirlikEkle(4.8)

            background: Rectangle {
                radius: 6
                color: agirlikEkle48Butonu.pressed ? "#1e3a8a" : (agirlikEkle48Butonu.hovered ? "#1e3a8a" :
"#1d4ed8")
                border.color: "#3b82f6"
                border.width: 1
            }

            contentItem: Text {
                text: agirlikEkle48Butonu.text
                color: "#dce8f5"
                font: agirlikEkle48Butonu.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            id: agirlikAzaltButonu
            width: (parent.width - 3 * parent.spacing) / 4
            height: 38
            text: txt("geriButon")
            font.family: "Segoe UI"
            font.pixelSize: 12
            enabled: agirlikAdedi > 0

            onClicked: agirlikGeriAl()

            background: Rectangle {
                radius: 6
                color: agirlikAzaltButonu.pressed ? "#78350f" : (agirlikAzaltButonu.hovered ? "#a16207" :
"#92400e")
                opacity: agirlikAzaltButonu.enabled ? 1.0 : 0.4
                border.color: "#d97706"
                border.width: 1
            }

            contentItem: Text {
                text: agirlikAzaltButonu.text
                color: "#dce8f5"
                opacity: agirlikAzaltButonu.enabled ? 1.0 : 0.6
                font: agirlikAzaltButonu.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            id: sifirlaButonu
            width: (parent.width - 3 * parent.spacing) / 4
            height: 38
            text: txt("sifirlaButon")
```

```
                        font.family: "Segoe UI"
                        font.pixelSize: 12
                        enabled: agirlikAdedi > 0

                        onClicked: agirlikListesi = []

                        background: Rectangle {
                            radius: 6
                            color: sifirlaButonu.pressed ? "#7f1d1d" : (sifirlaButonu.hovered ? "#b91c1c" : "#991b1b")
                            opacity: sifirlaButonu.enabled ? 1.0 : 0.4
                            border.color: "#dc2626"
                            border.width: 1
                        }

                        contentItem: Text {
                            text: sifirlaButonu.text
                            color: "#dce8f5"
                            opacity: sifirlaButonu.enabled ? 1.0 : 0.6
                            font: sifirlaButonu.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                        }
                    }
                }

                Rectangle {
                    width: parent.width
                    height: 34
                    radius: 6
                    color: "#0a0a0d"
                    border.color: "#1e2a3f"
                    border.width: 1

                    Text {
                        anchors.centerIn: parent
                        text: agirlikOzeti()
                        color: agirlikAdedi > 0 ? "#dce8f5" : "#4b5563"
                        font.family: "Segoe UI"
                        font.pixelSize: 13
                        font.bold: true
                    }
                }
            }

            Item { width: parent.width; height: 10 }

            Button {
                id: baslatButonu
                width: parent.width
                height: 44
                text: txt("olcumuBaslatButon")
                font.family: "Segoe UI"
                font.pixelSize: 14
                font.bold: true
                enabled: aktifOlcumId <= 0

                onClicked: baslatOlcumu()

                background: Rectangle {
                    radius: 8
                    color: !baslatButonu.enabled ? "#14532d" : (baslatButonu.pressed ? "#15803d" : (baslatButonu.hovered ?
"#22c55e" : "#16a34a"))
                    opacity: baslatButonu.enabled ? 1.0 : 0.5
                    Behavior on color { ColorAnimation { duration: 120 } }
                }

                contentItem: Text {
                    text: baslatButonu.text
                    color: "#dce8f5"
                    opacity: baslatButonu.enabled ? 1.0 : 0.6
```

```
                        font: baslatButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Text {
                    width: parent.width
                    visible: uyariMesaji.length > 0
                    text: uyariMesaji
                    color: "#f87171"
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                    wrapMode: Text.WordWrap
                }

                Button {
                    id: hazirlaButonu
                    width: parent.width
                    height: 40
                    enabled: aktifOlcumId > 0 && !calculator.duraklatildi && calculator.durum !== "INIYOR"
                    text: txt("hazirlaButon")
                    font.family: "Segoe UI"
                    font.pixelSize: 13

                    onClicked: calculator.hazirla()

                    background: Rectangle {
                        radius: 8
                        color: !hazirlaButonu.enabled ? "#1e3a8a" : (hazirlaButonu.pressed ? "#1d4ed8" : (hazirlaButonu.hovered ?
"#3b82f6" : "#2563eb"))
                        opacity: hazirlaButonu.enabled ? 1.0 : 0.5
                    }

                    contentItem: Text {
                        text: hazirlaButonu.text
                        color: "#dce8f5"
                        opacity: hazirlaButonu.enabled ? 1.0 : 0.6
                        font: hazirlaButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Row {
                    width: parent.width
                    spacing: 10

                    Button {
                        id: duraklatButonu
                        width: (parent.width - parent.spacing) / 2
                        height: 40
                        text: calculator.duraklatildi ? txt("devamEtButon") : txt("duraklatButon")
                        font.family: "Segoe UI"
                        font.pixelSize: 13

                        onClicked: {
                            if (calculator.duraklatildi) {
                                calculator.devamEt()
                            } else {
                                calculator.duraklat()
                            }
                        }

                        background: Rectangle {
                            radius: 8
                            color: duraklatButonu.pressed ? "#78350f" : (duraklatButonu.hovered ? "#a16207" : "#92400e")
                            border.color: "#d97706"
                            border.width: 1
                        }
```

```
            contentItem: Text {
                text: duraklatButonu.text
                color: "#dce8f5"
                font: duraklatButonu.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }

        Button {
            id: bitirButonu
            width: (parent.width - parent.spacing) / 2
            height: 40
            text: txt("bitirButon")
            font.family: "Segoe UI"
            font.pixelSize: 13

            onClicked: bitirTalebiGoster()

            background: Rectangle {
                radius: 8
                color: bitirButonu.pressed ? "#7f1d1d" : (bitirButonu.hovered ? "#b91c1c" : "#991b1b")
                border.color: "#dc2626"
                border.width: 1
            }

            contentItem: Text {
                text: bitirButonu.text
                color: "#dce8f5"
                font: bitirButonu.font
                horizontalAlignment: Text.AlignHCenter
                verticalAlignment: Text.AlignVCenter
            }
        }
    }
}

Rectangle {
    id: durumKarti
    anchors.top: formAlanlari.bottom
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.topMargin: 14
    anchors.leftMargin: 20
    anchors.rightMargin: 20
    height: 52
    radius: 10
    color: "#0a0a0d"
    border.color: "#1e2a3f"
    border.width: 1

    Row {
        anchors.left: parent.left
        anchors.verticalCenter: parent.verticalCenter
        anchors.leftMargin: 14
        spacing: 10

        Rectangle {
            id: trafikIsigi
            width: 12
            height: 12
            radius: 6
            anchors.verticalCenter: parent.verticalCenter
            color: {
                if (!sensorManager.veriGecerli) return "#dc2626"
                if (calculator.duraklatildi) return "#6b7280"
                if (calculator.hazir) return "#16a34a"
                if (calculator.hazirlaniyor) return "#f59e0b"
                if (aktifOlcumId > 0 && kayitDurumu === "BEKLEMEDE") return "#3b82f6"
```

```
                if (aktifOlcumId > 0 && kayitDurumu === "DONDU") return "#6b7280"
                if (calculator.durum === "YUKARIDA") return "#16a34a"
                if (calculator.durum === "INIYOR") return "#f59e0b"
                return "#dc2626"
            }

            Rectangle {
                anchors.centerIn: parent
                width: parent.width + 8
                height: parent.height + 8
                radius: width / 2
                color: "transparent"
                border.color: parent.color
                border.width: 1
                opacity: 0.35
            }
        }

        Text {
            anchors.verticalCenter: parent.verticalCenter
            text: {
                if (!sensorManager.veriGecerli) return txt("veriYok")
                if (calculator.duraklatildi) return txt("duraklatildiDurum")
                if (calculator.hazir) return txt("hazirDurum")
                if (calculator.hazirlaniyor) return txt("hazirlaniyorDurum")
                if (aktifOlcumId > 0 && kayitDurumu === "BEKLEMEDE") {
                    return ustKonumda(sensorManager.konum) ? txt("hazirlayinDurum") : txt("ustKonumBekleniyor")
                }
                if (aktifOlcumId > 0 && kayitDurumu === "DONDU") return txt("grafikDonduDurum")
                return calculator.durum
            }
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 13
            font.bold: true
        }
    }

    Row {
        anchors.right: parent.right
        anchors.verticalCenter: parent.verticalCenter
        anchors.rightMargin: 14
        spacing: 6

        Text {
            anchors.verticalCenter: parent.verticalCenter
            text: "STROKE"
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 10
            font.letterSpacing: 1
        }

        Text {
            anchors.verticalCenter: parent.verticalCenter
            text: calculator.strokeSayisi
            color: "#4f8cf7"
            font.family: "Segoe UI"
            font.pixelSize: 16
            font.bold: true
        }
    }
}

Text {
    id: anlikDegerlerBaslik
    anchors.top: durumKarti.bottom
    anchors.left: parent.left
    anchors.topMargin: 16
    anchors.leftMargin: 20
```

```
    text: txt("anlikDegerler")
    color: "#6b7280"
    font.family: "Segoe UI"
    font.pixelSize: 12
    font.bold: true
    font.letterSpacing: 1
}

Grid {
    id: anlikDegerlerIzgarasi
    anchors.top: anlikDegerlerBaslik.bottom
    anchors.left: parent.left
    anchors.topMargin: 12
    anchors.leftMargin: 20
    columns: 2
    spacing: 10

    Rectangle {
        id: basincKarti
        width: 115
        height: 90
        radius: 10
        color: "#0a0a0d"
        border.color: "#1e2a3f"
        border.width: 1
        clip: true

        Rectangle { width: 4; height: parent.height; color: "#3b82f6" }

        Column {
            anchors.top: parent.top
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.left: parent.left
            anchors.topMargin: 10
            anchors.rightMargin: 10
            anchors.bottomMargin: 10
            anchors.leftMargin: 16
            spacing: 6

            Text {
                text: txt("basincEtiket")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.letterSpacing: 1
            }

            Text {
                text: gosterilenVeriGecerli ? gosterilenBasinc.toFixed(1) : "--"
                color: "#3b82f6"
                font.family: "Segoe UI"
                font.pixelSize: 22
                font.bold: true
            }

            Text {
                text: "mbar"
                color: "#4b5563"
                font.family: "Segoe UI"
                font.pixelSize: 10
            }
        }
    }

    Rectangle {
        width: 115
        height: 90
        radius: 10
        color: "#0a0a0d"
```

```
        border.color: "#1e2a3f"
        border.width: 1
        clip: true

        Rectangle { width: 4; height: parent.height; color: "#9333ea" }

        Column {
            anchors.top: parent.top
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.left: parent.left
            anchors.topMargin: 10
            anchors.rightMargin: 10
            anchors.bottomMargin: 10
            anchors.leftMargin: 16
            spacing: 6

            Text {
                text: txt("konumEtiket")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.letterSpacing: 1
            }

            Text {
                text: gosterilenVeriGecerli ? gosterilenKonum.toFixed(1) : "--"
                color: "#9333ea"
                font.family: "Segoe UI"
                font.pixelSize: 22
                font.bold: true
            }

            Text {
                text: "mm"
                color: "#4b5563"
                font.family: "Segoe UI"
                font.pixelSize: 10
            }
        }
    }

    Rectangle {
        width: 115
        height: 90
        radius: 10
        color: "#0a0a0d"
        border.color: "#1e2a3f"
        border.width: 1
        clip: true

        Rectangle { width: 4; height: parent.height; color: "#f59e0b" }

        Column {
            anchors.top: parent.top
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.left: parent.left
            anchors.topMargin: 10
            anchors.rightMargin: 10
            anchors.bottomMargin: 10
            anchors.leftMargin: 16
            spacing: 6

            Text {
                text: txt("hizEtiket")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.letterSpacing: 1
```

```
            }

            Text {
                text: gosterilenVeriGecerli ? gosterilenHiz.toFixed(2) : "--"
                color: "#f59e0b"
                font.family: "Segoe UI"
                font.pixelSize: 22
                font.bold: true
            }

            Text {
                text: "m/s"
                color: "#4b5563"
                font.family: "Segoe UI"
                font.pixelSize: 10
            }
        }
    }

    Rectangle {
        width: 115
        height: 90
        radius: 10
        color: "#0a0a0d"
        border.color: "#1e2a3f"
        border.width: 1
        clip: true

        Rectangle { width: 4; height: parent.height; color: "#16a34a" }

        Column {
            anchors.top: parent.top
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.left: parent.left
            anchors.topMargin: 10
            anchors.rightMargin: 10
            anchors.bottomMargin: 10
            anchors.leftMargin: 16
            spacing: 6

            Text {
                text: txt("debiEtiket")
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 10
                font.letterSpacing: 1
            }

            Text {
                text: gosterilenVeriGecerli ? gosterilenDebi.toFixed(1) : "--"
                color: "#16a34a"
                font.family: "Segoe UI"
                font.pixelSize: 22
                font.bold: true
            }

            Text {
                text: "m3/h"
                color: "#4b5563"
                font.family: "Segoe UI"
                font.pixelSize: 10
            }
        }
    }
}

Row {
    id: daraSatiri
    anchors.top: anlikDegerlerIzgarasi.bottom
```

```
                anchors.left: parent.left
                anchors.topMargin: 10
                anchors.leftMargin: 20
                width: anlikDegerlerIzgarasi.width
                spacing: 8

                Button {
                    id: daraAlButonu
                    width: daraSatiri.width - daraSifirlaButonu.width - daraSatiri.spacing
                    height: 34
                    enabled: wifiManager.baglandi && sensorManager.veriGecerli && daraDegistirilebilir()
                    text: txt("daraAlButon")
                    font.family: "Segoe UI"
                    font.pixelSize: 11
                    font.bold: true
                    font.letterSpacing: 1

                    onClicked: daraAl()

                    background: Rectangle {
                        radius: 8
                        color: !daraAlButonu.enabled ? "#1f2937" : (daraAlButonu.pressed ? "#334155" : (daraAlButonu.hovered ?
"#475569" : "#374151"))
                        border.color: "#1e2a3f"
                        border.width: 1
                        opacity: daraAlButonu.enabled ? 1.0 : 0.5
                    }

                    contentItem: Text {
                        text: daraAlButonu.text
                        color: "#dce8f5"
                        opacity: daraAlButonu.enabled ? 1.0 : 0.6
                        font: daraAlButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }

                Button {
                    id: daraSifirlaButonu
                    width: 40
                    height: 34
                    enabled: wifiManager.daraBasinciMbar !== 0 && daraDegistirilebilir()
                    text: txt("daraSifirlaButon")
                    font.family: "Segoe UI"
                    font.pixelSize: 14

                    onClicked: daraSifirla()

                    background: Rectangle {
                        radius: 8
                        color: !daraSifirlaButonu.enabled ? "#1f2937" : (daraSifirlaButonu.pressed ? "#7f1d1d" :
(daraSifirlaButonu.hovered ? "#b91c1c" : "#991b1b"))
                        border.color: "#1e2a3f"
                        border.width: 1
                        opacity: daraSifirlaButonu.enabled ? 1.0 : 0.4
                    }

                    contentItem: Text {
                        text: daraSifirlaButonu.text
                        color: "#dce8f5"
                        opacity: daraSifirlaButonu.enabled ? 1.0 : 0.6
                        font: daraSifirlaButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }

            Text {
```

```
        id: daraDurumu
        anchors.top: daraSatiri.bottom
        anchors.left: parent.left
        anchors.topMargin: 6
        anchors.leftMargin: 20
        width: anlikDegerlerIzgarasi.width
        text: wifiManager.daraBasinciMbar !== 0
              ? txt("daraEtiket") + wifiManager.daraBasinciMbar.toFixed(2) + " mbar"
              : txt("daraYok")
        color: wifiManager.daraBasinciMbar !== 0 ? "#fbbf24" : "#4b5563"
        font.family: "Segoe UI"
        font.pixelSize: 11
        wrapMode: Text.WordWrap
    }
}

Rectangle {
    id: panelAyraci
    width: 3
    height: parent.height
    color: "#060607"

    Rectangle {
        anchors.right: parent.right
        anchors.top: parent.top
        anchors.bottom: parent.bottom
        width: 1
        color: "#403b82f6"
    }
}

Item {
    width: parent.width - testBilgileriPaneli.width - panelAyraci.width
    height: parent.height

    Rectangle {
        id: sonStrokeCubugu
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.margins: 16
        height: 52
        radius: 10
        color: "#12121a"
        border.color: calculator.sonStroke.no === undefined ? "#1e2a3f"
                    : (calculator.sonStroke.gecerli ? "#166534" : "#7f1d1d")
        border.width: 1

        readonly property var s: calculator.sonStroke
        readonly property bool var_: s.no !== undefined

        function nedenMetni(kod) {
            if (kod === "hiz") return txt("nedenHiz")
            if (kod === "basinc") return txt("nedenBasinc")
            if (kod === "ornek") return txt("nedenOrnek")
            if (kod === "kesinti") return txt("nedenKesinti")
            return ""
        }

        Text {
            anchors.centerIn: parent
            visible: !sonStrokeCubugu.var_
            text: (aktifOlcumId > 0 && kayitDurumu === "BEKLEMEDE") ? txt("ustKonumIpucu") : txt("strokeYok")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 12
        }

        Row {
            anchors.left: parent.left
```

```
                    anchors.leftMargin: 16
                    anchors.verticalCenter: parent.verticalCenter
                    spacing: 22
                    visible: sonStrokeCubugu.var_

                    Text {
                        anchors.verticalCenter: parent.verticalCenter
                        text: txt("sonStroke") + "  #" + (sonStrokeCubugu.var_ ? sonStrokeCubugu.s.no : "")
                        color: "#6b7280"
                        font.family: "Segoe UI"
                        font.pixelSize: 11
                        font.bold: true
                        font.letterSpacing: 1
                    }

                    Repeater {
                        model: sonStrokeCubugu.var_ ? [
                            { e: txt("sure"), d: sonStrokeCubugu.s.sure.toFixed(2) + " s" },
                            { e: "Pmax", d: sonStrokeCubugu.s.pMaks.toFixed(1) + " mbar" },
                            { e: "P0l", d: sonStrokeCubugu.s.p0l.toFixed(1) + " mbar" },
                            { e: "P0r", d: sonStrokeCubugu.s.p0r.toFixed(1) + " mbar" },
                            { e: "p", d: sonStrokeCubugu.s.basinc.toFixed(2) + " mbar" },
                            { e: "Q", d: sonStrokeCubugu.s.debi.toFixed(2) + " m³/h" },
                            { e: "v", d: sonStrokeCubugu.s.hiz.toFixed(3) + " m/s" }
                        ] : []

                        Column {
                            anchors.verticalCenter: parent.verticalCenter
                            spacing: 1
                            Text { text: modelData.e; color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 10 }
                            Text { text: modelData.d; color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 13; font.bold:
true }
                        }
                    }
                }

                Rectangle {
                    anchors.right: parent.right
                    anchors.rightMargin: 14
                    anchors.verticalCenter: parent.verticalCenter
                    visible: sonStrokeCubugu.var_
                    width: gecerlilikMetni.implicitWidth + 18
                    height: 24
                    radius: 12
                    color: sonStrokeCubugu.var_ && sonStrokeCubugu.s.gecerli ? "#123321" : "#2a1414"

                    Text {
                        id: gecerlilikMetni
                        anchors.centerIn: parent
                        text: !sonStrokeCubugu.var_ ? "" : (sonStrokeCubugu.s.gecerli ? txt("gecerliStroke")
                              : txt("hataliStroke") + " (" + sonStrokeCubugu.nedenMetni(sonStrokeCubugu.s.gecersizNedeni) + ")")
                        color: sonStrokeCubugu.var_ && sonStrokeCubugu.s.gecerli ? "#4ade80" : "#f87171"
                        font.family: "Segoe UI"
                        font.pixelSize: 11
                        font.bold: true
                    }
                }
            }

            Grid {
                anchors.top: sonStrokeCubugu.bottom
                anchors.left: parent.left
                anchors.right: parent.right
                anchors.bottom: parent.bottom
                anchors.margins: 10
                anchors.topMargin: 8
                columns: 2
                spacing: 8

                Rectangle {
```

```
width: (parent.width - parent.spacing) / 2
height: (parent.height - parent.spacing) / 2
radius: 10
color: "#12121a"
border.color: "#1b1b23"
border.width: 1

Item {
    id: basincBaslik
    anchors.top: parent.top
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.margins: 12
    height: 20

    Text {
        anchors.left: parent.left
        anchors.verticalCenter: parent.verticalCenter
        text: txt("basincZaman")
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 14
        font.bold: true
    }
}

ChartView {
    anchors.top: basincBaslik.bottom
    anchors.left: parent.left
    anchors.right: parent.right
    anchors.bottom: parent.bottom
    anchors.topMargin: 4
    anchors.bottomMargin: 6
    backgroundColor: "transparent"
    legend.visible: false
    antialiasing: true
    margins.top: 4
    margins.bottom: 4
    margins.left: 4
    margins.right: 12

    ValueAxis {
        id: xEkseni
        min: 0
        max: 60
        gridLineColor: "#1a1a20"
        labelsColor: "#6b7280"
        labelsFont.pixelSize: 12
        lineVisible: false
        minorGridVisible: false
    }

    ValueAxis {
        id: yEkseni
        min: 0
        max: 1100
        gridLineColor: "#1a1a20"
        labelsColor: "#6b7280"
        labelsFont.pixelSize: 12
        lineVisible: false
        minorGridVisible: false
    }

    LineSeries {
        id: basincSerisi
        axisX: xEkseni
        axisY: yEkseni
        color: "#3b82f6"
        width: 3
    }
```

```
    }
}

Rectangle {
    width: (parent.width - parent.spacing) / 2
    height: (parent.height - parent.spacing) / 2
    radius: 10
    color: "#12121a"
    border.color: "#241a38"
    border.width: 1

    Item {
        id: konumBaslik
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.margins: 12
        height: 20

        Text {
            anchors.left: parent.left
            anchors.verticalCenter: parent.verticalCenter
            text: txt("konumZaman")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 14
            font.bold: true
        }
    }

    ChartView {
        anchors.top: konumBaslik.bottom
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        anchors.topMargin: 4
        anchors.bottomMargin: 6
        backgroundColor: "transparent"
        legend.visible: false
        antialiasing: true
        margins.top: 4
        margins.bottom: 4
        margins.left: 4
        margins.right: 12

        ValueAxis {
            id: konumXEkseni
            min: 0
            max: 60
            gridLineColor: "#1a1a20"
            labelsColor: "#6b7280"
            labelsFont.pixelSize: 12
            lineVisible: false
            minorGridVisible: false
        }

        ValueAxis {
            id: konumYEkseni
            min: 0
            max: 500
            gridLineColor: "#1a1a20"
            labelsColor: "#6b7280"
            labelsFont.pixelSize: 12
            lineVisible: false
            minorGridVisible: false
        }

        LineSeries {
            id: konumSerisi
            axisX: konumXEkseni
```

```
                            axisY: konumYEkseni
                            color: "#9333ea"
                            width: 3
                        }

                        Connections {
                            target: calculator
                            function onDurumChanged() {
                                if (calculator.durum === "INIYOR") strokeAgirligi = agirlikToplam
                            }
                            function onStrokeTamamlandi(stroke) {
                                if (aktifOlcumId > 0) {
                                    var kayitId = database.strokeKaydet(aktifOlcumId, stroke, strokeAgirligi >= 0 ?
strokeAgirligi : agirlikToplam)
                                    if (kayitId <= 0) {
                                        uyariMesaji = txt("uyariStrokeKaydedilemedi").arg(stroke.no)
                                    }
                                }
                                strokeAgirligi = -1
                            }
                        }
                    }
                }

                Rectangle {
                    width: (parent.width - parent.spacing) / 2
                    height: (parent.height - parent.spacing) / 2
                    radius: 10
                    color: "#12121a"
                    border.color: "#1e2a3f"
                    border.width: 1

                    Item {
                        id: hizBaslik
                        anchors.top: parent.top
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.margins: 12
                        height: 20

                        Text {
                            anchors.left: parent.left
                            anchors.verticalCenter: parent.verticalCenter
                            text: txt("hizZaman")
                            color: "#dce8f5"
                            font.family: "Segoe UI"
                            font.pixelSize: 14
                            font.bold: true
                        }
                    }

                    ChartView {
                        anchors.top: hizBaslik.bottom
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.bottom: parent.bottom
                        anchors.topMargin: 4
                        anchors.bottomMargin: 6
                        backgroundColor: "transparent"
                        legend.visible: false
                        antialiasing: true
                        margins.top: 4
                        margins.bottom: 4
                        margins.left: 4
                        margins.right: 12

                        ValueAxis {
                            id: hizXEkseni
                            min: 0
                            max: 60
```

```
            gridLineColor: "#1a1a20"
            labelsColor: "#6b7280"
            labelsFont.pixelSize: 12
            lineVisible: false
            minorGridVisible: false
        }

        ValueAxis {
            id: hizYEkseni
            min: 0
            max: 4
            gridLineColor: "#1a1a20"
            labelsColor: "#6b7280"
            labelsFont.pixelSize: 12
            lineVisible: false
            minorGridVisible: false
        }

        LineSeries {
            id: hizSerisi
            axisX: hizXEkseni
            axisY: hizYEkseni
            color: "#f59e0b"
            width: 3
        }
    }
}

Rectangle {
    width: (parent.width - parent.spacing) / 2
    height: (parent.height - parent.spacing) / 2
    radius: 10
    color: "#12121a"
    border.color: "#163321"
    border.width: 1

    Item {
        id: debiBaslik
        anchors.top: parent.top
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.margins: 12
        height: 20

        Text {
            anchors.left: parent.left
            anchors.verticalCenter: parent.verticalCenter
            text: txt("debiZaman")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 14
            font.bold: true
        }
    }

    ChartView {
        anchors.top: debiBaslik.bottom
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        anchors.topMargin: 4
        anchors.bottomMargin: 6
        backgroundColor: "transparent"
        legend.visible: false
        antialiasing: true
        margins.top: 4
        margins.bottom: 4
        margins.left: 4
        margins.right: 12
```

```
                    ValueAxis {
                        id: debiXEkseni
                        min: 0
                        max: 60
                        gridLineColor: "#1a1a20"
                        labelsColor: "#6b7280"
                        labelsFont.pixelSize: 12
                        lineVisible: false
                        minorGridVisible: false
                    }

                    ValueAxis {
                        id: debiYEkseni
                        min: 0
                        max: 180
                        gridLineColor: "#1a1a20"
                        labelsColor: "#6b7280"
                        labelsFont.pixelSize: 12
                        lineVisible: false
                        minorGridVisible: false
                    }

                    LineSeries {
                        id: debiSerisi
                        axisX: debiXEkseni
                        axisY: debiYEkseni
                        color: "#16a34a"
                        width: 3
                    }
                }
            }
        }
    }
}

Rectangle {
    id: veriKesintiUyarisi
    visible: wifiManager.baglandi && !sensorManager.veriGecerli
    anchors.top: parent.top
    anchors.horizontalCenter: parent.horizontalCenter
    anchors.topMargin: 12
    width: kesintiMetni.implicitWidth + 32
    height: 36
    radius: 8
    color: "#7f1d1d"
    border.color: "#dc2626"
    border.width: 1
    z: 20
    Text {
        id: kesintiMetni
        anchors.centerIn: parent
        text: {
            if (sensorManager.mesafeSensoruArizali && sensorManager.yukSensoruArizali) return txt("ikiSensorArizali")
            if (sensorManager.mesafeSensoruArizali) return txt("mesafeSensoruArizali")
            if (sensorManager.yukSensoruArizali) return txt("yukSensoruArizali")
            return txt("veriAkisiKesildi")
        }
        color: "#fecaca"
        font.family: "Segoe UI"
        font.pixelSize: 13
        font.bold: true
    }
}

Rectangle {
    id: sesliKomutRozeti
    visible: voiceCommandManager.etkin
    anchors.top: parent.top
    anchors.right: parent.right
    anchors.margins: 16
```

```
    width: sesliKomutIcerik.implicitWidth + 24
    height: sesliKomutIcerik.implicitHeight + 16
    radius: 10
    color: "#0a0a0d"
    border.color: voiceCommandManager.dinliyor ? "#16a34a" : "#1e2a3f"
    border.width: 1
    z: 10

    Column {
        id: sesliKomutIcerik
        anchors.centerIn: parent
        spacing: 4

        Row {
            spacing: 6
            Rectangle {
                width: 8
                height: 8
                radius: 4
                anchors.verticalCenter: parent.verticalCenter
                color: voiceCommandManager.dinliyor ? "#16a34a" : "#6b7280"
            }
            Text {
                text: voiceCommandManager.dinliyor ? txt("sesliDinleniyor") : txt("sesliHazirlaniyor")
                color: "#9ca3af"
                font.family: "Segoe UI"
                font.pixelSize: 11
            }
        }

        Text {
            visible: voiceCommandManager.anlikMetin.length > 0
            text: txt("duyulanEtiket") + voiceCommandManager.anlikMetin
            color: "#4b5563"
            font.family: "Segoe UI"
            font.pixelSize: 10
            font.italic: true
        }
    }
}

Popup {
    id: bitirOnayPopup
    modal: true
    focus: true
    anchors.centerIn: parent
    width: 340
    padding: 24
    closePolicy: Popup.CloseOnEscape | Popup.CloseOnPressOutside

    background: Rectangle {
        color: "#12121a"
        radius: 12
        border.color: "#1e2a3f"
        border.width: 1
    }

    Overlay.modal: Rectangle {
        color: "#a6000000"
    }

    contentItem: Column {
        spacing: 20

        Text {
            width: 292
            text: txt("bitirOnaySorusu")
            color: "#dce8f5"
            font.family: "Segoe UI"
            font.pixelSize: 14
```

```
    font.bold: true
    wrapMode: Text.WordWrap
}

Column {
    width: 292
    spacing: 10

    Button {
        id: kaydetBitirButonu
        width: parent.width
        height: 40
        text: txt("kaydetVeBitir")
        font.family: "Segoe UI"
        font.pixelSize: 13

        onClicked: {
            bitirOnayPopup.close()
            var bitenId = aktifOlcumId
            console.log("Olcum kaydedilip sonlandirildi, id:", bitenId)
            resetMeasurementScreen()
            olcumTamamlandi(bitenId)
        }

        background: Rectangle {
            radius: 8
            color: kaydetBitirButonu.pressed ? "#14532d" : (kaydetBitirButonu.hovered ? "#15803d" : "#16a34a")
            border.color: "#22c55e"
            border.width: 1
        }

        contentItem: Text {
            text: kaydetBitirButonu.text
            color: "#dce8f5"
            font: kaydetBitirButonu.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
    }

    Button {
        id: silBitirButonu
        width: parent.width
        height: 40
        text: txt("testiSilVeBitir")
        font.family: "Segoe UI"
        font.pixelSize: 13

        onClicked: {
            bitirOnayPopup.close()
            var silinecekId = aktifOlcumId
            var basarili = database.olcumSil(silinecekId)
            console.log("Olcum silinip sonlandirildi, id:", silinecekId, "basarili:", basarili)
            resetMeasurementScreen()
        }

        background: Rectangle {
            radius: 8
            color: silBitirButonu.pressed ? "#7f1d1d" : (silBitirButonu.hovered ? "#b91c1c" : "#991b1b")
            border.color: "#dc2626"
            border.width: 1
        }

        contentItem: Text {
            text: silBitirButonu.text
            color: "#dce8f5"
            font: silBitirButonu.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
        }
```

```
                }

                Button {
                    id: vazgecButonu
                    width: parent.width
                    height: 40
                    text: txt("vazgecTesteDevam")
                    font.family: "Segoe UI"
                    font.pixelSize: 13

                    onClicked: {
                        bitirOnayPopup.close()
                        if (bitirIcinDuraklatildi) {
                            calculator.devamEt()
                            bitirIcinDuraklatildi = false
                        }
                    }

                    background: Rectangle {
                        radius: 8
                        color: vazgecButonu.pressed ? "#1e2a3f" : (vazgecButonu.hovered ? "#243349" : "#1a1a20")
                        border.color: "#2c3b52"
                        border.width: 1
                    }

                    contentItem: Text {
                        text: vazgecButonu.text
                        color: "#dce8f5"
                        font: vazgecButonu.font
                        horizontalAlignment: Text.AlignHCenter
                        verticalAlignment: Text.AlignVCenter
                    }
                }
            }
        }
    }
}
```

qml\pages\RehberMockup.qml

04.11.2026 11:45

```
import QtQuick 6.7

Rectangle {
    id: root

    property string sahneAdi: "olcum"
    property string vurguAnahtari: ""
    property color vurguRengi: "#3b82f6"

    color: "#0a0a0d"
    radius: 10
    border.color: "#1e2a3f"
    border.width: 1
    clip: true

    readonly property var sahneler: ({
        navBaglanti: {
            w: 150, h: 260,
            ogeler: [
                { x: 35, y: 6, w: 80, h: 16, tip: "merkezMetin", metin: "LIYA", renk: "#c0392b", boyut: 11, kalin: true },
                { x: 8, y: 30, w: 28, h: 14, tip: "buton", metin: "TR", renk: "#12121a", kenar: "#3b82f6" },
                { x: 40, y: 30, w: 28, h: 14, tip: "buton", metin: "EN", renk: "#12121a", kenar: "#1e2a3f" },
                { x: 8, y: 53, w: 8, h: 8, tip: "nokta", renk: "#dc2626" },
                { x: 20, y: 51, w: 110, h: 12, tip: "etiket", metin: "Bağlı Değil" },
                { x: 8, y: 69, w: 8, h: 8, tip: "nokta", renk: "#dc2626" },
                { x: 20, y: 67, w: 120, h: 12, tip: "etiket", metin: "Sesli Komut: Kapalı" },
                { x: 8, y: 88, w: 134, h: 1, tip: "cizgi" },
                { x: 8, y: 98, w: 134, h: 24, tip: "navSatir", etiket: "Ölçüm", aktif: true },
                { x: 8, y: 126, w: 134, h: 24, tip: "navSatir", etiket: "Sonuçlar", aktif: false },
                { x: 8, y: 154, w: 134, h: 24, tip: "navSatir", etiket: "Geçmiş", aktif: false },
                { x: 8, y: 182, w: 134, h: 24, tip: "navSatir", etiket: "Kalibrasyon", aktif: false },
                { x: 8, y: 210, w: 134, h: 24, tip: "navSatir", etiket: "Rehber", aktif: false }
            ],
            bolgeler: { baglanti: { x: 5, y: 47, w: 130, h: 20 } }
        },
        olcum: {
            w: 460, h: 260,
            ogeler: [
                { x: 8, y: 6, w: 150, h: 10, tip: "etiket", metin: "TEST BİLGİLERİ" },
                { x: 8, y: 20, w: 154, h: 16, tip: "alan", metin: "Müşteri" },
                { x: 8, y: 40, w: 154, h: 16, tip: "alan", metin: "Reçete Seçin ▾" },
                { x: 8, y: 62, w: 130, h: 9, tip: "etiket", metin: "Eklenen Ağırlık (kg)" },
                { x: 8, y: 74, w: 50, h: 18, tip: "buton", metin: "+1.6", renk: "#1d4ed8", kenar: "#3b82f6" },
                { x: 62, y: 74, w: 52, h: 18, tip: "buton", metin: "− Geri", renk: "#78350f", kenar: "#92400e" },
                { x: 118, y: 74, w: 44, h: 18, tip: "buton", metin: "↺", renk: "#7f1d1d", kenar: "#991b1b" },
                { x: 8, y: 96, w: 154, h: 14, tip: "merkezMetin", metin: "0 x 1.6 = 0.0 kg", renk: "#6b7280", boyut: 7 },
                { x: 8, y: 114, w: 154, h: 20, tip: "buton", metin: "▶ Ölçümü Başlat", renk: "#15803d", kenar: "#22c55e" },
                { x: 8, y: 138, w: 75, h: 18, tip: "buton", metin: "⏸ Duraklat", renk: "#92400e", kenar: "#a16207" },
                { x: 87, y: 138, w: 75, h: 18, tip: "buton", metin: "■ Bitir", renk: "#991b1b", kenar: "#b91c1c" },
                { x: 8, y: 160, w: 154, h: 24, tip: "durumKart", solRenk: "#dc2626", solMetin: "VERİ YOK", sagMetin: "STROKE 0"
},
                { x: 8, y: 190, w: 130, h: 9, tip: "etiket", metin: "ANLIK DEĞERLER" },
                { x: 8, y: 202, w: 154, h: 54, tip: "izgara2x2", grid: [
                    { renk: "#3b82f6", etiket: "BASINÇ" }, { renk: "#9333ea", etiket: "KONUM" },
                    { renk: "#f59e0b", etiket: "HIZ" }, { renk: "#16a34a", etiket: "DEBİ" }
                ] },
                { x: 178, y: 6, w: 130, h: 118, tip: "grafikKart", renk: "#3b82f6", baslik: "Basınç-Zaman", birim: "mbar" },
                { x: 316, y: 6, w: 130, h: 118, tip: "grafikKart", renk: "#9333ea", baslik: "Konum-Zaman", birim: "mm" },
                { x: 178, y: 132, w: 130, h: 118, tip: "grafikKart", renk: "#f59e0b", baslik: "Hız-Zaman", birim: "m/s" },
                { x: 316, y: 132, w: 130, h: 118, tip: "grafikKart", renk: "#16a34a", baslik: "Debi-Zaman", birim: "m3/h" }
            ],
            bolgeler: {
                musteriRecete: { x: 5, y: 17, w: 160, h: 42 },
                baslat: { x: 5, y: 111, w: 160, h: 26 },
                agirlikDuraklat: { x: 5, y: 59, w: 160, h: 100 },
                bitir: { x: 84, y: 135, w: 81, h: 24 }
            }
        },
```

```
        onay: {
            w: 280, h: 170,
            ogeler: [
                { x: 16, y: 14, w: 248, h: 20, tip: "merkezMetin", metin: "Ölçümü bitirmek istediğinize emin misiniz?", renk:
"#9ca3af", boyut: 8 },
                { x: 16, y: 46, w: 248, h: 20, tip: "buton", metin: "💾 Kaydet ve Bitir", renk: "#15803d", kenar: "#22c55e" },
                { x: 16, y: 72, w: 248, h: 20, tip: "buton", metin: "🗑 Testi Sil ve Bitir", renk: "#991b1b", kenar: "#b91c1c" },
                { x: 16, y: 98, w: 248, h: 20, tip: "buton", metin: "Vazgeç, Teste Devam Et", renk: "#1a1a20", kenar: "#1e2a3f" }
            ],
            bolgeler: { secenekler: { x: 12, y: 42, w: 256, h: 80 } }
        },
        sonuclar: {
            w: 460, h: 300,
            ogeler: [
                { x: 8, y: 6, w: 150, h: 9, tip: "etiket", metin: "SONUÇ ÖZETİ" },
                { x: 8, y: 18, w: 154, h: 34, tip: "kart", baslik: "P-Q KESİŞİMİ (A)", altMetin: "—", renk: "#3b82f6" },
                { x: 8, y: 56, w: 154, h: 34, tip: "kart", baslik: "P-Q EĞİMİ (B)", altMetin: "—", renk: "#9333ea" },
                { x: 8, y: 94, w: 154, h: 34, tip: "kart", baslik: "UYUM KALİTESİ (R²)", altMetin: "—", renk: "#16a34a" },
                { x: 8, y: 136, w: 150, h: 9, tip: "etiket", metin: "BORU HATTI TAHMİNİ" },
                { x: 8, y: 148, w: 154, h: 16, tip: "alan", metin: "125" },
                { x: 8, y: 168, w: 154, h: 16, tip: "alan", metin: "Boru Uzunluğu (m)" },
                { x: 8, y: 188, w: 154, h: 18, tip: "buton", metin: "Tahmini Hesapla", renk: "#1d4ed8", kenar: "#3b82f6" },
                { x: 8, y: 210, w: 154, h: 20, tip: "buton", metin: "✓ POMPALANABİLİR", renk: "#14532d", kenar: "#22c55e" },
                { x: 8, y: 236, w: 130, h: 9, tip: "etiket", metin: "RAPOR" },
                { x: 8, y: 248, w: 154, h: 16, tip: "buton", metin: "📄 PDF Rapor Önizle", renk: "#1d4ed8", kenar: "#3b82f6" },
                { x: 8, y: 266, w: 154, h: 16, tip: "buton", metin: "📊 Excel (CSV) Önizle", renk: "#15803d", kenar: "#22c55e"
},
                { x: 8, y: 284, w: 154, h: 16, tip: "buton", metin: "📑 Excel (XML) Dışa Aktar", renk: "#1d4ed8", kenar:
"#3b82f6" },
                { x: 178, y: 6, w: 274, h: 170, tip: "grafikKart", renk: "#3b82f6", baslik: "P-Q Dağılım Grafiği", legend: true
},
                { x: 178, y: 184, w: 274, h: 110, tip: "tablo", baslik: "Stroke Tablosu" }
            ],
            bolgeler: {
                ozet: { x: 5, y: 15, w: 160, h: 116 },
                boru: { x: 5, y: 133, w: 160, h: 100 },
                disaAktar: { x: 5, y: 233, w: 160, h: 70 },
                grafikTablo: { x: 175, y: 3, w: 280, h: 294 }
            }
        },
        gecmis: {
            w: 420, h: 260,
            ogeler: [
                { x: 8, y: 6, w: 280, h: 18, tip: "alan", metin: "🔍 Müşteri veya reçete ara..." },
                { x: 296, y: 6, w: 80, h: 18, tip: "buton", metin: "Tüm Testler ▾", renk: "#12121a", kenar: "#1e2a3f" },
                { x: 384, y: 6, w: 28, h: 18, tip: "buton", metin: "⚙", renk: "#12121a", kenar: "#3b82f6" },
                { x: 8, y: 30, w: 200, h: 12, tip: "etiket", metin: "Özel Tarih:" },
                { x: 8, y: 50, w: 404, h: 32, tip: "kayitSatiri" },
                { x: 8, y: 86, w: 404, h: 32, tip: "kayitSatiri" },
                { x: 8, y: 122, w: 404, h: 32, tip: "kayitSatiri" },
                { x: 8, y: 158, w: 404, h: 32, tip: "kayitSatiri" }
            ],
            bolgeler: {
                arama: { x: 5, y: 3, w: 294, h: 24 },
                gelismis: { x: 381, y: 3, w: 34, h: 24 },
                goruntule: { x: 340, y: 47, w: 72, h: 38 },
                sil: { x: 296, y: 47, w: 36, h: 38 }
            }
        },
        kalibrasyonSecim: {
            w: 460, h: 190,
            ogeler: [
                { x: 8, y: 4, w: 300, h: 10, tip: "etiket", metin: "KALİBRASYON — Sensör Seçin" },
                { x: 8, y: 20, w: 142, h: 160, tip: "kart", baslik: "Eğim Sensörü", altMetin: "Yatay hizalama kalibrasyonu",
renk: "#14b8a6", rozet: "✓ Kalibre Edildi" },
                { x: 158, y: 20, w: 142, h: 160, tip: "kart", baslik: "Load Cell", altMetin: "Basınç sensörü kalibrasyonu", renk:
"#3b82f6", rozet: "✓ Kalibre Edildi" },
                { x: 308, y: 20, w: 142, h: 160, tip: "kart", baslik: "Mesafe Sensörü", altMetin: "Konum ölçümü kalibrasyonu",
renk: "#9333ea", rozet: "✓ Kalibre Edildi" }
            ],
```

```
            bolgeler: { kartlar: { x: 5, y: 17, w: 450, h: 166 } }
        },
        kalibrasyonEgim: {
            w: 300, h: 306,
            ogeler: [
                { x: 0, y: 6, w: 60, h: 14, tip: "buton", metin: "← Geri", renk: "#12121a", kenar: "#1e2a3f" },
                { x: 68, y: 8, w: 8, h: 8, tip: "nokta", renk: "#14b8a6" },
                { x: 80, y: 6, w: 200, h: 14, tip: "etiket", metin: "Eğim Sensörü Kalibrasyonu" },
                { x: 40, y: 32, w: 220, h: 12, tip: "merkezMetin", metin: "Son kalibrasyon: 21.08.2026", renk: "#6b7280", boyut:
7 },
                { x: 20, y: 52, w: 260, h: 16, tip: "merkezMetin", metin: "Cihazı düz ve sabit bir yüzeye yerleştirin", renk:
"#9ca3af", boyut: 7 },
                { x: 75, y: 74, w: 150, h: 150, tip: "seviye" },
                { x: 75, y: 232, w: 150, h: 28, tip: "kutu", renk: "#14b8a6", dolgu: "#0f2e2a", metin: "X: 0.0°   Y: 0.0°",
metinRenk: "#2dd4bf" },
                { x: 20, y: 266, w: 260, h: 10, tip: "merkezMetin", metin: "Cihaz bağlı değil — kalibrasyon kaydedilemez", renk:
"#f87171", boyut: 6 },
                { x: 30, y: 280, w: 240, h: 20, tip: "buton", metin: "Sıfırla (Zero)", renk: "#12121a", kenar: "#1e2a3f" }
            ],
            bolgeler: { sifirla: { x: 27, y: 277, w: 246, h: 26 } }
        },
        kalibrasyonNoktaSayisi: {
            w: 320, h: 170,
            ogeler: [
                { x: 0, y: 6, w: 60, h: 14, tip: "buton", metin: "← Geri", renk: "#12121a", kenar: "#1e2a3f" },
                { x: 20, y: 40, w: 280, h: 14, tip: "merkezMetin", metin: "Kaç noktalı kalibrasyon yapmak istersiniz?", renk:
"#dce8f5", boyut: 8, kalin: true },
                { x: 20, y: 58, w: 280, h: 20, tip: "merkezMetin", metin: "Daha fazla nokta, daha hassas kalibrasyon sağlar",
renk: "#6b7280", boyut: 6 },
                { x: 40, y: 90, w: 240, h: 40, tip: "noktaSecici" }
            ],
            bolgeler: { secim: { x: 37, y: 87, w: 246, h: 46 } }
        },
        kalibrasyonNoktaGir: {
            w: 420, h: 170,
            ogeler: [
                { x: 0, y: 4, w: 60, h: 14, tip: "buton", metin: "← Geri", renk: "#12121a", kenar: "#1e2a3f" },
                { x: 10, y: 26, w: 170, h: 10, tip: "merkezMetin", metin: "Nokta 1 / 8", renk: "#3b82f6", boyut: 8, kalin: true
},
                { x: 10, y: 40, w: 170, h: 4, tip: "kutu", renk: "#1e2a3f", dolgu: "#16161e" },
                { x: 10, y: 50, w: 170, h: 22, tip: "merkezMetin", metin: "Hedef ağırlığı gir, sonra o ağırlığı load cell'e koy",
renk: "#9ca3af", boyut: 6 },
                { x: 10, y: 76, w: 170, h: 18, tip: "alan", metin: "kg" },
                { x: 10, y: 98, w: 170, h: 26, tip: "kutu", renk: "#dc2626", dolgu: "#2a1414", metin: "Ham değer: 0", metinRenk:
"#f87171" },
                { x: 10, y: 130, w: 170, h: 20, tip: "buton", metin: "Bu Noktayı Kaydet", renk: "#12121a", kenar: "#1e2a3f" },
                { x: 200, y: 26, w: 210, h: 10, tip: "etiket", metin: "KAYDEDİLEN NOKTALAR" },
                { x: 200, y: 42, w: 210, h: 22, tip: "kart", baslik: "① 0.00 kg", altMetin: "ADC: 81541", renk: "#3b82f6" },
                { x: 200, y: 68, w: 210, h: 22, tip: "kart", baslik: "② 1.00 kg", altMetin: "ADC: 127417", renk: "#3b82f6" },
                { x: 200, y: 94, w: 210, h: 22, tip: "kart", baslik: "③ 2.00 kg", altMetin: "ADC: 183259", renk: "#3b82f6" }
            ],
            bolgeler: {
                kaydet: { x: 7, y: 127, w: 176, h: 26 },
                hedefAlan: { x: 7, y: 73, w: 176, h: 24 }
            }
        },
        kalibrasyonMevcut: {
            w: 360, h: 170,
            ogeler: [
                { x: 0, y: 4, w: 60, h: 14, tip: "buton", metin: "← Geri", renk: "#12121a", kenar: "#1e2a3f" },
                { x: 40, y: 24, w: 280, h: 12, tip: "merkezMetin", metin: "Bu sensör için kayıtlı bir kalibrasyon bulundu", renk:
"#dce8f5", boyut: 8, kalin: true },
                { x: 30, y: 44, w: 145, h: 110, tip: "kart", baslik: "👁 Mevcut Kalibrasyonu Görüntüle", altMetin: "Kayıtlı
noktaları incele", renk: "#3b82f6" },
                { x: 185, y: 44, w: 145, h: 110, tip: "kart", baslik: "↻ Yeniden Kalibre Et", altMetin: "Sıfırdan yeni ölçüm
al", renk: "#6b7280" }
            ],
            bolgeler: { kartlar: { x: 27, y: 41, w: 303, h: 116 } }
        }
    })
```

```
readonly property var sahne: sahneler[sahneAdi] || sahneler.olcum
readonly property var vurguKutu: (sahne.bolgeler && sahne.bolgeler[vurguAnahtari]) ? sahne.bolgeler[vurguAnahtari] : null

Item {
    id: disCerceve
    anchors.fill: parent
    anchors.margins: 8
    readonly property real olcek: Math.min(width / root.sahne.w, height / root.sahne.h, 1.4)

    Item {
        id: tuval
        width: root.sahne.w
        height: root.sahne.h
        scale: disCerceve.olcek
        transformOrigin: Item.TopLeft
        x: Math.round((disCerceve.width - width * disCerceve.olcek) / 2)
        y: Math.round((disCerceve.height - height * disCerceve.olcek) / 2)

        Repeater {
            model: root.sahne.ogeler

            Item {
                id: oge
                property var ov: modelData
                x: ov.x
                y: ov.y
                width: ov.w
                height: ov.h

                Rectangle {
                    anchors.fill: parent
                    visible: oge.ov.tip === "alan" || oge.ov.tip === "buton" || oge.ov.tip === "kart"
                    radius: oge.ov.tip === "buton" ? 5 : 4
                    color: oge.ov.tip === "buton" ? (oge.ov.renk || "#1a1a20")
                         : oge.ov.tip === "kart" ? Qt.darker(oge.ov.renk || "#3b82f6", 4.5)
                         : "#0a0a0d"
                    border.color: oge.ov.tip === "buton" ? (oge.ov.kenar || "#1e2a3f")
                                : oge.ov.tip === "kart" ? (oge.ov.renk || "#3b82f6")
                                : "#1e2a3f"
                    border.width: 1

                    Rectangle {
                        visible: oge.ov.tip === "kart"
                        anchors.top: parent.top
                        width: parent.width
                        height: 3
                        radius: 1.5
                        color: oge.ov.renk || "#3b82f6"
                    }

                    Text {
                        visible: oge.ov.tip === "alan" || oge.ov.tip === "buton"
                        anchors.left: parent.left
                        anchors.right: parent.right
                        anchors.verticalCenter: oge.ov.tip === "buton" ? parent.verticalCenter : undefined
                        anchors.top: oge.ov.tip === "alan" ? parent.top : undefined
                        anchors.topMargin: oge.ov.tip === "alan" ? 3 : 0
                        anchors.leftMargin: 6
                        anchors.rightMargin: 6
                        horizontalAlignment: oge.ov.tip === "buton" ? Text.AlignHCenter : Text.AlignLeft
                        text: oge.ov.metin || ""
                        color: oge.ov.tip === "buton" ? "#f0e9d8" : "#6b7280"
                        font.pixelSize: 8
                        font.bold: oge.ov.tip === "buton"
                        font.family: "Segoe UI"
                        wrapMode: Text.WordWrap
                    }

                    Column {
```

```
        visible: oge.ov.tip === "kart"
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.top: parent.top
        anchors.topMargin: 9
        anchors.leftMargin: 6
        anchors.rightMargin: 6
        spacing: 3

        Text {
            width: parent.width
            text: oge.ov.baslik || ""
            color: oge.ov.renk || "#dce8f5"
            font.pixelSize: 8
            font.bold: true
            font.family: "Segoe UI"
            wrapMode: Text.WordWrap
        }
        Text {
            visible: oge.ov.altMetin !== undefined
            width: parent.width
            text: oge.ov.altMetin || ""
            color: "#6b7280"
            font.pixelSize: 7
            font.family: "Segoe UI"
            wrapMode: Text.WordWrap
        }
    }

    Rectangle {
        visible: oge.ov.tip === "kart" && oge.ov.rozet !== undefined
        anchors.bottom: parent.bottom
        anchors.horizontalCenter: parent.horizontalCenter
        anchors.bottomMargin: 8
        width: rozetYazi.implicitWidth + 12
        height: 14
        radius: 7
        color: "#123321"
        Text {
            id: rozetYazi
            anchors.centerIn: parent
            text: oge.ov.rozet || ""
            color: "#4ade80"
            font.pixelSize: 6
            font.bold: true
            font.family: "Segoe UI"
        }
    }
}

Text {
    visible: oge.ov.tip === "etiket"
    text: oge.ov.metin || ""
    color: "#6b7280"
    font.pixelSize: 7
    font.bold: true
    font.letterSpacing: 0.6
    font.family: "Segoe UI"
}

Text {
    visible: oge.ov.tip === "merkezMetin"
    anchors.fill: parent
    text: oge.ov.metin || ""
    color: oge.ov.renk || "#9ca3af"
    font.pixelSize: oge.ov.boyut || 8
    font.bold: oge.ov.kalin === true
    font.family: "Segoe UI"
    wrapMode: Text.WordWrap
    horizontalAlignment: Text.AlignHCenter
```

```
                        verticalAlignment: Text.AlignVCenter
                    }

                    Rectangle {
                        visible: oge.ov.tip === "nokta"
                        anchors.fill: parent
                        radius: width / 2
                        color: oge.ov.renk || "#16a34a"
                    }

                    Rectangle {
                        visible: oge.ov.tip === "cizgi"
                        anchors.fill: parent
                        color: "#1e2a3f"
                    }

                    Rectangle {
                        visible: oge.ov.tip === "kutu"
                        anchors.fill: parent
                        radius: 5
                        color: oge.ov.dolgu || "#12121a"
                        border.color: oge.ov.renk || "#1e2a3f"
                        border.width: 1

                        Text {
                            visible: oge.ov.metin !== undefined
                            anchors.centerIn: parent
                            text: oge.ov.metin || ""
                            color: oge.ov.metinRenk || "#9ca3af"
                            font.pixelSize: 8
                            font.bold: true
                            font.family: "Segoe UI"
                        }
                    }

                    Rectangle {
                        visible: oge.ov.tip === "durumKart"
                        anchors.fill: parent
                        radius: 5
                        color: "#0a0a0d"
                        border.color: "#1e2a3f"
                        border.width: 1

                        Row {
                            anchors.left: parent.left
                            anchors.verticalCenter: parent.verticalCenter
                            anchors.leftMargin: 8
                            spacing: 6
                            Rectangle { width: 8; height: 8; radius: 4; anchors.verticalCenter: parent.verticalCenter; color:
oge.ov.solRenk || "#dc2626" }
                            Text { text: oge.ov.solMetin || ""; color: "#dce8f5"; font.pixelSize: 7; font.bold: true;
font.family: "Segoe UI" }
                        }

                        Text {
                            anchors.right: parent.right
                            anchors.verticalCenter: parent.verticalCenter
                            anchors.rightMargin: 8
                            text: oge.ov.sagMetin || ""
                            color: "#4f8cf7"
                            font.pixelSize: 7
                            font.bold: true
                            font.family: "Segoe UI"
                        }
                    }

                    Grid {
                        visible: oge.ov.tip === "izgara2x2"
                        anchors.fill: parent
                        columns: 2
```

```
                        rows: 2
                        spacing: 6
                        Repeater {
                            model: oge.ov.grid || []
                            Rectangle {
                                width: (oge.width - 6) / 2
                                height: (oge.height - 6) / 2
                                radius: 3
                                color: "#0a0a0d"
                                border.color: "#1e2a3f"
                                border.width: 1

                                Rectangle {
                                    anchors.left: parent.left
                                    anchors.top: parent.top
                                    anchors.bottom: parent.bottom
                                    width: 2
                                    color: modelData.renk
                                }
                                Column {
                                    anchors.left: parent.left
                                    anchors.verticalCenter: parent.verticalCenter
                                    anchors.leftMargin: 6
                                    spacing: 1
                                    Text { text: modelData.etiket; color: "#6b7280"; font.pixelSize: 6; font.family: "Segoe UI" }
                                    Text { text: "--"; color: "#dce8f5"; font.pixelSize: 8; font.bold: true; font.family: "Segoe
UI" }
                                }
                            }
                        }
                    }

                    Rectangle {
                        visible: oge.ov.tip === "grafikKart"
                        anchors.fill: parent
                        radius: 6
                        color: "#12121a"
                        border.color: "#1e2a3f"
                        border.width: 1

                        Row {
                            anchors.top: parent.top
                            anchors.left: parent.left
                            anchors.right: parent.right
                            anchors.margins: 6
                            spacing: 5

                            Rectangle { width: 6; height: 6; radius: 3; anchors.verticalCenter: parent.verticalCenter; color:
oge.ov.renk || "#3b82f6" }
                            Text { text: oge.ov.baslik || ""; color: "#dce8f5"; font.pixelSize: 7; font.bold: true; font.family:
"Segoe UI" }
                        }

                        Row {
                            visible: oge.ov.legend === true
                            anchors.top: parent.top
                            anchors.right: parent.right
                            anchors.margins: 6
                            spacing: 8
                            Row {
                                spacing: 3
                                Rectangle { width: 5; height: 5; radius: 2.5; anchors.verticalCenter: parent.verticalCenter;
color: oge.ov.renk || "#3b82f6" }
                                Text { text: "Ölçüm"; color: "#6b7280"; font.pixelSize: 6; font.family: "Segoe UI" }
                            }
                            Row {
                                spacing: 3
                                Rectangle { width: 8; height: 2; anchors.verticalCenter: parent.verticalCenter; color: "#4f8cf7"
}
                                Text { text: "Regresyon"; color: "#6b7280"; font.pixelSize: 6; font.family: "Segoe UI" }
```

```
        }
    }

    Text {
        visible: oge.ov.legend !== true
        anchors.top: parent.top
        anchors.right: parent.right
        anchors.margins: 6
        text: "-- " + (oge.ov.birim || "")
        color: oge.ov.renk || "#3b82f6"
        font.pixelSize: 6
        font.family: "Segoe UI"
    }

    Rectangle {
        anchors.left: parent.left
        anchors.right: parent.right
        anchors.bottom: parent.bottom
        anchors.margins: 10
        height: parent.height - 26
        color: "transparent"
        border.color: "#1a2436"
        border.width: 1

        Rectangle {
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.leftMargin: 4
            anchors.rightMargin: 4
            anchors.bottomMargin: 4
            height: 1
            rotation: -12
            transformOrigin: Item.Left
            color: "#4f8cf7"
            opacity: 0.7
        }

        Repeater {
            model: 4
            Rectangle {
                readonly property real oran: index / 3
                x: 8 + oran * (parent.width - 16)
                y: parent.height - 8 - oran * (parent.height * 0.55) - (index % 2) * 6
                width: 4
                height: 4
                radius: 2
                color: oge.ov.renk || "#3b82f6"
            }
        }
    }
}

Rectangle {
    visible: oge.ov.tip === "tablo"
    anchors.fill: parent
    radius: 6
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    Column {
        anchors.fill: parent
        anchors.margins: 8
        spacing: 6

        Text {
            visible: oge.ov.baslik !== undefined
            text: oge.ov.baslik || ""
            color: "#dce8f5"
```

```
            font.pixelSize: 7
            font.bold: true
            font.family: "Segoe UI"
        }

        Repeater {
            model: 3
            Row {
                spacing: 6
                Rectangle { width: 34; height: 6; radius: 2; color: "#1e2a3f" }
                Rectangle { width: 60; height: 6; radius: 2; color: "#1e2a3f" }
                Rectangle {
                    width: 40; height: 10; radius: 5
                    color: index === 1 ? "#2a1414" : "#123321"
                    Text {
                        anchors.centerIn: parent
                        text: index === 1 ? "Hatalı" : "Geçerli"
                        color: index === 1 ? "#f87171" : "#4ade80"
                        font.pixelSize: 6
                    }
                }
            }
        }
    }
}

Rectangle {
    visible: oge.ov.tip === "kayitSatiri"
    anchors.fill: parent
    radius: 6
    color: "#12121a"
    border.color: "#1e2a3f"
    border.width: 1

    Row {
        anchors.left: parent.left
        anchors.verticalCenter: parent.verticalCenter
        anchors.leftMargin: 8
        spacing: 8

        Rectangle {
            width: 18; height: 18; radius: 9
            color: "#16161e"
            border.color: "#3b82f6"
            border.width: 1
            anchors.verticalCenter: parent.verticalCenter
        }

        Column {
            anchors.verticalCenter: parent.verticalCenter
            spacing: 4
            Rectangle { width: 70; height: 6; radius: 2; color: "#2a3548" }
            Rectangle { width: 46; height: 5; radius: 2; color: "#1e2a3f" }
        }
    }

    Row {
        anchors.right: parent.right
        anchors.verticalCenter: parent.verticalCenter
        anchors.rightMargin: 8
        spacing: 6

        Rectangle {
            width: 18; height: 18; radius: 4
            color: "#1a1420"
            border.color: "#3f1d24"
            border.width: 1
        }
        Rectangle {
            width: 34; height: 18; radius: 4
```

```
                        color: "#16161e"
                        border.color: "#3b82f6"
                        border.width: 1
                    }
                }
            }

            Rectangle {
                visible: oge.ov.tip === "navSatir"
                anchors.fill: parent
                radius: 5
                color: oge.ov.aktif ? "#16233c" : "transparent"

                Rectangle {
                    visible: oge.ov.aktif === true
                    anchors.left: parent.left
                    anchors.top: parent.top
                    anchors.bottom: parent.bottom
                    width: 2
                    color: "#3b82f6"
                }

                Row {
                    anchors.left: parent.left
                    anchors.verticalCenter: parent.verticalCenter
                    anchors.leftMargin: 8
                    spacing: 7
                    Rectangle { width: 8; height: 8; radius: 2; anchors.verticalCenter: parent.verticalCenter; color:
oge.ov.aktif ? "#3b82f6" : "#4b5563" }
                    Text { text: oge.ov.etiket || ""; color: oge.ov.aktif ? "#dce8f5" : "#6b7280"; font.pixelSize: 7;
font.bold: oge.ov.aktif === true; font.family: "Segoe UI" }
                }
            }

            Rectangle {
                visible: oge.ov.tip === "seviye"
                anchors.fill: parent
                radius: width / 2
                color: "transparent"
                border.color: "#1e2a3f"
                border.width: 1

                Rectangle { anchors.centerIn: parent; width: parent.width - 2; height: 1; color: "#1e2a3f" }
                Rectangle { anchors.centerIn: parent; width: 1; height: parent.height - 2; color: "#1e2a3f" }
                Rectangle { anchors.centerIn: parent; width: 12; height: 12; radius: 6; color: "#16a34a" }
            }

            Row {
                visible: oge.ov.tip === "noktaSecici"
                anchors.fill: parent
                spacing: 6
                Repeater {
                    model: ["4", "6", "8", "10"]
                    Rectangle {
                        width: (oge.width - 18) / 4
                        height: oge.height
                        radius: 5
                        color: index === 1 ? "#16161e" : "#0a0a0d"
                        border.color: index === 1 ? "#3b82f6" : "#1e2a3f"
                        border.width: 1
                        Text {
                            anchors.centerIn: parent
                            text: modelData
                            color: index === 1 ? "#3b82f6" : "#6b7280"
                            font.pixelSize: 9
                            font.bold: index === 1
                            font.family: "Segoe UI"
                        }
                    }
                }
```

```
                    }
                }
            }

            Rectangle {
                id: vurgu
                visible: root.vurguKutu !== null
                x: root.vurguKutu ? root.vurguKutu.x : 0
                y: root.vurguKutu ? root.vurguKutu.y : 0
                width: root.vurguKutu ? root.vurguKutu.w : 0
                height: root.vurguKutu ? root.vurguKutu.h : 0
                radius: 7
                color: Qt.rgba(root.vurguRengi.r, root.vurguRengi.g, root.vurguRengi.b, 0.14)
                border.color: root.vurguRengi
                border.width: 1.6

                SequentialAnimation {
                    running: vurgu.visible
                    loops: Animation.Infinite
                    NumberAnimation { target: vurgu; property: "border.width"; to: 2.6; duration: 700; easing.type:
Easing.InOutQuad }
                    NumberAnimation { target: vurgu; property: "border.width"; to: 1.4; duration: 700; easing.type:
Easing.InOutQuad }
                }
            }
        }
    }
}
```

qml\pages\RehberPage.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Layouts 6.7
import sliper

Rectangle {
    color: "#0a0e17"

    readonly property var metinler: ({
        baslik: { tr: "REHBER — SLIPER Nasıl Kullanılır", en: "GUIDE — How to Use SLIPER" },
        geriButon: { tr: "← Geri", en: "← Back" },
        adimSonEk: { tr: " Adım", en: " Steps" }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    readonly property color konuRengi: "#ef4444"

    readonly property var bolumler: [
        {
            renk: "#ef4444",
            baslikTr: "Ölçüm Nasıl Başlatılır",
            baslikEn: "How to Start a Measurement",
            ozetTr: "Yeni bir test için müşteri bilgisi, reçete ve cihaz bağlantısı",
            ozetEn: "Customer info, mix design and device connection for a new test",
            adimlar: [
                {
                    baslikTr: "Müşteri ve reçete bilgisini girin",
                    baslikEn: "Enter the customer and mix design",
                    metinTr: "Sol menüden Ölçüm sayfasını açın. \"Ölçüm Yeri\" ve \"Müşteri\" alanlarını doldurup \"Beton
Reçetesi\" listesinden ilgili reçeteyi seçin.",
                    metinEn: "Open the Measurement page from the left menu. Fill in \"Place of Measure\" and \"Customer\", then
select the recipe from the \"Concrete Mix Design\" list.",
                    mockup: { tur: "olcum", bolge: "musteriRecete" }
                },
                {
                    baslikTr: "Cihaz bağlantısını kontrol edin",
                    baslikEn: "Check the device connection",
                    metinTr: "Sol alt köşedeki bağlantı göstergesi yeşilse SLIPER-ESP32 bağlıdır. Bağlı değilse üzerine
tıklayarak bağlanmayı deneyin.",
                    metinEn: "If the connection indicator in the bottom-left corner is green, the SLIPER-ESP32 is connected. If
not, click it to attempt a connection.",
                    mockup: { tur: "navBaglanti", bolge: "baglanti" }
                },
                {
                    baslikTr: "\"▶ Ölçümü Başlat\" butonuna basın",
                    baslikEn: "Press \"▶ Start Measurement\"",
                    metinTr: "Tüm bilgiler girildikten sonra bu butonla ölçüm başlar; anlık basınç, konum, hız ve debi değerleri
ekranda canlı olarak izlenir.",
                    metinEn: "Once all information is filled in, this button starts the measurement; live pressure, position,
speed and flow rate values appear on screen.",
                    mockup: { tur: "olcum", bolge: "baslat" }
                },
                {
                    baslikTr: "Ağırlık ekleyin, gerekirse duraklatın",
                    baslikEn: "Add weight, pause if needed",
                    metinTr: "Boruyu üst konuma kaldırıp kilitleyin, \"● Hazırla\"ya basın ve HAZIR görününce serbest bırakın:
durağan basınç (P0l) hazırlama sırasında alınır, stroke otomatik algılanır. Her stroke'tan sonra yeniden hazırlamak gerekir.
Üstteki \"SON STROKE\" çubuğunda Pmax, P0l, P0r, p ve Q gösterilir. Her yük için en az 3 stroke atın; yükü \"+1.6\" / \"+4.8\"
butonlarıyla girin (her stroke o anki ağırlıkla kaydedilir). \"⏸ Duraklat\" stroke algılamayı durdurur, canlı veri akmaya devam
eder.",
                    metinEn: "Lift the pipe to the top position, lock it and release it: the stroke is detected automatically and
the \"LAST STROKE\" bar shows Pmax, P0l, P0r, p and Q. Do at least 3 strokes per load; enter the load with \"+1.6\" / \"+4.8\"
(each stroke is saved with the current weight). \"⏸ Pause\" stops stroke detection while live data keeps flowing.",
                    mockup: { tur: "olcum", bolge: "agirlikDuraklat" }
                }
```

```
            ]
        },
        {
            renk: "#ef4444",
            baslikTr: "Test Nasıl Bitirilir",
            baslikEn: "How to Finish a Test",
            ozetTr: "Ölçümü kaydetme veya silme",
            ozetEn: "Saving or discarding the measurement",
            adimlar: [
                {
                    baslikTr: "\"■ Bitir\" butonuna basın",
                    baslikEn: "Press \"■ Finish\"",
                    metinTr: "Ölçüm tamamlandığında bu butona basın. Test aktif değilse veya cihaz bağlı değilse önce ölçümü
başlatmanız istenir.",
                    metinEn: "Press this button once the measurement is complete. If no test is active or the device is not
connected, you'll be asked to start a measurement first.",
                    mockup: { tur: "olcum", bolge: "bitir" }
                },
                {
                    baslikTr: "Onay penceresinden seçim yapın",
                    baslikEn: "Choose an option in the confirmation dialog",
                    metinTr: "\"Ölçümü bitirmek istediğinize emin misiniz?\" sorusuna karşılık \"💾 Kaydet ve Bitir\", \"🗑 Testi
Sil ve Bitir\" veya \"Vazgeç, Teste Devam Et\" seçeneklerinden birini seçin.",
                    metinEn: "In response to \"Are you sure you want to finish the measurement?\", choose \"💾 Save and
Finish\", \"🗑 Delete Test and Finish\", or \"Cancel, Continue Testing\".",
                    mockup: { tur: "onay", bolge: "secenekler" }
                },
                {
                    baslikTr: "Sonuçlar sayfasına yönlendirilirsiniz",
                    baslikEn: "You are taken to the Results page",
                    metinTr: "Testi kaydettiğinizde uygulama otomatik olarak Sonuçlar sayfasını açar ve hesaplanan değerleri
gösterir.",
                    metinEn: "When you save the test, the app automatically opens the Results page and shows the calculated
values.",
                    mockup: { tur: "sonuclar", bolge: "ozet" }
                }
            ]
        },
        {
            renk: "#ef4444",
            baslikTr: "Sonuçlar Nasıl Görüntülenir",
            baslikEn: "How to View Results",
            ozetTr: "Akma gerilmesi, viskozite, grafikler ve raporlar",
            ozetEn: "Yield stress, viscosity, charts and reports",
            adimlar: [
                {
                    baslikTr: "Sonuç özetini inceleyin",
                    baslikEn: "Review the result summary",
                    metinTr: "\"SONUÇ ÖZETİ\" kartında P–Q doğrusunun kesişimi (A, mbar), eğimi (B, mbar·h/m³) ve uyum kalitesi
(R²) listelenir. A yağlama tabakasının akma direncini, B viskoz direncini gösterir.",
                    metinEn: "The \"RESULT SUMMARY\" card lists the P–Q line intercept (A, mbar), slope (B, mbar·h/m³), and fit
quality (R²). A reflects the yield resistance of the lubricating layer, B its viscous resistance.",
                    mockup: { tur: "sonuclar", bolge: "ozet" }
                },
                {
                    baslikTr: "Boru hattı tahminini hesaplayın",
                    baslikEn: "Calculate the pipeline estimate",
                    metinTr: "\"TAHMİN AYARLARI\" bölümüne şantiyedeki hattı girin: boru iç çapı, hedef debi, hat uzunluğu,
pompalama yüksekliği (aşağı pompalamada negatif), 90° ve 45° dirsek sayıları ve pompanın maksimum basıncı. \"Tahmini Hesapla ve
Kaydet\"e basın. Gereken basınç pompa kapasitesi içindeyse sonuç \"POMPALANABİLİR\", değilse \"POMPALAMA SORUNU\" olarak
gösterilir.",
                    metinEn: "In \"FORECAST PREFERENCES\" enter the site pipeline: pipe inner diameter, target flow, line length,
pumping height (negative for downward pumping), number of 90° and 45° bends and the pump max. pressure. Press \"Calculate and
Save Forecast\". If the required pressure is within the pump capacity the result is \"PUMPABLE\", otherwise \"PUMPING ISSUE\".",
                    mockup: { tur: "sonuclar", bolge: "boru" }
                },
                {
                    baslikTr: "Grafik ve stroke tablosunu inceleyin",
                    baslikEn: "Review the chart and stroke table",
                    metinTr: "Sağdaki sekmeler: \"P–Q Grafiği\" (noktalar ve regresyon), \"Stroke Tablosu\" (kutucukla stroke'u
```

```
tahmine dahil et/çıkar, 🗑 ile sil; hatalı stroke'lar \"Wrong Stroke\" olarak işaretlenir), \"Stroke Eğrileri\" (her strokun basınç ve mesafe/hız eğrisi), \"Tahmin Tablosu\" ve \"Tahmin Grafiği\".",
                    metinEn: "Tabs on the right: \"p–Q Chart\" (points and regression), \"Table of Strokes\" (include/exclude a stroke with the checkbox, delete with 🗑; invalid strokes are marked \"Wrong Stroke\"), \"Stroke Curves\" (pressure and distance/speed curve of each stroke), \"Forecast Table\" and \"Forecast Chart\".",
                    mockup: { tur: "sonuclar", bolge: "grafikTablo" }
                },
                {
                    baslikTr: "Rapor olarak dışa aktarın",
                    baslikEn: "Export as a report",
                    metinTr: "\"📄 PDF Rapor Önizle\" veya \"📊 Excel (CSV) Önizle\" ile raporu önizleyebilir, \"📑 Excel (XML) Dışa Aktar\" ile doğrudan dosyaya kaydedebilirsiniz.",
                    metinEn: "Preview the report with \"📄 Preview PDF Report\" or \"📊 Preview Excel (CSV)\", or save it directly to a file with \"📑 Export Excel (XML)\".",
                    mockup: { tur: "sonuclar", bolge: "disaAktar" }
                }
            ]
        },
        {
            renk: "#ef4444",
            baslikTr: "Geçmiş Nasıl İncelenir",
            baslikEn: "How to Review History",
            ozetTr: "Eski testleri arama, filtreleme ve yönetme",
            ozetEn: "Searching, filtering and managing past tests",
            adimlar: [
                {
                    baslikTr: "Kayıtlarda arama ve filtreleme yapın",
                    baslikEn: "Search and filter records",
                    metinTr: "Sol menüden Geçmiş sayfasını açın. Üstteki arama kutusuna müşteri veya reçete adı yazabilir, \"Tüm Testler\", \"Son 7 Gün\", \"Son 30 Gün\" veya \"Özel Tarih\" filtrelerini kullanabilirsiniz.",
                    metinEn: "Open the History page from the left menu. Type a customer or mix design name into the search box at the top, and use the \"All Tests\", \"Last 7 Days\", \"Last 30 Days\" or \"Custom Date\" filters.",
                    mockup: { tur: "gecmis", bolge: "arama" }
                },
                {
                    baslikTr: "Bir testi görüntüleyin",
                    baslikEn: "View a test",
                    metinTr: "İlgili kaydın yanındaki \"Görüntüle\" butonuna basarak o testin Sonuçlar sayfasını açabilirsiniz.",
                    metinEn: "Press the \"View\" button next to a record to open that test's Results page.",
                    mockup: { tur: "gecmis", bolge: "goruntule" }
                },
                {
                    baslikTr: "Bir kaydı silin",
                    baslikEn: "Delete a record",
                    metinTr: "\"Kaydı Sil\" seçeneğiyle şifre girerek istenmeyen bir testi kalıcı olarak silebilirsiniz. Bu işlem geri alınamaz.",
                    metinEn: "Use \"Delete Record\" and enter the password to permanently delete an unwanted test. This action cannot be undone.",
                    mockup: { tur: "gecmis", bolge: "sil" }
                },
                {
                    baslikTr: "Veritabanını yedekleyin veya geri yükleyin",
                    baslikEn: "Back up or restore the database",
                    metinTr: "\"Gelişmiş Ayarlar\" bölümünden veritabanını dışa aktararak yedekleyebilir, bir .db dosyasından içe aktararak geri yükleyebilirsiniz. Sensör kalibrasyonları bu işlemlerden etkilenmez.",
                    metinEn: "From \"Advanced Settings\" you can export the database as a backup or restore it by importing a .db file. Sensor calibrations are not affected by these operations.",
                    mockup: { tur: "gecmis", bolge: "gelismis" }
                }
            ]
        },
        {
            renk: "#ef4444",
            baslikTr: "Kalibrasyon Nasıl Yapılır",
            baslikEn: "How to Calibrate Sensors",
            ozetTr: "Eğim, Load Cell ve Mesafe sensörlerinin kalibrasyonu",
            ozetEn: "Calibrating the incline, load cell and distance sensors",
            adimlar: [
                {
                    baslikTr: "Kalibre edilecek sensörü seçin",
```

```
                    baslikEn: "Select the sensor to calibrate",
                    metinTr: "Sol menüden Kalibrasyon sayfasını açın ve Eğim Sensörü, Load Cell, Mesafe Sensörü veya Stroke Konumları kartlarından birine tıklayın. Stroke Konumları'nda borunun üst ve alt konumunu kaydedin; stroke'lar bu referanslarla algılanır.",
                    metinEn: "Open the Calibration page from the left menu and click Incline Sensor, Load Cell, Distance Sensor or Stroke Positions. In Stroke Positions save the top and bottom positions of the pipe; strokes are detected using these references.",
                    mockup: { tur: "kalibrasyonSecim", bolge: "kartlar" }
                },
                {
                    baslikTr: "Eğim sensörünü sıfırlayın",
                    baslikEn: "Zero the incline sensor",
                    metinTr: "Cihazı düz ve sabit bir yüzeye yerleştirin, su terazisi göstergesinin ortalanmasını bekleyin ve \"Sıfırla (Zero)\" butonuna basın.",
                    metinEn: "Place the device on a flat, stable surface, wait for the level indicator to center, then press \"Zero (Reset)\".",
                    mockup: { tur: "kalibrasyonEgim", bolge: "sifirla" }
                },
                {
                    baslikTr: "Load Cell / Mesafe sensörü için nokta sayısını seçin",
                    baslikEn: "Choose the point count for the Load Cell / Distance sensor",
                    metinTr: "Kaç noktalı kalibrasyon yapılacağını (4, 6, 8 veya 10) seçin — daha fazla nokta daha hassas bir kalibrasyon sağlar.",
                    metinEn: "Choose how many calibration points to use (4, 6, 8 or 10) — more points give a more precise calibration.",
                    mockup: { tur: "kalibrasyonNoktaSayisi", bolge: "secim" }
                },
                {
                    baslikTr: "Her nokta için hedef değeri girip kaydedin",
                    baslikEn: "Enter and save the target value for each point",
                    metinTr: "Hedef ağırlık veya mesafeyi girin, o değeri cihaza uygulayın ve noktayı kaydedin. Son noktadan sonra \"Kaydet ve Bitir\" ile kalibrasyon tamamlanır.",
                    metinEn: "Enter the target weight or distance, apply that value on the device, and save the point. After the last point, calibration is completed with \"Save and Finish\".",
                    mockup: { tur: "kalibrasyonNoktaGir", bolge: "kaydet" }
                },
                {
                    baslikTr: "Mevcut kalibrasyonu görüntüleyin veya yenileyin",
                    baslikEn: "View or refresh an existing calibration",
                    metinTr: "Daha önce kaydedilmiş bir kalibrasyon varsa \"Mevcut Kalibrasyonu Görüntüle\" ile noktaları inceleyebilir veya \"Yeniden Kalibre Et\" ile sıfırdan yeni bir ölçüm alabilirsiniz.",
                    metinEn: "If a calibration was already saved, use \"View Current Calibration\" to review the points, or \"Recalibrate\" to take a fresh measurement from scratch.",
                    mockup: { tur: "kalibrasyonMevcut", bolge: "kartlar" }
                }
            ]
        }
    ]

    property int seciliKonu: -1

    Component {
        id: rehberMockupBileseni
        RehberMockup {}
    }

    Component {
        id: konuIkonu

        Canvas {
            property int konu: -1
            property color cizgiRengi: "#6b7280"
            onCizgiRengiChanged: requestPaint()
            onKonuChanged: requestPaint()
            Component.onCompleted: requestPaint()

            onPaint: {
                var ctx = getContext("2d")
                ctx.clearRect(0, 0, width, height)
                ctx.strokeStyle = cizgiRengi
```

```
                ctx.fillStyle = cizgiRengi
                ctx.lineWidth = width / 11
                ctx.lineCap = "round"
                ctx.lineJoin = "round"

                var k = width / 20

                if (konu === 0) {
                    ctx.fillRect(3 * k, 12 * k, 3.4 * k, 5 * k)
                    ctx.fillRect(8.3 * k, 8 * k, 3.4 * k, 9 * k)
                    ctx.fillRect(13.6 * k, 3.5 * k, 3.4 * k, 13.5 * k)
                } else if (konu === 1) {
                    ctx.beginPath(); ctx.arc(10 * k, 10 * k, 8 * k, 0, Math.PI * 2, false); ctx.stroke()
                    ctx.beginPath(); ctx.moveTo(6 * k, 10.3 * k); ctx.lineTo(8.8 * k, 13 * k); ctx.lineTo(14 * k, 6.8 * k);
ctx.stroke()
                } else if (konu === 2) {
                    ctx.beginPath()
                    ctx.moveTo(3 * k, 15.5 * k); ctx.lineTo(8 * k, 10.5 * k); ctx.lineTo(12 * k, 12.5 * k); ctx.lineTo(17 * k,
4.5 * k)
                    ctx.stroke()
                    ;[[3, 15.5], [8, 10.5], [12, 12.5], [17, 4.5]].forEach(function(p) {
                        ctx.beginPath(); ctx.arc(p[0] * k, p[1] * k, 1.3 * k, 0, Math.PI * 2, false); ctx.fill()
                    })
                } else if (konu === 3) {
                    ctx.beginPath(); ctx.arc(10 * k, 10.5 * k, 7.2 * k, 0, Math.PI * 2, false); ctx.stroke()
                    ctx.beginPath(); ctx.moveTo(10 * k, 10.5 * k); ctx.lineTo(10 * k, 5.8 * k); ctx.stroke()
                    ctx.beginPath(); ctx.moveTo(10 * k, 10.5 * k); ctx.lineTo(13.2 * k, 12.3 * k); ctx.stroke()
                } else {
                    ;[5.5, 10, 14.5].forEach(function(y, i) {
                        ctx.beginPath(); ctx.moveTo(2.5 * k, y * k); ctx.lineTo(17.5 * k, y * k); ctx.stroke()
                        var knobX = [7.5, 12.5, 6.5][i]
                        ctx.beginPath(); ctx.arc(knobX * k, y * k, 2.1 * k, 0, Math.PI * 2, false); ctx.fill()
                    })
                }
            }
        }
    }

    Component {
        id: konuKartBileseni

        Rectangle {
            id: konuKarti
            property int gercekIndex: 0
            readonly property var modelData: bolumler[gercekIndex]
            width: 280
            height: 300
            radius: 12
            color: "#0f1420"
            border.color: konuKarti.hovered ? modelData.renk : "#1e2a3f"
            border.width: 1
            property bool hovered: false
            scale: hovered ? 1.02 : 1.0
            Behavior on border.color { ColorAnimation { duration: 120 } }
            Behavior on scale { NumberAnimation { duration: 120; easing.type: Easing.OutQuad } }

            Column {
                anchors.centerIn: parent
                width: parent.width - 28
                spacing: 16

                Loader {
                    anchors.horizontalCenter: parent.horizontalCenter
                    width: 30
                    height: 30
                    sourceComponent: konuIkonu
                    onLoaded: {
                        item.konu = konuKarti.gercekIndex
                        item.cizgiRengi = Qt.binding(function() {
                            return konuKarti.hovered ? modelData.renk : "#9ca3af"
```

```
                    })
                }
            }

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                text: Translations.turkish ? modelData.baslikTr : modelData.baslikEn
                color: "#dce8f5"
                font.family: "Segoe UI"
                font.pixelSize: 15
                font.bold: true
                wrapMode: Text.WordWrap
            }

            Text {
                width: parent.width
                horizontalAlignment: Text.AlignHCenter
                text: Translations.turkish ? modelData.ozetTr : modelData.ozetEn
                color: "#6b7280"
                font.family: "Segoe UI"
                font.pixelSize: 12
                wrapMode: Text.WordWrap
                lineHeight: 1.2
            }

            Rectangle {
                anchors.horizontalCenter: parent.horizontalCenter
                width: adimSayisiYazi.implicitWidth + 16
                height: 20
                radius: 10
                color: "#1e2a3f"

                Text {
                    id: adimSayisiYazi
                    anchors.centerIn: parent
                    text: modelData.adimlar.length + txt("adimSonEk")
                    color: modelData.renk
                    font.pixelSize: 10
                    font.bold: true
                }
            }
        }

        MouseArea {
            anchors.fill: parent
            hoverEnabled: true
            cursorShape: Qt.PointingHandCursor
            onEntered: konuKarti.hovered = true
            onExited: konuKarti.hovered = false
            onClicked: seciliKonu = konuKarti.gercekIndex
        }
    }
}

StackLayout {
    anchors.fill: parent
    currentIndex: seciliKonu === -1 ? 0 : 1

    ScrollView {
        id: konularScroll
        clip: true
        ScrollBar.horizontal.policy: ScrollBar.AlwaysOff

        Column {
            width: konularScroll.availableWidth - 40
            x: 20
            spacing: 20
            topPadding: 20
            bottomPadding: 20
```

```
        Text {
            text: txt("baslik")
            color: "#6b7280"
            font.family: "Segoe UI"
            font.pixelSize: 12
            font.bold: true
            font.letterSpacing: 1
        }

        Column {
            width: parent.width
            spacing: 18

            Row {
                anchors.horizontalCenter: parent.horizontalCenter
                spacing: 18

                Repeater {
                    model: 3

                    Loader {
                        sourceComponent: konuKartBileseni
                        onLoaded: item.gercekIndex = index
                    }
                }
            }

            Row {
                anchors.horizontalCenter: parent.horizontalCenter
                spacing: 18

                Repeater {
                    model: 2

                    Loader {
                        sourceComponent: konuKartBileseni
                        onLoaded: item.gercekIndex = index + 3
                    }
                }
            }
        }
    }
}

ScrollView {
    id: detayScroll
    clip: true
    ScrollBar.horizontal.policy: ScrollBar.AlwaysOff

    Column {
        width: detayScroll.availableWidth - 40
        x: 20
        spacing: 18
        topPadding: 20
        bottomPadding: 24

        Row {
            spacing: 12

            Button {
                text: txt("geriButon")
                font.pixelSize: 13

                background: Rectangle {
                    radius: 6
                    color: parent.hovered ? "#1e2a3f" : "#0f1420"
                    border.color: "#1e2a3f"
                    border.width: 1
                    Behavior on color { ColorAnimation { duration: 120 } }
```

```
                        }

                        contentItem: Text {
                            text: parent.text
                            color: "#dce8f5"
                            font: parent.font
                            horizontalAlignment: Text.AlignHCenter
                            verticalAlignment: Text.AlignVCenter
                            leftPadding: 10
                            rightPadding: 10
                        }

                        onClicked: seciliKonu = -1
                    }

                    Rectangle {
                        anchors.verticalCenter: parent.verticalCenter
                        width: 8
                        height: 8
                        radius: 4
                        color: seciliKonu >= 0 ? bolumler[seciliKonu].renk : "#6b7280"
                    }

                    Text {
                        anchors.verticalCenter: parent.verticalCenter
                        text: seciliKonu >= 0 ? (Translations.turkish ? bolumler[seciliKonu].baslikTr :
bolumler[seciliKonu].baslikEn) : ""
                        color: "#dce8f5"
                        font.family: "Segoe UI"
                        font.pixelSize: 16
                        font.bold: true
                    }
                }

                Text {
                    visible: seciliKonu >= 0
                    width: parent.width
                    text: seciliKonu >= 0 ? (Translations.turkish ? bolumler[seciliKonu].ozetTr : bolumler[seciliKonu].ozetEn) :
""
                    color: "#9ca3af"
                    font.family: "Segoe UI"
                    font.pixelSize: 13
                    wrapMode: Text.WordWrap
                }

                Rectangle {
                    width: parent.width
                    height: 1
                    color: "#1e2a3f"
                }

                Column {
                    width: parent.width
                    spacing: 26

                    Repeater {
                        model: seciliKonu >= 0 ? bolumler[seciliKonu].adimlar : []

                        RowLayout {
                            width: parent.width
                            spacing: 18

                            Rectangle {
                                Layout.preferredWidth: 26
                                Layout.preferredHeight: 26
                                Layout.alignment: Qt.AlignTop
                                radius: 13
                                color: Qt.darker(bolumler[seciliKonu].renk, 3.2)
                                border.color: bolumler[seciliKonu].renk
                                border.width: 1
```

```
                            Text {
                                anchors.centerIn: parent
                                text: index + 1
                                color: bolumler[seciliKonu].renk
                                font.pixelSize: 12
                                font.bold: true
                            }
                        }

                        Column {
                            Layout.preferredWidth: 300
                            Layout.alignment: Qt.AlignTop
                            spacing: 5

                            Text {
                                width: parent.width
                                text: Translations.turkish ? modelData.baslikTr : modelData.baslikEn
                                color: "#dce8f5"
                                font.family: "Segoe UI"
                                font.pixelSize: 14
                                font.bold: true
                                wrapMode: Text.WordWrap
                            }

                            Text {
                                width: parent.width
                                text: Translations.turkish ? modelData.metinTr : modelData.metinEn
                                color: "#9ca3af"
                                font.family: "Segoe UI"
                                font.pixelSize: 12
                                wrapMode: Text.WordWrap
                                lineHeight: 1.3
                            }
                        }

                        Loader {
                            id: mockupYukleyici
                            Layout.fillWidth: true
                            Layout.maximumWidth: 460
                            Layout.preferredHeight: 260
                            Layout.alignment: Qt.AlignTop
                            asynchronous: true
                            sourceComponent: rehberMockupBileseni

                            property var adimModeli: modelData
                            property color konuRengi: bolumler[seciliKonu].renk

                            onLoaded: {
                                item.sahneAdi = adimModeli.mockup.tur
                                item.vurguAnahtari = adimModeli.mockup.bolge
                                item.vurguRengi = konuRengi
                            }

                            Rectangle {
                                anchors.fill: parent
                                visible: mockupYukleyici.status !== Loader.Ready
                                radius: 10
                                color: "#0a0e17"
                                border.color: "#1e2a3f"
                                border.width: 1
                            }
                        }
                    }
                }
            }
        }
    }
}
```

qml\pages\SonuclarPage.qml

04.11.2026 11:45

```
import QtQuick 6.7
import QtQuick.Controls 6.7
import QtQuick.Layouts 6.7
import QtCharts 6.7
import sliper

Rectangle {
    id: kok
    color: "#0a0a0d"

    property int olcumId: -1
    property var bilgi: ({})
    property var sonuc: ({ yeterliVeri: false, tau0: 0, mu: 0, r2: 0, n: 0, toplamStroke: 0, uyarilar: [] })
    property var strokeListesi: []
    property var tahminAyarlari: ({})
    property var tahminSatirlari: []
    property int seciliStrokeIndex: -1
    property int aktifSekme: 0
    property int playbackAdimi: -1

    readonly property real hesaplananTau0: sonuc.tau0 || 0
    readonly property real hesaplananMu: sonuc.mu || 0

    readonly property var metinler: ({
        sonucOzeti: { tr: "SONUÇ ÖZETİ", en: "RESULT SUMMARY" },
        duzenle: { tr: "✎ Düzenle", en: "✎ Edit" },
        yer: { tr: "Yer", en: "Place" },
        musteri: { tr: "Müşteri", en: "Customer" },
        recete: { tr: "Reçete", en: "Formula" },
        yorum: { tr: "Yorum", en: "Comment" },
        kesisimA: { tr: "P–Q KESİŞİMİ (A)", en: "P–Q INTERCEPT (A)" },
        egimB: { tr: "P–Q EĞİMİ (B)", en: "P–Q SLOPE (B)" },
        yieldA: { tr: "a (Yield Pressure)", en: "a (Yield Pressure)" },
        gradyanB: { tr: "b (Pressure Gradient ×1000)", en: "b (Pressure Gradient ×1000)" },
        uyumKalitesi: { tr: "UYUM (R²)", en: "FIT (R²)" },
        kullanilanStroke: { tr: "KULLANILAN STROKE", en: "STROKES USED" },
        tahminAyarlari: { tr: "TAHMİN AYARLARI", en: "FORECAST PREFERENCES" },
        hedefDebi: { tr: "Hedef debi Q (m³/h)", en: "Target flow Q (m³/h)" },
        cap: { tr: "Boru iç çapı D (mm)", en: "Pipe inner diameter D (mm)" },
        hatUzunlugu: { tr: "Hat uzunluğu (m)", en: "Line length (m)" },
        yukseklik: { tr: "Pompalama yüksekliği (m)", en: "Pumping height (m)" },
        dirsek90: { tr: "90° dirsek sayısı", en: "90° bends" },
        dirsek45: { tr: "45° dirsek sayısı", en: "45° bends" },
        hesapUzunlugu: { tr: "Hesap uzunluğu (m)", en: "Calc. length (m)" },
        uygunBolge: { tr: "Uygun bölge", en: "Feasible region" },
        hataBandi: { tr: "Alt – Üst sınır", en: "Lower – upper bound" },
        hedefNokta: { tr: "Hedef debi", en: "Target flow" },
        yogunluk: { tr: "Yoğunluk ρ (kg/m³)", en: "Density ρ (kg/m³)" },
        hataPayi: { tr: "Hata payı (%)", en: "Error bar tol. (%)" },
        pompaMaks: { tr: "Pompa maks. (bar)", en: "Pump max. (bar)" },
        hesaplaKaydet: { tr: "Tahmini Hesapla ve Kaydet", en: "Calculate and Save Forecast" },
        gecersizGirdi: { tr: "Geçersiz değer: tüm alanlar sayı olmalı, D > 0.", en: "Invalid value: all fields must be numbers, D
> 0." },
        kaydedildi: { tr: "✓ Tahmin ayarları kaydedildi", en: "✓ Forecast preferences saved" },
        pompalanabilir: { tr: "POMPALANABİLİR", en: "PUMPABLE" },
        pompalamaSorunu: { tr: "POMPALAMA SORUNU", en: "PUMPING ISSUE" },
        pompalanabilirGuvenilmez: { tr: "POMPALANABİLİR — GÜVENİLİR DEĞİL", en: "PUMPABLE — UNRELIABLE" },
        durumEgimNegatif: { tr: "P–Q eğimi (B) negatif: tahmin fiziksel değil, pompalanabilirlik kararı verilemez. Ölçümü
tekrarlayın.", en: "p–Q slope (B) is negative: forecast is not physical, no pumpability decision. Repeat the measurement." },
        durumKesisimNegatif: { tr: "P–Q kesişimi (A) negatif: tahmin fiziksel değil, pompalanabilirlik kararı verilemez. Ölçümü
tekrarlayın.", en: "p–Q intercept (A) is negative: forecast is not physical, no pumpability decision. Repeat the measurement." },
        durumDusukR2: { tr: "Gereken basınç en fazla %1 bar, pompa kapasitesi içinde, ancak uyum kalitesi düşük (R² = %2): sonuç
güvenilir değil.", en: "Required pressure up to %1 bar is within pump capacity, but fit quality is low (R² = %2): result is
unreliable." },
        gecersiz: { tr: "geçersiz", en: "invalid" },
        degerlendirilmedi: { tr: "DEĞERLENDİRİLMEDİ", en: "NOT EVALUATED" },
        durumYetersizVeri: { tr: "Yeterli stroke verisi yok.", en: "Not enough stroke data." },
        durumPompaYok: { tr: "Pompalanabilirlik kararı için pompa maks. basıncını girip tahmini hesaplayın.", en: "Enter the pump
```

```
max. pressure and calculate the forecast to evaluate pumpability." },
        durumUygun: { tr: "Gereken basınç en fazla %1 bar (Q = %2 m³/h, hesap uzunluğu %3 m), pompa kapasitesi kullanımı %%4.",
en: "Required pressure up to %1 bar (Q = %2 m³/h, calc. length %3 m), %4% of pump capacity." },
        durumSorun: { tr: "Gereken basınç %1 bar, pompa kapasitesi %2 bar.", en: "Required pressure %1 bar, pump capacity %2
bar." },
        oneriAkmaBaslik: { tr: "Karışım sert", en: "Mix is stiff" },
        oneriViskozBaslik: { tr: "Karışım yapışkan", en: "Mix is sticky" },
        oneriYukseklikBaslik: { tr: "Reçeteyle çözülmez", en: "Cannot be solved by the mix" },
        oneriKatkiArtir: { tr: "Katkıyı (akışkanlaştırıcı) artırın", en: "Increase the admixture (plasticizer)" },
        oneriInceKumArtir: { tr: "İnce kum / harç oranını artırın", en: "Increase fine sand / mortar content" },
        oneriIriAgregaAzalt: { tr: "İri agregayı azaltın", en: "Reduce coarse aggregate" },
        oneriViskoziteKatkisiAzalt: { tr: "Viskozite düzenleyici katkıyı azaltın", en: "Reduce the viscosity-modifying admixture"
},
        oneriTozAzalt: { tr: "Toz / ince malzemeyi azaltın", en: "Reduce powder / fines" },
        oneriPompa: { tr: "Daha yüksek basınçlı pompa gerekir", en: "A higher-pressure pump is required" },
        oneriTekrarOlc: { tr: "Değişiklikten sonra tekrar ölçün.", en: "Measure again after the change." },
        pdfRaporOnizle: { tr: "📄 PDF Rapor Önizle", en: "📄 Preview PDF Report" },
        excelCsvOnizle: { tr: "📊 Excel (CSV) Önizle", en: "📊 Preview Excel (CSV)" },
        excelXmlDisaAktar: { tr: "📑 Excel (XML) Dışa Aktar", en: "📑 Export Excel (XML)" },
        disaAktarildi: { tr: "✓ Dışa aktarıldı: ", en: "✓ Exported: " },
        disaAktarilamadi: { tr: "✕ Dışa aktarılamadı", en: "✕ Export failed" },
        sekmePq: { tr: "P–Q Grafiği", en: "p–Q Chart" },
        sekmeTablo: { tr: "Stroke Tablosu", en: "Table of Strokes" },
        sekmeEgri: { tr: "Stroke Eğrileri", en: "Stroke Curves" },
        sekmeTahminTablo: { tr: "Tahmin Tablosu", en: "Forecast Table" },
        sekmeTahminGrafik: { tr: "Tahmin Grafiği", en: "Forecast Chart" },
        dahilNokta: { tr: "Tahmine dahil", en: "In forecast" },
        haricNokta: { tr: "Hariç / hatalı", en: "Excluded / wrong" },
        regresyon: { tr: "Regresyon", en: "Regression" },
        oynat: { tr: "▶ Oynat", en: "▶ Play" },
        durdur: { tr: "⏸ Durdur", en: "⏸ Pause" },
        no: { tr: "No", en: "No" },
        saat: { tr: "Saat", en: "Time" },
        sure: { tr: "Süre s", en: "Dur. s" },
        agirlik: { tr: "Ağırlık kg", en: "Weight kg" },
        durum: { tr: "Durum", en: "Status" },
        dahil: { tr: "Dahil", en: "Included" },
        haric: { tr: "Hariç", en: "Excluded" },
        otomatikHaricTakilma: { tr: "Otomatik hariç", en: "Auto-excluded" },
        hatali: { tr: "Wrong Stroke", en: "Wrong Stroke" },
        strokeYok: { tr: "Bu ölçümde stroke yok", en: "No strokes in this measurement" },
        strokeSilSoru: { tr: "Stroke %1 kalıcı olarak silinsin mi? Bu işlem geri alınamaz.", en: "Delete stroke %1 permanently?
This cannot be undone." },
        sil: { tr: "Sil", en: "Delete" },
        vazgec: { tr: "Vazgeç", en: "Cancel" },
        kaydet: { tr: "Kaydet", en: "Save" },
        basincEgrisi: { tr: "Basınç Eğrisi", en: "Pressure Curve" },
        konumHizEgrisi: { tr: "Mesafe ve Hız Eğrisi", en: "Distance and Speed Curve" },
        hamVeriYok: { tr: "Bu stroke için ham eğri verisi yok (eski sürümle kaydedilmiş).", en: "No raw curve data for this
stroke (saved with an older version)." },
        zaman: { tr: "Zaman (s)", en: "Time (s)" },
        basinc: { tr: "Basınç (mbar)", en: "Pressure (mbar)" },
        mesafe: { tr: "Mesafe (mm)", en: "Distance (mm)" },
        hiz: { tr: "Hız (m/s)", en: "Speed (m/s)" },
        debi: { tr: "Debi Q (m³/h)", en: "Flow Q (m³/h)" },
        uzunluk: { tr: "Uzunluk L (m)", en: "Length L (m)" },
        basincBar: { tr: "P (bar)", en: "P (bar)" },
        aralik: { tr: "Alt – Üst (bar)", en: "Lower – Upper (bar)" },
        pompa: { tr: "Pompa", en: "Pump" },
        uygun: { tr: "uygun", en: "ok" },
        asildi: { tr: "aşıldı", en: "exceeded" },
        tahminYok: { tr: "Tahmin için en az 2 geçerli stroke gerekir.", en: "At least 2 valid strokes are required for a
forecast." },
        tahmin: { tr: "Tahmin", en: "Forecast" },
        pompaCizgi: { tr: "Pompa maks.", en: "Pump max." },
        olcumBilgisi: { tr: "Ölçüm Bilgileri", en: "Measurement Information" },
        pdfRaporOnizlemeBaslik: { tr: "PDF Rapor Önizleme", en: "PDF Report Preview" },
        indirPdf: { tr: "💾 İndir (PDF)", en: "💾 Download (PDF)" },
        pdfKaydedildi: { tr: "PDF kaydedildi: ", en: "PDF saved: " },
        pdfOlusturulamadi: { tr: "PDF oluşturulamadı.", en: "Could not generate PDF." },
```

```
        kapat: { tr: "Kapat", en: "Close" },
        excelCsvOnizlemeBaslik: { tr: "Excel (CSV) Önizleme", en: "Excel (CSV) Preview" },
        indirCsv: { tr: "💾  İndir (CSV)", en: "💾  Download (CSV)" },
        csvKaydedildi: { tr: "CSV kaydedildi: ", en: "CSV saved: " },
        csvOlusturulamadi: { tr: "CSV oluşturulamadı.", en: "Could not generate CSV." },
        olcumSecilmedi: { tr: "Geçmiş sayfasından bir ölçüm seçin veya yeni bir ölçüm tamamlayın.", en: "Select a measurement
from History or complete a new measurement." }
    })

    function txt(anahtar) {
        return Translations.turkish ? metinler[anahtar].tr : metinler[anahtar].en
    }

    function sayi(deger, basamak) {
        return (deger === undefined || deger === null || isNaN(deger)) ? "—" : Number(deger).toFixed(basamak)
    }

    function schleibingerA(kesisimA) { return calculator.schleibingerA(kesisimA) }
    function schleibingerB(egimB) { return calculator.schleibingerB(egimB) }

    onOlcumIdChanged: verileriYukle()
    Component.onCompleted: verileriYukle()
    onVisibleChanged: if (visible) verileriYukle()

    function verileriYukle() {
        playbackTimer.stop()
        playbackAdimi = -1
        if (olcumId <= 0) {
            bilgi = ({})
            strokeListesi = []
            tahminSatirlari = []
            return
        }
        bilgi = database.olcumBilgisiGetir(olcumId)
        sonuc = database.binghamHesapla(olcumId)
        strokeListesi = database.strokeVerileriGetir(olcumId)
        tahminAyarlari = database.tahminAyarlariGetir(olcumId)
        ayarAlanlariniDoldur()
        if (seciliStrokeIndex >= strokeListesi.length || seciliStrokeIndex < 0)
            seciliStrokeIndex = strokeListesi.length > 0 ? 0 : -1
        pqGrafiginiCiz()
        tahminiGuncelle()
        strokeEgrisiniCiz()
    }

    function pqGrafiginiCiz() {
        pqDahil.clear()
        pqHaric.clear()
        pqDogru.clear()
        oynatmaIsaretci.clear()
        var enBuyukDebi = 5, enBuyukBasinc = 10, enKucukBasinc = 0
        for (var i = 0; i < strokeListesi.length; i++) {
            var stroke = strokeListesi[i]
            if (stroke.gecerli && stroke.secili) pqDahil.append(stroke.debi, stroke.basinc)
            else pqHaric.append(stroke.debi, stroke.basinc)
            enBuyukDebi = Math.max(enBuyukDebi, stroke.debi)
            enBuyukBasinc = Math.max(enBuyukBasinc, stroke.basinc)
            enKucukBasinc = Math.min(enKucukBasinc, stroke.basinc)
        }
        if (sonuc.yeterliVeri) {
            var kesisimA = sonuc.tau0, egimB = sonuc.mu
            var cizgiSonuDebi = enBuyukDebi * 1.1
            var cizgiSonuBasinc = kesisimA + egimB * cizgiSonuDebi
            pqDogru.append(0, kesisimA)
            pqDogru.append(cizgiSonuDebi, cizgiSonuBasinc)
            enBuyukBasinc = Math.max(enBuyukBasinc, cizgiSonuBasinc)
            enKucukBasinc = Math.min(enKucukBasinc, kesisimA)
        }
        pqQEkseni.max = Math.ceil(enBuyukDebi * 1.15)
        pqPEkseni.min = Math.floor(enKucukBasinc * 1.1)
```

```
    pqPEkseni.max = Math.ceil(enBuyukBasinc * 1.15)
}

function ayarAlanlariniDoldur() {
    var ayar = tahminAyarlari
    var alanlar = ayarAlanlari()
    for (var anahtar in alanlar) alanlar[anahtar].metin = ayar[anahtar]
    pompaAlani.metin = ayar.pompaMaks > 0 ? ayar.pompaMaks : ""
    ayarBildirimi.text = ""
}

function ayarAlanlari() {
    return { cap: capAlani, hatUzunlugu: hatUzunluguAlani, yukseklik: yukseklikAlani,
             dirsek90: dirsek90Alani, dirsek45: dirsek45Alani, hedefDebi: hedefDebiAlani }
}

function ayarlariKaydet() {
    var alanlar = ayarAlanlari()
    var yeni = Object.assign({}, tahminAyarlari)
    for (var anahtar in alanlar) {
        var deger = parseFloat(String(alanlar[anahtar].metin).replace(",", "."))
        if (isNaN(deger)) { ayarBildirimi.hata = true; ayarBildirimi.text = txt("gecersizGirdi"); return }
        yeni[anahtar] = deger
    }
    var pompa = parseFloat(String(pompaAlani.metin).replace(",", "."))
    yeni.pompaMaks = (isNaN(pompa) || pompa < 0) ? 0 : pompa
    if (yeni.cap <= 0 || yeni.hedefDebi < 0 || yeni.hatUzunlugu < 0 || yeni.dirsek90 < 0 || yeni.dirsek45 < 0) {
        ayarBildirimi.hata = true; ayarBildirimi.text = txt("gecersizGirdi"); return
    }
    if (database.tahminAyarlariKaydet(olcumId, yeni)) {
        tahminAyarlari = database.tahminAyarlariGetir(olcumId)
        ayarBildirimi.hata = false
        ayarBildirimi.text = txt("kaydedildi")
        tahminiGuncelle()
    }
}

function tahminiGuncelle() {
    tahminSatirlari = sonuc.yeterliVeri
        ? calculator.tahminTablosuHesapla(sonuc.tau0, sonuc.mu, tahminAyarlari) : []
    tahminGrafiginiCiz()
}

function tahminGrafiginiCiz() {
    var seriler = [hatCizgisi, hatAltSinir, hatUstSinir, hedefNoktasi, hedefCubugu, bolgeUst, bolgeAlt]
    for (var i = 0; i < seriler.length; i++) seriler[i].clear()
    pompaCizgisi.clear()
    if (!sonuc.yeterliVeri || tahminSatirlari.length === 0) return
    var ayar = tahminAyarlari
    var hedef = tahminSatirlari[0]
    function tahminEt(debi) {
        return calculator.boruHattiTahminHesapla(sonuc.tau0, sonuc.mu, debi, ayar.cap, hedef.uzunluk,
                                                 ayar.yukseklik, ayar.yogunluk, ayar.hataPayi)
    }
    var grafikSonuDebi = Math.max(ayar.hedefDebi * 1.5, 10)
    var sifirDebide = tahminEt(0)
    var grafikSonunda = tahminEt(grafikSonuDebi)
    hatCizgisi.append(0, sifirDebide.basincBar)
    hatCizgisi.append(grafikSonuDebi, grafikSonunda.basincBar)
    if (ayar.hataPayi > 0) {
        hatAltSinir.append(0, sifirDebide.altSinirBar)
        hatAltSinir.append(grafikSonuDebi, grafikSonunda.altSinirBar)
        hatUstSinir.append(0, sifirDebide.ustSinirBar)
        hatUstSinir.append(grafikSonuDebi, grafikSonunda.ustSinirBar)
        hedefCubugu.append(hedef.debi, hedef.altSinirBar)
        hedefCubugu.append(hedef.debi, hedef.ustSinirBar)
    }
    hedefNoktasi.append(hedef.debi, hedef.basincBar)
    var enBuyukBasinc = Math.max(1, grafikSonunda.ustSinirBar, sifirDebide.ustSinirBar)
    var enKucukBasinc = Math.min(0, sifirDebide.altSinirBar, grafikSonunda.altSinirBar)
```

```
        if (ayar.pompaMaks > 0) {
            pompaCizgisi.append(0, ayar.pompaMaks)
            pompaCizgisi.append(grafikSonuDebi, ayar.pompaMaks)
            enBuyukBasinc = Math.max(enBuyukBasinc, ayar.pompaMaks)
            var bolgeAdimSayisi = 60
            for (i = 0; i <= bolgeAdimSayisi; i++) {
                var debi = grafikSonuDebi * i / bolgeAdimSayisi
                var hatBasinci = tahminEt(debi).ustSinirBar
                if (hatBasinci > ayar.pompaMaks) break
                bolgeUst.append(debi, ayar.pompaMaks)
                bolgeAlt.append(debi, hatBasinci)
            }
        }
        tgQEkseni.max = Math.ceil(grafikSonuDebi)
        tgPEkseni.min = Math.floor(enKucukBasinc * 1.1)
        tgPEkseni.max = Math.ceil(enBuyukBasinc * 1.1)
    }

    readonly property var durumBilgisi: {
        if (!sonuc.yeterliVeri) return { kod: "yetersiz" }
        var guvenilmezNedeni = calculator.tahminGuvenilmezNedeni(sonuc.tau0, sonuc.mu, sonuc.r2)
        if (calculator.tahminKarariVerilemez(guvenilmezNedeni)) return { kod: "gecersizModel", neden: guvenilmezNedeni }
        var pompa = tahminAyarlari.pompaMaks || 0
        if (pompa <= 0 || tahminSatirlari.length === 0) return { kod: "yok" }
        var enKotu = tahminSatirlari[0], asilan = 0
        for (var i = 0; i < tahminSatirlari.length; i++) {
            var satir = tahminSatirlari[i]
            if (satir.ustSinirBar > enKotu.ustSinirBar) enKotu = satir
            if (satir.pompaAsildi) asilan++
        }
        var kod = asilan > 0 ? "sorun" : (guvenilmezNedeni === "dusukR2" ? "guvenilmez" : "uygun")
        return { kod: kod, enKotu: enKotu, asilan: asilan, pompa: pompa,
                 sorunNedeni: calculator.pompaSorunuNedeni(enKotu, pompa) }
    }

    readonly property var receteOnerisi: {
        if (durumBilgisi.kod !== "sorun") return null
        if (durumBilgisi.sorunNedeni === "yukseklik")
            return { baslik: txt("oneriYukseklikBaslik"), maddeler: [txt("oneriPompa")], tekrarOlc: false }
        if (durumBilgisi.sorunNedeni === "viskoz")
            return { baslik: txt("oneriViskozBaslik"), tekrarOlc: true,
                     maddeler: [txt("oneriKatkiArtir"), txt("oneriViskoziteKatkisiAzalt"), txt("oneriTozAzalt")] }
        return { baslik: txt("oneriAkmaBaslik"), tekrarOlc: true,
                 maddeler: [txt("oneriKatkiArtir"), txt("oneriInceKumArtir"), txt("oneriIriAgregaAzalt")] }
    }

    function strokeEgrisiniCiz() {
        var seriler = [egriBasinc, egriP0l, egriP0r, egriPmaks, egriBaslangic, egriBitis, egriKonum, egriHiz, egriBaslangic2,
egriBitis2]
        for (var i = 0; i < seriler.length; i++) seriler[i].clear()
        egriVeriYok.visible = false
        if (seciliStrokeIndex < 0 || seciliStrokeIndex >= strokeListesi.length) return
        var stroke = strokeListesi[seciliStrokeIndex]
        var hamVeri = database.strokeHamVeriGetir(stroke.id)
        if (!hamVeri.t || hamVeri.t.length < 2) { egriVeriYok.visible = true; return }
        var zamanlar = hamVeri.t, konumlar = hamVeri.x, basinclar = hamVeri.p
        var ornekSayisi = zamanlar.length
        var sonZaman = zamanlar[ornekSayisi - 1]
        var enKucukBasinc = 1e9, enBuyukBasinc = -1e9, enBuyukKonum = 10, enBuyukHiz = 0.1
        for (i = 0; i < ornekSayisi; i++) {
            egriBasinc.append(zamanlar[i], basinclar[i])
            egriKonum.append(zamanlar[i], konumlar[i])
            enKucukBasinc = Math.min(enKucukBasinc, basinclar[i])
            enBuyukBasinc = Math.max(enBuyukBasinc, basinclar[i])
            enBuyukKonum = Math.max(enBuyukKonum, konumlar[i])
        }
        var konumYonu = hamVeri.yon || 1
        var hizlar = calculator.hizEgrisiHesapla(zamanlar, konumlar, konumYonu)
        for (i = 0; i < hizlar.length; i++) {
            egriHiz.append(zamanlar[i], hizlar[i])
```

```
        enBuyukHiz = Math.max(enBuyukHiz, hizlar[i])
    }
    egriP0l.append(0, stroke.p0l); egriP0l.append(hamVeri.tBaslangic, stroke.p0l)
    egriP0r.append(hamVeri.tBitis, stroke.p0r); egriP0r.append(sonZaman, stroke.p0r)
    egriPmaks.append(hamVeri.tBaslangic, stroke.pMaks); egriPmaks.append(hamVeri.tBitis, stroke.pMaks)
    var basincAralik = enBuyukBasinc - enKucukBasinc
    var basincEkseniAlt = Math.floor(enKucukBasinc - basincAralik * 0.1 - 1)
    var basincEkseniUst = Math.ceil(enBuyukBasinc + basincAralik * 0.15 + 1)
    egriBaslangic.append(hamVeri.tBaslangic, basincEkseniAlt); egriBaslangic.append(hamVeri.tBaslangic, basincEkseniUst)
    egriBitis.append(hamVeri.tBitis, basincEkseniAlt); egriBitis.append(hamVeri.tBitis, basincEkseniUst)
    egriBaslangic2.append(hamVeri.tBaslangic, 0); egriBaslangic2.append(hamVeri.tBaslangic, enBuyukKonum * 1.1)
    egriBitis2.append(hamVeri.tBitis, 0); egriBitis2.append(hamVeri.tBitis, enBuyukKonum * 1.1)
    egTEkseni.max = Math.ceil(sonZaman); egPEkseni.min = basincEkseniAlt; egPEkseni.max = basincEkseniUst
    egT2Ekseni.max = Math.ceil(sonZaman); egXEkseni.max = Math.ceil(enBuyukKonum * 1.1 / 10) * 10
    egVEkseni.max = Math.ceil(enBuyukHiz * 1.2 * 10) / 10
}

onSeciliStrokeIndexChanged: strokeEgrisiniCiz()

function strokeDahilDegistir(id, secili) {
    database.strokeSeciliAyarla(id, secili)
    verileriYukle()
}

Timer {
    id: playbackTimer
    interval: 1000
    repeat: true
    onTriggered: {
        if (playbackAdimi < strokeListesi.length - 1) {
            playbackAdimi++
            var s = strokeListesi[playbackAdimi]
            oynatmaIsaretci.clear()
            oynatmaIsaretci.append(s.debi, s.basinc)
        } else {
            stop()
        }
    }
}

component Kart: Rectangle {
    radius: 10
    color: "#0a0a0d"
    border.color: "#1e2a3f"
    border.width: 1
}

component KartBaslik: Text {
    color: "#6b7280"
    font.family: "Segoe UI"
    font.pixelSize: 10
    font.bold: true
    font.letterSpacing: 1
}

component Girdi: Column {
    id: girdiKoku
    property string etiket: ""
    property alias metin: alan.text
    property string ipucu: ""
    spacing: 2
    Text { text: girdiKoku.etiket; color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 10 }
    TextField {
        id: alan
        width: girdiKoku.width
        height: 28
        color: "#dce8f5"
        font.pixelSize: 12
        leftPadding: 8
        placeholderText: girdiKoku.ipucu
```

```
            placeholderTextColor: "#4b5563"
            verticalAlignment: TextInput.AlignVCenter
            selectByMouse: true
            background: Rectangle {
                color: "#07070a"
                radius: 5
                border.color: alan.activeFocus ? "#3b82f6" : "#1e2a3f"
                border.width: 1
            }
        }
    }

    component Dugme: Button {
        id: dugmeKoku
        property color renk: "#3b82f6"
        height: 36
        font.pixelSize: 12
        font.bold: true
        background: Rectangle {
            radius: 8
            color: dugmeKoku.enabled ? (dugmeKoku.hovered ? Qt.lighter(dugmeKoku.renk, 1.15) : dugmeKoku.renk) : "#1e2a3f"
            Behavior on color { ColorAnimation { duration: 120 } }
        }
        contentItem: Text {
            text: dugmeKoku.text
            color: dugmeKoku.enabled ? "#dce8f5" : "#6b7280"
            font: dugmeKoku.font
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
            elide: Text.ElideRight
        }
    }

    component DegerSatiri: Item {
        id: satirKoku
        property string etiket: ""
        property string deger: "—"
        property color cizgi: "#3b82f6"
        height: 34
        Rectangle { width: 3; height: satirKoku.height - 8; anchors.verticalCenter: parent.verticalCenter; color:
satirKoku.cizgi; radius: 2 }
        Text { anchors.left: parent.left; anchors.leftMargin: 12; anchors.verticalCenter: parent.verticalCenter; text:
satirKoku.etiket; color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 10 }
        Text { anchors.right: parent.right; anchors.rightMargin: 4; anchors.verticalCenter: parent.verticalCenter; text:
satirKoku.deger; color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 14; font.bold: true }
    }

    component Hucre: Text {
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 12
        elide: Text.ElideRight
        verticalAlignment: Text.AlignVCenter
    }

    component BaslikHucre: Text {
        color: "#6b7280"
        font.family: "Segoe UI"
        font.pixelSize: 10
        font.bold: true
        font.letterSpacing: 0.5
        elide: Text.ElideRight
    }

    component Eksen: ValueAxis {
        gridLineColor: "#1a1a20"
        labelsColor: "#6b7280"
        labelsFont.pixelSize: 9
        titleFont.pixelSize: 10
        lineVisible: false
```

```
    minorGridVisible: false
}

component PopupArka: Rectangle {
    color: "#12121a"
    radius: 12
    border.color: "#1e2a3f"
    border.width: 1
}

Row {
    anchors.fill: parent
    spacing: 0

    Rectangle {
        id: solPanel
        width: 300
        height: parent.height
        color: "#12121a"

        Rectangle { anchors.right: parent.right; width: 1; height: parent.height; color: "#1e2a3f" }

        Flickable {
            anchors.fill: parent
            anchors.margins: 16
            contentWidth: width
            contentHeight: solIcerik.height
            clip: true
            boundsBehavior: Flickable.StopAtBounds
            ScrollBar.vertical: ScrollBar { policy: ScrollBar.AsNeeded }

            Column {
                id: solIcerik
                width: parent.width
                spacing: 10

                Item {
                    width: parent.width
                    height: 16
                    KartBaslik { anchors.left: parent.left; text: txt("sonucOzeti") }
                    Text {
                        anchors.right: parent.right
                        text: txt("duzenle")
                        color: olcumId > 0 ? "#4f8cf7" : "#374151"
                        font.family: "Segoe UI"
                        font.pixelSize: 11
                        MouseArea {
                            anchors.fill: parent
                            enabled: olcumId > 0
                            cursorShape: Qt.PointingHandCursor
                            onClicked: bilgiPopup.ac()
                        }
                    }
                }

                Column {
                    width: parent.width
                    spacing: 2
                    Text {
                        width: parent.width
                        text: bilgi.musteri ? bilgi.musteri : "—"
                        color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 16; font.bold: true
                        elide: Text.ElideRight
                    }
                    Text {
                        width: parent.width
                        text: (bilgi.recete ? bilgi.recete : "—") + (bilgi.tarih ? "  ·  " + bilgi.tarih : "")
                        color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 11; elide: Text.ElideRight
                    }
                    Text {
```

```
                        width: parent.width
                        visible: !!bilgi.yer
                        text: "📍 " + (bilgi.yer || "")
                        color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 11; elide: Text.ElideRight
                    }
                    Text {
                        width: parent.width
                        visible: !!bilgi.yorum
                        text: "💬 " + (bilgi.yorum || "")
                        color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 11; wrapMode: Text.WordWrap
                    }
                }

                Text {
                    width: parent.width
                    visible: olcumId <= 0
                    text: txt("olcumSecilmedi")
                    color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 12; wrapMode: Text.WordWrap
                }

                Kart {
                    width: parent.width
                    height: sonucSutunu.implicitHeight + 16
                    visible: olcumId > 0
                    Column {
                        id: sonucSutunu
                        anchors.left: parent.left; anchors.right: parent.right; anchors.top: parent.top
                        anchors.margins: 8
                        DegerSatiri { width: sonucSutunu.width; etiket: txt("kesisimA"); deger: sonuc.yeterliVeri ?
sayi(sonuc.tau0, 2) + " mbar" : "—"; cizgi: "#3b82f6" }
                        DegerSatiri { width: sonucSutunu.width; etiket: txt("egimB"); deger: sonuc.yeterliVeri ?
sayi(sonuc.mu, 3) + " mbar·h/m³" : "—"; cizgi: "#9333ea" }
                        DegerSatiri { width: sonucSutunu.width; etiket: txt("yieldA"); deger: sonuc.yeterliVeri ?
sayi(schleibingerA(sonuc.tau0), 3) + " mbar" : "—"; cizgi: "#2563eb" }
                        DegerSatiri { width: sonucSutunu.width; etiket: txt("gradyanB"); deger: sonuc.yeterliVeri ?
sayi(schleibingerB(sonuc.mu) * 1000, 3) : "—"; cizgi: "#7c3aed" }
                        DegerSatiri { width: sonucSutunu.width; etiket: txt("uyumKalitesi"); deger: sonuc.yeterliVeri ?
sayi(sonuc.r2, 3) : "—"; cizgi: "#16a34a" }
                        DegerSatiri { width: sonucSutunu.width; etiket: txt("kullanilanStroke"); deger: (sonuc.n || 0) + " /
" + (sonuc.toplamStroke || 0); cizgi: "#6b7280" }
                    }
                }

                Kart {
                    width: parent.width
                    height: ayarSutunu.implicitHeight + 20
                    visible: olcumId > 0
                    Column {
                        id: ayarSutunu
                        anchors.left: parent.left; anchors.right: parent.right; anchors.top: parent.top
                        anchors.margins: 10
                        spacing: 8
                        KartBaslik { text: txt("tahminAyarlari") }
                        Grid {
                            id: ayarIzgarasi
                            width: parent.width
                            columns: 2
                            columnSpacing: 8
                            rowSpacing: 6
                            readonly property real hucre: (width - columnSpacing) / 2
                            Girdi { id: capAlani; width: ayarIzgarasi.hucre; etiket: txt("cap") }
                            Girdi { id: hedefDebiAlani; width: ayarIzgarasi.hucre; etiket: txt("hedefDebi") }
                            Girdi { id: hatUzunluguAlani; width: ayarIzgarasi.hucre; etiket: txt("hatUzunlugu") }
                            Girdi { id: yukseklikAlani; width: ayarIzgarasi.hucre; etiket: txt("yukseklik") }
                            Girdi { id: dirsek90Alani; width: ayarIzgarasi.hucre; etiket: txt("dirsek90") }
                            Girdi { id: dirsek45Alani; width: ayarIzgarasi.hucre; etiket: txt("dirsek45") }
                            Girdi { id: pompaAlani; width: ayarIzgarasi.hucre; etiket: txt("pompaMaks"); ipucu: "85–130" }
                        }
                        Dugme {
                            width: parent.width
```

```
                            text: txt("hesaplaKaydet")
                            onClicked: ayarlariKaydet()
                        }
                        Text {
                            id: ayarBildirimi
                            property bool hata: false
                            width: parent.width
                            visible: text.length > 0
                            color: hata ? "#f87171" : "#4ade80"
                            font.family: "Segoe UI"; font.pixelSize: 11; wrapMode: Text.WordWrap
                        }
                    }
                }

                Kart {
                    id: durumKutusu
                    width: parent.width
                    height: durumSutunu.implicitHeight + 20
                    visible: olcumId > 0
                    property color vurgu: durumBilgisi.kod === "uygun" ? "#16a34a"
                                        : (durumBilgisi.kod === "sorun" ? "#dc2626"
                                        : (durumBilgisi.kod === "guvenilmez" ? "#f59e0b" : "#6b7280"))
                    border.color: Qt.darker(vurgu, 1.6)
                    Rectangle { width: 4; height: parent.height; radius: 2; color: durumKutusu.vurgu }
                    Column {
                        id: durumSutunu
                        anchors.left: parent.left; anchors.right: parent.right; anchors.top: parent.top
                        anchors.margins: 10; anchors.leftMargin: 16
                        spacing: 4
                        Text {
                            width: parent.width
                            wrapMode: Text.WordWrap
                            text: durumBilgisi.kod === "uygun" ? "✓ " + txt("pompalanabilir")
                                  : (durumBilgisi.kod === "sorun" ? "! " + txt("pompalamaSorunu")
                                  : (durumBilgisi.kod === "guvenilmez" ? "? " + txt("pompalanabilirGuvenilmez")
                                  : "? " + txt("degerlendirilmedi")))
                            color: durumKutusu.vurgu; font.family: "Segoe UI"; font.pixelSize: 14; font.bold: true;
font.letterSpacing: 1
                        }
                        Text {
                            width: parent.width
                            wrapMode: Text.WordWrap
                            color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 11
                            text: {
                                var d = durumBilgisi
                                if (d.kod === "yetersiz") return txt("durumYetersizVeri")
                                if (d.kod === "yok") return txt("durumPompaYok")
                                if (d.kod === "gecersizModel") return txt(d.neden === "egimNegatif" ? "durumEgimNegatif" :
"durumKesisimNegatif")
                                var bar = sayi(d.enKotu.ustSinirBar, 1)
                                if (d.kod === "guvenilmez")
                                    return txt("durumDusukR2").arg(bar).arg(sayi(sonuc.r2, 2))
                                if (d.kod === "uygun")
                                    return txt("durumUygun").arg(bar).arg(sayi(d.enKotu.debi, 1)).arg(sayi(d.enKotu.uzunluk,
0))
                                                            .arg(sayi(d.enKotu.ustSinirBar / d.pompa * 100, 0))
                                return txt("durumSorun").arg(bar).arg(sayi(d.pompa, 1))
                            }
                        }
                        Column {
                            width: parent.width
                            visible: receteOnerisi !== null
                            spacing: 3
                            topPadding: 4
                            Text {
                                width: parent.width
                                wrapMode: Text.WordWrap
                                text: receteOnerisi ? receteOnerisi.baslik : ""
                                color: "#e5e7eb"; font.family: "Segoe UI"; font.pixelSize: 12; font.bold: true
                            }
```

```
                            Repeater {
                                model: receteOnerisi ? receteOnerisi.maddeler : []
                                Text {
                                    required property string modelData
                                    width: parent.width
                                    wrapMode: Text.WordWrap
                                    text: "•  " + modelData
                                    color: "#d1d5db"; font.family: "Segoe UI"; font.pixelSize: 12
                                }
                            }
                            Text {
                                width: parent.width
                                visible: receteOnerisi ? receteOnerisi.tekrarOlc : false
                                wrapMode: Text.WordWrap
                                text: txt("oneriTekrarOlc")
                                color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 11; font.italic: true
                            }
                        }
                    }
                }

                Dugme {
                    width: parent.width
                    text: txt("pdfRaporOnizle")
                    enabled: olcumId > 0
                    onClicked: {
                        pdfOnizlemePopup.onizlemeHtml = reportManager.pdfOnizlemeHtml(olcumId)
                        pdfOnizlemePopup.open()
                    }
                }
                Dugme {
                    width: parent.width
                    text: txt("excelCsvOnizle")
                    renk: "#16a34a"
                    enabled: olcumId > 0
                    onClicked: csvOnizlemePopup.open()
                }
                Dugme {
                    width: parent.width
                    text: txt("excelXmlDisaAktar")
                    renk: "#0d9488"
                    enabled: olcumId > 0
                    onClicked: {
                        var yol = database.xmlDisaAktar(olcumId)
                        bildirimKutusu.goster(yol.length > 0 ? txt("disaAktarildi") + yol : txt("disaAktarilamadi"),
yol.length === 0)
                    }
                }
                Item { width: 1; height: 8 }
            }
        }
    }

    Item {
        width: parent.width - solPanel.width
        height: parent.height

        Row {
            id: sekmeler
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.margins: 16
            spacing: 6
            Repeater {
                model: ["sekmePq", "sekmeTablo", "sekmeEgri", "sekmeTahminTablo", "sekmeTahminGrafik"]
                Rectangle {
                    width: sekmeMetni.implicitWidth + 28
                    height: 34
                    radius: 8
                    color: aktifSekme === index ? "#17263d" : (sekmeFare.containsMouse ? "#141419" : "#12121a")
```

```
                    border.color: aktifSekme === index ? "#3b82f6" : "#1e2a3f"
                    border.width: 1
                    Text {
                        id: sekmeMetni
                        anchors.centerIn: parent
                        text: txt(modelData)
                        color: aktifSekme === index ? "#dce8f5" : "#9ca3af"
                        font.family: "Segoe UI"; font.pixelSize: 12; font.bold: aktifSekme === index
                    }
                    MouseArea {
                        id: sekmeFare
                        anchors.fill: parent
                        hoverEnabled: true
                        cursorShape: Qt.PointingHandCursor
                        onClicked: aktifSekme = index
                    }
                }
            }
        }

        Rectangle {
            anchors.top: sekmeler.bottom
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            anchors.margins: 16
            anchors.topMargin: 10
            radius: 10
            color: "#12121a"
            border.color: "#1e2a3f"
            border.width: 1

            StackLayout {
                anchors.fill: parent
                anchors.margins: 10
                currentIndex: aktifSekme

                Item {
                    Item {
                        id: pqUstBilgi
                        anchors.top: parent.top
                        anchors.left: parent.left
                        anchors.right: parent.right
                        height: 30
                        Text {
                            anchors.left: parent.left
                            anchors.verticalCenter: parent.verticalCenter
                            text: sonuc.yeterliVeri
                                  ? "A = " + sayi(sonuc.tau0, 2) + " mbar    B = " + sayi(sonuc.mu, 3) + " mbar·h/m³    a = "
                                    + sayi(schleibingerA(sonuc.tau0), 3) + " mbar    b = " + sayi(schleibingerB(sonuc.mu) *
1000, 3)
                                    + "    R² = " + sayi(sonuc.r2, 3)
                                  : txt("tahminYok")
                            color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12
                        }
                        Row {
                            anchors.right: parent.right
                            anchors.verticalCenter: parent.verticalCenter
                            spacing: 10
                            Text {
                                anchors.verticalCenter: parent.verticalCenter
                                visible: playbackAdimi >= 0
                                text: "Stroke " + (playbackAdimi + 1) + " / " + strokeListesi.length
                                color: "#f59e0b"; font.family: "Segoe UI"; font.pixelSize: 12
                            }
                            Dugme {
                                width: 100; height: 28
                                text: playbackTimer.running ? txt("durdur") : txt("oynat")
                                enabled: strokeListesi.length > 0
                                onClicked: {
```

```
                                if (playbackTimer.running) { playbackTimer.stop(); return }
                                if (playbackAdimi >= strokeListesi.length - 1) playbackAdimi = -1
                                playbackTimer.start()
                            }
                        }
                    }
                }

                ChartView {

                    theme: ChartView.ChartThemeDark
                    anchors.top: pqUstBilgi.bottom
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
                    backgroundColor: "transparent"
                    antialiasing: true
                    legend.visible: true
                    legend.labelColor: "#9ca3af"
                    legend.alignment: Qt.AlignBottom

                    Eksen { id: pqQEkseni; min: 0; max: 20; titleText: txt("debi") }
                    Eksen { id: pqPEkseni; min: 0; max: 100; titleText: "p (mbar)" }

                    ScatterSeries { id: pqDahil; name: txt("dahilNokta"); axisX: pqQEkseni; axisY: pqPEkseni; color:
"#3b82f6"; borderColor: "#1d4ed8"; markerSize: 11 }
                    ScatterSeries { id: pqHaric; name: txt("haricNokta"); axisX: pqQEkseni; axisY: pqPEkseni; color:
"#4b5563"; borderColor: "#6b7280"; markerSize: 9 }
                    LineSeries { id: pqDogru; name: txt("regresyon"); axisX: pqQEkseni; axisY: pqPEkseni; color:
"#dce8f5"; width: 2 }
                    ScatterSeries { id: oynatmaIsaretci; name: "▶"; axisX: pqQEkseni; axisY: pqPEkseni; color:
"#f59e0b"; borderColor: "#f59e0b"; markerSize: 18 }
                }
            }

            Item {
                id: tabloSekmesi
                readonly property var kolonlar: [0.05, 0.05, 0.1, 0.07, 0.08, 0.08, 0.08, 0.08, 0.09, 0.09, 0.19, 0.04]

                Row {
                    id: tabloBasligi
                    anchors.top: parent.top
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.leftMargin: 6
                    readonly property var k: tabloSekmesi.kolonlar
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[0]; text: "✓" }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[1]; text: txt("no") }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[2]; text: txt("saat") }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[3]; text: txt("sure") }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[4]; text: txt("agirlik") }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[5]; text: "Pmax" }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[6]; text: "P0l" }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[7]; text: "P0r" }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[8]; text: "p mbar" }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[9]; text: "Q m³/h" }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[10]; text: txt("durum") }
                    BaslikHucre { width: tabloBasligi.width * tabloBasligi.k[11]; text: "" }
                }

                Rectangle { id: tabloCizgi; anchors.top: tabloBasligi.bottom; anchors.topMargin: 6; width: parent.width;
height: 1; color: "#1e2a3f" }

                Text {
                    anchors.centerIn: parent
                    visible: strokeListesi.length === 0
                    text: txt("strokeYok")
                    color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 13
                }
```

```
                        ListView {
                            id: strokeTablosu
                            anchors.top: tabloCizgi.bottom
                            anchors.topMargin: 4
                            anchors.left: parent.left
                            anchors.right: parent.right
                            anchors.bottom: parent.bottom
                            clip: true
                            model: strokeListesi
                            ScrollBar.vertical: ScrollBar { policy: ScrollBar.AsNeeded }

                            delegate: Rectangle {
                                id: strokeSatiri
                                required property var modelData
                                required property int index
                                readonly property var s: modelData
                                readonly property var k: tabloSekmesi.kolonlar
                                width: strokeTablosu.width
                                height: 30
                                radius: 5
                                color: index === seciliStrokeIndex ? "#17263d" : (index % 2 === 0 ? "transparent" : "#0e0e14")

                                MouseArea {
                                    anchors.fill: parent
                                    cursorShape: Qt.PointingHandCursor
                                    onClicked: seciliStrokeIndex = strokeSatiri.index
                                    onDoubleClicked: { seciliStrokeIndex = strokeSatiri.index; aktifSekme = 2 }
                                }

                                Row {
                                    id: satirDuzeni
                                    anchors.fill: parent
                                    anchors.leftMargin: 6
                                    readonly property real g: width

                                    Item {
                                        width: satirDuzeni.g * strokeSatiri.k[0]; height: strokeSatiri.height
                                        Rectangle {
                                            anchors.verticalCenter: parent.verticalCenter
                                            width: 16; height: 16; radius: 3
                                            color: strokeSatiri.s.secili && strokeSatiri.s.gecerli ? "#1d4ed8" : "transparent"
                                            border.color: strokeSatiri.s.gecerli ? "#3b82f6" : "#374151"
                                            opacity: strokeSatiri.s.gecerli ? 1 : 0.5
                                            Text { anchors.centerIn: parent; text: strokeSatiri.s.secili &&
strokeSatiri.s.gecerli ? "✓" : ""; color: "#dce8f5"; font.pixelSize: 11; font.bold: true }
                                            MouseArea {
                                                anchors.fill: parent
                                                anchors.margins: -4
                                                enabled: strokeSatiri.s.gecerli
                                                cursorShape: Qt.PointingHandCursor
                                                onClicked: strokeDahilDegistir(strokeSatiri.s.id, !strokeSatiri.s.secili)
                                            }
                                        }
                                    }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[1]; height: strokeSatiri.height; text:
strokeSatiri.s.stroke }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[2]; height: strokeSatiri.height; text:
strokeSatiri.s.saat || "—" }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[3]; height: strokeSatiri.height; text:
sayi(strokeSatiri.s.sure, 2) }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[4]; height: strokeSatiri.height; text:
sayi(strokeSatiri.s.agirlik, 1) }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[5]; height: strokeSatiri.height; text:
sayi(strokeSatiri.s.pMaks, 1) }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[6]; height: strokeSatiri.height; text:
sayi(strokeSatiri.s.p0l, 1) }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[7]; height: strokeSatiri.height; text:
sayi(strokeSatiri.s.p0r, 1) }
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[8]; height: strokeSatiri.height; text:
sayi(strokeSatiri.s.basinc, 2); font.bold: true }
```

```
                                    Hucre { width: satirDuzeni.g * strokeSatiri.k[9]; height: strokeSatiri.height; text:
sayi(strokeSatiri.s.debi, 2); font.bold: true }
                                    Item {
                                        width: satirDuzeni.g * strokeSatiri.k[10]; height: strokeSatiri.height
                                        Rectangle {
                                            anchors.verticalCenter: parent.verticalCenter
                                            width: durumYazisi.implicitWidth + 14; height: 20; radius: 10
                                            color: !strokeSatiri.s.gecerli ? "#2a1414" : (strokeSatiri.s.secili ? "#123321" :
"#1f2937")
                                            Text {
                                                id: durumYazisi
                                                anchors.centerIn: parent
                                                text: !strokeSatiri.s.gecerli ? txt("hatali")
                                                      : strokeSatiri.s.secili ? txt("dahil")
                                                      : strokeSatiri.s.otomatikHaric ? txt("otomatikHaricTakilma") : txt("haric")
                                                color: !strokeSatiri.s.gecerli ? "#f87171"
                                                       : strokeSatiri.s.secili ? "#4ade80"
                                                       : strokeSatiri.s.otomatikHaric ? "#fbbf24" : "#9ca3af"
                                                font.pixelSize: 10; font.bold: true
                                            }
                                        }
                                    }
                                    Item {
                                        width: satirDuzeni.g * strokeSatiri.k[11]; height: strokeSatiri.height
                                        Text {
                                            anchors.centerIn: parent
                                            text: "🗑"
                                            font.pixelSize: 13
                                            opacity: silFare.containsMouse ? 1 : 0.5
                                            MouseArea {
                                                id: silFare
                                                anchors.fill: parent
                                                anchors.margins: -4
                                                hoverEnabled: true
                                                cursorShape: Qt.PointingHandCursor
                                                onClicked: {
                                                    strokeSilPopup.strokeId = strokeSatiri.s.id
                                                    strokeSilPopup.strokeNo = strokeSatiri.s.stroke
                                                    strokeSilPopup.open()
                                                }
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }

                    Item {
                        Row {
                            id: egriUst
                            anchors.top: parent.top
                            anchors.left: parent.left
                            height: 30
                            spacing: 10
                            Dugme { width: 34; height: 28; text: "◀"; enabled: seciliStrokeIndex > 0; onClicked:
seciliStrokeIndex-- }
                            Text {
                                anchors.verticalCenter: parent.verticalCenter
                                text: strokeListesi.length > 0 ? "Stroke " + (seciliStrokeIndex + 1) + " / " +
strokeListesi.length : txt("strokeYok")
                                color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 14; font.bold: true
                            }
                            Dugme { width: 34; height: 28; text: "▶"; enabled: seciliStrokeIndex < strokeListesi.length - 1;
onClicked: seciliStrokeIndex++ }
                            Text {
                                id: egriOzet
                                anchors.verticalCenter: parent.verticalCenter
                                readonly property bool var_: seciliStrokeIndex >= 0 && seciliStrokeIndex < strokeListesi.length
                                readonly property var s: var_ ? strokeListesi[seciliStrokeIndex] : ({})
```

```
                                text: var_ ? "Pmax " + sayi(s.pMaks, 1) + "   P0l " + sayi(s.p0l, 1) + "   P0r " + sayi(s.p0r, 1)
                                           + "   p " + sayi(s.basinc, 2) + " mbar   Q " + sayi(s.debi, 2) + " m³/h   " +
sayi(s.sure, 2) + " s"
                                           + (s.gecerli ? "" : "   ✕ " + txt("hatali")) : ""
                                color: var_ && !s.gecerli ? "#f87171" : "#9ca3af"
                                font.family: "Segoe UI"; font.pixelSize: 12
                            }
                        }

                        Text {
                            id: egriVeriYok
                            anchors.centerIn: parent
                            visible: false
                            text: txt("hamVeriYok")
                            color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 13
                            z: 5
                        }

                        Column {
                            anchors.top: egriUst.bottom
                            anchors.topMargin: 6
                            anchors.left: parent.left
                            anchors.right: parent.right
                            anchors.bottom: parent.bottom

                            ChartView {

                                theme: ChartView.ChartThemeDark
                                width: parent.width
                                height: parent.height / 2
                                title: txt("basincEgrisi")
                                titleColor: "#9ca3af"
                                backgroundColor: "transparent"
                                antialiasing: true
                                legend.visible: true
                                legend.labelColor: "#9ca3af"
                                legend.alignment: Qt.AlignRight

                                Eksen { id: egTEkseni; min: 0; max: 10; titleText: txt("zaman") }
                                Eksen { id: egPEkseni; min: 0; max: 200; titleText: txt("basinc") }

                                LineSeries { id: egriBasinc; name: txt("basinc"); axisX: egTEkseni; axisY: egPEkseni; color:
"#3b82f6"; width: 1.5 }
                                LineSeries { id: egriP0l; name: "P0l"; axisX: egTEkseni; axisY: egPEkseni; color: "#4ade80";
width: 2; style: Qt.DashLine }
                                LineSeries { id: egriP0r; name: "P0r"; axisX: egTEkseni; axisY: egPEkseni; color: "#14b8a6";
width: 2; style: Qt.DashLine }
                                LineSeries { id: egriPmaks; name: "Pmax"; axisX: egTEkseni; axisY: egPEkseni; color: "#f87171";
width: 2; style: Qt.DashLine }
                                LineSeries { id: egriBaslangic; name: ""; axisX: egTEkseni; axisY: egPEkseni; color: "#6b7280";
width: 1; style: Qt.DotLine }
                                LineSeries { id: egriBitis; name: ""; axisX: egTEkseni; axisY: egPEkseni; color: "#6b7280";
width: 1; style: Qt.DotLine }
                            }

                            ChartView {

                                theme: ChartView.ChartThemeDark
                                width: parent.width
                                height: parent.height / 2
                                title: txt("konumHizEgrisi")
                                titleColor: "#9ca3af"
                                backgroundColor: "transparent"
                                antialiasing: true
                                legend.visible: true
                                legend.labelColor: "#9ca3af"
                                legend.alignment: Qt.AlignRight

                                Eksen { id: egT2Ekseni; min: 0; max: 10; titleText: txt("zaman") }
                                Eksen { id: egXEkseni; min: 0; max: 600; titleText: txt("mesafe") }
```

```
                            Eksen { id: egVEkseni; min: 0; max: 1; titleText: txt("hiz") }

                            LineSeries { id: egriKonum; name: txt("mesafe"); axisX: egT2Ekseni; axisY: egXEkseni; color:
"#9333ea"; width: 2 }
                            LineSeries { id: egriHiz; name: txt("hiz"); axisX: egT2Ekseni; axisYRight: egVEkseni; color:
"#f59e0b"; width: 1.5 }
                            LineSeries { id: egriBaslangic2; name: ""; axisX: egT2Ekseni; axisY: egXEkseni; color: "#6b7280";
width: 1; style: Qt.DotLine }
                            LineSeries { id: egriBitis2; name: ""; axisX: egT2Ekseni; axisY: egXEkseni; color: "#6b7280";
width: 1; style: Qt.DotLine }
                        }
                    }
                }

                Item {
                    Column {
                        id: tahminTabloSutunu
                        anchors.fill: parent
                        anchors.margins: 10
                        spacing: 10

                        Text {
                            width: parent.width
                            wrapMode: Text.WordWrap
                            text: txt("tahmin") + "  —  D = " + sayi(tahminAyarlari.cap, 0) + " mm,  "
                                  + txt("hatUzunlugu") + " " + sayi(tahminAyarlari.hatUzunlugu, 0)
                                  + ",  h = " + sayi(tahminAyarlari.yukseklik, 1)
                                  + " m,  90°×" + sayi(tahminAyarlari.dirsek90, 0) + ",  45°×" +
sayi(tahminAyarlari.dirsek45, 0)
                                  + ",  ρ = " + sayi(tahminAyarlari.yogunluk, 0) + " kg/m³,  ±%" +
sayi(tahminAyarlari.hataPayi, 0)
                                  + (tahminAyarlari.pompaMaks > 0 ? ",  " + txt("pompaCizgi") + " " +
sayi(tahminAyarlari.pompaMaks, 0) + " bar" : "")
                            color: "#9ca3af"; font.family: "Segoe UI"; font.pixelSize: 12
                        }

                        Text {
                            visible: tahminSatirlari.length === 0
                            text: txt("tahminYok")
                            color: "#6b7280"; font.family: "Segoe UI"; font.pixelSize: 13
                        }

                        Column {
                            id: tahminTablosu
                            visible: tahminSatirlari.length > 0
                            width: Math.min(tahminTabloSutunu.width, 760)
                            spacing: 0

                            Row {
                                width: tahminTablosu.width
                                height: 30
                                BaslikHucre { width: tahminTablosu.width * 0.18; text: txt("debi"); anchors.verticalCenter:
parent.verticalCenter }
                                BaslikHucre { width: tahminTablosu.width * 0.18; text: txt("hesapUzunlugu");
anchors.verticalCenter: parent.verticalCenter }
                                BaslikHucre { width: tahminTablosu.width * 0.18; text: txt("basincBar");
anchors.verticalCenter: parent.verticalCenter }
                                BaslikHucre { width: tahminTablosu.width * 0.28; text: txt("aralik"); anchors.verticalCenter:
parent.verticalCenter }
                                BaslikHucre { width: tahminTablosu.width * 0.18; text: txt("pompa"); anchors.verticalCenter:
parent.verticalCenter }
                            }
                            Rectangle { width: tahminTablosu.width; height: 1; color: "#1e2a3f" }

                            Repeater {
                                model: tahminSatirlari
                                Rectangle {
                                    id: tahminSatiri
                                    required property var modelData
                                    required property int index
```

```
                                    width: tahminTablosu.width
                                    height: 34
                                    color: index % 2 === 0 ? "transparent" : "#0e0e14"
                                    readonly property bool pompaVar: tahminAyarlari.pompaMaks > 0
                                    readonly property bool satirGecersiz: durumBilgisi.kod === "gecersizModel" ||
tahminSatiri.modelData.basincNegatif
                                    readonly property bool satirGuvenilmez: durumBilgisi.kod === "guvenilmez"
                                    Row {
                                        anchors.fill: parent
                                        Hucre { width: tahminTablosu.width * 0.18; height: tahminSatiri.height; text:
sayi(tahminSatiri.modelData.debi, 1) }
                                        Hucre { width: tahminTablosu.width * 0.18; height: tahminSatiri.height; text:
sayi(tahminSatiri.modelData.uzunluk, 0) }
                                        Hucre { width: tahminTablosu.width * 0.18; height: tahminSatiri.height; text:
sayi(tahminSatiri.modelData.basincBar, 2); font.bold: true; font.pixelSize: 14 }
                                        Hucre { width: tahminTablosu.width * 0.28; height: tahminSatiri.height; text:
sayi(tahminSatiri.modelData.altSinirBar, 2) + " – " + sayi(tahminSatiri.modelData.ustSinirBar, 2); color: "#9ca3af" }
                                        Hucre {
                                            width: tahminTablosu.width * 0.18; height: tahminSatiri.height
                                            text: !tahminSatiri.pompaVar ? "—"
                                                  : (tahminSatiri.satirGecersiz ? "✕ " + txt("gecersiz")
                                                  : (tahminSatiri.modelData.pompaAsildi ? "✕ " + txt("asildi")
                                                  : (tahminSatiri.satirGuvenilmez ? "? " + txt("uygun") : "✓ " +
txt("uygun"))))
                                            color: !tahminSatiri.pompaVar ? "#6b7280"
                                                   : (tahminSatiri.satirGecersiz ? "#6b7280"
                                                   : (tahminSatiri.modelData.pompaAsildi ? "#f87171"
                                                   : (tahminSatiri.satirGuvenilmez ? "#f59e0b" : "#4ade80")))
                                            font.bold: true
                                        }
                                    }
                                }
                            }
                        }
                    }

                    Item {
                        ChartView {
                            theme: ChartView.ChartThemeDark
                            anchors.fill: parent
                            backgroundColor: "transparent"
                            antialiasing: true
                            legend.visible: true
                            legend.labelColor: "#9ca3af"
                            legend.alignment: Qt.AlignBottom

                            Eksen { id: tgQEkseni; min: 0; max: 100; titleText: txt("debi") }
                            Eksen { id: tgPEkseni; min: 0; max: 100; titleText: "P (bar)" }

                            AreaSeries {
                                name: txt("uygunBolge"); axisX: tgQEkseni; axisY: tgPEkseni
                                color: "#3316a34a"; borderColor: "transparent"; borderWidth: 0
                                upperSeries: LineSeries { id: bolgeUst }
                                lowerSeries: LineSeries { id: bolgeAlt }
                            }
                            LineSeries { id: hatCizgisi; name: txt("tahmin") + " " + sayi(tahminSatirlari.length > 0 ?
tahminSatirlari[0].uzunluk : 0, 0) + " m"; axisX: tgQEkseni; axisY: tgPEkseni; color: "#3b82f6"; width: 2.5 }
                            LineSeries { id: hatAltSinir; name: txt("hataBandi"); axisX: tgQEkseni; axisY: tgPEkseni; color:
"#64748b"; width: 1; style: Qt.DashLine }
                            LineSeries { id: hatUstSinir; name: ""; axisX: tgQEkseni; axisY: tgPEkseni; color: "#64748b"; width:
1; style: Qt.DashLine }
                            LineSeries { id: pompaCizgisi; name: txt("pompaCizgi"); axisX: tgQEkseni; axisY: tgPEkseni; color:
"#f59e0b"; width: 2; style: Qt.DashDotLine }
                            LineSeries { id: hedefCubugu; name: ""; axisX: tgQEkseni; axisY: tgPEkseni; color: "#ef4444"; width:
2 }
                            ScatterSeries { id: hedefNoktasi; name: txt("hedefNokta"); axisX: tgQEkseni; axisY: tgPEkseni; color:
"#ef4444"; borderColor: "#ef4444"; markerSize: 12 }
                        }
                    }
```

```
            }
        }
    }
}

Rectangle {
    id: bildirimKutusu
    property bool hataMi: false

    function goster(mesaj, hata) {
        bildirimMetni.text = mesaj
        hataMi = hata
        opacity = 1
        bildirimTimer.restart()
    }

    visible: opacity > 0
    opacity: 0
    z: 100
    anchors.bottom: parent.bottom
    anchors.horizontalCenter: parent.horizontalCenter
    anchors.bottomMargin: 24
    width: Math.min(bildirimMetni.implicitWidth + 40, parent.width - 40)
    height: 44
    radius: 10
    color: hataMi ? "#7f1d1d" : "#14532d"
    border.color: hataMi ? "#dc2626" : "#22c55e"
    border.width: 1

    Behavior on opacity { NumberAnimation { duration: 200 } }

    Text {
        id: bildirimMetni
        anchors.centerIn: parent
        width: parent.width - 20
        horizontalAlignment: Text.AlignHCenter
        elide: Text.ElideMiddle
        color: "#dce8f5"
        font.family: "Segoe UI"
        font.pixelSize: 13
        font.bold: true
    }

    Timer {
        id: bildirimTimer
        interval: 4000
        onTriggered: bildirimKutusu.opacity = 0
    }
}

Popup {
    id: strokeSilPopup
    property int strokeId: -1
    property int strokeNo: 0
    modal: true
    anchors.centerIn: parent
    width: 380
    padding: 22
    background: PopupArka {}
    Overlay.modal: Rectangle { color: "#a6000000" }
    contentItem: Column {
        spacing: 18
        Text {
            width: 336
            text: txt("strokeSilSoru").arg(strokeSilPopup.strokeNo)
            color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 14; wrapMode: Text.WordWrap
        }
        Row {
            spacing: 10
            Dugme {
```

```
                    width: 163; text: txt("sil"); renk: "#b91c1c"
                    onClicked: {
                        database.strokeSil(strokeSilPopup.strokeId)
                        strokeSilPopup.close()
                        verileriYukle()
                    }
                }
                Dugme { width: 163; text: txt("vazgec"); renk: "#1e2a3f"; onClicked: strokeSilPopup.close() }
            }
        }
    }

    Popup {
        id: bilgiPopup
        modal: true
        anchors.centerIn: parent
        width: 420
        padding: 22
        background: PopupArka {}
        Overlay.modal: Rectangle { color: "#a6000000" }

        function ac() {
            yerDuzenle.metin = bilgi.yer || ""
            musteriDuzenle.metin = bilgi.musteri || ""
            receteDuzenle.metin = bilgi.recete || ""
            yorumDuzenle.metin = bilgi.yorum || ""
            open()
        }

        contentItem: Column {
            spacing: 10
            Text { text: txt("olcumBilgisi"); color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 15; font.bold: true }
            Girdi { id: yerDuzenle; width: 376; etiket: txt("yer") }
            Girdi { id: musteriDuzenle; width: 376; etiket: txt("musteri") }
            Girdi { id: receteDuzenle; width: 376; etiket: txt("recete") }
            Girdi { id: yorumDuzenle; width: 376; etiket: txt("yorum") }
            Row {
                spacing: 10
                Dugme {
                    width: 183; text: txt("kaydet")
                    onClicked: {
                        database.olcumBilgisiGuncelle(olcumId, yerDuzenle.metin, musteriDuzenle.metin, receteDuzenle.metin,
yorumDuzenle.metin)
                        bilgiPopup.close()
                        verileriYukle()
                    }
                }
                Dugme { width: 183; text: txt("vazgec"); renk: "#1e2a3f"; onClicked: bilgiPopup.close() }
            }
        }
    }

    Popup {
        id: pdfOnizlemePopup
        modal: true
        focus: true
        anchors.centerIn: parent
        width: 720
        height: Math.min(820, parent.height - 40)
        padding: 0
        closePolicy: Popup.CloseOnEscape
        property string onizlemeHtml: ""
        background: PopupArka {}
        Overlay.modal: Rectangle { color: "#a6000000" }

        contentItem: Column {
            width: pdfOnizlemePopup.width
            spacing: 0

            Rectangle {
```

```
                width: parent.width
                height: 52
                color: "#0a0a0d"
                radius: 12
                Text {
                    anchors.left: parent.left
                    anchors.leftMargin: 20
                    anchors.verticalCenter: parent.verticalCenter
                    text: txt("pdfRaporOnizlemeBaslik")
                    color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 14; font.bold: true
                }
            }

            Rectangle {
                width: parent.width
                height: pdfOnizlemePopup.height - 52 - 64
                color: "#ffffff"

                Flickable {
                    anchors.fill: parent
                    anchors.margins: 20
                    contentWidth: width
                    contentHeight: onizlemeMetni.implicitHeight
                    clip: true
                    boundsBehavior: Flickable.StopAtBounds
                    ScrollBar.vertical: ScrollBar { policy: ScrollBar.AsNeeded }

                    TextEdit {
                        id: onizlemeMetni
                        width: parent.width
                        readOnly: true
                        selectByMouse: true
                        textFormat: TextEdit.RichText
                        wrapMode: Text.WordWrap
                        text: pdfOnizlemePopup.onizlemeHtml
                        color: "#14141a"
                        font.pixelSize: 12
                    }
                }
            }

            Rectangle {
                width: parent.width
                height: 64
                color: "#0a0a0d"
                radius: 12
                Row {
                    anchors.centerIn: parent
                    spacing: 12
                    Dugme {
                        width: 150
                        text: txt("indirPdf")
                        onClicked: {
                            var yol = reportManager.pdfOlustur(olcumId)
                            pdfOnizlemePopup.close()
                            bildirimKutusu.goster(yol.length > 0 ? txt("pdfKaydedildi") + yol : txt("pdfOlusturulamadi"),
yol.length === 0)
                        }
                    }
                    Dugme { width: 100; text: txt("kapat"); renk: "#1e2a3f"; onClicked: pdfOnizlemePopup.close() }
                }
            }
        }
    }

    Popup {
        id: csvOnizlemePopup
        modal: true
        focus: true
        anchors.centerIn: parent
```

```
        width: 860
        height: Math.min(680, parent.height - 40)
        padding: 0
        closePolicy: Popup.CloseOnEscape
        background: PopupArka {}
        Overlay.modal: Rectangle { color: "#a6000000" }

        contentItem: Column {
            width: csvOnizlemePopup.width
            spacing: 0

            Rectangle {
                width: parent.width
                height: 52
                color: "#0a0a0d"
                radius: 12
                Text {
                    anchors.left: parent.left
                    anchors.leftMargin: 20
                    anchors.right: parent.right
                    anchors.rightMargin: 20
                    anchors.verticalCenter: parent.verticalCenter
                    elide: Text.ElideRight
                    text: txt("excelCsvOnizlemeBaslik") + "  —  " + (bilgi.musteri || "") + "  ·  " + (bilgi.recete || "") + "  ·
" + (bilgi.tarih || "")
                    color: "#dce8f5"; font.family: "Segoe UI"; font.pixelSize: 14; font.bold: true
                }
            }

            Rectangle {
                width: parent.width
                height: csvOnizlemePopup.height - 52 - 64
                color: "#0a0a0d"

                Row {
                    id: csvBaslik
                    anchors.top: parent.top
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.margins: 14
                    readonly property var k: [0.06, 0.12, 0.08, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.14]
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[0]; text: txt("no") }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[1]; text: txt("saat") }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[2]; text: txt("sure") }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[3]; text: txt("agirlik") }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[4]; text: "Pmax" }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[5]; text: "P0l" }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[6]; text: "P0r" }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[7]; text: "p mbar" }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[8]; text: "Q m³/h" }
                    BaslikHucre { width: csvBaslik.width * csvBaslik.k[9]; text: txt("durum") }
                }

                ListView {
                    id: csvListesi
                    anchors.top: csvBaslik.bottom
                    anchors.left: parent.left
                    anchors.right: parent.right
                    anchors.bottom: parent.bottom
                    anchors.margins: 14
                    anchors.topMargin: 8
                    clip: true
                    model: strokeListesi
                    ScrollBar.vertical: ScrollBar { policy: ScrollBar.AsNeeded }

                    delegate: Row {
                        id: csvSatiri
                        required property var modelData
                        readonly property var k: csvBaslik.k
                        readonly property real g: csvListesi.width
```

```
                        width: csvListesi.width
                        height: 26
                        Hucre { width: csvSatiri.g * csvSatiri.k[0]; height: csvSatiri.height; text: csvSatiri.modelData.stroke }
                        Hucre { width: csvSatiri.g * csvSatiri.k[1]; height: csvSatiri.height; text: csvSatiri.modelData.saat ||
"—" }
                        Hucre { width: csvSatiri.g * csvSatiri.k[2]; height: csvSatiri.height; text:
sayi(csvSatiri.modelData.sure, 2) }
                        Hucre { width: csvSatiri.g * csvSatiri.k[3]; height: csvSatiri.height; text:
sayi(csvSatiri.modelData.agirlik, 1) }
                        Hucre { width: csvSatiri.g * csvSatiri.k[4]; height: csvSatiri.height; text:
sayi(csvSatiri.modelData.pMaks, 1) }
                        Hucre { width: csvSatiri.g * csvSatiri.k[5]; height: csvSatiri.height; text:
sayi(csvSatiri.modelData.p0l, 1) }
                        Hucre { width: csvSatiri.g * csvSatiri.k[6]; height: csvSatiri.height; text:
sayi(csvSatiri.modelData.p0r, 1) }
                        Hucre { width: csvSatiri.g * csvSatiri.k[7]; height: csvSatiri.height; text:
sayi(csvSatiri.modelData.basinc, 2) }
                        Hucre { width: csvSatiri.g * csvSatiri.k[8]; height: csvSatiri.height; text:
sayi(csvSatiri.modelData.debi, 2) }
                        Hucre {
                            width: csvSatiri.g * csvSatiri.k[9]; height: csvSatiri.height
                            text: !csvSatiri.modelData.gecerli ? txt("hatali") : (csvSatiri.modelData.secili ? txt("dahil") :
txt("haric"))
                            color: !csvSatiri.modelData.gecerli ? "#f87171" : (csvSatiri.modelData.secili ? "#4ade80" :
"#9ca3af")
                        }
                    }
                }
            }

            Rectangle {
                width: parent.width
                height: 64
                color: "#0a0a0d"
                radius: 12
                Row {
                    anchors.centerIn: parent
                    spacing: 12
                    Dugme {
                        width: 150
                        text: txt("indirCsv")
                        renk: "#16a34a"
                        onClicked: {
                            var yol = database.csvDisaAktar(olcumId)
                            csvOnizlemePopup.close()
                            bildirimKutusu.goster(yol.length > 0 ? txt("csvKaydedildi") + yol : txt("csvOlusturulamadi"),
yol.length === 0)
                        }
                    }
                    Dugme { width: 100; text: txt("kapat"); renk: "#1e2a3f"; onClicked: csvOnizlemePopup.close() }
                }
            }
        }
    }
}
```

qml\Translations.qml

04.11.2026 11:45

```
pragma Singleton
import QtQuick

QtObject {
    property bool turkish: true

    function toggle() {
        turkish = !turkish
    }
}
```

src\Backend.cpp

04.11.2026 11:45

```
#include "Backend.h"
#include <QDebug>

Backend::Backend(QObject *parent)
    : QObject(parent)
{
}

QString Backend::durum() const
{
    return "Baglanti bekleniyor...";
}

bool Backend::girisYap(const QString &kullaniciAdi, const QString &sifre)
{
    if (kullaniciAdi == "admin" && sifre == "admin123") {
        qDebug() << "Giris Basarili:" << kullaniciAdi;
        return true;
    }

    qDebug() << "Giris Basarisiz:" << kullaniciAdi;
    return false;
}

bool Backend::sifreDogrula(const QString &sifre)
{
    return sifre == "admin123";
}
```

src\Backend.h

04.11.2026 11:45

```
#pragma once
#include <QObject>

class Backend : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString durum READ durum CONSTANT)

public:
    explicit Backend(QObject *parent = nullptr);
    QString durum() const;
    Q_INVOKABLE bool girisYap(const QString &kullaniciAdi, const QString &sifre);
    Q_INVOKABLE bool sifreDogrula(const QString &sifre);
};
```

src\Calculator.cpp

04.11.2026 11:45

```
#include "Calculator.h"
#include "SliperModel.h"
#include <QtMath>
#include <algorithm>
#include <cmath>

using SliperModel::KONUM_TOLERANSI_MM;
using SliperModel::UST_YAKALAMA_TOLERANSI_MM;
using SliperModel::ALT_YAKALAMA_TOLERANSI_MM;
using SliperModel::DURGUN_TOLERANS_MM;
using SliperModel::DURMA_TOLERANSI_MM;
using SliperModel::INISTE_KALDIRMA_ESIGI_MM;
using SliperModel::DURMA_ONAYI_S;
using SliperModel::MIN_INIS_YOLU_MM;
using SliperModel::MIN_INIS_SURESI_S;
using SliperModel::KALDIRMA_ESIGI_MM;
using SliperModel::ONCESI_PENCERE_S;
using SliperModel::SONRASI_PENCERE_S;
using SliperModel::HAREKET_PAYI_S;
using SliperModel::P0R_OTURMA_S;
using SliperModel::UST_TAMPON_S;
using SliperModel::HAZIRLAMA_SURESI_S;
using SliperModel::HAZIRLAMA_BASINC_TOLERANSI_MBAR;

Calculator::Calculator(QObject *parent)
    : QObject(parent)
{
}

QString Calculator::durum() const { return m_durum; }
int Calculator::strokeSayisi() const { return m_strokeSayisi; }
bool Calculator::duraklatildi() const { return m_duraklatildi; }
bool Calculator::hazirlaniyor() const { return m_hazirlaniyor; }
bool Calculator::hazir() const { return m_hazir; }
bool Calculator::sonStrokeGecerli() const { return m_sonStrokeGecerli; }
QVariantMap Calculator::sonStroke() const { return m_sonStroke; }
double Calculator::ustKonumMm() const { return m_ustKonumMm; }
double Calculator::altKonumMm() const { return m_altKonumMm; }

void Calculator::setUstKonumMm(double deger)
{
    if (qFuzzyCompare(m_ustKonumMm, deger)) return;
    m_ustKonumMm = deger;
    emit konumSinirlariChanged();
}

void Calculator::setAltKonumMm(double deger)
{
    if (qFuzzyCompare(m_altKonumMm, deger)) return;
    m_altKonumMm = deger;
    emit konumSinirlariChanged();
}

double Calculator::ustEsik() const { return yon() * m_ustKonumMm - UST_YAKALAMA_TOLERANSI_MM; }

double Calculator::ustYakalamaToleransiMm() const { return UST_YAKALAMA_TOLERANSI_MM; }
double Calculator::minInisYoluMm() const { return MIN_INIS_YOLU_MM; }
double Calculator::altEsik() const { return yon() * m_altKonumMm + ALT_YAKALAMA_TOLERANSI_MM; }

void Calculator::durumAyarla(const QString &yeniDurum)
{
    if (yeniDurum != m_durum) {
        m_durum = yeniDurum;
        emit durumChanged();
    }
}

void Calculator::konumGuncelle(double zamanS, double konumMm, double basincMbar)
```

```
{
    if (m_duraklatildi) {
        return;
    }

    const double yukseklikMm = yon() * konumMm;
    const Ornek yeniOrnek { zamanS, yukseklikMm, basincMbar };

    if (m_hazirlaniyor) {
        hazirlamaOrnegiIsle(yeniOrnek);
    }

    if (m_bitisSonrasiToplaniyor) {
        const bool boruTekrarKaldirildi = yukseklikMm > m_bitisYuksekligiMm + KALDIRMA_ESIGI_MM;
        if (boruTekrarKaldirildi) {
            strokuBitir();
        } else {
            m_strokeOrnekleri.append(yeniOrnek);
            if (zamanS - m_bitisZamaniS >= SONRASI_PENCERE_S) {
                strokuBitir();
            }
            return;
        }
    }

    if (m_durum == "INIYOR" && kaldirilanInisiKapat(yukseklikMm)) {
        strokuBitir();
    }

    if (yukseklikMm > ustEsik()) {
        m_strokeOrnekleri.clear();
        m_ustteBekleyenOrnekler.append(yeniOrnek);
        while (!m_ustteBekleyenOrnekler.isEmpty()
               && zamanS - m_ustteBekleyenOrnekler.first().zamanS > UST_TAMPON_S) {
            m_ustteBekleyenOrnekler.removeFirst();
        }
        durumAyarla("YUKARIDA");
        return;
    }

    if (yukseklikMm > altEsik()) {
        if (m_durum == "YUKARIDA") {
            if (!m_hazir) {
                return;
            }
            strokuBaslat(zamanS);
            m_inisBaslangicYuksekligiMm = yukseklikMm;
        }
        if (m_durum == "YUKARIDA" || m_durum == "INIYOR") {
            m_strokeOrnekleri.append(yeniOrnek);
            durumAyarla("INIYOR");
            const int durmaIndeksi = durmaBaslangicIndeksi(zamanS);
            if (durmaIndeksi >= 0) {
                inisiKapat(m_strokeOrnekleri[durmaIndeksi].zamanS, durmaIndeksi);
            }
        }
        return;
    }

    if (m_durum == "INIYOR") {
        m_strokeOrnekleri.append(yeniOrnek);
        inisiKapat(zamanS);
        return;
    }
    if (m_durum != "TAMAMLANDI") {
        durumAyarla("BEKLENIYOR");
    }
}

int Calculator::durmaBaslangicIndeksi(double zamanS) const
```

```
{
    if (m_strokeOrnekleri.isEmpty()) return -1;

    const double inilenYolMm = m_inisBaslangicYuksekligiMm - m_strokeOrnekleri.last().yukseklikMm;
    if (inilenYolMm < MIN_INIS_YOLU_MM) return -1;
    if (zamanS - m_baslangicZamaniS < MIN_INIS_SURESI_S) return -1;

    double enAzMm = m_strokeOrnekleri.last().yukseklikMm;
    double enCokMm = enAzMm;
    int durmaBasiIndeksi = m_strokeOrnekleri.size() - 1;
    for (int i = m_strokeOrnekleri.size() - 1; i >= 0; --i) {
        const double adayEnAzMm = std::min(enAzMm, m_strokeOrnekleri[i].yukseklikMm);
        const double adayEnCokMm = std::max(enCokMm, m_strokeOrnekleri[i].yukseklikMm);
        if (adayEnCokMm - adayEnAzMm >= DURMA_TOLERANSI_MM) break;
        enAzMm = adayEnAzMm;
        enCokMm = adayEnCokMm;
        durmaBasiIndeksi = i;
    }

    const double durmaSuresiS = zamanS - m_strokeOrnekleri[durmaBasiIndeksi].zamanS;
    if (durmaSuresiS < DURMA_ONAYI_S) return -1;

    return durmaBasiIndeksi;
}

bool Calculator::kaldirilanInisiKapat(double yeniYukseklikMm)
{
    if (m_strokeOrnekleri.isEmpty()) return false;

    int enAltIndeks = 0;
    for (int i = 1; i < m_strokeOrnekleri.size(); ++i) {
        if (m_strokeOrnekleri[i].yukseklikMm < m_strokeOrnekleri[enAltIndeks].yukseklikMm) {
            enAltIndeks = i;
        }
    }
    const double enAltYukseklikMm = m_strokeOrnekleri[enAltIndeks].yukseklikMm;

    const bool boruKaldirildi = yeniYukseklikMm > enAltYukseklikMm + INISTE_KALDIRMA_ESIGI_MM;
    if (!boruKaldirildi) return false;
    if (m_inisBaslangicYuksekligiMm - enAltYukseklikMm < MIN_INIS_YOLU_MM) return false;

    const double durgunUstSiniriMm = enAltYukseklikMm + DURMA_TOLERANSI_MM;
    int altaVarisIndeksi = 0;
    while (altaVarisIndeksi < enAltIndeks
           && m_strokeOrnekleri[altaVarisIndeksi].yukseklikMm > durgunUstSiniriMm) {
        ++altaVarisIndeksi;
    }
    int sonDurgunIndeks = m_strokeOrnekleri.size() - 1;
    while (sonDurgunIndeks > enAltIndeks
           && m_strokeOrnekleri[sonDurgunIndeks].yukseklikMm > durgunUstSiniriMm) {
        --sonDurgunIndeks;
    }
    m_strokeOrnekleri.resize(sonDurgunIndeks + 1);

    inisiKapat(m_strokeOrnekleri[altaVarisIndeksi].zamanS, altaVarisIndeksi);
    return true;
}

void Calculator::inisiKapat(double zamanS, int bitisIndeksi)
{
    m_altaVarisIndeksi = bitisIndeksi >= 0 ? bitisIndeksi : m_strokeOrnekleri.size() - 1;
    m_bitisYukseklikMm = m_strokeOrnekleri[m_altaVarisIndeksi].yukseklikMm;
    m_bitisZamaniS = zamanS;
    m_bitisTarihi = QDateTime::currentDateTime();
    m_bitisSonrasiToplaniyor = true;
    durumAyarla("TAMAMLANDI");
}

void Calculator::strokuBaslat(double zamanS)
{
```

```
    m_baslangicZamaniS = zamanS;
    m_strokeOrnekleri.clear();
    m_durgunOrnekler.clear();
    m_strokeP0lMbar = m_hazirlananP0lMbar;
    hazirligiSifirla();

    const QVector<Ornek> &ustte = m_ustteBekleyenOrnekler;

    int hareketBaslangicIndeksi = ustte.size();
    if (!ustte.isEmpty()) {
        double enYuksekMm = ustte.first().yukseklikMm;
        for (const Ornek &ornek : ustte) enYuksekMm = std::max(enYuksekMm, ornek.yukseklikMm);
        while (hareketBaslangicIndeksi > 0
               && ustte[hareketBaslangicIndeksi - 1].yukseklikMm < enYuksekMm - DURGUN_TOLERANS_MM) {
            --hareketBaslangicIndeksi;
        }
    }
    const double hareketZamaniS = hareketBaslangicIndeksi < ustte.size()
        ? ustte[hareketBaslangicIndeksi].zamanS : zamanS;

    int durgunBitisIndeksi = hareketBaslangicIndeksi;
    while (durgunBitisIndeksi > 0
           && hareketZamaniS - ustte[durgunBitisIndeksi - 1].zamanS < HAREKET_PAYI_S) {
        --durgunBitisIndeksi;
    }
    if (durgunBitisIndeksi == 0) durgunBitisIndeksi = hareketBaslangicIndeksi;

    for (int i = 0; i < hareketBaslangicIndeksi; ++i) {
        const bool pencereIcinde = hareketZamaniS - ustte[i].zamanS <= ONCESI_PENCERE_S + HAREKET_PAYI_S;
        if (!pencereIcinde) continue;
        if (i < durgunBitisIndeksi) m_durgunOrnekler.append(ustte[i]);
        else m_strokeOrnekleri.append(ustte[i]);
    }
    for (int i = hareketBaslangicIndeksi; i < ustte.size(); ++i) {
        m_strokeOrnekleri.append(ustte[i]);
    }
    m_ustteBekleyenOrnekler.clear();
}

void Calculator::strokuBitir()
{
    m_bitisSonrasiToplaniyor = false;
    const QVector<Ornek> &ornekler = m_strokeOrnekleri;
    if (m_altaVarisIndeksi < 1 || m_altaVarisIndeksi >= ornekler.size()) {
        m_strokeOrnekleri.clear();
        return;
    }
    const int sonInisIndeksi = m_altaVarisIndeksi;

    const double p0l = m_strokeP0lMbar;

    QVector<double> bitisSonrasiBasinclar;
    for (int i = sonInisIndeksi + 1; i < ornekler.size(); ++i) {
        if (ornekler[i].zamanS - m_bitisZamaniS >= P0R_OTURMA_S) {
            bitisSonrasiBasinclar.append(ornekler[i].basincMbar);
        }
    }
    if (bitisSonrasiBasinclar.isEmpty()) {
        for (int i = sonInisIndeksi + 1; i < ornekler.size(); ++i) {
            bitisSonrasiBasinclar.append(ornekler[i].basincMbar);
        }
    }
    const double p0r = bitisSonrasiBasinclar.isEmpty()
        ? p0l
        : SliperModel::durgunBasincMbar(bitisSonrasiBasinclar);

    QVector<double> inisZamanlari, inisYukseklikleri, inisBasinclari;
    for (int i = 0; i <= sonInisIndeksi; ++i) {
        inisZamanlari.append(ornekler[i].zamanS);
        inisYukseklikleri.append(ornekler[i].yukseklikMm);
```

```
        inisBasinclari.append(ornekler[i].basincMbar);
    }
    const double hizMs = SliperModel::strokeHiziMs(inisZamanlari, inisYukseklikleri);
    const double debiM3h = SliperModel::hizdanDebiM3h(hizMs);
    const double pMaks = SliperModel::filtreliMaksimum(inisZamanlari, inisBasinclari);
    const double basincMbar = SliperModel::strokeBasinciMbar(pMaks, p0l);

    const int inisOrnekSayisi = sonInisIndeksi + 1;
    const QString gecersizNedeni = SliperModel::strokeGecersizNedeni(
        inisOrnekSayisi, SliperModel::enBuyukOrnekAraligiS(inisZamanlari), hizMs, basincMbar);

    const double ilkZamanS = m_durgunOrnekler.isEmpty()
        ? ornekler.first().zamanS
        : m_durgunOrnekler.first().zamanS;
    QVariantList zamanListesi, konumListesi, basincListesi;
    auto hamVeriyeEkle = [&](const Ornek &ornek) {
        zamanListesi.append(std::round((ornek.zamanS - ilkZamanS) * 1000.0) / 1000.0);
        konumListesi.append(std::round(yon() * ornek.yukseklikMm * 10.0) / 10.0);
        basincListesi.append(std::round(ornek.basincMbar * 100.0) / 100.0);
    };
    for (const Ornek &ornek : m_durgunOrnekler) hamVeriyeEkle(ornek);
    for (const Ornek &ornek : ornekler) hamVeriyeEkle(ornek);

    QVariantMap hamVeri;
    hamVeri["t"] = zamanListesi;
    hamVeri["x"] = konumListesi;
    hamVeri["p"] = basincListesi;
    hamVeri["tBaslangic"] = m_baslangicZamaniS - ilkZamanS;
    hamVeri["tBitis"] = m_bitisZamaniS - ilkZamanS;
    hamVeri["yon"] = yon();

    QVariantMap stroke;
    stroke["tarih"] = m_bitisTarihi.toString(Qt::ISODate);
    stroke["sure"] = m_bitisZamaniS - m_baslangicZamaniS;
    stroke["pMaks"] = pMaks;
    stroke["p0l"] = p0l;
    stroke["p0r"] = p0r;
    stroke["basinc"] = basincMbar;
    stroke["debi"] = debiM3h;
    stroke["hiz"] = hizMs;
    stroke["konum"] = yon() * ornekler[sonInisIndeksi].yukseklikMm;
    stroke["gecerli"] = gecersizNedeni.isEmpty();
    stroke["gecersizNedeni"] = gecersizNedeni;
    stroke["hamVeri"] = hamVeri;

    m_strokeOrnekleri.clear();
    m_durgunOrnekler.clear();
    m_altaVarisIndeksi = -1;

    m_sonStrokeGecerli = gecersizNedeni.isEmpty();
    emit sonStrokeGecerliChanged();
    m_strokeSayisi++;
    emit strokeSayisiChanged();
    stroke["no"] = m_strokeSayisi;
    m_sonStroke = stroke;
    emit sonStrokeChanged();

    emit strokeTamamlandi(stroke);
}

void Calculator::hazirla()
{
    if (m_duraklatildi || m_durum == "INIYOR" || m_bitisSonrasiToplaniyor) {
        return;
    }
    m_hazir = false;
    m_hazirlamaOrnekleri.clear();
    m_hazirlaniyor = true;
    emit hazirlikChanged();
}
```

```
void Calculator::hazirlamaOrnegiIsle(const Ornek &yeniOrnek)
{
    if (yeniOrnek.yukseklikMm <= ustEsik()) {
        hazirlamayiIptalEt();
        return;
    }

    m_hazirlamaOrnekleri.append(yeniOrnek);
    const double pencereSuresiS = yeniOrnek.zamanS - m_hazirlamaOrnekleri.first().zamanS;
    if (pencereSuresiS < HAZIRLAMA_SURESI_S) {
        return;
    }

    double enDusukBasincMbar = yeniOrnek.basincMbar;
    double enYuksekBasincMbar = yeniOrnek.basincMbar;
    double enDusukYukseklikMm = yeniOrnek.yukseklikMm;
    double enYuksekYukseklikMm = yeniOrnek.yukseklikMm;
    QVector<double> pencereBasinclari;
    for (const Ornek &ornek : m_hazirlamaOrnekleri) {
        enDusukBasincMbar = std::min(enDusukBasincMbar, ornek.basincMbar);
        enYuksekBasincMbar = std::max(enYuksekBasincMbar, ornek.basincMbar);
        enDusukYukseklikMm = std::min(enDusukYukseklikMm, ornek.yukseklikMm);
        enYuksekYukseklikMm = std::max(enYuksekYukseklikMm, ornek.yukseklikMm);
        pencereBasinclari.append(ornek.basincMbar);
    }

    const bool basincSakin = enYuksekBasincMbar - enDusukBasincMbar <= HAZIRLAMA_BASINC_TOLERANSI_MBAR;
    const bool boruSakin = enYuksekYukseklikMm - enDusukYukseklikMm <= DURGUN_TOLERANS_MM;
    if (!basincSakin || !boruSakin) {
        m_hazirlamaOrnekleri.removeFirst();
        return;
    }

    m_hazirlananP0lMbar = SliperModel::durgunBasincMbar(pencereBasinclari);
    m_hazirlamaOrnekleri.clear();
    m_hazirlaniyor = false;
    m_hazir = true;
    emit hazirlikChanged();
}

void Calculator::hazirlamayiIptalEt()
{
    m_hazirlamaOrnekleri.clear();
    if (m_hazirlaniyor) {
        m_hazirlaniyor = false;
        emit hazirlikChanged();
    }
}

void Calculator::hazirligiSifirla()
{
    const bool degisti = m_hazirlaniyor || m_hazir;
    m_hazirlaniyor = false;
    m_hazir = false;
    m_hazirlamaOrnekleri.clear();
    if (degisti) {
        emit hazirlikChanged();
    }
}

void Calculator::duraklat()
{
    if (m_duraklatildi) {
        return;
    }
    hazirligiSifirla();

    if (m_bitisSonrasiToplaniyor) {
        strokuBitir();
```

```
    }
    m_duraklatildi = true;
    m_strokeOrnekleri.clear();
    m_ustteBekleyenOrnekler.clear();
    m_durgunOrnekler.clear();
    m_altaVarisIndeksi = -1;
    emit duraklatildiChanged();
}

void Calculator::devamEt()
{
    if (!m_duraklatildi) {
        return;
    }

    m_duraklatildi = false;
    durumAyarla("BEKLENIYOR");
    emit duraklatildiChanged();
}

void Calculator::sifirla()
{
    m_strokeSayisi = 0;
    m_durum = "BEKLENIYOR";
    m_duraklatildi = false;
    m_sonStrokeGecerli = true;
    m_sonStroke.clear();
    m_strokeOrnekleri.clear();
    m_ustteBekleyenOrnekler.clear();
    m_durgunOrnekler.clear();
    m_altaVarisIndeksi = -1;
    m_inisBaslangicYuksekligiMm = 0.0;
    m_bitisYuksekligiMm = 0.0;
    m_bitisSonrasiToplaniyor = false;
    hazirligiSifirla();
    emit strokeSayisiChanged();
    emit durumChanged();
    emit duraklatildiChanged();
    emit sonStrokeGecerliChanged();
    emit sonStrokeChanged();
}

QVariantMap Calculator::boruHattiTahminHesapla(double aKesisimMbar, double bEgimMbarHm3,
                                               double debiM3h, double boruCapiMm,
                                               double boruUzunluguM,
                                               double pompalamaYuksekligiM,
                                               double yogunlukKgM3,
                                               double hataPayiYuzde) const
{
    return SliperModel::basincTahmini(aKesisimMbar, bEgimMbarHm3, debiM3h, boruCapiMm,
                                      boruUzunluguM, pompalamaYuksekligiM, yogunlukKgM3,
                                      hataPayiYuzde);
}

QVariantList Calculator::tahminTablosuHesapla(double aKesisimMbar, double bEgimMbarHm3,
                                              const QVariantMap &ayarlar) const
{
    return SliperModel::tahminTablosu(aKesisimMbar, bEgimMbarHm3, ayarlar);
}

QVariantMap Calculator::varsayilanTahminAyarlari() const
{
    return SliperModel::varsayilanTahminAyarlari();
}

QString Calculator::tahminGuvenilmezNedeni(double kesisimA, double egimB, double r2) const
{
    return SliperModel::tahminGuvenilmezNedeni(kesisimA, egimB, r2);
}
```

```
bool Calculator::tahminKarariVerilemez(const QString &guvenilmezNedeni) const
{
    return SliperModel::tahminKarariVerilemez(guvenilmezNedeni);
}

QString Calculator::pompaSorunuNedeni(const QVariantMap &tahminSatiri, double pompaMaksBar) const
{
    return SliperModel::pompaSorunuNedeni(tahminSatiri, pompaMaksBar);
}

double Calculator::schleibingerA(double kesisimA) const { return SliperModel::schleibingerA(kesisimA); }
double Calculator::schleibingerB(double egimB) const { return SliperModel::schleibingerB(egimB); }
double Calculator::sliperBoruCapiMm() const { return SliperModel::BORU_CAPI_M * 1000.0; }
double Calculator::sliperBoruUzunluguMm() const { return SliperModel::BORU_UZUNLUGU_M * 1000.0; }

QVariantList Calculator::hizEgrisiHesapla(const QVariantList &zamanS, const QVariantList &konumMm,
                                          double konumYonu) const
{
    QVector<double> zamanlar, konumlar;
    for (const QVariant &zaman : zamanS) zamanlar.append(zaman.toDouble());
    for (const QVariant &konum : konumMm) konumlar.append(konum.toDouble());

    QVariantList egri;
    for (double hiz : SliperModel::hizEgrisiMs(zamanlar, konumlar, konumYonu)) egri.append(hiz);
    return egri;
}
```

src\Calculator.h

04.11.2026 11:45

```
#pragma once
#include <QObject>
#include <QVariantMap>
#include <QVector>
#include <QDateTime>

class Calculator : public QObject
{
    Q_OBJECT
    Q_PROPERTY(QString durum READ durum NOTIFY durumChanged)
    Q_PROPERTY(int strokeSayisi READ strokeSayisi NOTIFY strokeSayisiChanged)
    Q_PROPERTY(bool duraklatildi READ duraklatildi NOTIFY duraklatildiChanged)
    Q_PROPERTY(bool hazirlaniyor READ hazirlaniyor NOTIFY hazirlikChanged)
    Q_PROPERTY(bool hazir READ hazir NOTIFY hazirlikChanged)
    Q_PROPERTY(bool sonStrokeGecerli READ sonStrokeGecerli NOTIFY sonStrokeGecerliChanged)
    Q_PROPERTY(QVariantMap sonStroke READ sonStroke NOTIFY sonStrokeChanged)
    Q_PROPERTY(double ustKonumMm READ ustKonumMm WRITE setUstKonumMm NOTIFY konumSinirlariChanged)
    Q_PROPERTY(double altKonumMm READ altKonumMm WRITE setAltKonumMm NOTIFY konumSinirlariChanged)

public:
    explicit Calculator(QObject *parent = nullptr);
    QString durum() const;
    int strokeSayisi() const;
    bool duraklatildi() const;
    bool hazirlaniyor() const;
    bool hazir() const;
    bool sonStrokeGecerli() const;
    QVariantMap sonStroke() const;
    double ustKonumMm() const;
    double altKonumMm() const;
    void setUstKonumMm(double deger);
    void setAltKonumMm(double deger);

    Q_INVOKABLE void konumGuncelle(double zamanS, double konumMm, double basincMbar);
    Q_INVOKABLE void hazirla();
    Q_INVOKABLE void duraklat();
    Q_INVOKABLE void devamEt();
    Q_INVOKABLE void sifirla();

    Q_INVOKABLE QVariantMap boruHattiTahminHesapla(double aKesisimMbar, double bEgimMbarHm3,
                                                   double debiM3h, double boruCapiMm,
                                                   double boruUzunluguM,
                                                   double pompalamaYuksekligiM,
                                                   double yogunlukKgM3,
                                                   double hataPayiYuzde) const;
    Q_INVOKABLE QVariantList tahminTablosuHesapla(double aKesisimMbar, double bEgimMbarHm3,
                                                  const QVariantMap &ayarlar) const;
    Q_INVOKABLE QVariantMap varsayilanTahminAyarlari() const;
    Q_INVOKABLE QString tahminGuvenilmezNedeni(double kesisimA, double egimB, double r2) const;
    Q_INVOKABLE bool tahminKarariVerilemez(const QString &guvenilmezNedeni) const;
    Q_INVOKABLE QString pompaSorunuNedeni(const QVariantMap &tahminSatiri, double pompaMaksBar) const;
    Q_INVOKABLE double schleibingerA(double kesisimA) const;
    Q_INVOKABLE double schleibingerB(double egimB) const;
    Q_INVOKABLE double ustYakalamaToleransiMm() const;
    Q_INVOKABLE double minInisYoluMm() const;
    Q_INVOKABLE double sliperBoruCapiMm() const;
    Q_INVOKABLE double sliperBoruUzunluguMm() const;
    Q_INVOKABLE QVariantList hizEgrisiHesapla(const QVariantList &zamanS, const QVariantList &konumMm,
                                              double konumYonu) const;

signals:
    void durumChanged();
    void strokeSayisiChanged();
    void duraklatildiChanged();
    void hazirlikChanged();
    void sonStrokeGecerliChanged();
    void sonStrokeChanged();
    void konumSinirlariChanged();
```

```
    void strokeTamamlandi(const QVariantMap &stroke);

private:
    struct Ornek {
        double zamanS;
        double yukseklikMm;
        double basincMbar;
    };

    void strokuBaslat(double zamanS);
    void strokuBitir();
    void hazirlamaOrnegiIsle(const Ornek &yeniOrnek);
    void hazirlamayiIptalEt();
    void hazirligiSifirla();
    int durmaBaslangicIndeksi(double zamanS) const;
    bool kaldirilanInisiKapat(double yeniYukseklikMm);
    void inisiKapat(double zamanS, int bitisIndeksi = -1);
    void durumAyarla(const QString &yeniDurum);
    double yon() const { return m_ustKonumMm >= m_altKonumMm ? 1.0 : -1.0; }
    double ustEsik() const;
    double altEsik() const;

    QString m_durum = "BEKLENIYOR";
    int m_strokeSayisi = 0;
    bool m_duraklatildi = false;
    bool m_sonStrokeGecerli = true;
    QVariantMap m_sonStroke;
    double m_ustKonumMm = 40.0;
    double m_altKonumMm = 540.0;

    bool m_hazirlaniyor = false;
    bool m_hazir = false;
    QVector<Ornek> m_hazirlamaOrnekleri;
    double m_hazirlananP0lMbar = 0.0;
    double m_strokeP0lMbar = 0.0;
    QVector<Ornek> m_ustteBekleyenOrnekler;
    QVector<Ornek> m_durgunOrnekler;
    QVector<Ornek> m_strokeOrnekleri;
    int m_altaVarisIndeksi = -1;
    double m_inisBaslangicYuksekligiMm = 0.0;
    double m_bitisYuksekligiMm = 0.0;
    double m_baslangicZamaniS = 0.0;
    double m_bitisZamaniS = 0.0;
    QDateTime m_bitisTarihi;
    bool m_bitisSonrasiToplaniyor = false;
};
```

src\Database.cpp

04.11.2026 11:45

```
#include "Database.h"
#include <QSqlQuery>
#include <QSqlError>
#include <QDebug>
#include <QDateTime>
#include <QVector>
#include <QFile>
#include <QTextStream>
#include <QStandardPaths>
#include <QDir>
#include <QJsonDocument>
#include <QJsonObject>
#include <QSet>
#include <QHash>
#include <algorithm>
#include "SliperModel.h"

Database::Database(QObject *parent)
    : QObject(parent)
{
    const QString klasor = QStandardPaths::writableLocation(QStandardPaths::AppLocalDataLocation) + "/database";
    QDir().mkpath(klasor);

    m_veriTabaniDosyaYolu = klasor + "/sliper.db";

    m_db = QSqlDatabase::addDatabase("QSQLITE");
    m_db.setDatabaseName(m_veriTabaniDosyaYolu);

    if (!m_db.open()) {
        qWarning() << "Veritabani acilamadi:" << m_db.lastError().text();
    } else {
        qDebug() << "Veritabani basariyla acildi.";
        tablolariOlustur();
    }
}

void Database::tablolariOlustur()
{
    QSqlQuery sorgu;

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS Olcumler ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "tarih TEXT, "
        "musteri TEXT, "
        "recete TEXT, "
        "agirlik REAL"
        ")"
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS Strokelar ("
        "id INTEGER PRIMARY KEY AUTOINCREMENT, "
        "olcumId INTEGER, "
        "basinc REAL, "
        "konum REAL, "
        "debi REAL, "
        "gecerli INTEGER, "
        "FOREIGN KEY(olcumId) REFERENCES Olcumler(id)"
        ")"
    );

    auto kolonEkle = [](const QString &tablo, const QString &kolon, const QString &tanim) {
        QSqlQuery kontrol(QString("PRAGMA table_info(%1)").arg(tablo));
        while (kontrol.next()) {
            if (kontrol.value("name").toString() == kolon) return;
        }
        QSqlQuery migrasyon;
```

```
    if (!migrasyon.exec(QString("ALTER TABLE %1 ADD COLUMN %2 %3").arg(tablo, kolon, tanim))) {
        qWarning() << "Kolon eklenemedi:" << tablo << kolon << migrasyon.lastError().text();
    }
};

kolonEkle("Olcumler", "yer", "TEXT DEFAULT ''");
kolonEkle("Olcumler", "yorum", "TEXT DEFAULT ''");
kolonEkle("Olcumler", "tahminAyarlari", "TEXT DEFAULT ''");
kolonEkle("Olcumler", "daraBasinci", "REAL DEFAULT 0");

kolonEkle("Strokelar", "debi", "REAL DEFAULT 0");
kolonEkle("Strokelar", "tarih", "TEXT DEFAULT ''");
kolonEkle("Strokelar", "sure", "REAL DEFAULT 0");
kolonEkle("Strokelar", "pMaks", "REAL DEFAULT 0");
kolonEkle("Strokelar", "p0l", "REAL DEFAULT 0");
kolonEkle("Strokelar", "p0r", "REAL DEFAULT 0");
kolonEkle("Strokelar", "hiz", "REAL DEFAULT 0");
kolonEkle("Strokelar", "agirlik", "REAL DEFAULT 0");
kolonEkle("Strokelar", "secili", "INTEGER DEFAULT 1");
kolonEkle("Strokelar", "gecersizNedeni", "TEXT DEFAULT ''");
kolonEkle("Strokelar", "hamVeri", "TEXT DEFAULT ''");

bool otomatikElemeKolonlariVardi = false;
{
    QSqlQuery kontrol("PRAGMA table_info(Strokelar)");
    while (kontrol.next()) {
        if (kontrol.value("name").toString() == "kullaniciKarari") otomatikElemeKolonlariVardi = true;
    }
}
kolonEkle("Strokelar", "otomatikHaricNedeni", "TEXT DEFAULT ''");
kolonEkle("Strokelar", "kullaniciKarari", "INTEGER DEFAULT NULL");
if (!otomatikElemeKolonlariVardi) {
    QSqlQuery eskiKararlar;
    eskiKararlar.exec("UPDATE Strokelar SET kullaniciKarari = 0 WHERE secili = 0");
    QSqlQuery olcumler("SELECT DISTINCT olcumId FROM Strokelar");
    QList<int> olcumIdleri;
    while (olcumler.next()) olcumIdleri.append(olcumler.value(0).toInt());
    for (int olcumId : olcumIdleri) otomatikElemeUygula(olcumId);
}

sorgu.exec(
    "CREATE TABLE IF NOT EXISTS Ayarlar ("
    "anahtar TEXT PRIMARY KEY, "
    "deger TEXT"
    ")"
);

sorgu.exec(
    "CREATE TABLE IF NOT EXISTS Kalibrasyonlar ("
    "sensor TEXT PRIMARY KEY, "
    "deger1 REAL, "
    "deger2 REAL, "
    "tarih TEXT"
    ")"
);

sorgu.exec(
    "CREATE TABLE IF NOT EXISTS LoadCellNoktalari ("
    "id INTEGER PRIMARY KEY AUTOINCREMENT, "
    "hedefKg REAL, "
    "hamDeger REAL"
    ")"
);

sorgu.exec(
    "CREATE TABLE IF NOT EXISTS MesafeNoktalari ("
    "id INTEGER PRIMARY KEY AUTOINCREMENT, "
    "hedefMm REAL, "
    "hamDeger REAL"
    ")"
```

```
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS EgimKalibrasyon ("
        "id INTEGER PRIMARY KEY CHECK (id = 1), "
        "biasX REAL, biasY REAL, biasZ REAL, "
        "gainX REAL, gainY REAL, gainZ REAL, "
        "tarih TEXT"
        ")"
    );

    sorgu.exec(
        "CREATE TABLE IF NOT EXISTS KalibrasyonTarihleri ("
        "sensor TEXT PRIMARY KEY, "
        "tarih TEXT"
        ")"
    );

    qDebug() << "Tablolar hazir.";
}

int Database::olcumBaslat(const QString &musteri, const QString &recete, double agirlik,
                          const QString &yer, const QString &yorum)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO Olcumler (tarih, musteri, recete, agirlik, yer, yorum, tahminAyarlari) "
        "VALUES (:tarih, :musteri, :recete, :agirlik, :yer, :yorum, :tahminAyarlari)"
    );

    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));
    sorgu.bindValue(":musteri", musteri);
    sorgu.bindValue(":recete", recete);
    sorgu.bindValue(":agirlik", agirlik);
    sorgu.bindValue(":yer", yer);
    sorgu.bindValue(":yorum", yorum);
    sorgu.bindValue(":tahminAyarlari", jsonYaz(varsayilanTahminAyarlariGetir()));

    if (!sorgu.exec()) {
        qWarning() << "Olcum baslatilamadi:" << sorgu.lastError().text();
        return -1;
    }

    int yeniId = sorgu.lastInsertId().toInt();
    qDebug() << "Yeni olcum baslatildi, id:" << yeniId;
    return yeniId;
}

bool Database::olcumBilgisiGuncelle(int olcumId, const QString &yer, const QString &musteri,
                                    const QString &recete, const QString &yorum)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "UPDATE Olcumler SET yer = :yer, musteri = :musteri, recete = :recete, yorum = :yorum "
        "WHERE id = :id"
    );
    sorgu.bindValue(":yer", yer);
    sorgu.bindValue(":musteri", musteri);
    sorgu.bindValue(":recete", recete);
    sorgu.bindValue(":yorum", yorum);
    sorgu.bindValue(":id", olcumId);

    if (!sorgu.exec()) {
        qWarning() << "Olcum bilgisi guncellenemedi:" << sorgu.lastError().text();
        return false;
    }
    return true;
}

bool Database::olcumDarasiKaydet(int olcumId, double daraBasinciMbar)
```

```
{
    if (olcumId <= 0) return false;

    QSqlQuery sorgu;
    sorgu.prepare("UPDATE Olcumler SET daraBasinci = :daraBasinci WHERE id = :id");
    sorgu.bindValue(":daraBasinci", daraBasinciMbar);
    sorgu.bindValue(":id", olcumId);

    if (!sorgu.exec()) {
        qWarning() << "Olcum darasi kaydedilemedi:" << sorgu.lastError().text();
        return false;
    }
    return true;
}

int Database::strokeKaydet(int olcumId, const QVariantMap &stroke, double agirlik)
{
    if (olcumId <= 0) {
        qWarning() << "Gecersiz olcumId, stroke kaydedilemedi.";
        return -1;
    }

    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO Strokelar (olcumId, basinc, konum, debi, gecerli, tarih, sure, pMaks, "
        "p0l, p0r, hiz, agirlik, secili, gecersizNedeni, hamVeri) "
        "VALUES (:olcumId, :basinc, :konum, :debi, :gecerli, :tarih, :sure, :pMaks, "
        ":p0l, :p0r, :hiz, :agirlik, 1, :gecersizNedeni, :hamVeri)"
    );

    sorgu.bindValue(":olcumId", olcumId);
    sorgu.bindValue(":basinc", stroke.value("basinc").toDouble());
    sorgu.bindValue(":konum", stroke.value("konum").toDouble());
    sorgu.bindValue(":debi", stroke.value("debi").toDouble());
    sorgu.bindValue(":gecerli", stroke.value("gecerli").toBool() ? 1 : 0);
    sorgu.bindValue(":tarih", stroke.value("tarih").toString());
    sorgu.bindValue(":sure", stroke.value("sure").toDouble());
    sorgu.bindValue(":pMaks", stroke.value("pMaks").toDouble());
    sorgu.bindValue(":p0l", stroke.value("p0l").toDouble());
    sorgu.bindValue(":p0r", stroke.value("p0r").toDouble());
    sorgu.bindValue(":hiz", stroke.value("hiz").toDouble());
    sorgu.bindValue(":agirlik", agirlik);
    sorgu.bindValue(":gecersizNedeni", stroke.value("gecersizNedeni").toString());
    sorgu.bindValue(":hamVeri", jsonYaz(stroke.value("hamVeri").toMap()));

    if (!sorgu.exec()) {
        qWarning() << "Stroke kaydedilemedi:" << sorgu.lastError().text();
        return -1;
    }
    qDebug() << "Stroke kaydedildi, olcumId:" << olcumId;
    const int strokeId = sorgu.lastInsertId().toInt();
    otomatikElemeUygula(olcumId);
    return strokeId;
}

void Database::otomatikElemeUygula(int olcumId)
{
    QSqlQuery sorgu;
    sorgu.prepare("SELECT id, agirlik, hamVeri FROM Strokelar "
                  "WHERE olcumId = :olcumId AND gecerli = 1 ORDER BY id ASC");
    sorgu.bindValue(":olcumId", olcumId);
    if (!sorgu.exec()) return;

    QVector<int> strokeIdleri;
    QVector<qint64> agirlikAnahtarlari;
    QVector<double> takilmaOranlari;
    while (sorgu.next()) {
        strokeIdleri.append(sorgu.value("id").toInt());
        agirlikAnahtarlari.append(qRound64(sorgu.value("agirlik").toDouble() * 10.0));
```

```
        const QVariantMap hamVeri = jsonOku(sorgu.value("hamVeri").toString());
        const QVariantList zamanlar = hamVeri.value("t").toList();
        const QVariantList konumlar = hamVeri.value("x").toList();
        const double baslangicS = hamVeri.value("tBaslangic").toDouble();
        const double bitisS = hamVeri.value("tBitis").toDouble();
        QVector<double> inisZamanlari, inisKonumlari;
        for (int i = 0; i < std::min(zamanlar.size(), konumlar.size()); ++i) {
            const double zamanS = zamanlar[i].toDouble();
            if (zamanS < baslangicS || zamanS > bitisS) continue;
            inisZamanlari.append(zamanS);
            inisKonumlari.append(konumlar[i].toDouble());
        }
        takilmaOranlari.append(SliperModel::takilmaOrani(inisZamanlari, inisKonumlari));
    }

    const QVector<int> elenecekler = SliperModel::otomatikElenecekler(agirlikAnahtarlari, takilmaOranlari);

    for (int i = 0; i < strokeIdleri.size(); ++i) {
        const QString otomatikHaricNedeni = elenecekler.contains(i) ? QStringLiteral("takilma") : QString();
        QSqlQuery guncelle;
        guncelle.prepare(
            "UPDATE Strokelar SET otomatikHaricNedeni = :neden, "
            "secili = COALESCE(kullaniciKarari, :otomatikSecili) WHERE id = :id");
        guncelle.bindValue(":neden", otomatikHaricNedeni);
        guncelle.bindValue(":otomatikSecili", otomatikHaricNedeni.isEmpty() ? 1 : 0);
        guncelle.bindValue(":id", strokeIdleri[i]);
        if (!guncelle.exec()) {
            qWarning() << "Otomatik eleme kaydedilemedi:" << guncelle.lastError().text();
        }
    }
}

bool Database::strokeSeciliAyarla(int strokeId, bool secili)
{
    QSqlQuery sorgu;
    sorgu.prepare("UPDATE Strokelar SET secili = :secili, kullaniciKarari = :kullaniciKarari WHERE id = :id");
    sorgu.bindValue(":secili", secili ? 1 : 0);
    sorgu.bindValue(":kullaniciKarari", secili ? 1 : 0);
    sorgu.bindValue(":id", strokeId);
    if (!sorgu.exec()) {
        qWarning() << "Stroke secimi guncellenemedi:" << sorgu.lastError().text();
        return false;
    }
    return true;
}

bool Database::strokeSil(int strokeId)
{
    int olcumId = -1;
    QSqlQuery olcumSorgusu;
    olcumSorgusu.prepare("SELECT olcumId FROM Strokelar WHERE id = :id");
    olcumSorgusu.bindValue(":id", strokeId);
    if (olcumSorgusu.exec() && olcumSorgusu.next()) olcumId = olcumSorgusu.value(0).toInt();

    QSqlQuery sorgu;
    sorgu.prepare("DELETE FROM Strokelar WHERE id = :id");
    sorgu.bindValue(":id", strokeId);
    if (!sorgu.exec()) {
        qWarning() << "Stroke silinemedi:" << sorgu.lastError().text();
        return false;
    }
    if (olcumId > 0) otomatikElemeUygula(olcumId);
    return true;
}

bool Database::olcumSil(int olcumId)
{
    if (olcumId <= 0) {
        qWarning() << "Gecersiz olcumId, olcum silinemedi.";
        return false;
```

```
    }

    if (!m_db.transaction()) {
        qWarning() << "Olcum silme icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery strokeSil;
    strokeSil.prepare("DELETE FROM Strokelar WHERE olcumId = :olcumId");
    strokeSil.bindValue(":olcumId", olcumId);
    if (!strokeSil.exec()) {
        qWarning() << "Strokelar silinemedi:" << strokeSil.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery olcumSil;
    olcumSil.prepare("DELETE FROM Olcumler WHERE id = :id");
    olcumSil.bindValue(":id", olcumId);
    if (!olcumSil.exec()) {
        qWarning() << "Olcum silinemedi:" << olcumSil.lastError().text();
        m_db.rollback();
        return false;
    }

    if (!m_db.commit()) {
        qWarning() << "Olcum silme commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Olcum silindi, id:" << olcumId;
    return true;
}

QVariantList Database::tumOlcumleriGetir()
{
    QVariantList sonuclar;

    QSqlQuery sorgu(
        "SELECT o.id, o.tarih, o.musteri, o.recete, o.agirlik, o.yer, o.yorum, "
        "COUNT(s.id) AS strokeSayisi "
        "FROM Olcumler o "
        "LEFT JOIN Strokelar s ON s.olcumId = o.id "
        "GROUP BY o.id "
        "ORDER BY o.id DESC"
    );

    while (sorgu.next()) {
        QVariantMap satir;
        satir["id"] = sorgu.value("id");
        satir["tarih"] = sorgu.value("tarih");
        satir["musteri"] = sorgu.value("musteri");
        satir["recete"] = sorgu.value("recete");
        satir["agirlik"] = sorgu.value("agirlik");
        satir["yer"] = sorgu.value("yer").toString();
        satir["yorum"] = sorgu.value("yorum").toString();
        satir["strokeSayisi"] = sorgu.value("strokeSayisi");
        sonuclar.append(satir);
    }

    return sonuclar;
}

QVariantList Database::strokeVerileriGetir(int olcumId)
{
    QVariantList sonuclar;

    QSqlQuery sorgu;
    sorgu.prepare(
        "SELECT id, basinc, konum, debi, gecerli, tarih, sure, pMaks, p0l, p0r, hiz, "
```

```
        "agirlik, secili, gecersizNedeni, otomatikHaricNedeni, kullaniciKarari FROM Strokelar "
        "WHERE olcumId = :olcumId ORDER BY id ASC"
    );
    sorgu.bindValue(":olcumId", olcumId);

    if (!sorgu.exec()) {
        qWarning() << "Stroke verileri alinamadi:" << sorgu.lastError().text();
        return sonuclar;
    }

    int sira = 1;
    while (sorgu.next()) {
        QVariantMap satir;
        satir["id"] = sorgu.value("id").toInt();
        satir["stroke"] = sira++;
        satir["basinc"] = sorgu.value("basinc").toDouble();
        satir["konum"] = sorgu.value("konum").toDouble();
        satir["debi"] = sorgu.value("debi").toDouble();
        satir["gecerli"] = sorgu.value("gecerli").toBool();
        const QDateTime zaman = QDateTime::fromString(sorgu.value("tarih").toString(), Qt::ISODate);
        satir["tarih"] = zaman.isValid() ? zaman.toString("yyyy-MM-dd") : QString();
        satir["saat"] = zaman.isValid() ? zaman.toString("HH:mm:ss") : QString();
        satir["sure"] = sorgu.value("sure").toDouble();
        satir["pMaks"] = sorgu.value("pMaks").toDouble();
        satir["p0l"] = sorgu.value("p0l").toDouble();
        satir["p0r"] = sorgu.value("p0r").toDouble();
        satir["hiz"] = sorgu.value("hiz").toDouble();
        satir["agirlik"] = sorgu.value("agirlik").toDouble();
        satir["secili"] = sorgu.value("secili").isNull() ? true : sorgu.value("secili").toBool();
        satir["gecersizNedeni"] = sorgu.value("gecersizNedeni").toString();
        satir["otomatikHaric"] = sorgu.value("kullaniciKarari").isNull()
                                 && !sorgu.value("otomatikHaricNedeni").toString().isEmpty();
        satir["otomatikHaricNedeni"] = sorgu.value("otomatikHaricNedeni").toString();
        sonuclar.append(satir);
    }

    return sonuclar;
}

QVariantMap Database::strokeHamVeriGetir(int strokeId)
{
    QSqlQuery sorgu;
    sorgu.prepare("SELECT hamVeri FROM Strokelar WHERE id = :id");
    sorgu.bindValue(":id", strokeId);
    if (sorgu.exec() && sorgu.next()) {
        return jsonOku(sorgu.value("hamVeri").toString());
    }
    return QVariantMap();
}

QVariantMap Database::binghamHesapla(int olcumId)
{
    QVariantMap sonuc;

    QSqlQuery sorgu;
    sorgu.prepare(
        "SELECT basinc, debi, gecerli, secili, agirlik, pMaks FROM Strokelar WHERE olcumId = :olcumId"
    );
    sorgu.bindValue(":olcumId", olcumId);

    QVector<double> basinclarMbar;
    QVector<double> debilerM3h;
    QVector<qint64> agirlikAnahtarlari;
    QHash<qint64, int> agirlikBasinaStrokeSayisi;
    QHash<qint64, double> agirlikBasinaPmaksToplami;
    int toplamStrokeSayisi = 0;
    int gecersizStrokeSayisi = 0;

    if (sorgu.exec()) {
        while (sorgu.next()) {
```

```
            ++toplamStrokeSayisi;
            const bool gecerli = sorgu.value("gecerli").toBool();
            const bool tahmineDahil = sorgu.value("secili").isNull() || sorgu.value("secili").toBool();
            if (!gecerli) ++gecersizStrokeSayisi;
            if (!gecerli || !tahmineDahil) continue;
            basinclarMbar.append(sorgu.value("basinc").toDouble());
            debilerM3h.append(sorgu.value("debi").toDouble());
            const qint64 agirlikAnahtari = qRound64(sorgu.value("agirlik").toDouble() * 10.0);
            agirlikAnahtarlari.append(agirlikAnahtari);
            agirlikBasinaStrokeSayisi[agirlikAnahtari] += 1;
            agirlikBasinaPmaksToplami[agirlikAnahtari] += sorgu.value("pMaks").toDouble();
        }
    }

    QHash<qint64, double> agirlikBasinaOrtalamaPmaks;
    for (auto it = agirlikBasinaPmaksToplami.constBegin();
         it != agirlikBasinaPmaksToplami.constEnd(); ++it) {
        agirlikBasinaOrtalamaPmaks[it.key()] = it.value() / agirlikBasinaStrokeSayisi.value(it.key(), 1);
    }

    const int kullanilanStrokeSayisi = basinclarMbar.size();
    QStringList uyarilar;
    sonuc["n"] = kullanilanStrokeSayisi;
    sonuc["toplamStroke"] = toplamStrokeSayisi;
    sonuc["gecersizStroke"] = gecersizStrokeSayisi;

    if (kullanilanStrokeSayisi < 2) {
        sonuc["tau0"] = 0.0;
        sonuc["mu"] = 0.0;
        sonuc["r2"] = 0.0;
        sonuc["yeterliVeri"] = false;
        uyarilar << "azStroke";
        sonuc["uyarilar"] = uyarilar;
        return sonuc;
    }

    const SliperModel::DogruUyumu pqCizgisi = SliperModel::pqDogrusu(debilerM3h, basinclarMbar, agirlikAnahtarlari);
    uyarilar = SliperModel::olcumUyarilari(pqCizgisi, agirlikBasinaStrokeSayisi,
                                           agirlikBasinaOrtalamaPmaks,
                                           toplamStrokeSayisi, gecersizStrokeSayisi);

    sonuc["tau0"] = pqCizgisi.kesisimA;
    sonuc["mu"] = pqCizgisi.egimB;
    sonuc["r2"] = pqCizgisi.r2;
    sonuc["qMin"] = pqCizgisi.qMin;
    sonuc["qMaks"] = pqCizgisi.qMaks;
    sonuc["yeterliVeri"] = true;
    sonuc["uyarilar"] = uyarilar;
    return sonuc;
}

QVariantMap Database::tahminAyarlariGetir(int olcumId)
{
    QSqlQuery sorgu;
    sorgu.prepare("SELECT tahminAyarlari FROM Olcumler WHERE id = :id");
    sorgu.bindValue(":id", olcumId);
    if (sorgu.exec() && sorgu.next()) {
        const QString json = sorgu.value("tahminAyarlari").toString();
        if (!json.isEmpty()) {
            return SliperModel::tahminAyarlariniTamamla(jsonOku(json));
        }
    }
    return varsayilanTahminAyarlariGetir();
}

bool Database::tahminAyarlariKaydet(int olcumId, const QVariantMap &ayarlar)
{
    const QVariantMap tam = SliperModel::tahminAyarlariniTamamla(ayarlar);
    QSqlQuery sorgu;
    sorgu.prepare("UPDATE Olcumler SET tahminAyarlari = :a WHERE id = :id");
```

```
    sorgu.bindValue(":a", jsonYaz(tam));
    sorgu.bindValue(":id", olcumId);
    if (!sorgu.exec()) {
        qWarning() << "Tahmin ayarlari kaydedilemedi:" << sorgu.lastError().text();
        return false;
    }
    ayarKaydet("varsayilanTahminAyarlari", jsonYaz(tam));
    return true;
}

QVariantMap Database::varsayilanTahminAyarlariGetir()
{
    const QString json = ayarGetir("varsayilanTahminAyarlari");
    return SliperModel::tahminAyarlariniTamamla(json.isEmpty() ? QVariantMap() : jsonOku(json));
}

QVariantList Database::tahminTablosuGetir(int olcumId)
{
    const QVariantMap pqSonucu = binghamHesapla(olcumId);
    if (!pqSonucu.value("yeterliVeri").toBool()) return QVariantList();
    const double kesisimA = pqSonucu["tau0"].toDouble();
    const double egimB = pqSonucu["mu"].toDouble();
    return SliperModel::tahminTablosu(kesisimA, egimB, tahminAyarlariGetir(olcumId));
}

QString Database::ayarGetir(const QString &anahtar, const QString &varsayilan)
{
    QSqlQuery sorgu;
    sorgu.prepare("SELECT deger FROM Ayarlar WHERE anahtar = :a");
    sorgu.bindValue(":a", anahtar);
    if (sorgu.exec() && sorgu.next()) {
        return sorgu.value("deger").toString();
    }
    return varsayilan;
}

bool Database::ayarKaydet(const QString &anahtar, const QString &deger)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO Ayarlar (anahtar, deger) VALUES (:a, :d) "
        "ON CONFLICT(anahtar) DO UPDATE SET deger = :d"
    );
    sorgu.bindValue(":a", anahtar);
    sorgu.bindValue(":d", deger);
    if (!sorgu.exec()) {
        qWarning() << "Ayar kaydedilemedi:" << anahtar << sorgu.lastError().text();
        return false;
    }
    return true;
}

QString Database::jsonYaz(const QVariantMap &veri)
{
    return QString::fromUtf8(QJsonDocument(QJsonObject::fromVariantMap(veri)).toJson(QJsonDocument::Compact));
}

QVariantMap Database::jsonOku(const QString &json)
{
    return QJsonDocument::fromJson(json.toUtf8()).object().toVariantMap();
}

bool Database::kalibrasyonKaydet(const QString &sensor, double deger1, double deger2)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO Kalibrasyonlar (sensor, deger1, deger2, tarih) "
        "VALUES (:sensor, :deger1, :deger2, :tarih) "
        "ON CONFLICT(sensor) DO UPDATE SET "
        "deger1 = :deger1, deger2 = :deger2, tarih = :tarih"
```

```
    );

    sorgu.bindValue(":sensor", sensor);
    sorgu.bindValue(":deger1", deger1);
    sorgu.bindValue(":deger2", deger2);
    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));

    if (!sorgu.exec()) {
        qWarning() << "Kalibrasyon kaydedilemedi:" << sorgu.lastError().text();
        return false;
    }

    qDebug() << "Kalibrasyon kaydedildi:" << sensor;
    return true;
}

QVariantMap Database::kalibrasyonGetir(const QString &sensor)
{
    QVariantMap sonuc;

    QSqlQuery sorgu;
    sorgu.prepare("SELECT deger1, deger2, tarih FROM Kalibrasyonlar WHERE sensor = :sensor");
    sorgu.bindValue(":sensor", sensor);

    if (sorgu.exec() && sorgu.next()) {
        sonuc["deger1"] = sorgu.value("deger1");
        sonuc["deger2"] = sorgu.value("deger2");
        sonuc["tarih"] = sorgu.value("tarih");
        sonuc["mevcut"] = true;
    } else {
        sonuc["mevcut"] = false;
    }

    return sonuc;
}

void Database::kalibrasyonTarihiKaydet(const QString &sensor)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO KalibrasyonTarihleri (sensor, tarih) VALUES (:sensor, :tarih) "
        "ON CONFLICT(sensor) DO UPDATE SET tarih = :tarih"
    );

    sorgu.bindValue(":sensor", sensor);
    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));

    if (!sorgu.exec()) {
        qWarning() << "Kalibrasyon tarihi kaydedilemedi:" << sorgu.lastError().text();
    }
}

QString Database::kalibrasyonTarihiGetir(const QString &sensor)
{
    QSqlQuery sorgu;
    sorgu.prepare("SELECT tarih FROM KalibrasyonTarihleri WHERE sensor = :sensor");
    sorgu.bindValue(":sensor", sensor);

    if (sorgu.exec() && sorgu.next()) {
        return sorgu.value("tarih").toString();
    }

    return QString();
}

QVariantMap Database::olcumBilgisiGetir(int olcumId)
{
    QVariantMap sonuc;

    QSqlQuery sorgu;
```

```
    sorgu.prepare("SELECT tarih, musteri, recete, agirlik, yer, yorum, daraBasinci "
                  "FROM Olcumler WHERE id = :id");
    sorgu.bindValue(":id", olcumId);

    if (sorgu.exec() && sorgu.next()) {
        sonuc["tarih"] = sorgu.value("tarih");
        sonuc["musteri"] = sorgu.value("musteri");
        sonuc["recete"] = sorgu.value("recete");
        sonuc["agirlik"] = sorgu.value("agirlik");
        sonuc["yer"] = sorgu.value("yer").toString();
        sonuc["yorum"] = sorgu.value("yorum").toString();
        sonuc["daraBasinci"] = sorgu.value("daraBasinci").toDouble();
        sonuc["bulundu"] = true;
    } else {
        sonuc["bulundu"] = false;
    }

    return sonuc;
}

QString Database::xmlDisaAktar(int olcumId)
{
    QVariantMap bilgi = olcumBilgisiGetir(olcumId);
    if (!bilgi["bulundu"].toBool()) {
        qWarning() << "Olcum bulunamadi, xml olusturulamadi.";
        return QString();
    }

    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    const QVariantList strokelar = strokeVerileriGetir(olcumId);
    QString zamanDamgasi = QDateTime::currentDateTime().toString("yyyy-MM-dd_HH_mm_ss");
    if (!strokelar.isEmpty()) {
        const QVariantMap ilk = strokelar.first().toMap();
        if (!ilk["tarih"].toString().isEmpty()) {
            zamanDamgasi = ilk["tarih"].toString() + "_" + ilk["saat"].toString().replace(':', '_');
        }
    }
    QString dosyaAdi = klasor + QString("/SLIPER_%1_%2.xml").arg(olcumId).arg(zamanDamgasi);

    QFile dosya(dosyaAdi);
    if (!dosya.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qWarning() << "XML dosyasi acilamadi:" << dosyaAdi;
        return QString();
    }

    const QVariantMap pqSonucu = binghamHesapla(olcumId);
    const QVariantMap ayarlar = tahminAyarlariGetir(olcumId);
    const QVariantList tahminSatirlari = tahminTablosuGetir(olcumId);

    QTextStream akis(&dosya);
    akis.setEncoding(QStringConverter::Utf8);

    auto xmlKacis = [](QString yazi) {
        yazi.replace("&", "&amp;");
        yazi.replace("<", "&lt;");
        yazi.replace(">", "&gt;");
        return yazi;
    };
    auto metin = [&](const QString &yazi) {
        return QString("<Cell><Data ss:Type=\"String\">%1</Data></Cell>").arg(xmlKacis(yazi));
    };
    auto sayi = [](double deger) {
        return QString("<Cell><Data ss:Type=\"Number\">%1</Data></Cell>").arg(QString::number(deger, 'g', 10));
    };
    auto satir = [&](const QStringList &hucreler) {
```

```
    akis << "<Row>" << hucreler.join("") << "</Row>\n";
};

akis << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
akis << "<?mso-application progid=\"Excel.Sheet\"?>\n";
akis << "<Workbook xmlns=\"urn:schemas-microsoft-com:office:spreadsheet\" "
        "xmlns:o=\"urn:schemas-microsoft-com:office:office\" "
        "xmlns:x=\"urn:schemas-microsoft-com:office:excel\" "
        "xmlns:ss=\"urn:schemas-microsoft-com:office:spreadsheet\">\n";

akis << "<Worksheet ss:Name=\"Results\">\n<Table>\n";
const QDateTime simdi = QDateTime::currentDateTime();
satir({ metin("Export Date"), metin(simdi.toString("yyyy-MM-dd")) });
satir({ metin("Export Time"), metin(simdi.toString("HH:mm:ss")) });
satir({ metin("Measurement ID"), sayi(olcumId) });
satir({ metin("Measurement Date"), metin(bilgi["tarih"].toString()) });
satir({ metin("Comment"), metin(bilgi["yorum"].toString()) });
satir({ metin("Place"), metin(bilgi["yer"].toString()) });
satir({ metin("Customer"), metin(bilgi["musteri"].toString()) });
satir({ metin("Formula"), metin(bilgi["recete"].toString()) });
satir({});

satir({ metin("Stroke"), metin("Date [yyyy-mm-dd]"), metin("Time [hh:mm:ss]"),
        metin("Duration [sec]"), metin("pmax [mbar]"), metin("P0l [mbar]"),
        metin("P0r [mbar]"), metin("p [mbar]"), metin("Q [m3/h]"),
        metin("Weight [kg]"), metin("Valid"), metin("In forecast") });
for (const QVariant &kayit : strokelar) {
    const QVariantMap stroke = kayit.toMap();
    satir({ sayi(stroke["stroke"].toInt()), metin(stroke["tarih"].toString()), metin(stroke["saat"].toString()),
            sayi(stroke["sure"].toDouble()), sayi(stroke["pMaks"].toDouble()), sayi(stroke["p0l"].toDouble()),
            sayi(stroke["p0r"].toDouble()), sayi(stroke["basinc"].toDouble()), sayi(stroke["debi"].toDouble()),
            sayi(stroke["agirlik"].toDouble()),
            metin(stroke["gecerli"].toBool() ? "yes" : "Wrong Stroke"),
            metin(stroke["secili"].toBool() ? "yes" : "no") });
}
satir({});

satir({ metin("Forecast at following input parameters") });
satir({ metin("Target output Q [m3/h]"), sayi(ayarlar["hedefDebi"].toDouble()) });
satir({ metin("Diameter of pipe D2 [m]"), sayi(ayarlar["cap"].toDouble() / 1000.0) });
satir({ metin("Line length [m]"), sayi(ayarlar["hatUzunlugu"].toDouble()) });
satir({ metin("Pumping height h [m]"), sayi(ayarlar["yukseklik"].toDouble()) });
satir({ metin("90 deg bends"), sayi(ayarlar["dirsek90"].toDouble()) });
satir({ metin("90 deg bend equivalent length [m]"), sayi(SliperModel::DIRSEK_90_ESDEGER_M) });
satir({ metin("45 deg bends"), sayi(ayarlar["dirsek45"].toDouble()) });
satir({ metin("45 deg bend equivalent length [m]"), sayi(SliperModel::DIRSEK_45_ESDEGER_M) });
satir({ metin("Calculation length L [m]"), sayi(SliperModel::hesapUzunluguM(ayarlar)) });
satir({ metin("bulk density [kg/m3]"), sayi(ayarlar["yogunluk"].toDouble()) });
satir({ metin("Error Bar Tolerance [%]"), sayi(ayarlar["hataPayi"].toDouble()) });
satir({ metin("Pump max. pressure [bar]"), sayi(ayarlar["pompaMaks"].toDouble()) });
satir({});

const double kesisimA = pqSonucu["tau0"].toDouble();
const double egimB = pqSonucu["mu"].toDouble();
satir({ metin("A - p-Q intercept [mbar]"), sayi(kesisimA) });
satir({ metin("B - p-Q slope [mbar*h/m3]"), sayi(egimB) });
satir({ metin("a - Yield Pressure [mbar]"), sayi(SliperModel::schleibingerA(kesisimA)) });
satir({ metin("b - Pressure Gradient [x1000]"), sayi(SliperModel::schleibingerB(egimB) * 1000.0) });
satir({ metin("R2"), sayi(pqSonucu["r2"].toDouble()) });
satir({ metin("Strokes used"), sayi(pqSonucu["n"].toInt()) });
satir({});

satir({ metin("Output Q [m3/h]"), metin("Length of pipe [m]"), metin("Pressure [bar]"),
        metin("Lower [bar]"), metin("Upper [bar]"), metin("Pump capacity") });
const bool pompaGirildi = ayarlar["pompaMaks"].toDouble() > 0.0;
for (const QVariant &kayit : tahminSatirlari) {
    const QVariantMap tahmin = kayit.toMap();
    satir({ sayi(tahmin["debi"].toDouble()), sayi(tahmin["uzunluk"].toDouble()),
            sayi(tahmin["basincBar"].toDouble()), sayi(tahmin["altSinirBar"].toDouble()),
            sayi(tahmin["ustSinirBar"].toDouble()),
```

```
                metin(!pompaGirildi ? "-" : (tahmin["pompaAsildi"].toBool() ? "exceeded" : "ok")) });
    }
    satir({});

    const QVariantMap konumSinirlari = kalibrasyonGetir("konum_sinirlari");
    const bool konumKalibreEdildi = konumSinirlari["mevcut"].toBool();
    satir({ metin("Device parameters") });
    satir({ metin("Diameter of rheometer [m]"), sayi(SliperModel::BORU_CAPI_M) });
    satir({ metin("Length of rheometer [m]"), sayi(SliperModel::BORU_UZUNLUGU_M) });
    satir({ metin("Top position [mm]"), konumKalibreEdildi ? sayi(konumSinirlari["deger1"].toDouble()) : metin("-") });
    satir({ metin("Bottom position [mm]"), konumKalibreEdildi ? sayi(konumSinirlari["deger2"].toDouble()) : metin("-") });
    satir({ metin("Load cell calibration"), metin(kalibrasyonTarihiGetir("loadcell")) });
    satir({ metin("Distance sensor calibration"), metin(kalibrasyonTarihiGetir("mesafe_olcum")) });
    akis << "</Table>\n</Worksheet>\n";

    for (const QVariant &kayit : strokelar) {
        const QVariantMap stroke = kayit.toMap();
        const QVariantMap hamVeri = strokeHamVeriGetir(stroke["id"].toInt());
        const QVariantList zamanlar = hamVeri["t"].toList();
        const QVariantList konumlar = hamVeri["x"].toList();
        const QVariantList basinclar = hamVeri["p"].toList();

        akis << "<Worksheet ss:Name=\"Stroke_" << stroke["stroke"].toInt() << "\">\n<Table>\n";
        satir({ metin("Start [s]"), sayi(hamVeri["tBaslangic"].toDouble()),
                metin("End [s]"), sayi(hamVeri["tBitis"].toDouble()) });
        satir({ metin("Time [s]"), metin("Distance [mm]"), metin("Pressure [mbar]") });
        const int ornekSayisi = std::min({ zamanlar.size(), konumlar.size(), basinclar.size() });
        for (int i = 0; i < ornekSayisi; ++i) {
            satir({ sayi(zamanlar[i].toDouble()), sayi(konumlar[i].toDouble()), sayi(basinclar[i].toDouble()) });
        }
        akis << "</Table>\n</Worksheet>\n";
    }

    akis << "</Workbook>\n";

    dosya.close();
    qDebug() << "XML disa aktarildi:" << dosyaAdi;
    return dosyaAdi;
}

QString Database::veriTabaniDisaAktar()
{
    if (m_veriTabaniDosyaYolu.isEmpty() || !QFile::exists(m_veriTabaniDosyaYolu)) {
        qWarning() << "Disa aktarilacak veritabani bulunamadi.";
        return QString();
    }

    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    QSqlQuery checkpoint(m_db);
    checkpoint.exec("PRAGMA wal_checkpoint(FULL)");

    QString hedefDosya = klasor + QString("/sliper_yedek_%1.db")
        .arg(QDateTime::currentDateTime().toString("ddMMyyyy_HHmmss"));

    if (!QFile::copy(m_veriTabaniDosyaYolu, hedefDosya)) {
        qWarning() << "Veritabani disa aktarilamadi.";
        return QString();
    }

    qDebug() << "Veritabani disa aktarildi:" << hedefDosya;
    return hedefDosya;
}

bool Database::veriTabaniIcaAktar(const QString &kaynakDosyaYolu)
{
```

```
    if (!QFile::exists(kaynakDosyaYolu)) {
        qWarning() << "Ice aktarilacak dosya bulunamadi:" << kaynakDosyaYolu;
        return false;
    }

    veriTabaniDisaAktar();

    m_db.close();

    QFile hedef(m_veriTabaniDosyaYolu);
    if (hedef.exists() && !hedef.remove()) {
        qWarning() << "Eski veritabani silinemedi, ice aktarim iptal edildi.";
        m_db.open();
        return false;
    }

    if (!QFile::copy(kaynakDosyaYolu, m_veriTabaniDosyaYolu)) {
        qWarning() << "Veritabani ice aktarilamadi.";
        m_db.open();
        return false;
    }

    if (!m_db.open()) {
        qWarning() << "Ice aktarim sonrasi veritabani acilamadi:" << m_db.lastError().text();
        return false;
    }

    tablolariOlustur();
    qDebug() << "Veritabani ice aktarildi:" << kaynakDosyaYolu;
    return true;
}

bool Database::tumVeriyiSil()
{
    if (!m_db.transaction()) {
        qWarning() << "Tum veriyi silme icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery strokeSil;
    if (!strokeSil.exec("DELETE FROM Strokelar")) {
        qWarning() << "Strokelar silinemedi:" << strokeSil.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery olcumSil;
    if (!olcumSil.exec("DELETE FROM Olcumler")) {
        qWarning() << "Olcumler silinemedi:" << olcumSil.lastError().text();
        m_db.rollback();
        return false;
    }

    if (!m_db.commit()) {
        qWarning() << "Tum veriyi silme commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Tum olcum verileri silindi.";
    return true;
}

QString Database::csvDisaAktar(int olcumId)
{
    QVariantMap bilgi = olcumBilgisiGetir(olcumId);
    if (!bilgi["bulundu"].toBool()) {
        qWarning() << "Olcum bulunamadi, csv olusturulamadi.";
        return QString();
    }
```

```
    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    QString dosyaAdi = klasor + QString("/SLIPER_Veri_%1_%2.csv")
        .arg(olcumId)
        .arg(QDateTime::currentDateTime().toString("ddMMyyyy_HHmmss"));

    QFile dosya(dosyaAdi);
    if (!dosya.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qWarning() << "CSV dosyasi acilamadi:" << dosyaAdi;
        return QString();
    }

    QTextStream akis(&dosya);
    akis.setEncoding(QStringConverter::Utf8);
    auto alan = [](QString yazi) {
        if (yazi.contains(';') || yazi.contains('"') || yazi.contains('\n')) {
            yazi.replace("\"", "\"\"");
            yazi = "\"" + yazi + "\"";
        }
        return yazi;
    };

    akis << "Olcum ID;" << olcumId << "\n";
    akis << "Tarih;" << alan(bilgi["tarih"].toString()) << "\n";
    akis << "Yer;" << alan(bilgi["yer"].toString()) << "\n";
    akis << "Musteri;" << alan(bilgi["musteri"].toString()) << "\n";
    akis << "Recete;" << alan(bilgi["recete"].toString()) << "\n";
    akis << "Yorum;" << alan(bilgi["yorum"].toString()) << "\n";
    akis << "\n";
    akis << "Stroke;Tarih;Saat;Sure (s);Pmax (mbar);P0l (mbar);P0r (mbar);p (mbar);Q (m3/h);"
            "Agirlik (kg);Gecerli;Tahmine Dahil\n";

    const QVariantList strokelar = strokeVerileriGetir(olcumId);
    for (const QVariant &kayit : strokelar) {
        const QVariantMap stroke = kayit.toMap();
        akis << stroke["stroke"].toInt() << ";"
             << stroke["tarih"].toString() << ";"
             << stroke["saat"].toString() << ";"
             << stroke["sure"].toDouble() << ";"
             << stroke["pMaks"].toDouble() << ";"
             << stroke["p0l"].toDouble() << ";"
             << stroke["p0r"].toDouble() << ";"
             << stroke["basinc"].toDouble() << ";"
             << stroke["debi"].toDouble() << ";"
             << stroke["agirlik"].toDouble() << ";"
             << (stroke["gecerli"].toBool() ? "Evet" : "Hatali Stroke") << ";"
             << (stroke["secili"].toBool() ? "Evet" : "Hayir") << "\n";
    }

    const QVariantMap pqSonucu = binghamHesapla(olcumId);
    const double kesisimA = pqSonucu["tau0"].toDouble();
    const double egimB = pqSonucu["mu"].toDouble();
    akis << "\n";
    akis << "A (mbar);" << kesisimA << "\n";
    akis << "B (mbar.h/m3);" << egimB << "\n";
    akis << "a (mbar);" << SliperModel::schleibingerA(kesisimA) << "\n";
    akis << "b (x1000);" << SliperModel::schleibingerB(egimB) * 1000.0 << "\n";
    akis << "R2;" << pqSonucu["r2"].toDouble() << "\n";

    dosya.close();
    qDebug() << "CSV disa aktarildi:" << dosyaAdi;
    return dosyaAdi;
}

bool Database::loadCellNoktalariKaydet(const QVariantList &noktalar)
{
```

```
    if (!m_db.transaction()) {
        qWarning() << "Load cell noktalari icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery temizle;
    if (!temizle.exec("DELETE FROM LoadCellNoktalari")) {
        qWarning() << "Load cell noktalari temizlenemedi:" << temizle.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT INTO LoadCellNoktalari (hedefKg, hamDeger) "
        "VALUES (:hedefKg, :hamDeger)"
    );

    for (const QVariant &v : noktalar) {
        QVariantMap nokta = v.toMap();
        sorgu.bindValue(":hedefKg", nokta["hedefKg"].toDouble());
        sorgu.bindValue(":hamDeger", nokta["hamDeger"].toDouble());
        if (!sorgu.exec()) {
            qWarning() << "Load cell noktasi kaydedilemedi:" << sorgu.lastError().text();
            m_db.rollback();
            return false;
        }
    }

    if (!m_db.commit()) {
        qWarning() << "Load cell noktalari commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Load cell" << noktalar.size() << "nokta kaydedildi.";
    kalibrasyonTarihiKaydet("loadcell");
    return true;
}

QVariantList Database::loadCellNoktalariGetir()
{
    QVariantList sonuclar;

    QSqlQuery sorgu("SELECT hedefKg, hamDeger FROM LoadCellNoktalari ORDER BY hamDeger ASC");

    while (sorgu.next()) {
        QVariantMap satir;
        satir["hedefKg"] = sorgu.value("hedefKg");
        satir["hamDeger"] = sorgu.value("hamDeger");
        sonuclar.append(satir);
    }

    return sonuclar;
}

bool Database::mesafeNoktalariKaydet(const QVariantList &noktalar)
{
    if (!m_db.transaction()) {
        qWarning() << "Mesafe noktalari icin transaction baslatilamadi:" << m_db.lastError().text();
        return false;
    }

    QSqlQuery temizle;
    if (!temizle.exec("DELETE FROM MesafeNoktalari")) {
        qWarning() << "Mesafe noktalari temizlenemedi:" << temizle.lastError().text();
        m_db.rollback();
        return false;
    }

    QSqlQuery sorgu;
```

```
    sorgu.prepare(
        "INSERT INTO MesafeNoktalari (hedefMm, hamDeger) "
        "VALUES (:hedefMm, :hamDeger)"
    );

    for (const QVariant &v : noktalar) {
        QVariantMap nokta = v.toMap();
        sorgu.bindValue(":hedefMm", nokta["hedefMm"].toDouble());
        sorgu.bindValue(":hamDeger", nokta["hamDeger"].toDouble());
        if (!sorgu.exec()) {
            qWarning() << "Mesafe noktasi kaydedilemedi:" << sorgu.lastError().text();
            m_db.rollback();
            return false;
        }
    }

    if (!m_db.commit()) {
        qWarning() << "Mesafe noktalari commit edilemedi:" << m_db.lastError().text();
        return false;
    }

    qDebug() << "Mesafe" << noktalar.size() << "nokta kaydedildi.";
    kalibrasyonTarihiKaydet("mesafe_olcum");
    return true;
}

QVariantList Database::mesafeNoktalariGetir()
{
    QVariantList sonuclar;

    QSqlQuery sorgu("SELECT hedefMm, hamDeger FROM MesafeNoktalari ORDER BY hamDeger ASC");

    while (sorgu.next()) {
        QVariantMap satir;
        satir["hedefMm"] = sorgu.value("hedefMm");
        satir["hamDeger"] = sorgu.value("hamDeger");
        sonuclar.append(satir);
    }

    return sonuclar;
}

bool Database::egimKalibrasyonuKaydet(double biasX, double biasY, double biasZ,
                                      double gainX, double gainY, double gainZ)
{
    QSqlQuery sorgu;
    sorgu.prepare(
        "INSERT OR REPLACE INTO EgimKalibrasyon (id, biasX, biasY, biasZ, gainX, gainY, gainZ, tarih) "
        "VALUES (1, :biasX, :biasY, :biasZ, :gainX, :gainY, :gainZ, :tarih)"
    );

    sorgu.bindValue(":biasX", biasX);
    sorgu.bindValue(":biasY", biasY);
    sorgu.bindValue(":biasZ", biasZ);
    sorgu.bindValue(":gainX", gainX);
    sorgu.bindValue(":gainY", gainY);
    sorgu.bindValue(":gainZ", gainZ);
    sorgu.bindValue(":tarih", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm"));

    if (!sorgu.exec()) {
        qWarning() << "Egim kalibrasyonu kaydedilemedi:" << sorgu.lastError().text();
        return false;
    }

    qDebug() << "Egim kalibrasyonu kaydedildi.";
    return true;
}

QVariantMap Database::egimKalibrasyonuGetir()
{
```

```
    QVariantMap sonuc;

    QSqlQuery sorgu("SELECT biasX, biasY, biasZ, gainX, gainY, gainZ, tarih FROM EgimKalibrasyon WHERE id = 1");

    if (sorgu.exec() && sorgu.next()) {
        sonuc["biasX"] = sorgu.value("biasX");
        sonuc["biasY"] = sorgu.value("biasY");
        sonuc["biasZ"] = sorgu.value("biasZ");
        sonuc["gainX"] = sorgu.value("gainX");
        sonuc["gainY"] = sorgu.value("gainY");
        sonuc["gainZ"] = sorgu.value("gainZ");
        sonuc["tarih"] = sorgu.value("tarih");
        sonuc["mevcut"] = true;
    } else {
        sonuc["mevcut"] = false;
    }

    return sonuc;
}
```

src\Database.h

04.11.2026 11:45

```
#pragma once
#include <QObject>
#include <QSqlDatabase>
#include <QVariantList>
#include <QVariantMap>

class Database : public QObject
{
    Q_OBJECT

public:
    explicit Database(QObject *parent = nullptr);

    Q_INVOKABLE int olcumBaslat(const QString &musteri, const QString &recete, double agirlik,
                                const QString &yer = QString(), const QString &yorum = QString());
    Q_INVOKABLE bool olcumBilgisiGuncelle(int olcumId, const QString &yer, const QString &musteri,
                                          const QString &recete, const QString &yorum);
    Q_INVOKABLE bool olcumSil(int olcumId);
    Q_INVOKABLE bool olcumDarasiKaydet(int olcumId, double daraBasinciMbar);
    Q_INVOKABLE QVariantList tumOlcumleriGetir();
    Q_INVOKABLE QVariantMap olcumBilgisiGetir(int olcumId);

    Q_INVOKABLE int strokeKaydet(int olcumId, const QVariantMap &stroke, double agirlik);
    Q_INVOKABLE QVariantList strokeVerileriGetir(int olcumId);
    Q_INVOKABLE QVariantMap strokeHamVeriGetir(int strokeId);
    Q_INVOKABLE bool strokeSeciliAyarla(int strokeId, bool secili);
    Q_INVOKABLE bool strokeSil(int strokeId);

    Q_INVOKABLE QVariantMap binghamHesapla(int olcumId);
    Q_INVOKABLE QVariantMap tahminAyarlariGetir(int olcumId);
    Q_INVOKABLE bool tahminAyarlariKaydet(int olcumId, const QVariantMap &ayarlar);
    Q_INVOKABLE QVariantMap varsayilanTahminAyarlariGetir();
    Q_INVOKABLE QVariantList tahminTablosuGetir(int olcumId);

    Q_INVOKABLE QString ayarGetir(const QString &anahtar, const QString &varsayilan = QString());
    Q_INVOKABLE bool ayarKaydet(const QString &anahtar, const QString &deger);

    Q_INVOKABLE QString csvDisaAktar(int olcumId);
    Q_INVOKABLE QString xmlDisaAktar(int olcumId);
    Q_INVOKABLE QString veriTabaniDisaAktar();
    Q_INVOKABLE bool veriTabaniIcaAktar(const QString &kaynakDosyaYolu);
    Q_INVOKABLE bool tumVeriyiSil();

    Q_INVOKABLE bool kalibrasyonKaydet(const QString &sensor, double deger1, double deger2);
    Q_INVOKABLE QVariantMap kalibrasyonGetir(const QString &sensor);
    Q_INVOKABLE bool loadCellNoktalariKaydet(const QVariantList &noktalar);
    Q_INVOKABLE QVariantList loadCellNoktalariGetir();
    Q_INVOKABLE bool mesafeNoktalariKaydet(const QVariantList &noktalar);
    Q_INVOKABLE QVariantList mesafeNoktalariGetir();
    Q_INVOKABLE bool egimKalibrasyonuKaydet(double biasX, double biasY, double biasZ,
                                            double gainX, double gainY, double gainZ);
    Q_INVOKABLE QVariantMap egimKalibrasyonuGetir();
    Q_INVOKABLE void kalibrasyonTarihiKaydet(const QString &sensor);
    Q_INVOKABLE QString kalibrasyonTarihiGetir(const QString &sensor);

private:
    void tablolariOlustur();
    void otomatikElemeUygula(int olcumId);
    static QString jsonYaz(const QVariantMap &veri);
    static QVariantMap jsonOku(const QString &json);
    QSqlDatabase m_db;
    QString m_veriTabaniDosyaYolu;
};
```

src\ReportManager.cpp

04.11.2026 11:45

```
#include "ReportManager.h"
#include "Database.h"
#include "SliperModel.h"
#include <QPrinter>
#include <QPainter>
#include <QPainterPath>
#include <QTextDocument>
#include <QStandardPaths>
#include <QDir>
#include <QDateTime>
#include <QFile>
#include <QUrl>
#include <QDebug>
#include <cmath>
#include <algorithm>

ReportManager::ReportManager(Database *database, QObject *parent)
    : QObject(parent)
    , m_database(database)
{
}

namespace {

QString sayi(double deger, int basamak) { return QString::number(deger, 'f', basamak); }
QString tarihVeyaCizgi(const QString &tarih) { return tarih.isEmpty() ? QStringLiteral("—") : tarih; }

double okunakliEksenAdimi(double aralik, int hedefCizgiSayisi)
{
    if (aralik <= 0.0) return 1.0;
    const double kabaAdim = aralik / hedefCizgiSayisi;
    const double onunKuvveti = std::pow(10.0, std::floor(std::log10(kabaAdim)));
    const double ilkRakam = kabaAdim / onunKuvveti;
    double yuvarlakRakam = 10.0;
    if (ilkRakam <= 1.0) yuvarlakRakam = 1.0;
    else if (ilkRakam <= 2.0) yuvarlakRakam = 2.0;
    else if (ilkRakam <= 5.0) yuvarlakRakam = 5.0;
    return yuvarlakRakam * onunKuvveti;
}

QString uyariMetni(const QString &kod)
{
    static const QMap<QString, QString> metinler {
        { "azStroke", "Tahmin için yeterli geçerli stroke yok (en az 3 önerilir; her yük için 3 stroke idealdir)." },
        { "egimNegatif", "P–Q eğimi (B) negatif: debi arttıkça basınç düşüyor. Ayrışma/blokaj veya ölçüm hatası olabilir." },
        { "kesisimNegatif", "P–Q kesişimi (A) negatif: durağan basınç (P0) hatalı ölçülmüş olabilir." },
        { "dusukR2", QString("Uyum kalitesi düşük (R² < %1): stroke noktaları doğrudan çok sapıyor.")
                        .arg(SliperModel::MIN_R2, 0, 'f', 2) },
        { "darDebiAraligi", "Debi aralığı çok dar: farklı ağırlıklarla stroke atılmalı, aksi halde eğim güvenilir değil." },
        { "yukBasinaAzStroke", "Bazı yüklerde 3'ten az geçerli stroke var; kılavuz her yük için en az 3 stroke önerir." },
        { "tekYuk", "Tüm stroke'lar aynı ek ağırlıkla atılmış: en az iki farklı yük kullanılmalı." },
        { "cokGecersiz", "Stroke'ların %30'undan fazlası geçersiz: boru sıkışması veya ayrışma belirtisi olabilir." },
        { "agirlikTepkisiZayif", "Eklenen ağırlık Pmax'ta görünmüyor: her kg Pmax'ı 7.6 mbar yükseltmeliydi. "
                                 "Ağırlık load cell'e ulaşmıyor (kızak sürtünmesi, ağırlığın boruya oturmaması "
                                 "veya load cell kalibrasyonu). Bu ölçümde eğim ölçülmüş değildir." }
    };
    return metinler.value(kod, kod);
}

}

QImage ReportManager::grafikCiz(const QString &baslik, const QString &xEtiket, const QString &yEtiket,
                                const QVector<Seri> &seriler)
{
    const int gorselGenislik = 1000, gorselYukseklik = 560;
    const QRectF cizimAlani(90, 50, gorselGenislik - 120, gorselYukseklik - 130);
    QImage gorsel(gorselGenislik, gorselYukseklik, QImage::Format_ARGB32);
    gorsel.fill(Qt::white);
```

```
double xMin = 0, xMaks = 1, yMin = 0, yMaks = 1;
bool ilkNokta = true;
auto araligaKat = [&](const QPointF &nokta) {
    if (ilkNokta) { xMaks = std::max(1e-9, nokta.x()); yMaks = std::max(1e-9, nokta.y()); ilkNokta = false; }
    xMin = std::min(xMin, nokta.x()); xMaks = std::max(xMaks, nokta.x());
    yMin = std::min(yMin, nokta.y()); yMaks = std::max(yMaks, nokta.y());
};
for (const Seri &seri : seriler) {
    for (const QPointF &nokta : seri.noktalar) araligaKat(nokta);
    for (const auto &cubuk : seri.hataCubuklari) { araligaKat(cubuk.first); araligaKat(cubuk.second); }
}
const double xAdim = okunakliEksenAdimi(xMaks - xMin, 8);
const double yAdim = okunakliEksenAdimi(yMaks - yMin, 6);
xMin = std::floor(xMin / xAdim) * xAdim; xMaks = std::ceil(xMaks * 1.05 / xAdim) * xAdim;
yMin = std::floor(yMin / yAdim) * yAdim; yMaks = std::ceil(yMaks * 1.08 / yAdim) * yAdim;
if (xMaks <= xMin) xMaks = xMin + xAdim;
if (yMaks <= yMin) yMaks = yMin + yAdim;

auto pikseleCevir = [&](const QPointF &veriNoktasi) {
    return QPointF(cizimAlani.left() + (veriNoktasi.x() - xMin) / (xMaks - xMin) * cizimAlani.width(),
                   cizimAlani.bottom() - (veriNoktasi.y() - yMin) / (yMaks - yMin) * cizimAlani.height());
};

QPainter kalem(&gorsel);
kalem.setRenderHint(QPainter::Antialiasing);
QFont normalYazi("Segoe UI", 11);
kalem.setFont(normalYazi);

kalem.setPen(QPen(QColor("#e5e7eb"), 1));
for (double x = xMin; x <= xMaks + xAdim * 0.01; x += xAdim) {
    const QPointF piksel = pikseleCevir({ x, yMin });
    kalem.drawLine(QPointF(piksel.x(), cizimAlani.top()), QPointF(piksel.x(), cizimAlani.bottom()));
}
for (double y = yMin; y <= yMaks + yAdim * 0.01; y += yAdim) {
    const QPointF piksel = pikseleCevir({ xMin, y });
    kalem.drawLine(QPointF(cizimAlani.left(), piksel.y()), QPointF(cizimAlani.right(), piksel.y()));
}
kalem.setPen(QColor("#374151"));
for (double x = xMin; x <= xMaks + xAdim * 0.01; x += xAdim) {
    const QPointF piksel = pikseleCevir({ x, yMin });
    kalem.drawText(QRectF(piksel.x() - 40, cizimAlani.bottom() + 6, 80, 20), Qt::AlignHCenter | Qt::AlignTop,
                   QString::number(x, 'g', 6));
}
for (double y = yMin; y <= yMaks + yAdim * 0.01; y += yAdim) {
    const QPointF piksel = pikseleCevir({ xMin, y });
    kalem.drawText(QRectF(cizimAlani.left() - 86, piksel.y() - 10, 80, 20), Qt::AlignRight | Qt::AlignVCenter,
                   QString::number(y, 'g', 6));
}
kalem.setPen(QPen(QColor("#6b7280"), 1.5));
kalem.drawRect(cizimAlani);

kalem.drawText(QRectF(cizimAlani.left(), gorselYukseklik - 50, cizimAlani.width(), 24), Qt::AlignCenter, xEtiket);
kalem.save();
kalem.translate(18, cizimAlani.center().y());
kalem.rotate(-90);
kalem.drawText(QRectF(-cizimAlani.height() / 2, -12, cizimAlani.height(), 24), Qt::AlignCenter, yEtiket);
kalem.restore();
QFont baslikYazisi("Segoe UI", 14, QFont::Bold);
kalem.setFont(baslikYazisi);
kalem.setPen(QColor("#111827"));
kalem.drawText(QRectF(0, 10, gorselGenislik, 30), Qt::AlignCenter, baslik);
kalem.setFont(normalYazi);

kalem.setClipRect(cizimAlani.adjusted(-6, -6, 6, 6));
for (const Seri &seri : seriler) {
    if (seri.noktaMi) {
        kalem.setPen(QPen(seri.renk.darker(130), 1));
        kalem.setBrush(seri.renk);
        for (const QPointF &nokta : seri.noktalar) kalem.drawEllipse(pikseleCevir(nokta), 6, 6);
```

```
        } else if (seri.noktalar.size() >= 2) {
            kalem.setPen(QPen(seri.renk, 2.5, seri.stil));
            kalem.setBrush(Qt::NoBrush);
            QPainterPath cizgi(pikseleCevir(seri.noktalar.first()));
            for (int i = 1; i < seri.noktalar.size(); ++i) cizgi.lineTo(pikseleCevir(seri.noktalar[i]));
            kalem.drawPath(cizgi);
        }
        kalem.setPen(QPen(seri.renk, 1.8));
        for (const auto &cubuk : seri.hataCubuklari) {
            const QPointF altUc = pikseleCevir(cubuk.first), ustUc = pikseleCevir(cubuk.second);
            kalem.drawLine(altUc, ustUc);
            kalem.drawLine(QPointF(altUc.x() - 7, altUc.y()), QPointF(altUc.x() + 7, altUc.y()));
            kalem.drawLine(QPointF(ustUc.x() - 7, ustUc.y()), QPointF(ustUc.x() + 7, ustUc.y()));
        }
    }
    kalem.setClipping(false);

    double lejantX = cizimAlani.left() + 12;
    const double lejantY = cizimAlani.top() + 12;
    for (const Seri &seri : seriler) {
        if (seri.ad.isEmpty()) continue;
        kalem.setPen(QPen(seri.renk, 2.5, seri.stil));
        kalem.setBrush(seri.renk);
        if (seri.noktaMi) kalem.drawEllipse(QPointF(lejantX + 8, lejantY + 8), 5, 5);
        else kalem.drawLine(QPointF(lejantX, lejantY + 8), QPointF(lejantX + 22, lejantY + 8));
        kalem.setPen(QColor("#111827"));
        const double yaziGenisligi = kalem.fontMetrics().horizontalAdvance(seri.ad);
        kalem.drawText(QPointF(lejantX + 28, lejantY + 13), seri.ad);
        lejantX += 40 + yaziGenisligi;
    }
    return gorsel;
}

QString ReportManager::raporHtmlOlustur(int olcumId, QMap<QString, QImage> &gorseller)
{
    if (!m_database) return QString();

    const QVariantMap bilgi = m_database->olcumBilgisiGetir(olcumId);
    const QVariantList strokelar = m_database->strokeVerileriGetir(olcumId);
    const QVariantMap pqSonucu = m_database->binghamHesapla(olcumId);
    const QVariantMap ayarlar = m_database->tahminAyarlariGetir(olcumId);
    const QVariantList tahminSatirlari = m_database->tahminTablosuGetir(olcumId);
    const double kesisimA = pqSonucu.value("tau0").toDouble();
    const double egimB = pqSonucu.value("mu").toDouble();
    const bool yeterliVeri = pqSonucu.value("yeterliVeri").toBool();

    const QString hucreStili = "style='border:1px solid #d1d5db; padding:3pt;'";
    const QString baslikHucreStili = "style='border:1px solid #d1d5db; padding:3pt; background:#f3f4f6; text-align:left;'";
    auto satir = [&](const QStringList &hucreler, bool basliksatiri = false) {
        QString satirHtml = "<tr>";
        for (const QString &hucre : hucreler) {
            satirHtml += QString("<%1 %2>%3</%1>").arg(basliksatiri ? "th" : "td",
                                                       basliksatiri ? baslikHucreStili : hucreStili, hucre);
        }
        return satirHtml + "</tr>";
    };
    auto tablo = [](const QString &icerik) {
        return "<table width='100%' cellspacing='0' cellpadding='3' style='border-collapse:collapse; font-size:9pt;'>" + icerik +
"</table>";
    };
    auto baslik = [](const QString &yazi) {
        return "<h2 style='color:#111827; margin-top:22px; margin-bottom:6px; font-size:12pt;'>" + yazi + "</h2>";
    };

    QString html = "<html><body style='font-family:Segoe UI, Arial, sans-serif; color:#111827;'>";
    html += "<h1 style='color:#3b82f6; margin-bottom:4px; font-size:18pt;'>SLIPER Analiz Raporu</h1>"
            "<p style='color:#6b7280; font-size:9pt; margin-top:0;'>Liya Laboratuvar Test Cihazları — Sliding Pipe Rheometer</p>"
            "<hr style='border:none; border-top:1px solid #d1d5db;'>";

    html += tablo(
```

```
        satir({ "<b>Ölçüm ID</b>", QString::number(olcumId) })
        + satir({ "<b>Ölçüm tarihi</b>", bilgi["tarih"].toString().toHtmlEscaped() })
        + satir({ "<b>Yer</b>", bilgi["yer"].toString().toHtmlEscaped() })
        + satir({ "<b>Müşteri</b>", bilgi["musteri"].toString().toHtmlEscaped() })
        + satir({ "<b>Reçete</b>", bilgi["recete"].toString().toHtmlEscaped() })
        + satir({ "<b>Yorum</b>", bilgi["yorum"].toString().toHtmlEscaped() })
        + satir({ "<b>Rapor tarihi</b>", QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm") }));

    html += baslik("P–Q Sonuçları");
    html += tablo(
        satir({ "Parametre", "Değer" }, true)
        + satir({ "P–Q kesişimi A", yeterliVeri ? sayi(kesisimA, 2) + " mbar" : "—" })
        + satir({ "P–Q eğimi B", yeterliVeri ? sayi(egimB, 3) + " mbar·h/m³" : "—" })
        + satir({ "a (Yield Pressure)", yeterliVeri ? sayi(SliperModel::schleibingerA(kesisimA), 3) + " mbar" : "—" })
        + satir({ "b (Pressure Gradient, ×1000)", yeterliVeri ? sayi(SliperModel::schleibingerB(egimB) * 1000.0, 3) : "—" })
        + satir({ "Uyum kalitesi R²", yeterliVeri ? sayi(pqSonucu["r2"].toDouble(), 3) : "—" })
        + satir({ "Kullanılan / toplam stroke", QString("%1 /
%2").arg(pqSonucu["n"].toInt()).arg(pqSonucu["toplamStroke"].toInt()) }));

    const QStringList uyarilar = pqSonucu.value("uyarilar").toStringList();
    if (!uyarilar.isEmpty()) {
        html += "<p style='color:#b45309; font-size:9pt;'><b>Uyarılar:</b><br>";
        for (const QString &uyariKodu : uyarilar) html += "• " + uyariMetni(uyariKodu).toHtmlEscaped() + "<br>";
        html += "</p>";
    }

    {
        Seri dahilNoktalar; dahilNoktalar.noktaMi = true; dahilNoktalar.renk = QColor("#3b82f6"); dahilNoktalar.ad = "Stroke
(tahmine dahil)";
        Seri haricNoktalar; haricNoktalar.noktaMi = true; haricNoktalar.renk = QColor("#9ca3af"); haricNoktalar.ad = "Hariç /
hatalı";
        double enBuyukDebi = 1.0;
        for (const QVariant &kayit : strokelar) {
            const QVariantMap stroke = kayit.toMap();
            const QPointF nokta(stroke["debi"].toDouble(), stroke["basinc"].toDouble());
            const bool tahmineDahil = stroke["gecerli"].toBool() && stroke["secili"].toBool();
            (tahmineDahil ? dahilNoktalar : haricNoktalar).noktalar.append(nokta);
            enBuyukDebi = std::max(enBuyukDebi, nokta.x());
        }
        QVector<Seri> seriler { dahilNoktalar };
        if (!haricNoktalar.noktalar.isEmpty()) seriler.append(haricNoktalar);
        if (yeterliVeri) {
            Seri pqCizgisi; pqCizgisi.renk = QColor("#111827"); pqCizgisi.ad = "Regresyon";
            const double cizgiSonuDebi = enBuyukDebi * 1.1;
            pqCizgisi.noktalar = { QPointF(0, kesisimA), QPointF(cizgiSonuDebi, kesisimA + egimB * cizgiSonuDebi) };
            seriler.append(pqCizgisi);
        }
        gorseller["pq"] = grafikCiz(QString("p–Q  (A = %1 mbar, B = %2 mbar·h/m³)").arg(sayi(kesisimA, 2), sayi(egimB, 3)),
                                    "Q [m³/h]", "p [mbar]", seriler);
        html += "<p align='center'><img src='__pq__' width='620'></p>";
    }

    html += baslik("Stroke Özeti");
    {
        QString icerik = satir({ "No", "Saat", "Süre [s]", "Ağırlık [kg]", "Pmax", "P0l", "P0r", "p [mbar]", "Q [m³/h]", "Durum"
}, true);
        for (const QVariant &kayit : strokelar) {
            const QVariantMap stroke = kayit.toMap();
            const QString durum = stroke["gecerli"].toBool()
                ? (stroke["secili"].toBool() ? "Dahil"
                   : stroke["otomatikHaric"].toBool() ? "Otomatik hariç" : "Hariç")
                : "Hatalı stroke";
            icerik += satir({ QString::number(stroke["stroke"].toInt()), stroke["saat"].toString(),
                              sayi(stroke["sure"].toDouble(), 2), sayi(stroke["agirlik"].toDouble(), 1),
                              sayi(stroke["pMaks"].toDouble(), 1), sayi(stroke["p0l"].toDouble(), 1),
                              sayi(stroke["p0r"].toDouble(), 1), sayi(stroke["basinc"].toDouble(), 2),
                              sayi(stroke["debi"].toDouble(), 2), durum });
        }
        html += tablo(icerik);
    }
```

```
    html += baslik("Pompa Basıncı Tahmini");
    html += tablo(
        satir({ "Girdi", "Değer" }, true)
        + satir({ "Hedef debi Q", sayi(ayarlar["hedefDebi"].toDouble(), 1) + " m³/h" })
        + satir({ "Boru iç çapı D", sayi(ayarlar["cap"].toDouble(), 0) + " mm" })
        + satir({ "Hat uzunluğu", sayi(ayarlar["hatUzunlugu"].toDouble(), 0) + " m" })
        + satir({ "Pompalama yüksekliği h", sayi(ayarlar["yukseklik"].toDouble(), 1) + " m" })
        + satir({ "90° dirsek", QString("%1 adet × %2 m").arg(sayi(ayarlar["dirsek90"].toDouble(), 0),
sayi(SliperModel::DIRSEK_90_ESDEGER_M, 1)) })
        + satir({ "45° dirsek", QString("%1 adet × %2 m").arg(sayi(ayarlar["dirsek45"].toDouble(), 0),
sayi(SliperModel::DIRSEK_45_ESDEGER_M, 1)) })
        + satir({ "Hesap uzunluğu L (hat + dirsekler)", sayi(SliperModel::hesapUzunluguM(ayarlar), 1) + " m" })
        + satir({ "Beton yoğunluğu ρ", sayi(ayarlar["yogunluk"].toDouble(), 0) + " kg/m³" })
        + satir({ "Hata payı", "±%" + sayi(ayarlar["hataPayi"].toDouble(), 0) })
        + satir({ "Pompa maks. basıncı", ayarlar["pompaMaks"].toDouble() > 0 ? sayi(ayarlar["pompaMaks"].toDouble(), 1) + " bar"
: "girilmedi" }));

    const double pompaMaksBar = ayarlar["pompaMaks"].toDouble();
    bool pompaAsildi = false;
    const QString guvenilmezNedeni = SliperModel::tahminGuvenilmezNedeni(kesisimA, egimB, pqSonucu["r2"].toDouble());
    const bool kararVerilemez = SliperModel::tahminKarariVerilemez(guvenilmezNedeni);
    const bool dusukGuven = guvenilmezNedeni == "dusukR2";
    double enYuksekTahminBar = 0.0;
    QVariantMap enKotuTahmin;
    if (!tahminSatirlari.isEmpty()) {
        QString icerik = satir({ "Q [m³/h]", "L [m]", "P [bar]", "Alt–Üst [bar]", "Pompa" }, true);
        for (const QVariant &kayit : tahminSatirlari) {
            const QVariantMap tahmin = kayit.toMap();
            if (enKotuTahmin.isEmpty() || tahmin["ustSinirBar"].toDouble() > enYuksekTahminBar) enKotuTahmin = tahmin;
            enYuksekTahminBar = std::max(enYuksekTahminBar, tahmin["ustSinirBar"].toDouble());
            pompaAsildi = pompaAsildi || tahmin["pompaAsildi"].toBool();
            QString pompaDurumu = "—";
            if (pompaMaksBar > 0) {
                if (kararVerilemez || tahmin["basincNegatif"].toBool())
                    pompaDurumu = "<span style='color:#6b7280;'>geçersiz</span>";
                else if (tahmin["pompaAsildi"].toBool())
                    pompaDurumu = "<span style='color:#dc2626;'><b>aşıldı</b></span>";
                else if (dusukGuven)
                    pompaDurumu = "<span style='color:#b45309;'>uygun?</span>";
                else
                    pompaDurumu = "<span style='color:#16a34a;'>uygun</span>";
            }
            icerik += satir({ sayi(tahmin["debi"].toDouble(), 1), sayi(tahmin["uzunluk"].toDouble(), 0),
                              "<b>" + sayi(tahmin["basincBar"].toDouble(), 2) + "</b>",
                              sayi(tahmin["altSinirBar"].toDouble(), 2) + " – " + sayi(tahmin["ustSinirBar"].toDouble(), 2),
                              pompaDurumu });
        }
        html += "<br>" + tablo(icerik);
        html += "<p style='color:#6b7280; font-size:8pt;'>Model: P = 4L/D·a + 16·L·Q/(π·D³)·b + ρ·g·h "
            + QString("(a = d·A/4l, b = B·π·d³/16l; d = %1 m, l = %2 m)</p>")
                  .arg(SliperModel::BORU_CAPI_M).arg(SliperModel::BORU_UZUNLUGU_M);

        const double hedefDebi = ayarlar["hedefDebi"].toDouble();
        const double grafikSonuDebi = std::max(hedefDebi * 1.5, 10.0);
        const double hatCapiMm = ayarlar["cap"].toDouble();
        const double hatUzunluguM = SliperModel::hesapUzunluguM(ayarlar);
        const double yukseklikM = ayarlar["yukseklik"].toDouble();
        const double yogunluk = ayarlar["yogunluk"].toDouble();
        const double hataPayi = ayarlar["hataPayi"].toDouble();
        QVector<Seri> seriler;
        Seri tahminCizgisi; tahminCizgisi.renk = QColor("#2563eb"); tahminCizgisi.stil = Qt::SolidLine;
        tahminCizgisi.ad = QString("Tahmin %1 m").arg(sayi(hatUzunluguM, 0));
        for (double debi : { 0.0, grafikSonuDebi }) {
            const QVariantMap tahmin = SliperModel::basincTahmini(kesisimA, egimB, debi, hatCapiMm,
                hatUzunluguM, yukseklikM, yogunluk, hataPayi);
            tahminCizgisi.noktalar.append(QPointF(debi, tahmin["basincBar"].toDouble()));
        }
        if (hataPayi > 0) {
            const QVariantMap tahmin = SliperModel::basincTahmini(kesisimA, egimB, hedefDebi, hatCapiMm,
```

```
                hatUzunluguM, yukseklikM, yogunluk, hataPayi);
            tahminCizgisi.hataCubuklari.append({ QPointF(hedefDebi, tahmin["altSinirBar"].toDouble()),
                                                 QPointF(hedefDebi, tahmin["ustSinirBar"].toDouble()) });
        }
        seriler.append(tahminCizgisi);
        Seri hedefNoktasi; hedefNoktasi.noktaMi = true; hedefNoktasi.renk = QColor("#dc2626"); hedefNoktasi.ad = "Hedef debi";
        hedefNoktasi.noktalar.append(QPointF(hedefDebi, SliperModel::basincTahmini(kesisimA, egimB, hedefDebi, hatCapiMm,
            hatUzunluguM, yukseklikM, yogunluk, hataPayi)["basincBar"].toDouble()));
        seriler.append(hedefNoktasi);
        if (pompaMaksBar > 0) {
            Seri pompaCizgisi; pompaCizgisi.renk = QColor("#f59e0b"); pompaCizgisi.stil = Qt::DashDotLine; pompaCizgisi.ad =
"Pompa maks.";
            pompaCizgisi.noktalar = { QPointF(0, pompaMaksBar), QPointF(grafikSonuDebi, pompaMaksBar) };
            seriler.append(pompaCizgisi);
        }
        gorseller["tahmin"] = grafikCiz("Tahmin Grafiği", "Q [m³/h]", "P [bar]", seriler);
        html += "<p align='center'><img src='__tahmin__' width='620'></p>";
    } else {
        html += "<p style='color:#6b7280;'>Tahmin için yeterli geçerli stroke yok.</p>";
    }

    QString durumRengi = "#6b7280";
    QString durumMetni = "DEĞERLENDİRİLMEDİ (pompa maks. basıncı girilmedi)";
    if (!yeterliVeri) {
        durumMetni = "DEĞERLENDİRİLEMEDİ (yetersiz stroke verisi)";
    } else if (kararVerilemez) {
        durumMetni = guvenilmezNedeni == "egimNegatif"
            ? "DEĞERLENDİRİLEMEDİ — P–Q eğimi (B) negatif, tahmin fiziksel değil. Ölçümü tekrarlayın."
            : "DEĞERLENDİRİLEMEDİ — P–Q kesişimi (A) negatif, tahmin fiziksel değil. Ölçümü tekrarlayın.";
    } else if (pompaMaksBar > 0) {
        if (pompaAsildi) {
            durumRengi = "#dc2626";
            durumMetni = QString("POMPA KAPASİTESİ YETERSİZ — gereken en fazla %1 bar / pompa %2 bar")
                .arg(sayi(enYuksekTahminBar, 1), sayi(pompaMaksBar, 1));
        } else if (dusukGuven) {
            durumRengi = "#b45309";
            durumMetni = QString("POMPALANABİLİR — GÜVENİLİR DEĞİL (R² = %1) — gereken en fazla %2 bar / pompa %3 bar")
                .arg(sayi(pqSonucu["r2"].toDouble(), 2), sayi(enYuksekTahminBar, 1), sayi(pompaMaksBar, 1));
        } else {
            durumRengi = "#16a34a";
            durumMetni = QString("POMPALANABİLİR — gereken en fazla %1 bar / pompa %2 bar")
                .arg(sayi(enYuksekTahminBar, 1), sayi(pompaMaksBar, 1));
        }
    }
    html += QString("<p style='margin-top:16px;'><b>Durum:</b> <span style='color:%1; font-weight:bold;'>%2</span></p>")
                .arg(durumRengi, durumMetni.toHtmlEscaped());

    if (yeterliVeri && !kararVerilemez && pompaMaksBar > 0 && pompaAsildi) {
        const QString sorunNedeni = SliperModel::pompaSorunuNedeni(enKotuTahmin, pompaMaksBar);
        QString oneriBasligi;
        QString gerekce;
        QStringList maddeler;
        if (sorunNedeni == "yukseklik") {
            oneriBasligi = "Reçeteyle çözülmez";
            gerekce = "Yalnızca pompalama yüksekliğinden gelen basınç (ρ·g·h) pompa kapasitesine ulaşıyor; "
                      "karışımın sürtünmesini azaltmak yeterli olmaz.";
            maddeler = { "Daha yüksek basınçlı pompa gerekir" };
        } else if (sorunNedeni == "viskoz") {
            oneriBasligi = "Karışım yapışkan";
            gerekce = "Sürtünmenin büyük kısmı viskoz terimden (b) geliyor: boru cidarındaki kayma tabakası koyu, "
                      "direnç debiyle hızla artıyor.";
            maddeler = { "Katkıyı (akışkanlaştırıcı) artırın", "Viskozite düzenleyici katkıyı azaltın",
                         "Toz / ince malzemeyi azaltın" };
        } else {
            oneriBasligi = "Karışım sert";
            gerekce = "Sürtünmenin büyük kısmı akma teriminden (a) geliyor: boru cidarında kayma tabakasını "
                      "oluşturacak harç yetersiz, betonu harekete geçirmek için yüksek basınç gerekiyor.";
            maddeler = { "Katkıyı (akışkanlaştırıcı) artırın", "İnce kum / harç oranını artırın",
                         "İri agregayı azaltın" };
        }
```

```
        html += baslik("Reçete Önerisi");
        html += "<p><b>" + oneriBasligi.toHtmlEscaped() + "</b></p><ul>";
        for (const QString &madde : maddeler) html += "<li>" + madde.toHtmlEscaped() + "</li>";
        html += "</ul>";
        if (sorunNedeni != "yukseklik") html += "<p>Değişiklikten sonra tekrar ölçün.</p>";
        html += "<p style='color:#6b7280; font-size:8pt;'>" + gerekce.toHtmlEscaped()
                + " Öneriler yön belirtir; miktar kullanılan malzemeye bağlıdır ve yeniden ölçümle doğrulanmalıdır.</p>";
    }

    const QVariantMap konumSinirlari = m_database->kalibrasyonGetir("konum_sinirlari");
    html += baslik("Cihaz Parametreleri");
    html += tablo(
        satir({ "Reometre boru çapı", sayi(SliperModel::BORU_CAPI_M * 1000.0, 0) + " mm" })
        + satir({ "Reometre dolum yüksekliği", sayi(SliperModel::BORU_UZUNLUGU_M * 1000.0, 0) + " mm" })
        + satir({ "Üst / alt konum referansı", konumSinirlari["mevcut"].toBool()
                  ? sayi(konumSinirlari["deger1"].toDouble(), 1) + " / " + sayi(konumSinirlari["deger2"].toDouble(), 1) + " mm"
                  : "kalibre edilmedi (varsayılan)" })
        + satir({ "Load cell kalibrasyonu", tarihVeyaCizgi(m_database->kalibrasyonTarihiGetir("loadcell")) })
        + satir({ "Mesafe sensörü kalibrasyonu", tarihVeyaCizgi(m_database->kalibrasyonTarihiGetir("mesafe_olcum")) })
        + satir({ "Uygulanan dara", bilgi["daraBasinci"].toDouble() != 0.0
                  ? sayi(bilgi["daraBasinci"].toDouble(), 2) + " mbar (A ve a bu kadar düşüktür)"
                  : "yok" }));

    html += "<hr style='border:none; border-top:1px solid #d1d5db; margin-top:20px;'>"
            "<p style='color:#6b7280; font-size:8pt;'>Liya Laboratuvar Test Cihazları - SLIPER Analiz Sistemi</p>"
            "</body></html>";
    return html;
}

QString ReportManager::pdfOnizlemeHtml(int olcumId)
{
    QMap<QString, QImage> gorseller;
    QString html = raporHtmlOlustur(olcumId, gorseller);

    const QString klasor = QStandardPaths::writableLocation(QStandardPaths::AppLocalDataLocation) + "/rapor_onizleme";
    QDir().mkpath(klasor);
    const QString damga = QString::number(QDateTime::currentMSecsSinceEpoch());
    for (auto it = gorseller.constBegin(); it != gorseller.constEnd(); ++it) {
        const QString yol = klasor + QString("/%1_%2_%3.png").arg(olcumId).arg(it.key(), damga);
        it.value().save(yol);
        html.replace("__" + it.key() + "__", QUrl::fromLocalFile(yol).toString());
    }
    return html;
}

QString ReportManager::pdfOlustur(int olcumId)
{
    QString klasor = QStandardPaths::writableLocation(QStandardPaths::DocumentsLocation) + "/SliperRaporlari";
    if (!QDir().mkpath(klasor)) {
        qWarning() << "Rapor klasoru olusturulamadi:" << klasor;
        return QString();
    }

    QString dosyaAdi = klasor + QString("/SLIPER_Rapor_%1_%2.pdf")
        .arg(olcumId)
        .arg(QDateTime::currentDateTime().toString("ddMMyyyy_HHmmss"));

    QMap<QString, QImage> gorseller;
    QString html = raporHtmlOlustur(olcumId, gorseller);

    QTextDocument belge;
    auto kaynaklariEkle = [&]() {
        for (auto it = gorseller.constBegin(); it != gorseller.constEnd(); ++it) {
            belge.addResource(QTextDocument::ImageResource, QUrl("gorsel://" + it.key()), it.value());
        }
    };
    for (auto it = gorseller.constBegin(); it != gorseller.constEnd(); ++it) {
        html.replace("__" + it.key() + "__", "gorsel://" + it.key());
    }
    kaynaklariEkle();
```

```
    belge.setHtml(html);
    kaynaklariEkle();

    QPrinter yazici(QPrinter::HighResolution);
    yazici.setOutputFormat(QPrinter::PdfFormat);
    yazici.setPageSize(QPageSize(QPageSize::A4));
    yazici.setOutputFileName(dosyaAdi);

    belge.print(&yazici);

    if (!QFile::exists(dosyaAdi)) {
        qWarning() << "PDF dosyasi yazilamadi:" << dosyaAdi;
        return QString();
    }

    qDebug() << "PDF rapor olusturuldu:" << dosyaAdi;
    return dosyaAdi;
}
```

src\ReportManager.h

04.11.2026 11:45

```
#pragma once
#include <QObject>
#include <QVariantMap>
#include <QImage>
#include <QPointF>
#include <QColor>
#include <QMap>

class Database;

class ReportManager : public QObject
{
    Q_OBJECT

public:
    explicit ReportManager(Database *database, QObject *parent = nullptr);

    Q_INVOKABLE QString pdfOnizlemeHtml(int olcumId);
    Q_INVOKABLE QString pdfOlustur(int olcumId);

    struct Seri {
        QVector<QPointF> noktalar;
        QColor renk;
        bool noktaMi = false;
        Qt::PenStyle stil = Qt::SolidLine;
        QString ad;
        QVector<QPair<QPointF, QPointF>> hataCubuklari;
    };

private:
    QString raporHtmlOlustur(int olcumId, QMap<QString, QImage> &gorseller);
    static QImage grafikCiz(const QString &baslik, const QString &xEtiket, const QString &yEtiket,
                            const QVector<Seri> &seriler);

    Database *m_database = nullptr;
};
```

src\SensorManager.cpp

04.11.2026 11:45

```
#include "SensorManager.h"

SensorManager::SensorManager(QObject *parent)
    : QObject(parent)
{
}

double SensorManager::basinc() const { return m_basinc; }
double SensorManager::konum() const { return m_konum; }
double SensorManager::hiz() const { return m_hiz; }
double SensorManager::debi() const { return m_debi; }
double SensorManager::egimX() const { return m_egimX; }
double SensorManager::egimY() const { return m_egimY; }
double SensorManager::hamAgirlik() const { return m_hamAgirlik; }
double SensorManager::hamMesafe() const { return m_hamMesafe; }
double SensorManager::hamAccelX() const { return m_hamAccelX; }
double SensorManager::hamAccelY() const { return m_hamAccelY; }
double SensorManager::hamAccelZ() const { return m_hamAccelZ; }
bool SensorManager::veriGecerli() const { return m_veriGecerli; }
bool SensorManager::mesafeSensoruArizali() const { return m_mesafeSensoruArizali; }
bool SensorManager::yukSensoruArizali() const { return m_yukSensoruArizali; }
double SensorManager::bataryaVoltaj() const { return m_bataryaVoltaj; }
double SensorManager::zamanS() const { return m_zamanS; }

void SensorManager::veriGuncelle(double zamanS, double basinc, double konum, double hiz, double debi, double egimX, double egimY)
{
    const bool veriGecerliDegisti = !m_veriGecerli;
    m_veriGecerli = true;
    m_zamanS = zamanS;

    if (m_basinc != basinc) { m_basinc = basinc; emit basincChanged(); }
    if (m_konum != konum) { m_konum = konum; emit konumChanged(); }
    if (m_hiz != hiz) { m_hiz = hiz; emit hizChanged(); }
    if (m_debi != debi) { m_debi = debi; emit debiChanged(); }
    if (m_egimX != egimX) { m_egimX = egimX; emit egimXChanged(); }
    if (m_egimY != egimY) { m_egimY = egimY; emit egimYChanged(); }
    if (veriGecerliDegisti) { emit veriGecerliChanged(); }

    emit veriGuncellendi();
}

void SensorManager::hamAgirlikGuncelle(double hamAgirlik)
{
    if (m_hamAgirlik != hamAgirlik) {
        m_hamAgirlik = hamAgirlik;
        emit hamAgirlikChanged();
    }
}

void SensorManager::hamMesafeGuncelle(double hamMesafe)
{
    if (m_hamMesafe != hamMesafe) {
        m_hamMesafe = hamMesafe;
        emit hamMesafeChanged();
    }
}

void SensorManager::hamAccelGuncelle(double x, double y, double z)
{
    if (m_hamAccelX != x || m_hamAccelY != y || m_hamAccelZ != z) {
        m_hamAccelX = x;
        m_hamAccelY = y;
        m_hamAccelZ = z;
        emit hamAccelDegisti();
    }
}

void SensorManager::bataryaVoltajGuncelle(double voltaj)
```

```
{
    if (m_bataryaVoltaj != voltaj) {
        m_bataryaVoltaj = voltaj;
        emit bataryaVoltajChanged();
    }
}

void SensorManager::veriyiGecersizYap()
{
    if (!m_veriGecerli) return;
    m_veriGecerli = false;
    emit veriGecerliChanged();
}

void SensorManager::sensorArizasiGuncelle(bool mesafeSensoruArizali, bool yukSensoruArizali)
{
    if (m_mesafeSensoruArizali == mesafeSensoruArizali && m_yukSensoruArizali == yukSensoruArizali) return;
    m_mesafeSensoruArizali = mesafeSensoruArizali;
    m_yukSensoruArizali = yukSensoruArizali;
    emit sensorArizasiChanged();
}
```

src\SensorManager.h

04.11.2026 11:45

```
#pragma once
#include <QObject>

class SensorManager : public QObject
{
    Q_OBJECT
    Q_PROPERTY(double basinc READ basinc NOTIFY basincChanged)
    Q_PROPERTY(double konum READ konum NOTIFY konumChanged)
    Q_PROPERTY(double hiz READ hiz NOTIFY hizChanged)
    Q_PROPERTY(double debi READ debi NOTIFY debiChanged)
    Q_PROPERTY(double egimX READ egimX NOTIFY egimXChanged)
    Q_PROPERTY(double egimY READ egimY NOTIFY egimYChanged)
    Q_PROPERTY(double hamAgirlik READ hamAgirlik NOTIFY hamAgirlikChanged)
    Q_PROPERTY(double hamMesafe READ hamMesafe NOTIFY hamMesafeChanged)
    Q_PROPERTY(double hamAccelX READ hamAccelX NOTIFY hamAccelDegisti)
    Q_PROPERTY(double hamAccelY READ hamAccelY NOTIFY hamAccelDegisti)
    Q_PROPERTY(double hamAccelZ READ hamAccelZ NOTIFY hamAccelDegisti)
    Q_PROPERTY(bool veriGecerli READ veriGecerli NOTIFY veriGecerliChanged)
    Q_PROPERTY(bool mesafeSensoruArizali READ mesafeSensoruArizali NOTIFY sensorArizasiChanged)
    Q_PROPERTY(bool yukSensoruArizali READ yukSensoruArizali NOTIFY sensorArizasiChanged)
    Q_PROPERTY(double bataryaVoltaj READ bataryaVoltaj NOTIFY bataryaVoltajChanged)
    Q_PROPERTY(double zamanS READ zamanS NOTIFY veriGuncellendi)

public:
    explicit SensorManager(QObject *parent = nullptr);

    double basinc() const;
    double konum() const;
    double hiz() const;
    double debi() const;
    double egimX() const;
    double egimY() const;
    double hamAgirlik() const;
    double hamMesafe() const;
    double hamAccelX() const;
    double hamAccelY() const;
    double hamAccelZ() const;
    bool veriGecerli() const;
    bool mesafeSensoruArizali() const;
    bool yukSensoruArizali() const;
    double bataryaVoltaj() const;
    double zamanS() const;

    Q_INVOKABLE void veriGuncelle(double zamanS, double basinc, double konum, double hiz, double debi, double egimX, double
egimY);
    Q_INVOKABLE void hamAgirlikGuncelle(double hamAgirlik);
    Q_INVOKABLE void hamMesafeGuncelle(double hamMesafe);
    Q_INVOKABLE void hamAccelGuncelle(double x, double y, double z);
    Q_INVOKABLE void bataryaVoltajGuncelle(double voltaj);
    Q_INVOKABLE void veriyiGecersizYap();
    void sensorArizasiGuncelle(bool mesafeSensoruArizali, bool yukSensoruArizali);

signals:
    void basincChanged();
    void konumChanged();
    void hizChanged();
    void debiChanged();
    void egimXChanged();
    void egimYChanged();
    void hamAgirlikChanged();
    void hamMesafeChanged();
    void hamAccelDegisti();
    void veriGecerliChanged();
    void sensorArizasiChanged();
    void veriGuncellendi();
    void bataryaVoltajChanged();

private:
```

```
    double m_basinc = 0.0;
    double m_konum = 0.0;
    double m_hiz = 0.0;
    double m_debi = 0.0;
    double m_egimX = 0.0;
    double m_egimY = 0.0;
    double m_hamAgirlik = 0.0;
    double m_hamMesafe = 0.0;
    double m_hamAccelX = 0.0;
    double m_hamAccelY = 0.0;
    double m_hamAccelZ = 0.0;
    bool m_veriGecerli = false;
    bool m_mesafeSensoruArizali = false;
    bool m_yukSensoruArizali = false;
    double m_bataryaVoltaj = 0.0;
    double m_zamanS = 0.0;
};
```

src\SliperModel.cpp


```
#include "SliperModel.h"
#include <QMap>
#include <QSet>
#include <QtMath>
#include <algorithm>
#include <cmath>
#include <limits>

namespace SliperModel {
namespace {

struct Cizgi {
    bool hesaplandi = false;
    double egim = 0.0;
    double baslangic = 0.0;
    double uyumR2 = 0.0;
    double ortalamaX = 0.0;
};

Cizgi enUygunCizgi(const QVector<double> &xDegerleri, const QVector<double> &yDegerleri)
{
    Cizgi cizgi;
    const int noktaSayisi = std::min(xDegerleri.size(), yDegerleri.size());
    if (noktaSayisi < 2) return cizgi;

    double xToplami = 0.0, yToplami = 0.0;
    for (int i = 0; i < noktaSayisi; ++i) {
        xToplami += xDegerleri[i];
        yToplami += yDegerleri[i];
    }
    const double ortalamaX = xToplami / noktaSayisi;
    const double ortalamaY = yToplami / noktaSayisi;

    double birlikteSapmaToplami = 0.0;
    double xSapmaKareToplami = 0.0;
    for (int i = 0; i < noktaSayisi; ++i) {
        const double xSapmasi = xDegerleri[i] - ortalamaX;
        const double ySapmasi = yDegerleri[i] - ortalamaY;
        birlikteSapmaToplami += xSapmasi * ySapmasi;
        xSapmaKareToplami += xSapmasi * xSapmasi;
    }
    if (xSapmaKareToplami <= 0.0) return cizgi;

    cizgi.egim = birlikteSapmaToplami / xSapmaKareToplami;
    cizgi.baslangic = ortalamaY - cizgi.egim * ortalamaX;
    cizgi.ortalamaX = ortalamaX;

    double cizgidenKalanKareToplami = 0.0;
    double toplamSapmaKareToplami = 0.0;
    for (int i = 0; i < noktaSayisi; ++i) {
        const double cizgininDedigi = cizgi.baslangic + cizgi.egim * xDegerleri[i];
        const double cizgidenFark = yDegerleri[i] - cizgininDedigi;
        const double ortalamadanFark = yDegerleri[i] - ortalamaY;
        cizgidenKalanKareToplami += cizgidenFark * cizgidenFark;
        toplamSapmaKareToplami += ortalamadanFark * ortalamadanFark;
    }
    cizgi.uyumR2 = toplamSapmaKareToplami > 0.0
        ? 1.0 - cizgidenKalanKareToplami / toplamSapmaKareToplami
        : 0.0;
    cizgi.hesaplandi = true;
    return cizgi;
}

}

double kalibrasyonTablosundanOku(const QVector<double> &hamNoktalar,
                                 const QVector<double> &gercekNoktalar,
                                 double hamDeger)
```

```
{
    const int noktaSayisi = std::min(hamNoktalar.size(), gercekNoktalar.size());
    if (noktaSayisi == 0) return 0.0;
    if (noktaSayisi == 1) return gercekNoktalar[0];

    int solNokta;
    if (hamDeger <= hamNoktalar[0]) {
        solNokta = 0;
    } else if (hamDeger >= hamNoktalar[noktaSayisi - 1]) {
        solNokta = noktaSayisi - 2;
    } else {
        for (solNokta = 0; solNokta < noktaSayisi - 1; ++solNokta) {
            if (hamDeger <= hamNoktalar[solNokta + 1]) break;
        }
    }
    const int sagNokta = solNokta + 1;

    const double solHam = hamNoktalar[solNokta];
    const double sagHam = hamNoktalar[sagNokta];
    const double solGercek = gercekNoktalar[solNokta];
    const double sagGercek = gercekNoktalar[sagNokta];

    if (sagHam == solHam) return solGercek;
    const double hamBirimBasinaGercek = (sagGercek - solGercek) / (sagHam - solHam);
    return solGercek + (hamDeger - solHam) * hamBirimBasinaGercek;
}

double strokeHiziMs(const QVector<double> &zamanS, const QVector<double> &yukseklikMm)
{
    const int ornekSayisi = std::min(zamanS.size(), yukseklikMm.size());
    if (ornekSayisi < 2) return 0.0;

    const double baslangicYuksekligi = yukseklikMm.first();
    const double bitisYuksekligi = yukseklikMm[ornekSayisi - 1];
    const double toplamYolMm = baslangicYuksekligi - bitisYuksekligi;

    QVector<double> ortaZamanlar, ortaYukseklikler;
    for (int i = 0; i < ornekSayisi; ++i) {
        const double alinanYolOrani = toplamYolMm > 0.0
            ? (baslangicYuksekligi - yukseklikMm[i]) / toplamYolMm
            : 0.0;
        if (alinanYolOrani >= HIZ_UYUM_ALT && alinanYolOrani <= HIZ_UYUM_UST) {
            ortaZamanlar.append(zamanS[i]);
            ortaYukseklikler.append(yukseklikMm[i]);
        }
    }

    if (ortaZamanlar.size() >= 3) {
        const Cizgi cizgi = enUygunCizgi(ortaZamanlar, ortaYukseklikler);
        if (cizgi.hesaplandi) {
            const double yukseklikKaybiMmSaniye = -cizgi.egim;
            return yukseklikKaybiMmSaniye / 1000.0;
        }
    }

    const double toplamSureS = zamanS[ornekSayisi - 1] - zamanS.first();
    return toplamSureS > 0.0 ? (toplamYolMm / 1000.0) / toplamSureS : 0.0;
}

double filtreliMaksimum(const QVector<double> &zamanS, const QVector<double> &basincMbar,
                        double pencereS)
{
    const int ornekSayisi = std::min(zamanS.size(), basincMbar.size());
    if (ornekSayisi == 0) return 0.0;

    const double yariPencereS = pencereS / 2.0;
    double enBuyukOrtalama = -1e300;
    for (int merkez = 0; merkez < ornekSayisi; ++merkez) {
        const double merkezZamaniS = zamanS[merkez];
```

```
        int pencereBasi = merkez;
        while (pencereBasi > 0 && merkezZamaniS - zamanS[pencereBasi - 1] <= yariPencereS) {
            --pencereBasi;
        }
        int pencereSonu = merkez;
        while (pencereSonu < ornekSayisi - 1 && zamanS[pencereSonu + 1] - merkezZamaniS <= yariPencereS) {
            ++pencereSonu;
        }

        while (pencereSonu - pencereBasi + 1 < PMAKS_MIN_ORNEK
               && (pencereBasi > 0 || pencereSonu < ornekSayisi - 1)) {
            const bool solaGenislet = pencereBasi > 0
                && (pencereSonu == ornekSayisi - 1
                    || merkezZamaniS - zamanS[pencereBasi - 1] <= zamanS[pencereSonu + 1] - merkezZamaniS);
            if (solaGenislet) --pencereBasi;
            else ++pencereSonu;
        }

        double penceredekiToplam = 0.0;
        for (int i = pencereBasi; i <= pencereSonu; ++i) {
            penceredekiToplam += basincMbar[i];
        }
        const double penceredekiOrtalama = penceredekiToplam / (pencereSonu - pencereBasi + 1);

        enBuyukOrtalama = std::max(enBuyukOrtalama, penceredekiOrtalama);
    }
    return enBuyukOrtalama;
}

double durgunBasincMbar(const QVector<double> &basincMbar)
{
    if (basincMbar.isEmpty()) return 0.0;

    QVector<double> siraliBasinclar = basincMbar;
    std::sort(siraliBasinclar.begin(), siraliBasinclar.end());

    const int ornekSayisi = siraliBasinclar.size();
    const int ortaIndeks = ornekSayisi / 2;
    if (ornekSayisi % 2 == 1) return siraliBasinclar[ortaIndeks];
    return (siraliBasinclar[ortaIndeks - 1] + siraliBasinclar[ortaIndeks]) / 2.0;
}

double enBuyukOrnekAraligiS(const QVector<double> &zamanS)
{
    double enBuyukAralikS = 0.0;
    for (int i = 1; i < zamanS.size(); ++i) {
        enBuyukAralikS = std::max(enBuyukAralikS, zamanS[i] - zamanS[i - 1]);
    }
    return enBuyukAralikS;
}

QString strokeGecersizNedeni(int inisOrnekSayisi, double enBuyukOrnekAraligiS, double hizMs,
                             double basincMbar)
{
    if (enBuyukOrnekAraligiS > MAKS_ORNEK_ARALIGI_S) return "kesinti";
    if (inisOrnekSayisi < MIN_STROKE_ORNEGI) return "ornek";
    if (hizMs <= 0.0) return "hiz";
    if (basincMbar <= 0.0) return "basinc";
    return QString();
}

double takilmaOrani(const QVector<double> &zamanS, const QVector<double> &konumMm)
{
    const double hesaplanamadi = std::numeric_limits<double>::quiet_NaN();
    const int ornekSayisi = std::min(zamanS.size(), konumMm.size());
    if (ornekSayisi < 6) return hesaplanamadi;

    const double toplamYolMm = konumMm[ornekSayisi - 1] - konumMm.first();
    if (std::abs(toplamYolMm) < 1.0) return hesaplanamadi;
```

```
    QVector<double> araOrtaYolOranlari, araHizlari;
    for (int i = 1; i < ornekSayisi; ++i) {
        const double sureS = zamanS[i] - zamanS[i - 1];
        if (sureS <= 0.0) continue;
        const double oncekiOran = (konumMm[i - 1] - konumMm.first()) / toplamYolMm;
        const double sonrakiOran = (konumMm[i] - konumMm.first()) / toplamYolMm;
        araOrtaYolOranlari.append((oncekiOran + sonrakiOran) / 2.0);
        araHizlari.append((sonrakiOran - oncekiOran) / sureS);
    }

    QVector<double> pozitifHizlar;
    for (double hiz : araHizlari) {
        if (hiz > 0.0) pozitifHizlar.append(hiz);
    }
    if (pozitifHizlar.isEmpty()) return hesaplanamadi;
    std::sort(pozitifHizlar.begin(), pozitifHizlar.end());
    const int ortaIndeks = pozitifHizlar.size() / 2;
    const double tipikHiz = pozitifHizlar.size() % 2 == 1
        ? pozitifHizlar[ortaIndeks]
        : (pozitifHizlar[ortaIndeks - 1] + pozitifHizlar[ortaIndeks]) / 2.0;

    double enDusukHiz = std::numeric_limits<double>::max();
    for (int i = 0; i < araHizlari.size(); ++i) {
        if (araOrtaYolOranlari[i] >= TAKILMA_YOL_ALT && araOrtaYolOranlari[i] <= TAKILMA_YOL_UST) {
            enDusukHiz = std::min(enDusukHiz, araHizlari[i]);
        }
    }
    if (enDusukHiz == std::numeric_limits<double>::max()) return hesaplanamadi;
    return enDusukHiz / tipikHiz;
}

QVector<int> otomatikElenecekler(const QVector<qint64> &agirlikAnahtarlari,
                                 const QVector<double> &takilmaOranlari)
{
    const int strokeSayisi = std::min(agirlikAnahtarlari.size(), takilmaOranlari.size());

    QVector<int> adaylar;
    for (int i = 0; i < strokeSayisi; ++i) {
        if (!std::isnan(takilmaOranlari[i]) && takilmaOranlari[i] < TAKILMA_ESIGI) adaylar.append(i);
    }
    std::sort(adaylar.begin(), adaylar.end(), [&](int sol, int sag) {
        return takilmaOranlari[sol] < takilmaOranlari[sag];
    });

    const int enFazlaElenecek = std::max(1, strokeSayisi * MAKS_OTOMATIK_ELEME_YUZDE / 100);
    QVector<int> elenecekler;
    QSet<qint64> elemeYapilanAgirliklar;
    for (int aday : adaylar) {
        if (elenecekler.size() >= enFazlaElenecek) break;
        if (elemeYapilanAgirliklar.contains(agirlikAnahtarlari[aday])) continue;
        elemeYapilanAgirliklar.insert(agirlikAnahtarlari[aday]);
        elenecekler.append(aday);
    }
    return elenecekler;
}

DogruUyumu pqDogrusu(const QVector<double> &debiM3h, const QVector<double> &basincMbar,
                     const QVector<qint64> &agirlikAnahtarlari)
{
    DogruUyumu sonuc;
    const int noktaSayisi = std::min(debiM3h.size(), basincMbar.size());
    sonuc.n = noktaSayisi;
    if (noktaSayisi < 2) return sonuc;

    sonuc.qMin = *std::min_element(debiM3h.begin(), debiM3h.begin() + noktaSayisi);
    sonuc.qMaks = *std::max_element(debiM3h.begin(), debiM3h.begin() + noktaSayisi);

    const Cizgi cizgi = enUygunCizgi(debiM3h, basincMbar);
    double debiToplami = 0.0;
    for (int i = 0; i < noktaSayisi; ++i) debiToplami += debiM3h[i];
```

```
    sonuc.qOrt = debiToplami / noktaSayisi;

    if (cizgi.hesaplandi) {
        sonuc.kesisimA = cizgi.baslangic;
        sonuc.egimB = cizgi.egim;
        sonuc.r2 = cizgi.uyumR2;

        QMap<qint64, double> agirlikBasinaDebiToplami;
        QMap<qint64, double> agirlikBasinaBasincToplami;
        QMap<qint64, int> agirlikBasinaNoktaSayisi;
        const int gruplananNoktaSayisi = std::min<int>(noktaSayisi, agirlikAnahtarlari.size());
        for (int i = 0; i < gruplananNoktaSayisi; ++i) {
            agirlikBasinaDebiToplami[agirlikAnahtarlari[i]] += debiM3h[i];
            agirlikBasinaBasincToplami[agirlikAnahtarlari[i]] += basincMbar[i];
            agirlikBasinaNoktaSayisi[agirlikAnahtarlari[i]] += 1;
        }
        QVector<double> ortalamaDebiler;
        QVector<double> ortalamaBasinclar;
        for (auto it = agirlikBasinaNoktaSayisi.constBegin(); it != agirlikBasinaNoktaSayisi.constEnd(); ++it) {
            ortalamaDebiler.append(agirlikBasinaDebiToplami.value(it.key()) / it.value());
            ortalamaBasinclar.append(agirlikBasinaBasincToplami.value(it.key()) / it.value());
        }
        const Cizgi ortalamaCizgisi = enUygunCizgi(ortalamaDebiler, ortalamaBasinclar);
        if (ortalamaCizgisi.hesaplandi && ortalamaDebiler.size() >= MIN_ORTALAMA_R2_AGIRLIK_SAYISI) {
            sonuc.r2 = ortalamaCizgisi.uyumR2;
        }
    } else {
        double basincToplami = 0.0;
        for (int i = 0; i < noktaSayisi; ++i) basincToplami += basincMbar[i];
        sonuc.kesisimA = basincToplami / noktaSayisi;
    }
    return sonuc;
}

double agirlikTepkiOrani(const QHash<qint64, double> &agirlikBasinaOrtalamaPmax)
{
    if (agirlikBasinaOrtalamaPmax.size() < 2) return std::numeric_limits<double>::quiet_NaN();

    qint64 enHafifAnahtar = agirlikBasinaOrtalamaPmax.constBegin().key();
    qint64 enAgirAnahtar = enHafifAnahtar;
    for (auto it = agirlikBasinaOrtalamaPmax.constBegin();
         it != agirlikBasinaOrtalamaPmax.constEnd(); ++it) {
        enHafifAnahtar = std::min(enHafifAnahtar, it.key());
        enAgirAnahtar = std::max(enAgirAnahtar, it.key());
    }

    const double agirlikFarkiKg = (enAgirAnahtar - enHafifAnahtar) / 10.0;
    if (agirlikFarkiKg <= 0.0) return std::numeric_limits<double>::quiet_NaN();

    const double beklenenArtisMbar = agirliktanBasincMbar(agirlikFarkiKg);
    const double olculenArtisMbar = agirlikBasinaOrtalamaPmax.value(enAgirAnahtar)
                                  - agirlikBasinaOrtalamaPmax.value(enHafifAnahtar);
    return olculenArtisMbar / beklenenArtisMbar;
}

QStringList olcumUyarilari(const DogruUyumu &pqCizgisi,
                           const QHash<qint64, int> &agirlikBasinaStrokeSayisi,
                           const QHash<qint64, double> &agirlikBasinaOrtalamaPmax,
                           int toplamStrokeSayisi, int gecersizStrokeSayisi)
{
    QStringList uyarilar;
    if (pqCizgisi.n < MIN_STROKE_SAYISI) uyarilar << "azStroke";
    if (pqCizgisi.egimB < 0.0) uyarilar << "egimNegatif";
    if (pqCizgisi.kesisimA < 0.0) uyarilar << "kesisimNegatif";
    if (pqCizgisi.r2 < MIN_R2) uyarilar << "dusukR2";

    const double debiAraligi = pqCizgisi.qMaks - pqCizgisi.qMin;
    if (pqCizgisi.qOrt > 0.0 && debiAraligi < MIN_DEBI_ARALIGI_ORANI * pqCizgisi.qOrt) {
        uyarilar << "darDebiAraligi";
    }
```

```
    const double tepkiOrani = agirlikTepkiOrani(agirlikBasinaOrtalamaPmax);
    if (!std::isnan(tepkiOrani) && tepkiOrani < MIN_AGIRLIK_TEPKI_ORANI) {
        uyarilar.prepend("agirlikTepkisiZayif");
    }

    if (agirlikBasinaStrokeSayisi.size() < 2) uyarilar << "tekYuk";
    for (const int strokeSayisi : agirlikBasinaStrokeSayisi) {
        if (strokeSayisi < MIN_YUK_BASINA_STROKE) { uyarilar << "yukBasinaAzStroke"; break; }
    }

    if (toplamStrokeSayisi > 0
        && gecersizStrokeSayisi * 100 > toplamStrokeSayisi * MAKS_GECERSIZ_YUZDE) {
        uyarilar << "cokGecersiz";
    }
    return uyarilar;
}

QVariantMap varsayilanTahminAyarlari()
{
    QVariantMap ayar;
    ayar["cap"] = 125.0;
    ayar["hatUzunlugu"] = 100.0;
    ayar["yukseklik"] = 0.0;
    ayar["dirsek90"] = 0.0;
    ayar["dirsek45"] = 0.0;
    ayar["hedefDebi"] = 30.0;
    ayar["pompaMaks"] = 0.0;
    ayar["yogunluk"] = 2400.0;
    ayar["hataPayi"] = 10.0;
    return ayar;
}

QVariantMap tahminAyarlariniTamamla(const QVariantMap &ayarlar)
{
    QVariantMap tasinanAyarlar = ayarlar;
    if (!tasinanAyarlar.contains("hatUzunlugu")) {
        if (tasinanAyarlar.contains("yatayUzunluk")) {
            tasinanAyarlar["hatUzunlugu"] = tasinanAyarlar.value("yatayUzunluk").toDouble()
                                          + std::abs(tasinanAyarlar.value("yukseklik").toDouble());
        } else if (tasinanAyarlar.contains("l4")) {
            tasinanAyarlar["hatUzunlugu"] = tasinanAyarlar.value("l4");
        }
    }
    if (!tasinanAyarlar.contains("hedefDebi") && tasinanAyarlar.contains("q2")) {
        tasinanAyarlar["hedefDebi"] = tasinanAyarlar.value("q2");
    }

    QVariantMap tamAyarlar = varsayilanTahminAyarlari();
    for (auto alan = tasinanAyarlar.constBegin(); alan != tasinanAyarlar.constEnd(); ++alan) {
        if (!tamAyarlar.contains(alan.key()) || !alan.value().isValid()) continue;
        bool sayiMi = false;
        const double deger = alan.value().toDouble(&sayiMi);
        if (sayiMi) tamAyarlar[alan.key()] = deger;
    }
    return tamAyarlar;
}

double hesapUzunluguM(const QVariantMap &ayarlar)
{
    const QVariantMap ayar = tahminAyarlariniTamamla(ayarlar);
    return ayar["hatUzunlugu"].toDouble()
           + ayar["dirsek90"].toDouble() * DIRSEK_90_ESDEGER_M
           + ayar["dirsek45"].toDouble() * DIRSEK_45_ESDEGER_M;
}

QVariantMap basincTahmini(double kesisimA, double egimB, double debiM3h, double capMm,
                          double uzunlukM, double yukseklikM, double yogunlukKgM3,
                          double hataPayiYuzde)
{
```

```
    QVariantMap sonuc;
    sonuc["gecerliGiris"] = false;
    sonuc["basincBar"] = 0.0;
    sonuc["altSinirBar"] = 0.0;
    sonuc["ustSinirBar"] = 0.0;

    if (capMm <= 0.0 || uzunlukM < 0.0 || debiM3h < 0.0 || yogunlukKgM3 < 0.0) {
        return sonuc;
    }

    const double aMbar = schleibingerA(kesisimA);
    const double bMbarHm = schleibingerB(egimB);
    const double hatCapiM = capMm / 1000.0;
    const double hatUzunluguM = uzunlukM;
    const double hatCapiKupu = hatCapiM * hatCapiM * hatCapiM;

    const double akmaPayiMbar = 4.0 * hatUzunluguM / hatCapiM * aMbar;
    const double hizPayiMbar = 16.0 * hatUzunluguM * debiM3h / (M_PI * hatCapiKupu) * bMbarHm;
    const double yukseklikPayiMbar = yogunlukKgM3 * YERCEKIMI * yukseklikM / 100.0;

    const double toplamBar = (akmaPayiMbar + hizPayiMbar + yukseklikPayiMbar) / 1000.0;

    const double surtunmeBar = (akmaPayiMbar + hizPayiMbar) / 1000.0;
    const double hataOrani = qBound(0.0, hataPayiYuzde, 100.0) / 100.0;

    sonuc["gecerliGiris"] = true;
    sonuc["basincBar"] = toplamBar;
    const double hataBandiBar = std::abs(surtunmeBar) * hataOrani;
    sonuc["altSinirBar"] = toplamBar - hataBandiBar;
    sonuc["ustSinirBar"] = toplamBar + hataBandiBar;
    sonuc["akmaMbar"] = akmaPayiMbar;
    sonuc["viskozMbar"] = hizPayiMbar;
    sonuc["yukseklikMbar"] = yukseklikPayiMbar;
    sonuc["aMbar"] = aMbar;
    sonuc["bMbarHm"] = bMbarHm;
    return sonuc;
}

QVariantList tahminTablosu(double kesisimA, double egimB, const QVariantMap &ayarlar)
{
    const QVariantMap ayar = tahminAyarlariniTamamla(ayarlar);
    const double pompaMaksBar = ayar["pompaMaks"].toDouble();
    const double debi = ayar["hedefDebi"].toDouble();
    const double uzunluk = hesapUzunluguM(ayar);

    QVariantMap satir = basincTahmini(kesisimA, egimB, debi, ayar["cap"].toDouble(), uzunluk,
                                      ayar["yukseklik"].toDouble(), ayar["yogunluk"].toDouble(),
                                      ayar["hataPayi"].toDouble());
    satir["debi"] = debi;
    satir["uzunluk"] = uzunluk;
    satir["pompaAsildi"] = pompaMaksBar > 0.0 && satir["ustSinirBar"].toDouble() > pompaMaksBar;
    satir["basincNegatif"] = satir["basincBar"].toDouble() < 0.0;
    return { satir };
}

QString tahminGuvenilmezNedeni(double kesisimA, double egimB, double r2)
{
    if (egimB < 0.0) return "egimNegatif";
    if (kesisimA < 0.0) return "kesisimNegatif";
    if (r2 < MIN_R2) return "dusukR2";
    return QString();
}

bool tahminKarariVerilemez(const QString &guvenilmezNedeni)
{
    return guvenilmezNedeni == "egimNegatif" || guvenilmezNedeni == "kesisimNegatif";
}

QString pompaSorunuNedeni(const QVariantMap &tahminSatiri, double pompaMaksBar)
{
```

```
    const double yukseklikBar = tahminSatiri["yukseklikMbar"].toDouble() / 1000.0;
    if (yukseklikBar >= pompaMaksBar) return "yukseklik";
    if (tahminSatiri["viskozMbar"].toDouble() > tahminSatiri["akmaMbar"].toDouble()) return "viskoz";
    return "akma";
}

double penceredekiHizMs(const QVector<double> &zamanS, const QVector<double> &konumMm, double konumYonu)
{
    const Cizgi cizgi = enUygunCizgi(zamanS, konumMm);
    if (!cizgi.hesaplandi) return 0.0;
    return -konumYonu * cizgi.egim / 1000.0;
}

QVector<double> hizEgrisiMs(const QVector<double> &zamanS, const QVector<double> &konumMm,
                            double konumYonu)
{
    const int ornekSayisi = std::min(zamanS.size(), konumMm.size());
    const double yariPencereS = HIZ_PENCERESI_S / 2.0;
    QVector<double> egri;
    int pencereBasi = 0;
    int pencereSonu = 0;
    for (int merkez = 0; merkez < ornekSayisi; ++merkez) {
        while (zamanS[pencereBasi] < zamanS[merkez] - yariPencereS) ++pencereBasi;
        while (pencereSonu + 1 < ornekSayisi && zamanS[pencereSonu + 1] <= zamanS[merkez] + yariPencereS) {
            ++pencereSonu;
        }
        const int penceredekiOrnekSayisi = pencereSonu - pencereBasi + 1;
        egri.append(penceredekiHizMs(zamanS.mid(pencereBasi, penceredekiOrnekSayisi),
                                     konumMm.mid(pencereBasi, penceredekiOrnekSayisi), konumYonu));
    }
    return egri;
}

EgimAcilari egimAcilari(double ivmeX, double ivmeY, double ivmeZ)
{
    const double radyandanDereceye = 180.0 / PI;
    EgimAcilari acilar;
    acilar.xDerece = std::atan2(ivmeY, ivmeZ) * radyandanDereceye;
    acilar.yDerece = std::atan2(-ivmeX, std::sqrt(ivmeY * ivmeY + ivmeZ * ivmeZ)) * radyandanDereceye;
    return acilar;
}

}
```

src\SliperModel.h

04.11.2026 11:45

```
#pragma once
#include <QVariantMap>
#include <QVariantList>
#include <QVector>
#include <QHash>
#include <QString>
#include <QStringList>

namespace SliperModel {
constexpr double PI = 3.14159265358979323846;
constexpr double BORU_CAPI_M = 0.128;
constexpr double BORU_UZUNLUGU_M = 0.5;
constexpr double BORU_ALANI_M2 = PI * BORU_CAPI_M * BORU_CAPI_M / 4.0;
constexpr double YERCEKIMI = 9.80665;

double kalibrasyonTablosundanOku(const QVector<double> &hamNoktalar,
                                 const QVector<double> &gercekNoktalar,
                                 double hamDeger);

constexpr double KONUM_FILTRE_KATSAYISI = 0.35;
constexpr double konumFiltrele(double oncekiFiltreliKonumMm, double hamKonumMm)
{
    return KONUM_FILTRE_KATSAYISI * hamKonumMm
           + (1.0 - KONUM_FILTRE_KATSAYISI) * oncekiFiltreliKonumMm;
}

constexpr double AGIRLIK_FILTRE_KATSAYISI = 0.4;
constexpr double agirlikFiltrele(double oncekiFiltreliHamAgirlik, double hamAgirlik)
{
    return AGIRLIK_FILTRE_KATSAYISI * hamAgirlik
           + (1.0 - AGIRLIK_FILTRE_KATSAYISI) * oncekiFiltreliHamAgirlik;
}

constexpr double agirliktanBasincMbar(double kutleKg)
{
    return kutleKg * YERCEKIMI / BORU_ALANI_M2 / 100.0;
}

constexpr double DARA_UST_SINIRI_KG = 2.0;
constexpr double DARA_UST_SINIRI_MBAR = agirliktanBasincMbar(DARA_UST_SINIRI_KG);

constexpr double hizdanDebiM3h(double hizMs)
{
    return hizMs * BORU_ALANI_M2 * 3600.0;
}

constexpr double KONUM_TOLERANSI_MM = 5.0;
constexpr double UST_YAKALAMA_TOLERANSI_MM = 20.0;
constexpr double ALT_YAKALAMA_TOLERANSI_MM = 25.0;
constexpr double DURGUN_TOLERANS_MM = 12.0;
constexpr double DURMA_TOLERANSI_MM = 12.0;
constexpr double INISTE_KALDIRMA_ESIGI_MM = 30.0;
constexpr double DURMA_ONAYI_S = 2.0;
constexpr double MIN_INIS_YOLU_MM = 40.0;
constexpr double MIN_INIS_SURESI_S = 1.0;
constexpr double KALDIRMA_ESIGI_MM = 10.0;
constexpr double ONCESI_PENCERE_S = 2.0;
constexpr double SONRASI_PENCERE_S = 2.0;
constexpr double HAREKET_PAYI_S = 0.3;
constexpr double P0R_OTURMA_S = 0.5;
constexpr double UST_TAMPON_S = 4.0;
constexpr double HAZIRLAMA_SURESI_S = 2.0;
constexpr double HAZIRLAMA_BASINC_TOLERANSI_MBAR = 3.0;

constexpr double HIZ_UYUM_ALT = 0.2;
constexpr double HIZ_UYUM_UST = 0.8;
double strokeHiziMs(const QVector<double> &zamanS, const QVector<double> &yukseklikMm);
```

```
constexpr double PMAKS_PENCERE_S = 0.30;
constexpr int PMAKS_MIN_ORNEK = 3;
double filtreliMaksimum(const QVector<double> &zamanS, const QVector<double> &basincMbar,
                        double pencereS = PMAKS_PENCERE_S);

double durgunBasincMbar(const QVector<double> &basincMbar);

constexpr double strokeBasinciMbar(double pMaksMbar, double p0lMbar)
{
    return pMaksMbar - p0lMbar;
}

constexpr int MIN_STROKE_ORNEGI = 5;
constexpr double MAKS_ORNEK_ARALIGI_S = 0.3;
double enBuyukOrnekAraligiS(const QVector<double> &zamanS);
QString strokeGecersizNedeni(int inisOrnekSayisi, double enBuyukOrnekAraligiS, double hizMs,
                             double basincMbar);

constexpr double TAKILMA_ESIGI = 0.15;
constexpr double TAKILMA_YOL_ALT = 0.05;
constexpr double TAKILMA_YOL_UST = 0.80;
double takilmaOrani(const QVector<double> &zamanS, const QVector<double> &konumMm);

constexpr int MAKS_OTOMATIK_ELEME_YUZDE = 25;
QVector<int> otomatikElenecekler(const QVector<qint64> &agirlikAnahtarlari,
                                 const QVector<double> &takilmaOranlari);

constexpr int MIN_ORTALAMA_R2_AGIRLIK_SAYISI = 3;
struct DogruUyumu {
    int n = 0;
    double kesisimA = 0.0;
    double egimB = 0.0;
    double r2 = 0.0;
    double qMin = 0.0;
    double qMaks = 0.0;
    double qOrt = 0.0;
};
DogruUyumu pqDogrusu(const QVector<double> &debiM3h, const QVector<double> &basincMbar,
                     const QVector<qint64> &agirlikAnahtarlari);

constexpr int MIN_STROKE_SAYISI = 3;
constexpr double MIN_R2 = 0.85;
constexpr double MIN_DEBI_ARALIGI_ORANI = 0.3;
constexpr int MIN_YUK_BASINA_STROKE = 3;
constexpr int MAKS_GECERSIZ_YUZDE = 30;

constexpr double MIN_AGIRLIK_TEPKI_ORANI = 0.5;
double agirlikTepkiOrani(const QHash<qint64, double> &agirlikBasinaOrtalamaPmax);

QStringList olcumUyarilari(const DogruUyumu &pqCizgisi,
                           const QHash<qint64, int> &agirlikBasinaStrokeSayisi,
                           const QHash<qint64, double> &agirlikBasinaOrtalamaPmax,
                           int toplamStrokeSayisi, int gecersizStrokeSayisi);

constexpr double schleibingerA(double kesisimMbar)
{
    return BORU_CAPI_M * kesisimMbar / (4.0 * BORU_UZUNLUGU_M);
}
constexpr double schleibingerB(double egimMbarHm3)
{
    return egimMbarHm3 * PI * BORU_CAPI_M * BORU_CAPI_M * BORU_CAPI_M
           / (16.0 * BORU_UZUNLUGU_M);
}

constexpr double DIRSEK_90_ESDEGER_M = 3.0;
constexpr double DIRSEK_45_ESDEGER_M = 1.5;
QVariantMap varsayilanTahminAyarlari();
QVariantMap tahminAyarlariniTamamla(const QVariantMap &ayarlar);
double hesapUzunluguM(const QVariantMap &ayarlar);
```

```
QVariantMap basincTahmini(double kesisimA, double egimB, double debiM3h, double capMm,
                          double uzunlukM, double yukseklikM, double yogunlukKgM3,
                          double hataPayiYuzde);

QVariantList tahminTablosu(double kesisimA, double egimB, const QVariantMap &ayarlar);

QString tahminGuvenilmezNedeni(double kesisimA, double egimB, double r2);
bool tahminKarariVerilemez(const QString &guvenilmezNedeni);
QString pompaSorunuNedeni(const QVariantMap &tahminSatiri, double pompaMaksBar);

constexpr double HIZ_PENCERESI_S = 0.40;
double penceredekiHizMs(const QVector<double> &zamanS, const QVector<double> &konumMm, double konumYonu);
QVector<double> hizEgrisiMs(const QVector<double> &zamanS, const QVector<double> &konumMm,
                            double konumYonu);

struct EgimAcilari {
    double xDerece = 0.0;
    double yDerece = 0.0;
};
EgimAcilari egimAcilari(double ivmeX, double ivmeY, double ivmeZ);

constexpr double ADS1115_LSB_VOLT = 0.000125;
constexpr double BATARYA_BOLUCU_ORANI = 2.0;
constexpr double bataryaVoltaji(double hamDeger)
{
    return hamDeger * ADS1115_LSB_VOLT * BATARYA_BOLUCU_ORANI;
}

}
```

src\VoiceCommandManager.cpp

04.11.2026 11:45

```
#include "VoiceCommandManager.h"
#include "VoiceRecognitionWorker.h"

#include <QCoreApplication>
#include <QDir>
#include <QMetaObject>

VoiceCommandManager::VoiceCommandManager(QObject *parent)
    : QObject(parent)
{
    const QString modelYolu = QDir(QCoreApplication::applicationDirPath()).filePath("vosk-model-small-tr-0.3");

    m_worker = new VoiceRecognitionWorker(modelYolu);
    m_worker->moveToThread(&m_workerThread);

    connect(&m_workerThread, &QThread::finished, m_worker, &QObject::deleteLater);

    connect(m_worker, &VoiceRecognitionWorker::modelHazir, this, &VoiceCommandManager::workerModelHazir);
    connect(m_worker, &VoiceRecognitionWorker::dinliyorDegisti, this, &VoiceCommandManager::workerDinliyorDegisti);
    connect(m_worker, &VoiceRecognitionWorker::hataOlustu, this, &VoiceCommandManager::workerHataOlustu);
    connect(m_worker, &VoiceRecognitionWorker::komutAlgilandi, this, &VoiceCommandManager::workerKomutAlgilandi);
    connect(m_worker, &VoiceRecognitionWorker::anlikMetinDegisti, this, &VoiceCommandManager::workerAnlikMetinDegisti);

    m_workerThread.start();
    QMetaObject::invokeMethod(m_worker, "modeliYukle", Qt::QueuedConnection);
}

VoiceCommandManager::~VoiceCommandManager()
{
    m_workerThread.quit();
    m_workerThread.wait(3000);
}

bool VoiceCommandManager::etkin() const { return m_etkin; }

void VoiceCommandManager::setEtkin(bool etkin)
{
    if (m_etkin == etkin)
        return;
    m_etkin = etkin;
    emit etkinChanged();

    if (m_etkin)
        QMetaObject::invokeMethod(m_worker, "dinlemeyeBasla", Qt::QueuedConnection);
    else
        QMetaObject::invokeMethod(m_worker, "dinlemeyiDurdur", Qt::QueuedConnection);
}

bool VoiceCommandManager::dinliyor() const { return m_dinliyor; }
bool VoiceCommandManager::modelHazir() const { return m_modelHazir; }
QString VoiceCommandManager::sonKomutMetni() const { return m_sonKomutMetni; }
QString VoiceCommandManager::anlikMetin() const { return m_anlikMetin; }
QString VoiceCommandManager::sonHata() const { return m_sonHata; }

void VoiceCommandManager::workerModelHazir(bool hazir)
{
    m_modelHazir = hazir;
    emit modelHazirChanged();

    if (hazir && m_etkin)
        QMetaObject::invokeMethod(m_worker, "dinlemeyeBasla", Qt::QueuedConnection);
}

void VoiceCommandManager::workerDinliyorDegisti(bool dinliyor)
{
    m_dinliyor = dinliyor;
    emit dinliyorChanged();
}
```

```
void VoiceCommandManager::workerHataOlustu(const QString &mesaj)
{
    m_sonHata = mesaj;
    emit sonHataChanged();
}

void VoiceCommandManager::workerKomutAlgilandi(int komutTuru, const QString &metin)
{
    m_sonKomutMetni = metin;
    emit sonKomutMetniChanged();

    switch (static_cast<VoiceRecognitionWorker::KomutTuru>(komutTuru)) {
    case VoiceRecognitionWorker::KomutBaslat: emit baslatKomutu(); break;
    case VoiceRecognitionWorker::KomutDurdur: emit durdurKomutu(); break;
    case VoiceRecognitionWorker::KomutBitir:  emit bitirKomutu();  break;
    case VoiceRecognitionWorker::KomutDevam:  emit devamKomutu();  break;
    case VoiceRecognitionWorker::KomutEkle:   emit ekleKomutu();   break;
    }
}

void VoiceCommandManager::workerAnlikMetinDegisti(const QString &metin)
{
    m_anlikMetin = metin;
    emit anlikMetinChanged();
}
```

src\VoiceCommandManager.h

04.11.2026 11:45

```
#pragma once
#include <QObject>
#include <QString>
#include <QThread>

class VoiceRecognitionWorker;

class VoiceCommandManager : public QObject
{
    Q_OBJECT
    Q_PROPERTY(bool etkin READ etkin WRITE setEtkin NOTIFY etkinChanged)
    Q_PROPERTY(bool dinliyor READ dinliyor NOTIFY dinliyorChanged)
    Q_PROPERTY(bool modelHazir READ modelHazir NOTIFY modelHazirChanged)
    Q_PROPERTY(QString sonKomutMetni READ sonKomutMetni NOTIFY sonKomutMetniChanged)
    Q_PROPERTY(QString anlikMetin READ anlikMetin NOTIFY anlikMetinChanged)
    Q_PROPERTY(QString sonHata READ sonHata NOTIFY sonHataChanged)

public:
    explicit VoiceCommandManager(QObject *parent = nullptr);
    ~VoiceCommandManager() override;

    bool etkin() const;
    void setEtkin(bool etkin);
    bool dinliyor() const;
    bool modelHazir() const;
    QString sonKomutMetni() const;
    QString anlikMetin() const;
    QString sonHata() const;

signals:
    void etkinChanged();
    void dinliyorChanged();
    void modelHazirChanged();
    void sonKomutMetniChanged();
    void anlikMetinChanged();
    void sonHataChanged();

    void baslatKomutu();
    void durdurKomutu();
    void bitirKomutu();
    void devamKomutu();
    void ekleKomutu();

private slots:
    void workerModelHazir(bool hazir);
    void workerDinliyorDegisti(bool dinliyor);
    void workerHataOlustu(const QString &mesaj);
    void workerKomutAlgilandi(int komutTuru, const QString &metin);
    void workerAnlikMetinDegisti(const QString &metin);

private:
    QThread m_workerThread;
    VoiceRecognitionWorker *m_worker = nullptr;

    bool m_etkin = false;
    bool m_dinliyor = false;
    bool m_modelHazir = false;
    QString m_sonKomutMetni;
    QString m_anlikMetin;
    QString m_sonHata;
};
```

src\VoiceRecognitionWorker.cpp

04.11.2026 11:45

```
#include "VoiceRecognitionWorker.h"
#include "vosk_api.h"

#include <QAudioSource>
#include <QAudioFormat>
#include <QMediaDevices>
#include <QAudioDevice>
#include <QIODevice>
#include <QJsonDocument>
#include <QJsonObject>
#include <QJsonArray>
#include <QDebug>
#include <QLocale>
#include <QVector>
#include <algorithm>
#include <cstdlib>

namespace {
const QStringList &grammarKelimeleri()
{
    static const QStringList kelimeler = {
        "başlat", "başla",
        "durdur",
        "bitir", "bitti", "bitirdi", "bitiriyor", "bitirelim", "tamamla",
        "devam",
        "ekle"
    };
    return kelimeler;
}

QByteArray grammarJsonUret()
{
    QJsonArray dizi;
    for (const QString &kelime : grammarKelimeleri())
        dizi.append(kelime);
    dizi.append(QStringLiteral("[unk]"));
    return QJsonDocument(dizi).toJson(QJsonDocument::Compact);
}

QString sadelestir(const QString &metin)
{
    QString kucuk = QLocale(QLocale::Turkish).toLower(metin);
    QString sonuc;
    sonuc.reserve(kucuk.size());
    for (const QChar &ch : kucuk) {
        switch (ch.unicode()) {
        case u'ş': sonuc += 's'; break;
        case u'ç': sonuc += 'c'; break;
        case u'ğ': sonuc += 'g'; break;
        case u'ü': sonuc += 'u'; break;
        case u'ö': sonuc += 'o'; break;
        case u'ı': sonuc += 'i'; break;
        case u'İ': sonuc += 'i'; break;
        default:
            if (ch.isLetterOrNumber() || ch.isSpace())
                sonuc += ch;
            break;
        }
    }
    return sonuc;
}

int levenshtein(const QString &a, const QString &b)
{
    const int n = a.size();
    const int m = b.size();
    QVector<int> onceki(m + 1), simdiki(m + 1);
    for (int j = 0; j <= m; ++j) onceki[j] = j;
```

```
    for (int i = 1; i <= n; ++i) {
        simdiki[0] = i;
        for (int j = 1; j <= m; ++j) {
            int maliyet = (a[i - 1] == b[j - 1]) ? 0 : 1;
            simdiki[j] = std::min({ onceki[j] + 1, simdiki[j - 1] + 1, onceki[j - 1] + maliyet });
        }
        onceki.swap(simdiki);
    }
    return onceki[m];
}

bool kelimeUyuyorMu(const QString &token, const QStringList &adaylar, int tolerans)
{
    if (adaylar.contains(token))
        return true;
    for (const QString &aday : adaylar) {
        if (std::abs(token.size() - aday.size()) <= tolerans && levenshtein(token, aday) <= tolerans)
            return true;
    }
    return false;
}

}

VoiceRecognitionWorker::VoiceRecognitionWorker(const QString &modelYolu, QObject *parent)
    : QObject(parent)
    , m_modelYolu(modelYolu)
{
    m_sonKomutZamani.start();
}

VoiceRecognitionWorker::~VoiceRecognitionWorker()
{
    dinlemeyiDurdur();
    if (m_recognizer) vosk_recognizer_free(m_recognizer);
    if (m_model) vosk_model_free(m_model);
}

void VoiceRecognitionWorker::modeliYukle()
{
    vosk_set_log_level(-1);
    m_model = vosk_model_new(m_modelYolu.toUtf8().constData());
    if (!m_model) {
        emit hataOlustu(QStringLiteral("Ses tanima modeli yuklenemedi: ") + m_modelYolu);
        emit modelHazir(false);
        return;
    }
    emit modelHazir(true);
}

void VoiceRecognitionWorker::dinlemeyeBasla()
{
    if (!m_model) {
        emit hataOlustu(QStringLiteral("Ses tanima modeli hazir degil."));
        return;
    }
    if (m_audioSource)
        return;

    QAudioFormat format;
    format.setSampleRate(16000);
    format.setChannelCount(1);
    format.setSampleFormat(QAudioFormat::Int16);

    const QAudioDevice girisAygiti = QMediaDevices::defaultAudioInput();
    if (girisAygiti.isNull()) {
        emit hataOlustu(QStringLiteral("Mikrofon bulunamadi."));
        return;
    }
```

```
    if (!m_recognizer) {
        const QByteArray grammar = grammarJsonUret();
        m_recognizer = vosk_recognizer_new_grm(m_model, 16000.0f, grammar.constData());
        if (!m_recognizer) {
            emit hataOlustu(QStringLiteral("Ses tanima motoru baslatilamadi."));
            return;
        }
        vosk_recognizer_set_words(m_recognizer, 0);
    } else {
        vosk_recognizer_reset(m_recognizer);
    }

    m_audioSource = new QAudioSource(girisAygiti, format, this);
    m_audioDevice = m_audioSource->start();
    if (!m_audioDevice) {
        emit hataOlustu(QStringLiteral("Mikrofon acilamadi."));
        delete m_audioSource;
        m_audioSource = nullptr;
        return;
    }

    connect(m_audioDevice, &QIODevice::readyRead, this, &VoiceRecognitionWorker::sesVerisiHazir);
    emit dinliyorDegisti(true);
}

void VoiceRecognitionWorker::dinlemeyiDurdur()
{
    if (!m_audioSource)
        return;

    m_audioSource->stop();
    m_audioSource->deleteLater();
    m_audioSource = nullptr;
    m_audioDevice = nullptr;

    if (m_recognizer)
        vosk_recognizer_reset(m_recognizer);

    emit dinliyorDegisti(false);
    emit anlikMetinDegisti(QString());
}

void VoiceRecognitionWorker::sesVerisiHazir()
{
    if (!m_audioDevice || !m_recognizer)
        return;

    const QByteArray veri = m_audioDevice->readAll();
    if (veri.isEmpty())
        return;

    const int bitti = vosk_recognizer_accept_waveform(m_recognizer, veri.constData(), veri.size());
    if (bitti == 1) {
        sonucuIsle(QString::fromUtf8(vosk_recognizer_result(m_recognizer)));
    } else if (bitti == 0) {
        const QString kismi = QString::fromUtf8(vosk_recognizer_partial_result(m_recognizer));
        QJsonObject obj = QJsonDocument::fromJson(kismi.toUtf8()).object();
        emit anlikMetinDegisti(obj.value(QStringLiteral("partial")).toString());
    }
}

void VoiceRecognitionWorker::sonucuIsle(const QString &json)
{
    QJsonObject obj = QJsonDocument::fromJson(json.toUtf8()).object();
    const QString metin = obj.value(QStringLiteral("text")).toString().trimmed();
    emit anlikMetinDegisti(QString());
    if (metin.isEmpty())
        return;
```

```
    metniDegerlendir(metin);
}

void VoiceRecognitionWorker::metniDegerlendir(const QString &metin)
{
    static const QStringList baslatAdaylari = { "baslat", "basla" };
    static const QStringList durdurAdaylari = { "durdur" };
    static const QStringList bitirAdaylari = { "bitir", "bitti", "bitirdi", "bitiriyor", "bitirelim", "tamamla" };
    static const QStringList devamAdaylari = { "devam" };
    static const QStringList ekleAdaylari = { "ekle" };

    const QString sade = sadelestir(metin);
    const QStringList tokenlar = sade.split(' ', Qt::SkipEmptyParts);

    for (const QString &token : tokenlar) {
        if (token == "unk")
            continue;

        int komutTuru = -1;
        if (kelimeUyuyorMu(token, bitirAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutBitir;
        else if (kelimeUyuyorMu(token, durdurAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutDurdur;
        else if (kelimeUyuyorMu(token, devamAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutDevam;
        else if (kelimeUyuyorMu(token, ekleAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutEkle;
        else if (kelimeUyuyorMu(token, baslatAdaylari, 1)) komutTuru = VoiceRecognitionWorker::KomutBaslat;

        if (komutTuru >= 0) {
            if (m_sonKomutZamani.elapsed() < KOMUT_BEKLEME_MS)
                return;
            m_sonKomutZamani.restart();
            emit komutAlgilandi(komutTuru, metin);
            return;
        }
    }
}
```

src\VoiceRecognitionWorker.h

04.11.2026 11:45

```
#pragma once
#include <QObject>
#include <QString>
#include <QElapsedTimer>

class QAudioSource;
class QIODevice;
struct VoskModel;
struct VoskRecognizer;

class VoiceRecognitionWorker : public QObject
{
    Q_OBJECT

public:
    enum KomutTuru {
        KomutBaslat = 0,
        KomutDurdur = 1,
        KomutBitir = 2,
        KomutDevam = 3,
        KomutEkle = 4
    };
    Q_ENUM(KomutTuru)

    explicit VoiceRecognitionWorker(const QString &modelYolu, QObject *parent = nullptr);
    ~VoiceRecognitionWorker() override;

public slots:
    void modeliYukle();
    void dinlemeyeBasla();
    void dinlemeyiDurdur();

signals:
    void modelHazir(bool basarili);
    void dinliyorDegisti(bool dinliyor);
    void hataOlustu(const QString &mesaj);
    void komutAlgilandi(int komutTuru, const QString &metin);
    void anlikMetinDegisti(const QString &metin);

private slots:
    void sesVerisiHazir();

private:
    void sonucuIsle(const QString &json);
    void metniDegerlendir(const QString &metin);

    QString m_modelYolu;
    VoskModel *m_model = nullptr;
    VoskRecognizer *m_recognizer = nullptr;
    QAudioSource *m_audioSource = nullptr;
    QIODevice *m_audioDevice = nullptr;

    QElapsedTimer m_sonKomutZamani;
    static constexpr qint64 KOMUT_BEKLEME_MS = 1200;
};
```

src\WifiManager.cpp

04.11.2026 11:45

```
#include "WifiManager.h"
#include "SliperModel.h"
#include <QJsonDocument>
#include <QJsonObject>
#include <QDebug>
#include <cmath>

WifiManager::WifiManager(SensorManager *sensorManager, Database *database, QObject *parent)
    : QObject(parent)
    , m_sensorManager(sensorManager)
    , m_database(database)
{
    m_soket = new QTcpSocket(this);

    connect(m_soket, &QTcpSocket::connected, this, &WifiManager::soketBaglandi);
    connect(m_soket, &QTcpSocket::disconnected, this, &WifiManager::soketAyrildi);
    connect(m_soket, &QTcpSocket::errorOccurred, this, &WifiManager::soketHata);
    connect(m_soket, &QTcpSocket::readyRead, this, &WifiManager::veriHazir);

    m_hareketsizlikTimer.setInterval(2 * 60 * 1000);
    connect(&m_hareketsizlikTimer, &QTimer::timeout, this, &WifiManager::hareketsizlikKontrolEt);

    m_veriBekciTimer.setInterval(500);
    connect(&m_veriBekciTimer, &QTimer::timeout, this, &WifiManager::veriAkisiniKontrolEt);

    m_yenidenBaglanmaTimer.setSingleShot(true);
    m_yenidenBaglanmaTimer.setInterval(YENIDEN_BAGLANMA_ARALIGI_MS);
    connect(&m_yenidenBaglanmaTimer, &QTimer::timeout, this, &WifiManager::baglan);

    m_baglantiZamanAsimiTimer.setSingleShot(true);
    m_baglantiZamanAsimiTimer.setInterval(BAGLANTI_ZAMAN_ASIMI_MS);
    connect(&m_baglantiZamanAsimiTimer, &QTimer::timeout, this, &WifiManager::baglantiZamanAsimiOldu);
}

bool WifiManager::baglandi() const { return m_baglandi; }
bool WifiManager::baglaniyor() const { return m_baglaniyor; }
QString WifiManager::durumMesaji() const { return m_durumMesaji; }

void WifiManager::baglan()
{
    m_kullaniciKesti = false;
    m_yenidenBaglanmaTimer.stop();

    if (m_baglandi || m_baglaniyor) return;

    if (m_soket->state() != QAbstractSocket::UnconnectedState) {
        m_soket->abort();
    }

    m_baglaniyor = true;
    emit baglaniyorChanged();
    durumGuncelle("SLIPER-ESP32'ye baglaniliyor...");

    m_soket->connectToHost(QString(ESP32_IP), ESP32_PORT);
    m_baglantiZamanAsimiTimer.start();
}

void WifiManager::baglantiyiKes()
{
    m_kullaniciKesti = true;
    m_yenidenBaglanmaTimer.stop();
    m_baglantiZamanAsimiTimer.stop();
    m_soket->abort();
    if (m_baglandi || m_baglaniyor) {
        soketAyrildi();
    }
}
```

```
void WifiManager::yenidenBaglanmayiPlanla()
{
    if (m_kullaniciKesti) return;
    if (m_yenidenBaglanmaTimer.isActive()) return;

    qDebug() << "Baglanti koptu," << YENIDEN_BAGLANMA_ARALIGI_MS << "ms sonra yeniden denenecek.";
    m_yenidenBaglanmaTimer.start();
}

void WifiManager::baglantiZamanAsimiOldu()
{
    if (!m_baglaniyor) return;

    qWarning() << "Baglanti denemesi zaman asimina ugradi, soket iptal ediliyor.";
    m_soket->abort();
    m_baglaniyor = false;
    emit baglaniyorChanged();
    durumGuncelle("Baglanti kurulamadi, yeniden deneniyor...");
    yenidenBaglanmayiPlanla();
}

void WifiManager::kalibrasyonYenidenYukle()
{
    kalibrasyonYukle();
}

double WifiManager::daraBasinciMbar() const
{
    return m_daraBasinciMbar;
}

double WifiManager::daraUstSiniriMbar() const
{
    return SliperModel::DARA_UST_SINIRI_MBAR;
}

bool WifiManager::daraAl()
{
    if (!m_sensorManager || !m_sensorManager->veriGecerli()) {
        qWarning() << "Dara alinamadi: gecerli sensor verisi yok.";
        return false;
    }

    const double yeniDara = m_darasizBasincMbar;

    if (std::abs(yeniDara) > SliperModel::DARA_UST_SINIRI_MBAR) {
        qWarning() << "Dara reddedildi, ust sinir asildi:" << yeniDara << "mbar >"
                   << SliperModel::DARA_UST_SINIRI_MBAR << "mbar";
        return false;
    }

    m_daraBasinciMbar = yeniDara;
    emit daraBasinciMbarChanged();
    qDebug() << "Dara alindi:" << m_daraBasinciMbar << "mbar";
    return true;
}

void WifiManager::daraSifirla()
{
    if (m_daraBasinciMbar == 0.0) return;
    m_daraBasinciMbar = 0.0;
    emit daraBasinciMbarChanged();
    qDebug() << "Dara sifirlandi.";
}

void WifiManager::kalibrasyonYukle()
{
    if (!m_database) return;

    m_loadCellHamDegerleri.clear();
```

```
    m_loadCellKiloDegerleri.clear();
    const QVariantList loadCellNoktalari = m_database->loadCellNoktalariGetir();
    for (const QVariant &kayit : loadCellNoktalari) {
        const QVariantMap nokta = kayit.toMap();
        m_loadCellHamDegerleri.append(nokta["hamDeger"].toDouble());
        m_loadCellKiloDegerleri.append(nokta["hedefKg"].toDouble());
    }
    qDebug() << "Load cell" << m_loadCellHamDegerleri.size() << "nokta yuklendi.";

    m_mesafeHamDegerleri.clear();
    m_mesafeMmDegerleri.clear();
    const QVariantList mesafeNoktalari = m_database->mesafeNoktalariGetir();
    for (const QVariant &kayit : mesafeNoktalari) {
        const QVariantMap nokta = kayit.toMap();
        m_mesafeHamDegerleri.append(nokta["hamDeger"].toDouble());
        m_mesafeMmDegerleri.append(nokta["hedefMm"].toDouble());
    }
    qDebug() << "Mesafe" << m_mesafeHamDegerleri.size() << "nokta yuklendi.";

    const QVariantMap konumSinirlari = m_database->kalibrasyonGetir("konum_sinirlari");
    if (konumSinirlari.value("mevcut").toBool()) {
        const double ustKonumMm = konumSinirlari.value("deger1").toDouble();
        const double altKonumMm = konumSinirlari.value("deger2").toDouble();
        m_konumYonu = ustKonumMm < altKonumMm ? -1.0 : 1.0;
    } else {
        m_konumYonu = -1.0;
    }

    QVariantMap egimKal = m_database->egimKalibrasyonuGetir();
    if (egimKal.value("mevcut").toBool()) {
        m_egimBiasX = egimKal.value("biasX").toDouble();
        m_egimBiasY = egimKal.value("biasY").toDouble();
        m_egimBiasZ = egimKal.value("biasZ").toDouble();
        auto gecerliGain = [](const QVariant &kayitliDeger) {
            const double gain = kayitliDeger.toDouble();
            return std::isfinite(gain) && std::abs(gain) > 1e-9 ? gain : 1.0;
        };
        m_egimGainX = gecerliGain(egimKal.value("gainX"));
        m_egimGainY = gecerliGain(egimKal.value("gainY"));
        m_egimGainZ = gecerliGain(egimKal.value("gainZ"));
        qDebug() << "Egim ivme kalibrasyonu yuklendi.";
    }
}

void WifiManager::soketBaglandi()
{
    m_baglandi = true;
    m_baglaniyor = false;
    m_baglantiZamanAsimiTimer.stop();
    m_yenidenBaglanmaTimer.stop();
    emit baglandiChanged();
    emit baglaniyorChanged();
    durumGuncelle("SLIPER-ESP32 Bağlı");

    m_soket->setSocketOption(QAbstractSocket::LowDelayOption, 1);
    m_soket->setSocketOption(QAbstractSocket::KeepAliveOption, 1);

    m_ilkPaket = true;
    m_agirlikFiltresiBasladi = false;
    m_zamanlayici.restart();
    m_hareketsizlikZamanlayici.restart();
    m_hareketsizlikTimer.start();
    m_sonVeriZamani.restart();
    m_sonBekciTikZamani.restart();
    m_veriBekciTimer.start();
    kalibrasyonYukle();

    if (m_sensorManager) {
        m_sensorManager->veriyiGecersizYap();
        m_sensorManager->sensorArizasiGuncelle(false, false);
```

```
    }

    qDebug() << "ESP32'ye baglanildi:" << ESP32_IP << ESP32_PORT;
}

void WifiManager::soketAyrildi()
{
    m_baglandi = false;
    m_baglaniyor = false;
    m_hareketsizlikTimer.stop();
    m_veriBekciTimer.stop();
    m_baglantiZamanAsimiTimer.stop();
    emit baglandiChanged();
    emit baglaniyorChanged();
    durumGuncelle(m_kullaniciKesti ? "Bağlı Değil" : "Bağlantı koptu, yeniden deneniyor...");

    m_tamponVeri.clear();

    if (m_sensorManager) {
        m_sensorManager->veriyiGecersizYap();
    }

    qDebug() << "ESP32 baglantisi kesildi.";
    yenidenBaglanmayiPlanla();
}

void WifiManager::hareketsizlikKontrolEt()
{
    if (!m_baglandi) return;

    if (m_hareketsizlikZamanlayici.elapsed() >= HAREKETSIZLIK_LIMIT_MS) {
        qWarning() << "2 saatten uzun suredir veri akisi yok, baglanti otomatik kesiliyor.";
        emit hareketsizlikNedeniyleBaglantiKesildi();
        baglantiyiKes();
        durumGuncelle("Hareketsizlik nedeniyle baglanti kesildi");
    }
}

void WifiManager::veriAkisiniKontrolEt()
{
    if (!m_baglandi || !m_sonVeriZamani.isValid()) return;

    const qint64 bekciGecikmesiMs = m_sonBekciTikZamani.isValid()
        ? m_sonBekciTikZamani.elapsed() - m_veriBekciTimer.interval()
        : 0;
    m_sonBekciTikZamani.restart();

    if (bekciGecikmesiMs > BEKCI_GECIKME_TOLERANSI_MS) {
        qWarning() << "Arayuz" << bekciGecikmesiMs << "ms bloklandi, veri kesintisi sayilmiyor.";
        m_sonVeriZamani.restart();
        return;
    }

    const qint64 gecenSureMs = m_sonVeriZamani.elapsed();

    if (gecenSureMs > VERI_KESINTI_BAGLANTI_KES_MS) {
        qWarning() << "Uzun suredir (" << gecenSureMs << "ms) veri gelmiyor, baglanti sonlandiriliyor.";
        m_soket->abort();
        return;
    }

    if (gecenSureMs > VERI_KESINTI_MS) {
        if (m_sensorManager && m_sensorManager->veriGecerli()) {
            qWarning() << "Veri akisi kesildi (" << gecenSureMs << "ms paket yok).";
            m_sensorManager->veriyiGecersizYap();
            durumGuncelle("Veri akışı kesildi");
        }
    }
}
```

```
void WifiManager::soketHata(QAbstractSocket::SocketError hata)
{
    const QString hataMetni = m_soket->errorString();

    m_baglandi = false;
    m_baglaniyor = false;
    m_hareketsizlikTimer.stop();
    m_veriBekciTimer.stop();
    m_baglantiZamanAsimiTimer.stop();

    if (m_soket->state() != QAbstractSocket::UnconnectedState) {
        m_soket->abort();
    }
    m_tamponVeri.clear();

    emit baglandiChanged();
    emit baglaniyorChanged();
    durumGuncelle(m_kullaniciKesti ? "Baglanti hatasi: " + hataMetni
                                   : "Baglanti hatasi, yeniden deneniyor...");

    if (m_sensorManager) {
        m_sensorManager->veriyiGecersizYap();
    }

    qWarning() << "Wifi soket hatasi:" << hata << hataMetni;
    yenidenBaglanmayiPlanla();
}

void WifiManager::veriHazir()
{
    m_sonVeriZamani.restart();
    if (m_durumMesaji == "Veri akışı kesildi") {
        durumGuncelle("SLIPER-ESP32 Bağlı");
    }

    if (m_hareketsizlikZamanlayici.isValid()) {
        m_hareketsizlikZamanlayici.restart();
    }

    m_tamponVeri.append(m_soket->readAll());

    int satirSonu;
    while ((satirSonu = m_tamponVeri.indexOf('\n')) != -1) {
        QByteArray satir = m_tamponVeri.left(satirSonu).trimmed();
        m_tamponVeri.remove(0, satirSonu + 1);

        if (!satir.isEmpty()) {
            jsonSatiriIsle(satir);
        }
    }
}

void WifiManager::jsonSatiriIsle(const QByteArray &satir)
{
    QJsonParseError hata;
    QJsonDocument belge = QJsonDocument::fromJson(satir, &hata);

    if (hata.error != QJsonParseError::NoError || !belge.isObject()) {
        qWarning() << "JSON parse hatasi:" << hata.errorString() << satir;
        return;
    }

    const QJsonObject paket = belge.object();

    const bool mesafeSensoruArizali = !paket.value("tofGecerli").toBool(true);
    const bool yukSensoruArizali = !paket.value("hxGecerli").toBool(true);
    if (m_sensorManager) {
        m_sensorManager->sensorArizasiGuncelle(mesafeSensoruArizali, yukSensoruArizali);
    }
    if (mesafeSensoruArizali || yukSensoruArizali) {
```

```
    if (m_sensorManager) {
        m_sensorManager->veriyiGecersizYap();
    }
    return;
}

const double gelenHamAgirlik = paket.value("hamAgirlik").toDouble();
if (!m_agirlikFiltresiBasladi) {
    m_filtreliHamAgirlik = gelenHamAgirlik;
    m_agirlikFiltresiBasladi = true;
} else if (gelenHamAgirlik != m_oncekiGelenHamAgirlik) {
    m_filtreliHamAgirlik = SliperModel::agirlikFiltrele(m_filtreliHamAgirlik, gelenHamAgirlik);
}
m_oncekiGelenHamAgirlik = gelenHamAgirlik;
const double hamAgirlik = m_filtreliHamAgirlik;

const double hamMesafe = paket.contains("hamMesafe")
    ? paket.value("hamMesafe").toDouble()
    : paket.value("konum").toDouble();
const double accelX = paket.value("accelX").toDouble();
const double accelY = paket.value("accelY").toDouble();
const double accelZ = paket.value("accelZ").toDouble();

if (m_sensorManager) {
    m_sensorManager->hamAgirlikGuncelle(hamAgirlik);
    m_sensorManager->hamMesafeGuncelle(hamMesafe);
    m_sensorManager->hamAccelGuncelle(accelX, accelY, accelZ);

    if (BATARYA_OLCUMU_AKTIF && paket.contains("hamBatarya")) {
        const double hamBatarya = paket.value("hamBatarya").toDouble();
        m_sensorManager->bataryaVoltajGuncelle(SliperModel::bataryaVoltaji(hamBatarya));
    }
}

const bool mesafeDogrudanMm = paket.contains("mesafeMm");
const double sensorCmdenMmCarpani = 10.0;
const double konumKalibreliMm = (mesafeDogrudanMm || m_mesafeHamDegerleri.isEmpty())
    ? hamMesafe * sensorCmdenMmCarpani
    : SliperModel::kalibrasyonTablosundanOku(m_mesafeHamDegerleri, m_mesafeMmDegerleri, hamMesafe);

m_filtreliKonum = m_ilkPaket
    ? konumKalibreliMm
    : SliperModel::konumFiltrele(m_filtreliKonum, konumKalibreliMm);
const double konumMm = m_filtreliKonum;

const double agirlikKg =
    SliperModel::kalibrasyonTablosundanOku(m_loadCellHamDegerleri, m_loadCellKiloDegerleri, hamAgirlik);
m_darasizBasincMbar = SliperModel::agirliktanBasincMbar(agirlikKg);
const double basincMbar = m_darasizBasincMbar - m_daraBasinciMbar;

const double hamZamanMs = paket.contains("t")
    ? paket.value("t").toDouble()
    : static_cast<double>(m_zamanlayici.elapsed());
if (m_zamanBasladi && hamZamanMs + m_zamanOfsetiMs < m_oncekiZamanMs - 1000.0) {
    m_zamanOfsetiMs = m_oncekiZamanMs + 20.0 - hamZamanMs;
}
const double simdikiZamanMs = hamZamanMs + m_zamanOfsetiMs;
m_zamanBasladi = true;

if (m_ilkPaket) {
    m_ilkPaket = false;
    m_canliHizZamanlariS.clear();
    m_canliHizKonumlariMm.clear();
}
const double simdikiZamanS = simdikiZamanMs / 1000.0;
m_canliHizZamanlariS.append(simdikiZamanS);
m_canliHizKonumlariMm.append(konumMm);
while (m_canliHizZamanlariS.first() < simdikiZamanS - SliperModel::HIZ_PENCERESI_S) {
    m_canliHizZamanlariS.removeFirst();
    m_canliHizKonumlariMm.removeFirst();
```

```
    }

    const double hizMs = SliperModel::penceredekiHizMs(m_canliHizZamanlariS, m_canliHizKonumlariMm, m_konumYonu);
    const double debiM3h = SliperModel::hizdanDebiM3h(hizMs);

    m_oncekiZamanMs = simdikiZamanMs;

    const double duzeltilmisX = (accelX - m_egimBiasX) / m_egimGainX;
    const double duzeltilmisY = (accelY - m_egimBiasY) / m_egimGainY;
    const double duzeltilmisZ = (accelZ - m_egimBiasZ) / m_egimGainZ;

    const SliperModel::EgimAcilari egim =
        SliperModel::egimAcilari(duzeltilmisX, duzeltilmisY, duzeltilmisZ);

    if (m_sensorManager) {
        m_sensorManager->veriGuncelle(simdikiZamanMs / 1000.0, basincMbar, konumMm, hizMs, debiM3h,
                                      egim.xDerece, egim.yDerece);
    }
}

void WifiManager::durumGuncelle(const QString &mesaj)
{
    if (m_durumMesaji != mesaj) {
        m_durumMesaji = mesaj;
        emit durumMesajiChanged();
    }
}
```

src\WifiManager.h

04.11.2026 11:45

```
#pragma once
#include <QObject>
#include <QTcpSocket>
#include <QElapsedTimer>
#include <QTimer>
#include <QVector>
#include "SensorManager.h"
#include "Database.h"

class WifiManager : public QObject
{
    Q_OBJECT
    Q_PROPERTY(bool baglandi READ baglandi NOTIFY baglandiChanged)
    Q_PROPERTY(bool baglaniyor READ baglaniyor NOTIFY baglaniyorChanged)
    Q_PROPERTY(QString durumMesaji READ durumMesaji NOTIFY durumMesajiChanged)
    Q_PROPERTY(double daraBasinciMbar READ daraBasinciMbar NOTIFY daraBasinciMbarChanged)
    Q_PROPERTY(double daraUstSiniriMbar READ daraUstSiniriMbar CONSTANT)

public:
    explicit WifiManager(SensorManager *sensorManager, Database *database, QObject *parent = nullptr);

    bool baglandi() const;
    bool baglaniyor() const;
    QString durumMesaji() const;
    double daraBasinciMbar() const;
    double daraUstSiniriMbar() const;

    Q_INVOKABLE void baglan();
    Q_INVOKABLE void baglantiyiKes();
    Q_INVOKABLE void kalibrasyonYenidenYukle();

    Q_INVOKABLE bool daraAl();
    Q_INVOKABLE void daraSifirla();

signals:
    void baglandiChanged();
    void baglaniyorChanged();
    void durumMesajiChanged();
    void daraBasinciMbarChanged();
    void hareketsizlikNedeniyleBaglantiKesildi();

private slots:
    void soketBaglandi();
    void soketAyrildi();
    void soketHata(QAbstractSocket::SocketError hata);
    void veriHazir();
    void hareketsizlikKontrolEt();
    void veriAkisiniKontrolEt();
    void baglantiZamanAsimiOldu();

private:
    void durumGuncelle(const QString &mesaj);
    void jsonSatiriIsle(const QByteArray &satir);
    void kalibrasyonYukle();
    void yenidenBaglanmayiPlanla();

    QTcpSocket *m_soket = nullptr;
    SensorManager *m_sensorManager = nullptr;
    Database *m_database = nullptr;
    QByteArray m_tamponVeri;

    bool m_baglandi = false;
    bool m_baglaniyor = false;
    QString m_durumMesaji = "Bağlı Değil";

    bool m_kullaniciKesti = false;

    bool m_ilkPaket = true;
```

```
    double m_oncekiZamanMs = 0.0;
    QVector<double> m_canliHizZamanlariS;
    QVector<double> m_canliHizKonumlariMm;
    double m_filtreliKonum = 0.0;
    double m_filtreliHamAgirlik = 0.0;
    double m_oncekiGelenHamAgirlik = 0.0;
    bool m_agirlikFiltresiBasladi = false;

    double m_darasizBasincMbar = 0.0;
    double m_daraBasinciMbar = 0.0;
    double m_zamanOfsetiMs = 0.0;
    bool m_zamanBasladi = false;
    double m_konumYonu = -1.0;
    QElapsedTimer m_zamanlayici;
    QElapsedTimer m_hareketsizlikZamanlayici;

    QVector<double> m_loadCellHamDegerleri;
    QVector<double> m_loadCellKiloDegerleri;

    QVector<double> m_mesafeHamDegerleri;
    QVector<double> m_mesafeMmDegerleri;

    double m_egimBiasX = 0.0, m_egimBiasY = 0.0, m_egimBiasZ = 0.0;
    double m_egimGainX = 1.0, m_egimGainY = 1.0, m_egimGainZ = 1.0;

    static constexpr const char *ESP32_IP = "192.168.4.1";
    static constexpr quint16 ESP32_PORT = 8888;

    static constexpr bool BATARYA_OLCUMU_AKTIF = false;

    QTimer m_hareketsizlikTimer;
    QTimer m_veriBekciTimer;
    QElapsedTimer m_sonVeriZamani;
    QElapsedTimer m_sonBekciTikZamani;
    QTimer m_yenidenBaglanmaTimer;
    QTimer m_baglantiZamanAsimiTimer;
    static constexpr int VERI_KESINTI_MS = 1500;
    static constexpr int VERI_KESINTI_BAGLANTI_KES_MS = 5000;
    static constexpr int HAREKETSIZLIK_LIMIT_MS = 2 * 60 * 60 * 1000;
    static constexpr int BEKCI_GECIKME_TOLERANSI_MS = 2000;
    static constexpr int YENIDEN_BAGLANMA_ARALIGI_MS = 2000;
    static constexpr int BAGLANTI_ZAMAN_ASIMI_MS = 8000;
};
```

tools\esp32_baglanti_testi.py

04.11.2026 11:45

```
"""
SLIPER ESP32 ham bağlantı testi.

Qt uygulamasını aradan çıkarır: doğrudan 192.168.4.1:8888'e bağlanıp ham TCP
akışını dinler. Böylece "veri gelmiyor" sorununun kaynağı kesin ayrılır:

  - Hiç bağlanamıyorsa          -> Wi-Fi / AP tarafı (yanlış ağ, sinyal, reset)
  - Bağlanıp hiç bayt gelmiyorsa -> ESP32'nin loop() fonksiyonu kilitlenmiş
                                    (sensör okuması asılı kalmış; bkz. I2C)
  - Veri gelip arada kesiliyorsa -> sinyal / besleme kaynaklı kopma
  - Veri düzgün akıyorsa         -> sorun PC/Qt tarafında

Kullanım (önce PC'yi SLIPER-ESP32 ağına bağla):
    python tools/esp32_baglanti_testi.py
    python tools/esp32_baglanti_testi.py --sure 60
"""

import argparse
import json
import socket
import subprocess
import sys
import time

ESP32_IP = "192.168.4.1"
ESP32_PORT = 8888


def bagli_oldugumuz_agi_yazdir():
    """Testin yanlış ağda çalıştırılması en sık yapılan hata; önce onu gösterir."""
    try:
        cikti = subprocess.run(
            ["netsh", "wlan", "show", "interfaces"],
            capture_output=True, text=True, timeout=10,
        ).stdout
    except Exception as hata:
        print(f"Wi-Fi durumu okunamadi: {hata}")
        return

    for satir in cikti.splitlines():
        temiz = satir.strip()
        if temiz.startswith(("SSID", "Signal", "Rssi", "State")) and "BSSID" not in temiz:
            print("  " + temiz)


def ana():
    ayristirici = argparse.ArgumentParser(description="SLIPER ESP32 ham baglanti testi")
    ayristirici.add_argument("--sure", type=int, default=20, help="dinleme suresi (saniye)")
    ayristirici.add_argument("--ip", default=ESP32_IP)
    ayristirici.add_argument("--port", type=int, default=ESP32_PORT)
    argumanlar = ayristirici.parse_args()

    print("=== Wi-Fi durumu ===")
    bagli_oldugumuz_agi_yazdir()

    print(f"\n=== TCP baglantisi: {argumanlar.ip}:{argumanlar.port} ===")
    soket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    soket.settimeout(5.0)
    baslangic = time.time()
    try:
        soket.connect((argumanlar.ip, argumanlar.port))
    except Exception as hata:
        print(f"BAGLANAMADI: {hata}")
        print("-> PC SLIPER-ESP32 agina bagli degil ya da ESP32 kapali/resetleniyor.")
        return 1
    print(f"Baglanti kuruldu ({(time.time() - baslangic) * 1000:.0f} ms).")

    # --- Akışı dinle: gelen baytlar, satırlar ve en uzun sessizlik aralığı ---
```

```
print(f"\n=== {argumanlar.sure} saniye dinleniyor ===")
soket.settimeout(1.0)
tampon = b""
toplam_bayt = 0
satir_sayisi = 0
bozuk_satir = 0
ilk_satirlar = []
son_veri_zamani = time.time()
en_uzun_sessizlik = 0.0
ilk_zaman_damgasi = None
son_zaman_damgasi = None
dinleme_basi = time.time()

while time.time() - dinleme_basi < argumanlar.sure:
    try:
        parca = soket.recv(4096)
    except socket.timeout:
        sessizlik = time.time() - son_veri_zamani
        en_uzun_sessizlik = max(en_uzun_sessizlik, sessizlik)
        if sessizlik > 2.0:
            print(f"  [{time.time() - dinleme_basi:5.1f}s] {sessizlik:.1f} saniyedir veri yok")
        continue
    except Exception as hata:
        print(f"  Okuma hatasi: {hata}")
        break

    if not parca:
        print(f"  [{time.time() - dinleme_basi:5.1f}s] ESP32 baglantiyi kapatti.")
        break

    en_uzun_sessizlik = max(en_uzun_sessizlik, time.time() - son_veri_zamani)
    son_veri_zamani = time.time()
    toplam_bayt += len(parca)
    tampon += parca

    while b"\n" in tampon:
        satir, tampon = tampon.split(b"\n", 1)
        satir = satir.strip()
        if not satir:
            continue
        satir_sayisi += 1
        try:
            paket = json.loads(satir)
            zaman = paket.get("t")
            if zaman is not None:
                if ilk_zaman_damgasi is None:
                    ilk_zaman_damgasi = zaman
                son_zaman_damgasi = zaman
        except Exception:
            bozuk_satir += 1
        if len(ilk_satirlar) < 3:
            ilk_satirlar.append(satir.decode("utf-8", "replace"))

soket.close()
gecen_sure = time.time() - dinleme_basi

# --- Özet ve yorum ---
print("\n=== SONUC ===")
print(f"Sure            : {gecen_sure:.1f} s")
print(f"Gelen bayt      : {toplam_bayt}")
print(f"Gelen satir     : {satir_sayisi}  ({satir_sayisi / gecen_sure:.1f} paket/s, beklenen ~50)")
print(f"Bozuk satir     : {bozuk_satir}")
print(f"En uzun sessizlik: {en_uzun_sessizlik:.1f} s")
if ilk_zaman_damgasi is not None and son_zaman_damgasi is not None:
    print(f"ESP32 zamani    : {ilk_zaman_damgasi} -> {son_zaman_damgasi} ms")
    if son_zaman_damgasi < ilk_zaman_damgasi:
        print("  DIKKAT: zaman geri gitti -> ESP32 test sirasinda yeniden basladi (reset).")
for satir in ilk_satirlar:
    print(f"Ornek paket     : {satir}")
```

```
    print("\n=== YORUM ===")
    if toplam_bayt == 0:
        print("TCP baglandi ama TEK BAYT gelmedi.")
        print("-> Wi-Fi ve TCP katmani ESP32'de ayri bir gorevde calisir; bu yuzden")
        print("   Arduino loop() tamamen kilitlenmis olsa bile baglanti kurulur.")
        print("   Sorun ESP32 firmware'inde: sensor okumasi (buyuk ihtimalle ADS1115")
        print("   veya MPU6050 I2C) donusumu bekleyen sonsuz dongude asili kalmis.")
        print("   Yapilacak: ESP32'nin gucunu kesip acin; duzelmezse I2C (SDA/SCL) ve")
        print("   ADS1115 kablolarini kontrol edin, ardindan guncel firmware'i yukleyin.")
    elif satir_sayisi / gecen_sure < 25:
        print("Veri geliyor ama hiz beklenenin cok altinda (~50 paket/s olmali).")
        print("-> Zayif Wi-Fi sinyali veya ESP32 loop'unda araliklar.")
    elif en_uzun_sessizlik > 1.5:
        print("Akis genel olarak saglikli ama araliklarla kesiliyor.")
        print("-> Sinyal zayifligi ya da ESP32 beslemesi (brownout) incelenmeli.")
    else:
        print("Veri akisi saglikli. Sorun bu durumda PC/Qt tarafindadir.")

    return 0


if __name__ == "__main__":
    sys.exit(ana())
```

tools\esp32_seri_dinle.py

04.11.2026 11:45

```
"""
SLIPER ESP32 seri port dinleyici.

Kartın açılış mesajlarını ve saniyelik durum özetini okur. Özellikle şu iki
satır aranır:

  "Yeniden baslatma sebebi: ..."  -> kart beklenmedik şekilde yeniden
                                     başlıyorsa sebebi (watchdog / brownout)
  "[12s] istemci=1 gonderilen=600 ..." -> loop() çalışıyor; bu satır durursa
                                     döngü kilitlenmiş demektir

Kullanım:
    python tools/esp32_seri_dinle.py            # 20 saniye dinler
    python tools/esp32_seri_dinle.py --sure 60 --port COM14
"""

import argparse
import sys
import time

try:
    import serial  # pyserial
except ImportError:
    print("pyserial kurulu degil. Kurmak icin: python -m pip install pyserial")
    sys.exit(1)


def ana():
    ayristirici = argparse.ArgumentParser(description="SLIPER ESP32 seri port dinleyici")
    ayristirici.add_argument("--port", default="COM14")
    ayristirici.add_argument("--sure", type=int, default=20, help="dinleme suresi (saniye)")
    ayristirici.add_argument("--hiz", type=int, default=115200)
    argumanlar = ayristirici.parse_args()

    try:
        baglanti = serial.Serial(argumanlar.port, argumanlar.hiz, timeout=1)
    except Exception as hata:
        print(f"{argumanlar.port} acilamadi: {hata}")
        print("-> Arduino IDE'nin seri monitoru acik olabilir; kapatip tekrar deneyin.")
        return 1

    print(f"{argumanlar.port} dinleniyor ({argumanlar.sure} s). Kartin resetlenmesi icin bekleniyor...\n")

    baslangic = time.time()
    son_satir_zamani = time.time()
    en_uzun_sessizlik = 0.0
    satir_sayisi = 0

    try:
        while time.time() - baslangic < argumanlar.sure:
            ham = baglanti.readline()
            if not ham:
                en_uzun_sessizlik = max(en_uzun_sessizlik, time.time() - son_satir_zamani)
                continue
            son_satir_zamani = time.time()
            satir_sayisi += 1
            satir = ham.decode("utf-8", "replace").rstrip()
            print(f"[{time.time() - baslangic:6.2f}s] {satir}")
    except KeyboardInterrupt:
        pass
    finally:
        baglanti.close()

    print(f"\n--- {satir_sayisi} satir okundu, en uzun sessizlik {en_uzun_sessizlik:.1f} s ---")
    if satir_sayisi == 0:
        print("Hic cikti yok -> kart calismiyor ya da USB CDC kapali yuklenmis olabilir.")
    elif en_uzun_sessizlik > 3:
        print("Durum satiri uzun sure durmus -> loop() kilitleniyor.")
```

```
    return 0


if __name__ == "__main__":
    sys.exit(ana())
```

tools\pmaks_yeniden_hesapla.py

04.11.2026 11:45

```
"""Gecmis stroke kayitlarinda Pmax'i saklanan ham veriden yeniden turetir.

NEDEN: Pmax filtresinin penceresi ornek sayisiyla tanimliydi (PMAKS_PENCERE = 15)
ve "50 Hz'de 0.3 s" varsayimina dayaniyordu. Kayitli ham veri orneklerin ~4.5 Hz
geldigini gosteriyor; 15 ornek o hizda 3.3 saniyelik ortalama demekti, yani
stroke'un kendisinden uzundu. Basinc tepesi duz tabanla birlikte ortalanip
siliniyor, Pmax 30-70 mbar dusuk cikiyordu. Pencere SliperModel'de zaman tabanli
hale getirildi (PMAKS_PENCERE_S = 0.30); bu betik ayni duzeltmeyi gecmis
kayitlara uygular.

NE DEGISIR: yalnizca pMaks ve ondan turetilen basinc (p = Pmax - P0l).
Ham veri, P0l, P0r, debi, hiz ve konum ellenmez.

GERI ALMA: orijinal degerler pMaksOrijinal / basincOrijinal sutunlarinda saklanir,
ayrica calistirmadan once veritabaninin tam yedegi alinir. Betik tekrar
calistirilabilir; orijinal sutunlar bir kez doldurulur, uzerine yazilmaz.
"""

import json
import os
import shutil
import sqlite3
import sys
from datetime import datetime

PMAKS_PENCERE_S = 0.30
PMAKS_MIN_ORNEK = 3

VERITABANI_YOLU = os.path.join(
    os.environ.get("LOCALAPPDATA", ""), "SLIPER", "database", "sliper.db")


def filtreli_maksimum(zamanlar, basinclar, pencere_s=PMAKS_PENCERE_S):
    """SliperModel::filtreliMaksimum ile ayni: SURE ile tanimli merkezi hareketli
    ortalamalarin en buyugu, pencerede en az PMAKS_MIN_ORNEK ornek bulunmak sartiyla."""
    ornek_sayisi = min(len(zamanlar), len(basinclar))
    if ornek_sayisi == 0:
        return 0.0

    yari_pencere = pencere_s / 2.0
    en_buyuk = float("-inf")
    for merkez in range(ornek_sayisi):
        merkez_zamani = zamanlar[merkez]

        pencere_basi = merkez
        while pencere_basi > 0 and merkez_zamani - zamanlar[pencere_basi - 1] <= yari_pencere:
            pencere_basi -= 1
        pencere_sonu = merkez
        while (pencere_sonu < ornek_sayisi - 1
               and zamanlar[pencere_sonu + 1] - merkez_zamani <= yari_pencere):
            pencere_sonu += 1

        # Sure sarti az ornek biraktiysa pencere iki yana dengeli genisletilir
        while (pencere_sonu - pencere_basi + 1 < PMAKS_MIN_ORNEK
               and (pencere_basi > 0 or pencere_sonu < ornek_sayisi - 1)):
            sola_genislet = pencere_basi > 0 and (
                pencere_sonu == ornek_sayisi - 1
                or merkez_zamani - zamanlar[pencere_basi - 1]
                <= zamanlar[pencere_sonu + 1] - merkez_zamani)
            if sola_genislet:
                pencere_basi -= 1
            else:
                pencere_sonu += 1

        pencere = basinclar[pencere_basi:pencere_sonu + 1]
        en_buyuk = max(en_buyuk, sum(pencere) / len(pencere))
    return en_buyuk
```

```
def orijinal_sutunlari_ekle(imlec):
    mevcut_sutunlar = {satir[1] for satir in imlec.execute("PRAGMA table_info(Strokelar)")}
    for sutun in ("pMaksOrijinal", "basincOrijinal"):
        if sutun not in mevcut_sutunlar:
            imlec.execute(f"ALTER TABLE Strokelar ADD COLUMN {sutun} REAL")


def main():
    if not os.path.exists(VERITABANI_YOLU):
        sys.exit(f"Veritabani bulunamadi: {VERITABANI_YOLU}")

    damga = datetime.now().strftime("%Y%m%d_%H%M%S")
    yedek_yolu = f"{VERITABANI_YOLU}.yedek_{damga}"
    shutil.copy2(VERITABANI_YOLU, yedek_yolu)
    print(f"Yedek alindi: {yedek_yolu}\n")

    baglanti = sqlite3.connect(VERITABANI_YOLU)
    imlec = baglanti.cursor()
    orijinal_sutunlari_ekle(imlec)

    satirlar = imlec.execute(
        "SELECT id, olcumId, pMaks, p0l, basinc, gecerli, gecersizNedeni, hamVeri, "
        "pMaksOrijinal FROM Strokelar "
        "WHERE hamVeri IS NOT NULL AND hamVeri != '' ORDER BY id"
    ).fetchall()

    guncellenen = 0
    gecerliye_donen = 0
    toplam_kazanc = 0.0
    olcum_basina_kazanc = {}

    for (stroke_id, olcum_id, pmaks_eski, p0l, basinc_eski, gecerli,
         gecersiz_nedeni, ham, pmaks_orijinal) in satirlar:
        try:
            veri = json.loads(ham)
            zamanlar, basinclar = veri["t"], veri["p"]
            bitis_zamani = veri["tBitis"]
        except (ValueError, KeyError):
            print(f"  stroke {stroke_id}: ham veri okunamadi, atlandi")
            continue

        if not zamanlar or not basinclar:
            continue

        # Pmax yalnizca inis bolumunden hesaplanir (bitis sonrasi P0r penceresi haric)
        inis_sonu = next((i for i, t in enumerate(zamanlar) if t > bitis_zamani), len(zamanlar))
        inis_zamanlari = zamanlar[:inis_sonu]
        inis_basinclari = basinclar[:inis_sonu]
        if not inis_basinclari:
            continue

        pmaks_yeni = filtreli_maksimum(inis_zamanlari, inis_basinclari)
        basinc_yeni = pmaks_yeni - p0l

        # Orijinaller yalnizca ilk calistirmada yazilir
        if pmaks_orijinal is None:
            imlec.execute(
                "UPDATE Strokelar SET pMaksOrijinal = ?, basincOrijinal = ? WHERE id = ?",
                (pmaks_eski, basinc_eski, stroke_id))

        imlec.execute("UPDATE Strokelar SET pMaks = ?, basinc = ? WHERE id = ?",
                      (pmaks_yeni, basinc_yeni, stroke_id))

        # Pmax yukseldigi icin "basinc <= 0" diye gecersiz sayilan stroke'lar
        # artik gecerli olabilir (SliperModel::strokeGecersizNedeni)
        if gecersiz_nedeni == "basinc" and basinc_yeni > 0.0:
            imlec.execute(
                "UPDATE Strokelar SET gecerli = 1, gecersizNedeni = '' WHERE id = ?",
                (stroke_id,))
```

```
            gecerliye_donen += 1

        guncellenen += 1
        kazanc = pmaks_yeni - pmaks_eski
        toplam_kazanc += kazanc
        olcum_basina_kazanc.setdefault(olcum_id, []).append(kazanc)

    baglanti.commit()

    print(f"{guncellenen} stroke yeniden hesaplandi.")
    if gecerliye_donen:
        print(f"{gecerliye_donen} stroke 'basinc' gecersizliginden kurtuldu.")
    print(f"Ortalama Pmax kazanci: {toplam_kazanc / max(guncellenen, 1):+.1f} mbar\n")

    print("Olcum basina ortalama Pmax kazanci [mbar]:")
    for olcum_id in sorted(olcum_basina_kazanc):
        kazanclar = olcum_basina_kazanc[olcum_id]
        print(f"  {olcum_id:>4} : {sum(kazanclar) / len(kazanclar):+6.1f}  ({len(kazanclar)} stroke)")

    baglanti.close()


if __name__ == "__main__":
    main()
```